

Physlib Monthly Report

1–28 February 2025

Contributors: Joseph Tooby-Smith, kuotsanhsu

Generated 14 August 2026 from commit 733bf4f.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–28 February 2025 on branch `master` of [leanprover-community/physlib](#), between commits `6aab0ba` and `733bf4f`. Over that period 2 contributors landed 191 commits, changing 31,297 lines across 299 files and adding 1,032 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.Mathematics.Fin</code>	3
2.2	<code>PhysLean.Mathematics.Fin.Involutions</code>	7
2.3	<code>PhysLean.Mathematics.List</code>	11
2.4	<code>PhysLean.Mathematics.List.InsertIdx</code>	21
2.5	<code>PhysLean.Mathematics.List.InsertionSort</code>	24
2.6	<code>PhysLean.Mathematics.PiTensorProduct</code>	31
2.7	<code>PhysLean.Mathematics.RatComplexNum</code>	34
2.8	<code>PhysLean.Mathematics.SpecialFunctions.PhyscisisistsHermite</code>	36
2.9	<code>PhysLean.Meta.Basic</code>	42
2.10	<code>PhysLean.Meta.Informal.Post</code>	45
2.11	<code>PhysLean.Meta.Notes.Basic</code>	45
2.12	<code>PhysLean.Meta.Notes.HTMLNote</code>	45
2.13	<code>PhysLean.Meta.Notes.NoteFile</code>	46
2.14	<code>PhysLean.Meta.Notes.ToHTML</code>	46
2.15	<code>PhysLean.Meta.Remark.Basic</code>	48
2.16	<code>PhysLean.Meta.Remark.Properties</code>	48
2.17	<code>PhysLean.Meta.TODO.Basic</code>	48
2.18	<code>PhysLean.Particles.StandardModel.HiggsBoson.Basic</code>	49
2.19	<code>PhysLean.Particles.StandardModel.HiggsBoson.GaugeAction</code>	49
2.20	<code>PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Basic</code>	49
2.21	<code>PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Grading</code>	52

2.22	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.Basic . . .	56
2.23	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.Lemmas . .	57
2.24	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.WickContractions	59
2.25	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.StaticWickTerm	59
2.26	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.SuperCommute	61
2.27	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.TimeContraction	63
2.28	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.TimeOrder	64
2.29	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Universality	65
2.30	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.WicksTheoremNormal . .	66
2.31	PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.WickTerm	68
2.32	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic	70
2.33	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading	74
2.34	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder	77
2.35	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormTimeOrder . . .	82
2.36	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute . . .	82
2.37	PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder	89
2.38	PhysLean.QFT.PerturbationTheory.FieldSpecification.Basic	92
2.39	PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp	93
2.40	PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder	95
2.41	PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder	96
2.42	PhysLean.QFT.PerturbationTheory.FieldStatistics.OfFinset	97
2.43	PhysLean.QFT.PerturbationTheory.WickContraction.Basic	98
2.44	PhysLean.QFT.PerturbationTheory.WickContraction.Card	99
2.45	PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv	101
2.46	PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat	101
2.47	PhysLean.QFT.PerturbationTheory.WickContraction.Join	102
2.48	PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic	108
2.49	PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertNone . . .	108
2.50	PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertSome . . .	108
2.51	PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Join	109
2.52	PhysLean.QFT.PerturbationTheory.WickContraction.Singleton	112
2.53	PhysLean.QFT.PerturbationTheory.WickContraction.StaticContract	114
2.54	PhysLean.QFT.PerturbationTheory.WickContraction.SubContraction	115
2.55	PhysLean.QFT.PerturbationTheory.WickContraction.TimeCond	117
2.56	PhysLean.QFT.PerturbationTheory.WickContraction.TimeContract	121
2.57	PhysLean.QFT.PerturbationTheory.WickContraction.Uncontracted	123
2.58	PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedList . . .	123
2.59	PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Basic . . .	126
2.60	PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Completeness	128
2.61	PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction	130
2.62	PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TISE	133
2.63	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic	135
2.64	PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Parity	137
2.65	PhysLean.Relativity.Lorentz.ComplexTensor.Basic	137
2.66	PhysLean.Relativity.Lorentz.ComplexTensor.Basis	138
2.67	PhysLean.Relativity.Lorentz.ComplexTensor.Metrics.Basis	138
2.68	PhysLean.Relativity.Lorentz.ComplexTensor.OfRat	140
2.69	PhysLean.Relativity.Lorentz.ComplexTensor.Units.Basis	142
2.70	PhysLean.Relativity.Lorentz.ComplexVector.Basic	143
2.71	PhysLean.Relativity.Lorentz.PauliMatrices.Basic	144
2.72	PhysLean.Relativity.Lorentz.PauliMatrices.Basis	146

2.73	<code>PhysLean.Relativity.Lorentz.PauliMatrices.Relations</code>	147
2.74	<code>PhysLean.Relativity.Tensors.TensorSpecies.Basic</code>	148
2.75	<code>PhysLean.Relativity.Tensors.TensorSpecies.Basis</code>	148
2.76	<code>PhysLean.Relativity.Tensors.TensorSpecies.OfInt</code>	152
2.77	<code>PhysLean.Relativity.Tensors.Tree.Basic</code>	152
2.78	<code>scripts.check_file_imports</code>	154
2.79	<code>scripts.find_TODOs</code>	154
2.80	<code>scripts.hepLean_style_lint</code>	154
2.81	<code>scripts.MetaPrograms.informal</code>	154
2.82	<code>scripts.MetaPrograms.notes</code>	156

1 Introduction

During Feb 2025, 2 contributors submitted 191 commits that changed 31,297 lines (+21,688 / -9,609) across 299 files spanning 32 top-level areas of the repository. Of those, 68 files were newly added and 30 were removed outright. This report catalogues 1,032 newly added Lean declarations: 797 lemmas, 180 defs, 32 instances, 11 structures, 6 theorems, 3 abbrevs, and 3 inductives.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	38,010	190
kuotsanhsu	1,791	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

No reviewers identified for this month.

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/QFT	104	+27	–	+9,827	-1,363
docs/_data	2	+2	–	+4,071	-0
HepLean/PerturbationTheory	13	–	-13	+0	-3,953
PhysLean/Relativity	74	+10	–	+3,085	-477
HepLean/Lorentz	6	–	-6	+0	-1,807
PhysLean/QuantumMechanics	6	+6	–	+1,731	-0
PhysLean/Mathematics	11	+2	–	+895	-59
scripts/MetaPrograms	7	–	-1	+454	-492
HepLean/Meta	5	–	-5	+0	-656
PhysLean/Meta	12	+5	–	+486	-40
PhysLean/Particles	18	+4	–	+416	-98
docs/CuratedNotes	3	+2	–	+222	-14
PhysLean.lean	1	+1	–	+232	-0
HepLean.lean	1	–	-1	+0	-199
scripts	12	–	–	+83	-93
HepLean/BeyondTheStandardModel	3	–	-3	+0	-170
HepLean/StandardModel	1	–	-1	+0	-114
docs	5	+1	–	+55	-29
lake-manifest.json	1	–	–	+19	-19
README.md	1	–	–	+11	-10
PhysLean/ClassicalMechanics	1	+1	–	+17	-0
.github/workflows	2	–	–	+6	-6

PhysLean/Cosmology	1	+1	—	+12	-0
PhysLean/Electrodynamics	1	+1	—	+12	-0
PhysLean/Optics	1	+1	—	+12	-0
PhysLean/StatisticalMechanics	1	+1	—	+12	-0
PhysLean/Thermodynamics	1	+1	—	+12	-0
lakefile.toml	1	—	—	+6	-6
PhysLean/CondensedMatter	1	+1	—	+7	-0
docs/_layouts	1	—	—	+4	-3
lean-toolchain	1	—	—	+1	-1
docs/assets	1	+1	—	+0	-0

1.4 New declarations by kind

Kind	Count
lemma	797
def	180
instance	32
structure	11
theorem	6
abbrev	3
inductive	3

2 Details by file

2.1 PhysLean.Mathematics.Fin

[PhysLean/Mathematics/Fin.lean](#) on GitHub.

def predAboveI

[\[source\]](#)

Given a i and x in $\text{Fin } n.\text{succ}.\text{succ}$ returns an element of $\text{Fin } n.\text{succ}$ subtracting 1 if $i.\text{val} \leq x.\text{val}$ else casting x .

```
def predAboveI (i x : Fin n.succ.succ) : Fin n.succ
```

lemma predAboveI_self

[\[source\]](#)

```
lemma predAboveI_self (i : Fin n.succ.succ) : predAboveI i i = (i.val - 1, by omega)
```

lemma predAboveI_succAbove

[\[source\]](#)

@[simp]

```
lemma predAboveI_succAbove (i : Fin n.succ.succ) (x : Fin n.succ) :
  predAboveI i (Fin.succAbove i x) = x
```

lemma succsAbove_predAboveI

[\[source\]](#)

```
lemma succsAbove_predAboveI {i x : Fin n.succ.succ} (h : i ≠ x) :
  Fin.succAbove i (predAboveI i x) = x
```

lemma predAboveI_eq_iff[\[source\]](#)

```
lemma predAboveI_eq_iff {i x : Fin n.succ.succ} (h : i ≠ x) (y : Fin n.succ) :
  y = predAboveI i x ↔ i.succAbove y = x
```

lemma predAboveI_lt[\[source\]](#)

```
lemma predAboveI_lt {i x : Fin n.succ.succ} (h : x.val < i.val) :
  predAboveI i x = {x.val, by omega}
```

lemma predAboveI_ge[\[source\]](#)

```
lemma predAboveI_ge {i x : Fin n.succ.succ} (h : i.val < x.val) :
  predAboveI i x = {x.val - 1, by omega}
```

lemma succAbove_succAbove_predAboveI[\[source\]](#)

```
lemma succAbove_succAbove_predAboveI (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :
  i.succAbove (j.succAbove x) =
  (i.succAbove j).succAbove ((predAboveI (i.succAbove j) i).succAbove x)
```

def finExtractOne[\[source\]](#)

The equivalence between $\text{Fin } n.\text{succ}$ and $\text{Fin } 1 \oplus \text{Fin } n$ extracting the i th component.

```
def finExtractOne {n : ℕ} (i : Fin n.succ) : Fin n.succ ≈ Fin 1 ⊕ Fin n
```

lemma finExtractOne_apply_eq[\[source\]](#)

```
@[simp]
lemma finExtractOne_apply_eq {n : ℕ} (i : Fin n.succ) :
  finExtractOne i i = Sum.inl 0
```

lemma finExtractOne_symm_inr[\[source\]](#)

```
lemma finExtractOne_symm_inr {n : ℕ} (i : Fin n.succ) :
  (finExtractOne i).symm ∘ Sum.inr = i.succAbove
```

lemma finExtractOne_symm_inr_apply[\[source\]](#)

```
@[simp]
lemma finExtractOne_symm_inr_apply {n : ℕ} (i : Fin n.succ) (x : Fin n) :
  (finExtractOne i).symm (Sum.inr x) = i.succAbove x
```

lemma finExtractOne_symm_inl_apply[\[source\]](#)

```
@[simp]
lemma finExtractOne_symm_inl_apply {n : ℕ} (i : Fin n.succ) :
  (finExtractOne i).symm (Sum.inl 0) = i
```

lemma finExtractOne_apply_neq[\[source\]](#)

```
lemma finExtractOne_apply_neq {n : ℕ} (i j : Fin n.succ.succ) (hij : i ≠ j) :
  finExtractOne i j = Sum.inr (predAboveI i j)
```

def finExtractOnPermHom[\[source\]](#)

Given an equivalence $\text{Fin } n.\text{succ.succ} \approx \text{Fin } n.\text{succ.succ}$, and an $i : \text{Fin } n.\text{succ.succ}$, the map $\text{Fin } n.\text{succ} \rightarrow \text{Fin } n.\text{succ}$ obtained by dropping i and its image.

```
def finExtractOnPermHom {m : ℕ} (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  Fin n.succ → Fin m.succ
```

lemma finExtractOnPermHom_inv[\[source\]](#)

```
lemma finExtractOnPermHom_inv {m : ℕ} (i : Fin n.succ.succ)
  (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  (finExtractOnPermHom (σ i) σ.symm) ∘ (finExtractOnPermHom i σ) = id
```

def finExtractOnePerm[\[source\]](#)

Given an equivalence $\text{Fin } n.\text{succ.succ} \approx \text{Fin } n.\text{succ.succ}$, and an $i : \text{Fin } n.\text{succ.succ}$, the equivalence $\text{Fin } n.\text{succ} \approx \text{Fin } n.\text{succ}$ obtained by dropping i and its image.

```
def finExtractOnePerm {m : ℕ} (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin m.succ.succ) :
  Fin n.succ ≈ Fin m.succ where
```

lemma finExtractOnePerm_equiv[\[source\]](#)

```
lemma finExtractOnePerm_equiv {n m : ℕ} (e : Fin n.succ.succ ≈ Fin m.succ.succ)
  (i : Fin n.succ.succ) :
  e ∘ i.succAbove = (e i).succAbove ∘ finExtractOnePerm i e
```

lemma finExtractOnePerm_apply[\[source\]](#)

```
@[simp]
lemma finExtractOnePerm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : finExtractOnePerm i σ x = predAboveI (σ i)
  (σ ((finExtractOne i).symm (Sum.inr x)))
```

lemma finExtractOnePerm_symm_apply[\[source\]](#)

```
@[simp]
lemma finExtractOnePerm_symm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : (finExtractOnePerm i σ).symm x = predAboveI (σ.symm (σ i))
  (σ.symm ((finExtractOne (σ i)).symm (Sum.inr x)))
```

def finExtractTwo[\[source\]](#)

The equivalence of types $\text{Fin } n.\text{succ.succ} \approx (\text{Fin } 1 \oplus \text{Fin } 1) \oplus \text{Fin } n$ extracting the i and $(i.\text{succAbove } j)$.

```
def finExtractTwo {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  Fin n.succ.succ ≈ (Fin 1 ⊕ Fin 1) ⊕ Fin n
```

lemma finExtractTwo_apply_fst

[\[source\]](#)

```
@[simp]
lemma finExtractTwo_apply_fst {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  finExtractTwo i j i = Sum.inl (Sum.inl 0)
```

lemma finExtractTwo_symm_inr

[\[source\]](#)

```
lemma finExtractTwo_symm_inr {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm ∘ Sum.inr = i.succAbove ∘ j.succAbove
```

lemma finExtractTwo_symm_inr_apply

[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) (x : Fin n) :
  (finExtractTwo i j).symm (Sum.inr x) = i.succAbove (j.succAbove x)
```

lemma finExtractTwo_symm_inl_inr_apply

[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inl_inr_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm (Sum.inl (Sum.inr 0)) = i.succAbove j
```

lemma finExtractTwo_symm_inl_inl_apply

[\[source\]](#)

```
@[simp]
lemma finExtractTwo_symm_inl_inl_apply {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  (finExtractTwo i j).symm (Sum.inl (Sum.inl 0)) = i
```

lemma finExtractTwo_apply_snd

[\[source\]](#)

```
@[simp]
lemma finExtractTwo_apply_snd {n : ℕ} (i : Fin n.succ.succ) (j : Fin n.succ) :
  finExtractTwo i j (i.succAbove j) = Sum.inl (Sum.inr 0)
```

def finMapToEquiv

[\[source\]](#)

Takes two maps $\text{Fin } n \rightarrow \text{Fin } m$ and returns the equivalence they form.

```
def finMapToEquiv (f1 : Fin n → Fin m) (f2 : Fin m → Fin n)
  (h : ∀ x, f1 (f2 x) = x := by decide)
  (h' : ∀ x, f2 (f1 x) = x := by decide) : Fin n ≈ Fin m where
```

lemma finMapToEquiv_apply

[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin n) :
  finMapToEquiv f1 f2 h h' x = f1 x
```


lemma finMapToEquiv_symm_apply[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_symm_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin m) :
  (finMapToEquiv f1 f2 h h').symm x = f2 x
```

lemma finMapToEquiv_symm_eq[\[source\]](#)

```
lemma finMapToEquiv_symm_eq {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} :
  (finMapToEquiv f1 f2 h h').symm = finMapToEquiv f2 f1 h' h
```

def equivCons[\[source\]](#)

Given an equivalence between $\text{Fin } n$ and $\text{Fin } m$, the induced equivalence between $\text{Fin } n.\text{succ}$ and $\text{Fin } m.\text{succ}$ derived by $\text{Fin}.\text{cons}$.

```
def equivCons {n m : ℕ} (e : Fin n ≈ Fin m) : Fin n.succ ≈ Fin m.succ where
```

lemma equivCons_zero[\[source\]](#)

```
@[simp]
lemma equivCons_zero {n m : ℕ} (e : Fin n ≈ Fin m) :
  equivCons e 0 = 0
```

lemma equivCons_trans[\[source\]](#)

```
@[simp]
lemma equivCons_trans {n m k : ℕ} (e : Fin n ≈ Fin m) (f : Fin m ≈ Fin k) :
  Fin.equivCons (e.trans f) = (Fin.equivCons e).trans (Fin.equivCons f)
```

lemma equivCons_castOrderIso[\[source\]](#)

```
@[simp]
lemma equivCons_castOrderIso {n m : ℕ} (h : n = m) :
  (Fin.equivCons (Fin.castOrderIso h).toEquiv) = (Fin.castOrderIso (by simp [h])).toEquiv
```

lemma equivCons_symm_succ[\[source\]](#)

```
@[simp]
lemma equivCons_symm_succ {n m : ℕ} (e : Fin n ≈ Fin m) (i : ℕ) (hi : i + 1 < m.succ) :
  (Fin.equivCons e).symm (i + 1, hi) = (e.symm (i, Nat.succ_lt_succ_iff.mp hi)).succ
```

lemma equivCons_succ[\[source\]](#)

```
@[simp]
lemma equivCons_succ {n m : ℕ} (e : Fin n ≈ Fin m) (i : ℕ) (hi : i + 1 < n.succ) :
  (Fin.equivCons e) (i + 1, hi) = (e (i, Nat.succ_lt_succ_iff.mp hi)).succ
```

2.2 PhysLean.Mathematics.Fin.Involutions

[PhysLean/Mathematics/Fin/Involutions.lean](#) on GitHub.

def involutionCons[\[source\]](#)

There is an equivalence between involutions of $\text{Fin } n.\text{succ}$ and involutions of $\text{Fin } n$ and an optional valid choice of an element in $\text{Fin } n$ (which is where 0 in $\text{Fin } n.\text{succ}$ will be sent).

```
def involutionCons (n : ℕ) : {f : Fin n.succ → Fin n.succ // Function.Involutive f} ≈
  (f : {f : Fin n → Fin n // Function.Involutive f}) × {i : Option (Fin n) //
    ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} where
```

lemma involutionCons_ext[\[source\]](#)

```
lemma involutionCons_ext {n : ℕ} {f1 f2 : (f : {f : Fin n → Fin n // Function.Involutive f}) ×
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)}}
  (h1 : f1.1 = f2.1) (h2 : f1.2 = Equiv.subtypeEquivRight (by rw [h1]; simp) f2.2) : f1 = f2
```

def involutionAddEquiv[\[source\]](#)

Given an involution of $\text{Fin } n$, the optional choice of an element in $\text{Fin } n$ which maps to itself is equivalent to the optional choice of an element in Fin ($\text{Finset.univ.filter fun i => f.1 i = i}$).card.

```
def involutionAddEquiv {n : ℕ} (f : {f : Fin n → Fin n // Function.Involutive f}) :
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} ≈
  Option (Fin (Finset.univ.filter fun i => f.1 i = i).card)
```

lemma involutionAddEquiv_none_image_zero[\[source\]](#)

```
lemma involutionAddEquiv_none_image_zero {n : ℕ} :
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  → involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none
  → f.1 {0, Nat.zero_lt_succ n} = {0, Nat.zero_lt_succ n}
```

lemma involutionAddEquiv_cast[\[source\]](#)

```
lemma involutionAddEquiv_cast {n : ℕ} {f1 f2 : {f : Fin n → Fin n // Function.Involutive f}}
  (hf : f1 = f2) :
  involutionAddEquiv f1 = (Equiv.subtypeEquivRight (by rw [hf]; simp)).trans
    ((involutionAddEquiv f2).trans (Equiv.optionCongr (finCongr (by rw [hf]))))
```

lemma involutionAddEquiv_cast'[\[source\]](#)

```
lemma involutionAddEquiv_cast' {m : ℕ} {f1 f2 : {f : Fin m → Fin m // Function.Involutive f}}
  {N : ℕ} (hf : f1 = f2) (n : Option (Fin N))
  (hn1 : N = (Finset.filter (fun i => f1.1 i = i) Finset.univ).card)
  (hn2 : N = (Finset.filter (fun i => f2.1 i = i) Finset.univ).card) :
  HEq ((involutionAddEquiv f1).symm (Option.map (finCongr hn1) n))
    ((involutionAddEquiv f2).symm (Option.map (finCongr hn2) n))
```

lemma involutionAddEquiv_none_succ[\[source\]](#)

```
lemma involutionAddEquiv_none_succ {n : ℕ}
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  (h : involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none)
```

```
(x : Fin n) : f.1 x.succ = x.succ ↔ (involutionCons n f).1.1 x = x
```

lemma involutionAddEquiv_isSome_image_zero

[\[source\]](#)

```
lemma involutionAddEquiv_isSome_image_zero {n : ℕ} :
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  → (involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2).isSome
  → ¬ f.1 {0, Nat.zero_lt_succ n} = {0, Nat.zero_lt_succ n}
```

def involutionNoFixedEquivSum

[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ can be separated based on where they sent 0.

```
def involutionNoFixedEquivSum {n : ℕ} :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f
  ∧ ∀ i, f i ≠ i} ≈ Σ (k : Fin n), {f : Fin n.succ → Fin n.succ // Function.Involutive f
  ∧ (∀ i, f i ≠ i) ∧ f 0 = k.succ} where
```

def involutionNoFixedZeroSetEquivEquiv

[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f \ 0$, can be modified by conjugation with an equivalence.

```
def involutionNoFixedZeroSetEquivEquiv {n : ℕ}
  (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧ (∀ i, f i ≠ i) ∧ f 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} where
```

def involutionNoFixedZeroSetEquivSetEquiv

[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f \ 0$ given an equivalence e , can be modified so that only the condition on $f \ 0$ is up-to the equivalence e .

```
def involutionNoFixedZeroSetEquivSetEquiv {n : ℕ} (k : Fin n)
  (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
  (∀ i, f i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ}
```

def involutionNoFixedZeroSetEquivEquiv'

[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ fixing $(e.\text{symm} \circ f \circ e) = k.\text{succ}$ for a given e are equivalent to fixing $f \ (e \ 0) = e \ k.\text{succ}$.

```
def involutionNoFixedZeroSetEquivEquiv' {n : ℕ} (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
  (∀ i, f i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
  (∀ i, f i ≠ i) ∧ f (e 0) = e k.succ}
```

def involutionNoFixedZeroSetEquivSetOne[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ with $f \ 0 = k.\text{succ}$ are equivalent to fixed point involutions with $f \ 0 = 1$.

```
def involutionNoFixedZeroSetEquivSetOne {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = k.succ}
  ≈ {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = 1}
```

def involutionNoFixedSetOne[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ fixing $f \ 0 = 1$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedSetOne {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = 1} ≈ {f : Fin n → Fin n // Function.Involutive f ∧
    (∀ i, f i ≠ i)} where
```

def involutionNoFixedZeroSetEquiv[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ}.\text{succ}$ for fixed $f \ 0 = k.\text{succ}$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedZeroSetEquiv {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f 0 = k.succ}
  ≈ {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedEquivSumSame[\[source\]](#)

The type of fixed point free involutions of $\text{Fin } n.\text{succ}.\text{succ}$ is equivalent to the sum of $\text{Fin } n.\text{succ}$ copies of fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedEquivSumSame {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = Σ ( _ : Fin n.succ),
    {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedZeroEquivProd[\[source\]](#)

Every fixed-point free involutions of $\text{Fin } n.\text{succ}.\text{succ}$ can be decomposed into a element of $\text{Fin } n.\text{succ}$ (where 0 is sent) and a fixed-point free involution of $\text{Fin } n$.

```
def involutionNoFixedZeroEquivProd {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = Fin n.succ ×
    {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

instance Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }[\[source\]](#)

The type of fixed-point free involutions of $\text{Fin } n$ is finite.

```
instance {n : ℕ} : Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }
```

lemma involutionNoFixed_card_succ[\[source\]](#)

```

lemma involutionNoFixed_card_succ {n : ℕ} :
  Fintype.card
    {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = n.succ *
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}

```

lemma involutionNoFixed_card_mul_two[\[source\]](#)

```

lemma involutionNoFixed_card_mul_two : (n : ℕ) →
  Fintype.card {f : Fin (2 * n) → Fin (2 * n) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = (2 * n - 1)!!
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ]
  erw [involutionNoFixed_card_mul_two n]
  exact Eq.symm (Nat.doubleFactorial_add_one (Nat.mul 2 n))

```

lemma involutionNoFixed_card_mul_two_plus_one[\[source\]](#)

```

lemma involutionNoFixed_card_mul_two_plus_one : (n : ℕ) →
  Fintype.card {f : Fin (2 * n + 1) → Fin (2 * n + 1) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = 0
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ]
  erw [involutionNoFixed_card_mul_two_plus_one n]
  exact rfl

```

lemma involutionNoFixed_card_even[\[source\]](#)

```

lemma involutionNoFixed_card_even : (n : ℕ) → (he : Even n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = (n - 1)!!

```

lemma involutionNoFixed_card_odd[\[source\]](#)

```

lemma involutionNoFixed_card_odd : (n : ℕ) → (ho : Odd n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = 0

```

2.3 PhysLean.Mathematics.List

[PhysLean/Mathematics/List.lean](#) on GitHub.

lemma takeWhile_eraseIdx[\[source\]](#)

```

lemma takeWhile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
  (l : List I) → (i : ℕ) → (hi : ∀ (i j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
  List.takeWhile P (List.eraseIdx l i) = (List.takeWhile P l).eraseIdx i
| [], _, h => by
  rfl
| a :: [], 0, h => by
  simp only [List.takeWhile, List.eraseIdx_zero, List.nil_eq]
  split

```

```

    · rfl
    · rfl
| a :: [], Nat.succ n, h => by
  simp only [Nat.succ_eq_add_one, List.eraseIdx_cons_succ, List.eraseIdx_nil]
  rw [List.eraseIdx_of_length_le]
  have h1 : (List.takeWhile P [a]).length ≤ [a].length

```

lemma dropWhile_eraseIdx[\[source\]](#)

```

lemma dropWhile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
  (l : List I) → (i : ℕ) → (hi : ∀ (i j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
  List.dropWhile P (List.eraseIdx l i) =
    if (List.takeWhile P l).length ≤ i then
      (List.dropWhile P l).eraseIdx (i - (List.takeWhile P l).length)
    else (List.dropWhile P l)
| [], _, h => by
  simp
| a :: [], 0, h => by
  simp only [List.dropWhile, nonpos_iff_eq_zero, List.length_eq_zero, List.
    takeWhile_eq_nil_iff,
    List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue, List.get_eq_getElem,
    Fin.val_eq_zero, List.getElem_cons_zero, decide_eq_true_eq, forall_const, zero_le,
    Nat.sub_eq_zero_of_le, List.eraseIdx_zero, ite_not, List.nil_eq]
  simp_all only [List.length_cons, List.length_nil, List.get_eq_getElem, Fin.val_eq_zero,
    List.getElem_cons_zero, implies_true, List.tail_cons, List.dropWhile_nil, decide_true,
    decide_false, ite_self]
| a :: [], Nat.succ n, h => by
  simp only [List.dropWhile, List.eraseIdx_nil, List.takeWhile, Nat.succ_eq_add_one]
  rw [List.eraseIdx_of_length_le]
  simp_all only [List.length_singleton, List.get_eq_getElem, Fin.val_eq_zero,
    List.getElem_cons_zero, implies_true, ite_self]
  split
  next x heq =>
    simp_all only [decide_eq_true_eq, decide_true, List.dropWhile_cons_of_pos, List.
    dropWhile_nil,
    List.length_singleton, le_add_iff_nonneg_left, zero_le, ↪reduceIte,
    add_tsub_cancel_right, List.eraseIdx_nil]
  next x heq =>
    simp_all only [decide_eq_false_iff_not, decide_false, Bool.false_eq_true,
    not_false_eq_true,
    List.dropWhile_cons_of_neg, List.length_nil, le_add_iff_nonneg_left,
    zero_le, ↪reduceIte, tsub_zero, List.eraseIdx_cons_succ, List.eraseIdx_nil]
  exact Nat.le_add_left [a].length n
| a :: b :: l, 0, h => by
  simp only [List.dropWhile, List.takeWhile, nonpos_iff_eq_zero, List.length_eq_zero, zero_le
  ,
    Nat.sub_eq_zero_of_le, List.eraseIdx_zero]
  by_cases hPb : P b
  · have hPa : P a

```

lemma insertionSort_length[\[source\]](#)

```

lemma insertionSort_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (l : List I) :
  (List.insertionSort le1 l).length = l.length

```

def orderedInsertPos[\[source\]](#)

The position $r0$ ends up in r on adding it via `List.orderedInsert _ r0 r`.

```
def orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
  Fin (List.orderedInsert le1 r0 r).length
```

lemma orderedInsertPos_lt_length[\[source\]](#)

```
lemma orderedInsertPos_lt_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) : orderedInsertPos le1 r r0 < (r0 :: r).length
```

lemma orderedInsert_get_orderedInsertPos[\[source\]](#)

```
@[simp]
lemma orderedInsert_get_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r)[(orderedInsertPos le1 r r0).val] = r0
```

lemma orderedInsert_eraseIdx_orderedInsertPos[\[source\]](#)

```
@[simp]
lemma orderedInsert_eraseIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r).eraseIdx ↑(orderedInsertPos le1 r r0) = r
```

lemma orderedInsertPos_cons[\[source\]](#)

```
lemma orderedInsertPos_cons {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 r1 : I) :
  (orderedInsertPos le1 (r1 :: r) r0).val =
  if le1 r0 r1 then ⟨0, by simp⟩ else (Fin.succ (orderedInsertPos le1 r r0))
```

lemma orderedInsertPos_sigma[\[source\]](#)

```
lemma orderedInsertPos_sigma {I : Type} {f : I → Type}
  (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
  (k : I) (a : f k) :
  (orderedInsertPos (fun (i j : Σ i, f i) => le1 i.1 j.1) l ⟨k, a⟩).1 =
  (orderedInsertPos le1 (List.map (fun (i : Σ i, f i) => i.1) l) k).1
```

lemma orderedInsert_get_lt[\[source\]](#)

```
lemma orderedInsert_get_lt {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : i < orderedInsertPos le1 r r0) :
  (List.orderedInsert le1 r0 r)[i] = r.get (i, by
    simp only [orderedInsertPos] at hi
  have h1 : (List.takeWhile (fun b => decide ¬le1 r0 b) r).length ≤ r.length :=
    List.Sublist.length_le (List.takeWhile_sublist fun b => decide ¬le1 r0 b)
  omega)
```

lemma orderedInsertPos_take_orderedInsert[\[source\]](#)

```
lemma orderedInsertPos_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)) =
  List.takeWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_take_eq_orderedInsert[\[source\]](#)

```
lemma orderedInsertPos_take_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.take (orderedInsertPos le1 r r0) r =
  List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)
```

lemma orderedInsertPos_drop_eq_orderedInsert[\[source\]](#)

```
lemma orderedInsertPos_drop_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.drop (orderedInsertPos le1 r r0) r =
  List.drop (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)
```

lemma orderedInsertPos_take[\[source\]](#)

```
lemma orderedInsertPos_take {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.take (orderedInsertPos le1 r r0) r = List.takeWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_drop[\[source\]](#)

```
lemma orderedInsertPos_drop {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.drop (orderedInsertPos le1 r r0) r = List.dropWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_succ_take_orderedInsert[\[source\]](#)

```
lemma orderedInsertPos_succ_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  (List.take (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)) =
  List.takeWhile (fun b => decide ¬le1 r0 b) r ++ [r0]
```

lemma lt_orderedInsertPos_rel[\[source\]](#)

```
lemma lt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r0 : I) (r : List I) (n : Fin r.length)
  (hn : n.val < (orderedInsertPos le1 r r0).val) : ¬le1 r0 (r.get n)
```

lemma lt_orderedInsertPos_rel_fin[\[source\]](#)

```
lemma lt_orderedInsertPos_rel_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r0 : I) (r : List I) (n : Fin (List.orderedInsert le1 r0 r).length)
  (hn : n < (orderedInsertPos le1 r r0)) : ¬le1 r0 ((List.orderedInsert le1 r0 r).get n)
```


lemma gt_orderedInsertPos_rel[\[source\]](#)

```

lemma gt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [IsTotal I le1] [IsTrans I le1] (r0 : I) (r : List I) (hs : List.Sorted le1 r)
  (n : Fin r.length)
  (hn : ¬ n.val < (orderedInsertPos le1 r r0).val) : le1 r0 (r.get n)

```

lemma orderedInsert_eraseIdx_lt_orderedInsertPos[\[source\]](#)

```

lemma orderedInsert_eraseIdx_lt_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : i < orderedInsertPos le1 r r0)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx i = List.orderedInsert le1 r0 (r.eraseIdx i)

```

lemma orderedInsert_eraseIdx_orderedInsertPos_le[\[source\]](#)

```

lemma orderedInsert_eraseIdx_orderedInsertPos_le {I : Type} (le1 : I → I → Prop) [DecidableRel
  le1]
  (r : List I) (r0 : I) (i : ℕ)
  (hi : orderedInsertPos le1 r r0 ≤ i)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (Nat.succ i) =
  List.orderedInsert le1 r0 (r.eraseIdx i)

```

def orderedInsertEquiv[\[source\]](#)

The equivalence between $\text{Fin } (r0 :: r).length$ and $\text{Fin } (List.orderedInsert le1 r0 r).length$ according to where the elements map, i.e. θ is taken to $\text{orderedInsertPos le1 r r0}$.

```

def orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I)
  :
  Fin (r0 :: r).length ≈ Fin (List.orderedInsert le1 r0 r).length

```

lemma orderedInsertEquiv_zero[\[source\]](#)

```

lemma orderedInsertEquiv_zero {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) : orderedInsertEquiv le1 r r0 ⟨0, by simp⟩ = orderedInsertPos le1 r r0

```

lemma orderedInsertEquiv_succ[\[source\]](#)

```

lemma orderedInsertEquiv_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) (n : ℕ) (hn : Nat.succ n < (r0 :: r).length) :
  orderedInsertEquiv le1 r r0 (Nat.succ n, hn) =
  Fin.cast (List.orderedInsert_length le1 r r0).symm
  ((Fin.succAbove ((orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0))
  ⟨n, Nat.succ_lt_succ_iff.mp hn⟩)

```

lemma orderedInsertEquiv_fin_succ[\[source\]](#)

```

lemma orderedInsertEquiv_fin_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List
  I)

```

```
(r0 : I) (n : Fin r.length) :
orderedInsertEquiv le1 r r0 n.succ = Fin.cast (List.orderedInsert_length le1 r r0).symm
((Fin.succAbove ((orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0))
  (n, n.isLt))
```

lemma orderedInsertEquiv_monotone_fin_succ[\[source\]](#)

```
lemma orderedInsertEquiv_monotone_fin_succ {I : Type}
  (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) (n m : Fin r.length)
  (hx : orderedInsertEquiv le1 r r0 n.succ < orderedInsertEquiv le1 r r0 m.succ) :
  n < m
```

lemma orderedInsertEquiv_congr[\[source\]](#)

```
lemma orderedInsertEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (a : α)
  (l l' : List α) (h : l = l') :
  orderedInsertEquiv r l a = (Fin.castOrderIso (by simp [h])).toEquiv.trans
  ((orderedInsertEquiv r l' a).trans (Fin.castOrderIso (by simp [h])).toEquiv)
```

lemma get_eq_orderedInsertEquiv[\[source\]](#)

```
lemma get_eq_orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) :
  (r0 :: r).get = (List.orderedInsert le1 r0 r).get ∘ (orderedInsertEquiv le1 r r0)
```

lemma orderedInsertEquiv_get[\[source\]](#)

```
lemma orderedInsertEquiv_get {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) :
  (r0 :: r).get ∘ (orderedInsertEquiv le1 r r0).symm = (List.orderedInsert le1 r0 r).get
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero[\[source\]](#)

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 ⟨0, by simp⟩) = r
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ[\[source\]](#)

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : ℕ)
  (hn : Nat.succ n < (r0 :: r).length)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 ⟨Nat.succ n, hn⟩) =
  (List.orderedInsert le1 r0 (r.eraseIdx n))
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ[\[source\]](#)

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ
  {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : Fin r.length)
  (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
  (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 n.succ) =
```

```
(List.orderedInsert le1 r0 (r.eraseIdx n))
```

lemma orderedInsertEquiv_sigma[\[source\]](#)

```
lemma orderedInsertEquiv_sigma {I : Type} {f : I → Type}
  (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
  (i : I) (a : f i) :
  (orderedInsertEquiv (fun i j => le1 i.fst j.fst) l (i, a)) =
  (Fin.castOrderIso (by simp)).toEquiv.trans
  ((orderedInsertEquiv le1 (List.map (fun i => i.1) l) i).trans
  (Fin.castOrderIso (by simp [List.orderedInsert_length]))).toEquiv)
```

theorem length_insertIdx'[\[source\]](#)

This result is taken from: <https://github.com/leanprover/lean4/blob/master/src/Init/Data/List/Nat/InsertIdx.lean> with simple modification here to make it run. The file it was taken from is licensed under the Apache License, Version 2.0. and written by Parikshit Khanna, Jeremy Avigad, Leonardo de Moura, Floris van Doorn, Mario Carneiro.

Once PhysLean is updated to a more recent version of Lean this result will be removed.

```
theorem length_insertIdx' : ∀ n as, (List.insertIdx n a as).length =
  if n ≤ as.length then as.length + 1 else as.length
| 0, _ => by simp
| n + 1, [] => by rfl
| n + 1, a :: as => by
  simp only [List.insertIdx_succ_cons, List.length_cons, length_insertIdx',
    Nat.add_le_add_iff_right]
  split <;> rfl
```

lemma orderedInsert_eq_insertIdx_orderedInsertPos[\[source\]](#)

```
lemma orderedInsert_eq_insertIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r : List I) (r0 : I) :
  List.orderedInsert le1 r0 r = List.insertIdx (orderedInsertPos le1 r r0).1 r0 r
```

def insertionSortEquiv[\[source\]](#)

The equivalence between $\text{Fin } l.\text{length} \approx \text{Fin } (\text{List.insertionSort } r \ l).\text{length}$ induced by the sorting algorithm.

```
def insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r] : (l : List α) →
  Fin l.length ≈ Fin (List.insertionSort r l).length
| [] => Equiv.refl _
| a :: l =>
  (Fin.equivCons (insertionSortEquiv r l)).trans (orderedInsertEquiv r (List.insertionSort r l) a)
```

lemma insertionSortEquiv_get[\[source\]](#)

```
lemma insertionSortEquiv_get {α : Type} {r : α → α → Prop} [DecidableRel r] : (l : List α) →
  l.get ◦ (insertionSortEquiv r l).symm = (List.insertionSort r l).get
| [] => by rfl
| a :: l => by
```

```

rw [insertionSortEquiv]
change ((a :: l).get ◦ ((Fin.equivCons (insertionSortEquiv r l))).symm) ◦
  (orderedInsertEquiv r (List.insertionSort r l) a).symm = _
have hl : (a :: l).get ◦ ((Fin.equivCons (insertionSortEquiv r l))).symm =
  (a :: List.insertionSort r l).get

```

lemma insertionSortEquiv_congr[\[source\]](#)

```

lemma insertionSortEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (l l' : List α)
  (h : l = l') : insertionSortEquiv r l = (Fin.castOrderIso (by simp [h])).toEquiv.trans
  ((insertionSortEquiv r l').trans (Fin.castOrderIso (by simp [h])).toEquiv)

```

lemma insertionSort_get_comp_insertionSortEquiv[\[source\]](#)

```

lemma insertionSort_get_comp_insertionSortEquiv {α : Type} {r : α → α → Prop} [DecidableRel r]
  (l : List α) : (List.insertionSort r l).get ◦ (insertionSortEquiv r l) = l.get

```

lemma insertionSort_eq_ofFn[\[source\]](#)

```

lemma insertionSort_eq_ofFn {α : Type} {r : α → α → Prop} [DecidableRel r] (l : List α) :
  List.insertionSort r l = List.ofFn (l.get ◦ (insertionSortEquiv r l).symm)

```

lemma insertionSortEquiv_order[\[source\]](#)

```

lemma insertionSortEquiv_order {α : Type} {r : α → α → Prop} [DecidableRel r] :
  (l : List α) → (i : Fin l.length) → (j : Fin l.length) → (hij : i < j)
  → (hij' : insertionSortEquiv r l j < insertionSortEquiv r l i) →
  ¬ r l[i] l[j]
| [], i, _, _, _ => Fin.elim0 i
| a :: as, ⟨0, hi⟩, ⟨j + 1, hj⟩, hij, hij' => by
  simp only [List.length_cons, Fin.zero_eta, Fin.getElem_fin, Fin.val_zero,
    List.getElem_cons_zero, List.getElem_cons_succ]
  nth_rewrite 2 [insertionSortEquiv] at hij'
  simp only [List.insertionSort.eq_2, List.length_cons, Nat.succ_eq_add_one, Fin.zero_eta,
    Equiv.trans_apply, equivCons_zero] at hij'
have hx

```

def optionErase[\[source\]](#)

Optional erase of an element in a list. For none returns the list, for some i returns the list with the i'th element erased.

```

def optionErase {I : Type} (l : List I) (i : Option (Fin l.length)) : List I

```

lemma eraseIdx_length'[\[source\]](#)

```

lemma eraseIdx_length' {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length = l.length - 1

```

lemma eraseIdx_length[\[source\]](#)

```

lemma eraseIdx_length {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length + 1 = l.length

```

lemma eraseIdx_length_succ[\[source\]](#)

```
lemma eraseIdx_length_succ {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length.succ = l.length
```

lemma eraseIdx_cons_length[\[source\]](#)

```
lemma eraseIdx_cons_length {I : Type} (a : I) (l : List I) (i : Fin (a :: l).length) :
  (List.eraseIdx (a :: l) i).length = l.length
```

lemma eraseIdx_get[\[source\]](#)

```
lemma eraseIdx_get {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).get = l.get ∘ (Fin.cast (eraseIdx_length l i)) ∘
  (Fin.cast (eraseIdx_length l i).symm i).succAbove
```

lemma eraseIdx_insertionSort[\[source\]](#)

```
lemma eraseIdx_insertionSort {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [IsTotal I le1] [IsTrans I le1] :
  (n : ℕ) → (r : List I) → (hn : n < r.length) →
  (List.insertionSort le1 r).eraseIdx ↑((insertionSortEquiv le1 r) {n, hn})
  = List.insertionSort le1 (r.eraseIdx n)
| 0, [], _ => by rfl
| 0, (r0 :: r), hn => by
  simp only [List.insertionSort, List.insertionSort.eq_2, List.length_cons,
    insertionSortEquiv,
    Nat.succ_eq_add_one, Fin.zero_eta, Equiv.trans_apply, equivCons_zero, List.eraseIdx_zero,
    List.tail_cons]
  erw [orderedInsertEquiv_zero]
  simp
| Nat.succ n, [], hn => by rfl
| Nat.succ n, (r0 :: r), hn => by
  simp only [List.insertionSort, List.length_cons, insertionSortEquiv, Nat.succ_eq_add_one,
    Equiv.trans_apply, equivCons_succ]
  have h0r
```

lemma eraseIdx_insertionSort_fin[\[source\]](#)

```
lemma eraseIdx_insertionSort_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  [IsTotal I le1] [IsTrans I le1] (r : List I) (n : Fin r.length) :
  (List.insertionSort le1 r).eraseIdx ↑((PhysLean.List.insertionSortEquiv le1 r) n)
  = List.insertionSort le1 (r.eraseIdx n)
```

def insertionSortMinPos[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r . That is the first position of l such that for every element $(i :: l)[b]$ before that position $r ((i :: l)[b]) ((i :: l)[a])$ is not true. The use of $i :: l$ here rather than just l is to ensure that such a position exists. .

```
def insertionSortMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  Fin (i :: l).length
```

def insertionSortMin[\[source\]](#)

The element of $i :: l$ at insertionSortMinPos.

```
def insertionSortMin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  α
```

lemma insertionSortMin_eq_insertionSort_head[\[source\]](#)

```
lemma insertionSortMin_eq_insertionSort_head {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMin r i l = (List.insertionSort r (i :: l)).head (by
  refine List.ne_nil_of_length_pos ?_
  rw [insertionSort_length]
  exact Nat.zero_lt_succ l.length)
```

def insertionSortDropMinPos[\[source\]](#)

The list remaining after dropping the element at the position determined by insertionSortMinPos.

```
def insertionSortDropMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α)
  :
  List α
```

lemma insertionSort_eq_insertionSortMin_cons[\[source\]](#)

```
lemma insertionSort_eq_insertionSortMin_cons {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (i : α) (l : List α) :
  List.insertionSort r (i :: l) =
  (insertionSortMin r i l) :: List.insertionSort r (insertionSortDropMinPos r i l)
```

def optionEraseZ[\[source\]](#)

Optional erase of an element in a list, with addition for none. For none adds a to the front of the list, for some i removes the ith element of the list (does not add a). E.g. optionEraseZ [0, 1, 2] 4 none = [4, 0, 1, 2] and optionEraseZ [0, 1, 2] 4 (some 1) = [0, 2].

```
def optionEraseZ {I : Type} (l : List I) (a : I) (i : Option (Fin l.length)) : List I
```

lemma optionEraseZ_some_length[\[source\]](#)

```
@[simp]
lemma optionEraseZ_some_length {I : Type} (l : List I) (a : I) (i : (Fin l.length)) :
  (optionEraseZ l a (some i)).length = l.length - 1
```

lemma optionEraseZ_ext[\[source\]](#)

```
lemma optionEraseZ_ext {I : Type} {l l' : List I} {a a' : I} {i : Option (Fin l.length)}
  {i' : Option (Fin l'.length)} (hl : l = l') (ha : a = a')
  (hi : Option.map (Fin.cast (by rw [hl])) i = i') :
  optionEraseZ l a i = optionEraseZ l' a' i'
```

lemma mem_take_finrange[\[source\]](#)

```

lemma mem_take_finrange : (n m : ℕ) → (a : Fin n) → a ∈ List.take m (List.finRange n) ↔ a.val <
  m
| 0, m, a => Fin.elim0 a
| n+1, 0, a => by
  simp
| n +1, m + 1, ⟨0, h⟩ => by
  simp [List.finRange_succ]
| n +1, m + 1, ⟨i + 1, h⟩ => by
  simp only [List.finRange_succ, List.take_succ_cons, List.mem_cons, Fin.ext_iff, Fin.
    val_zero,
    AddLeftCancelMonoid.add_eq_zero, one_ne_zero, and_false, false_or, add_lt_add_iff_right]
  rw [← List.map_take]
  rw [← List.mem_map]
  apply Iff.intro
  · intro h
    obtain ⟨a, ha⟩

```

2.4 PhysLean.Mathematics.List.InsertIdx

[PhysLean/Mathematics/List/InsertIdx.lean](#) on GitHub.

lemma insertIdx_map[\[source\]](#)

```

lemma insertIdx_map {I J : Type} (f : I → J) : (i : ℕ) → (r : List I) → (r0 : I) →
  (List.insertIdx i r0 r).map f = List.insertIdx i (f r0) (r.map f)
| 0, [], r0 => by simp
| n+1, [], r0 => by simp
| 0, a::as, r0 => by simp
| n+1, a::as, r0 => by
  simp only [List.insertIdx_succ_cons, List.map_cons, List.cons.injEq, true_and]
  exact insertIdx_map f n as r0

```

lemma eraseIdx_sorted[\[source\]](#)

```

lemma eraseIdx_sorted {I : Type} (le : I → I → Prop) :
  (r : List I) → (n : ℕ) →
  List.Sorted le r → List.Sorted le (r.eraseIdx n)
| [], _, _ => by simp
| a::as, 0, h => by
  simp only [List.eraseIdx]
  simp only [List.sorted_cons] at h
  exact h.2
| a::as, n+1, h => by
  simp only [List.eraseIdx_cons_succ, List.sorted_cons]
  simp only [List.sorted_cons] at h
  refine And.intro ?_ (eraseIdx_sorted le as n h.2)
  intro b hb
  refine h.1 _ ?_
  exact List.mem_of_mem_eraseIdx hb

```

lemma mem_eraseIdx_nodup[\[source\]](#)

```

lemma mem_eraseIdx_nodup {I : Type} (i : I) :
  (l : List I) → (n : ℕ) → (hn : n < l.length) → (h : List.Nodup l) →

```

```

i ∈ l.eraseIdx n ↔ i ∈ l ∧ i ≠ l[n]
| [], _, _, _ => by simp
| a1 :: as, 0, _, h => by
  simp only [List.eraseIdx_zero, List.tail_cons, List.mem_cons, List.getElem_cons_zero, ne_eq]
  by_cases hi : i = a1
  · subst hi
    simp only [List.nodup_cons] at h
    simp [h]
  · simp [hi]
| a1 :: as, n+1, hn, h => by
  simp only [List.eraseIdx_cons_succ, List.mem_cons, List.getElem_cons_succ, ne_eq]
  simp only [List.nodup_cons] at h
  rw [mem_eraseIdx_nodup i as n (Nat.succ_lt_succ_iff.mp hn) h.2]
  simp_all only [ne_eq]
  obtain ⟨left, right⟩

```

lemma insertIdx_eq_take_drop[\[source\]](#)

```

lemma insertIdx_eq_take_drop {I : Type} (i : I) : (r : List I) → (n : Fin r.length.succ) →
  List.insertIdx n i r = List.take n r ++ i :: r.drop n
| [], 0 => by simp
| a :: as, 0 => by
  simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.drop_succ_cons, List.
    cons_append,
    List.cons.injEq, true_and]
  exact insertIdx_eq_take_drop i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩

```

lemma insertIdx_length_fin[\[source\]](#)

```

@[simp]
lemma insertIdx_length_fin {I : Type} (i : I) :
  (r : List I) → (n : Fin r.length.succ) →
  (List.insertIdx n i r).length = r.length.succ
| [], 0 => by simp
| a :: as, 0 => by simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one, add_left_inj]
  exact insertIdx_length_fin i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩

```

lemma insertIdx_getElem_fin[\[source\]](#)

```

@[simp]
lemma insertIdx_getElem_fin {I : Type} (i : I) :
  (r : List I) → (k : Fin r.length.succ) → (m : Fin r.length) →
  (List.insertIdx k i r)[(k.succAbove m).val] = r[m.val]
| [], 0, m => by exact Fin.elim0 m
| a :: as, 0, m => by simp
| a :: as, ⟨n + 1, h⟩, ⟨0, h0⟩ => by
  simp [Fin.succAbove, Fin.lt_def]
| a :: as, ⟨n + 1, h⟩, ⟨m+1, hm⟩ => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one,
    List.getElem_cons_succ]
  conv_rhs => rw [← insertIdx_getElem_fin i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩]

```



```

    (m, Nat.lt_of_succ_lt_succ hm)]
simp only [Fin.succAbove, Fin.castSucc_mk, Fin.lt_def, add_lt_add_iff_right, Fin.succ_mk,
  Nat.succ_eq_add_one]
split
· simp_all only [List.getElem_cons_succ]
· simp_all only [List.getElem_cons_succ]

```

lemma insertIdx_eraseIdx_fin[\[source\]](#)

```

lemma insertIdx_eraseIdx_fin {I : Type} :
  (r : List I) → (k : Fin r.length) →
  (List.eraseIdx r k).insertIdx k r[k] = r
| [], k => by exact Fin.elim0 k
| a :: as, ⟨0, h⟩ => by simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.length_cons, Fin.getElem_fin, List.getElem_cons_succ, List.
    eraseIdx_cons_succ,
    List.insertIdx_succ_cons, List.cons.injEq, true_and]
  exact insertIdx_eraseIdx_fin as ⟨n, Nat.lt_of_succ_lt_succ h⟩

```

lemma insertIdx_length_fst_append[\[source\]](#)

```

lemma insertIdx_length_fst_append {I : Type} (φ : I) : (φs φs' : List I) →
  List.insertIdx φs.length φ (φs ++ φs') = (φs ++ φ :: φs')
| [], φs' => by simp
| φ' :: φs, φs' => by
  simp only [List.length_cons, List.cons_append, List.insertIdx_succ_cons, List.cons.injEq,
    true_and]
  exact insertIdx_length_fst_append φ φs φs'

```

lemma get_eq_insertIdx_succAbove[\[source\]](#)

```

lemma get_eq_insertIdx_succAbove {I : Type} (i : I) (r : List I) (k : Fin r.length.succ) :
  r.get = (List.insertIdx k i r).get ◦
  (finCongr (insertIdx_length_fin i r k).symm) ◦ k.succAbove

```

lemma take_insert_same[\[source\]](#)

```

lemma take_insert_same {I : Type} (i : I) :
  (n : ℕ) → (r : List I) →
  List.take n (List.insertIdx n i r) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp
| n+1, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_insert_same i n as

```

lemma take_eraseIdx_same[\[source\]](#)

```

lemma take_eraseIdx_same {I : Type} :
  (n : ℕ) → (r : List I) →
  List.take n (List.eraseIdx r n) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp

```

```
| n+1, a::as => by
  simp only [List.eraseIdx_cons_succ, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_eraseIdx_same n as
```

lemma drop_eraseIdx_succ[\[source\]](#)

```
lemma drop_eraseIdx_succ {I : Type} :
  (n : ℕ) → (r : List I) → (hn : n < r.length) →
  r[n] :: List.drop n (List.eraseIdx r n) = List.drop n r
| 0, _, _ => by
  simp only [List.eraseIdx_zero, List.drop_tail, zero_add, List.drop_one, List.drop_zero]
  rw [List.getElem_zero]
  exact List.head_cons_tail _ _
| n+1, [], hn => by simp at hn
| n+1, a::as, hn => by
  simp only [List.getElem_cons_succ, List.eraseIdx_cons_succ, List.drop_succ_cons]
  refine drop_eraseIdx_succ n as _
```

lemma take_insert_gt[\[source\]](#)

```
lemma take_insert_gt {I : Type} (i : I) :
  (n m : ℕ) → (h : n < m) → (r : List I) →
  List.take n (List.insertIdx m i r) = List.take n r
| 0, 0, _, _ => by simp
| 0, m + 1, _, _ => by simp
| n+1, m + 1, _, [] => by simp
| n+1, m + 1, h, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  refine take_insert_gt i n m (Nat.succ_lt_succ_iff.mp h) as
```

lemma take_insert_let[\[source\]](#)

```
lemma take_insert_let {I : Type} (i : I) :
  (n m : ℕ) → (h : m ≤ n) → (r : List I) → (hm : m ≤ r.length) →
  (List.take (n + 1) (List.insertIdx m i r)).Perm (i :: List.take n r)
| 0, 0, h, _, _ => by simp
| m + 1, 0, h, r, _ => by simp
| n + 1, m + 1, h, [], hm => by
  simp at hm
| n + 1, m + 1, h, a::as, hm => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons]
  have hp : (i :: a :: List.take n as).Perm (a :: i :: List.take n as)
```

2.5 PhysLean.Mathematics.List.InsertionSort

[PhysLean/Mathematics/List/InsertionSort.lean](#) on GitHub.

lemma insertionSortMin_lt_length_succ[\[source\]](#)

```
lemma insertionSortMin_lt_length_succ {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMinPos r i l < (insertionSortDropMinPos r i l).length.succ
```

def insertionSortMinPosFin[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r as an element of $\text{Fin } (\text{insertionSortDropMinPos } r \ i \ l).length.succ$.

```
def insertionSortMinPosFin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  Fin (insertionSortDropMinPos r i l).length.succ
```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos[\[source\]](#)

```
lemma insertionSortMin_lt_mem_insertionSortDropMinPos {α : Type} (r : α → α → Prop) [
  DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length) :
  r (insertionSortMin r a l) ((insertionSortDropMinPos r a l)[i])
```

lemma insertionSortMinPos_insertionSortEquiv[\[source\]](#)

```
lemma insertionSortMinPos_insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r]
  (a : α) (l : List α) :
  insertionSortEquiv r (a :: l) (insertionSortMinPos r a l) =
  ⟨0, by simp [List.orderedInsert_length]⟩
```

lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos[\[source\]](#)

```
lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α) (k : Fin (a :: l).length)
  (hk : k ≠ insertionSortMinPos r a l) :
  ⟨0, by simp [List.orderedInsert_length]⟩ < insertionSortEquiv r (a :: l) k
```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt[\[source\]](#)

```
lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length)
  (h : (insertionSortMinPosFin r a l).succAbove i < insertionSortMinPosFin r a l) :
  ~ r ((insertionSortDropMinPos r a l)[i]) (insertionSortMin r a l)
```

lemma insertionSort_insertionSort[\[source\]](#)

```
lemma insertionSort_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (ll : List α) :
  List.insertionSort r (List.insertionSort r ll) = List.insertionSort r ll
```

lemma orderedInsert_commute[\[source\]](#)

```
lemma orderedInsert_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a b : α) (hr : ~ r a b) : (l : List α) →
  List.orderedInsert r a (List.orderedInsert r b l) =
  List.orderedInsert r b (List.orderedInsert r a l)
| [] => by
  have hrb : r b a
```

lemma insertionSort_orderedInsert_append[\[source\]](#)

```

lemma insertionSort_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
  List.insertionSort r (List.orderedInsert r a l1 ++ l2) = List.insertionSort r (a :: l1 ++
  l2)
| [], l2 => by
  simp
| b :: l1, l2 => by
  conv_lhs => simp
  by_cases h : r a b
  · simp [h]
  conv_lhs => simp [h]
  rw [insertionSort_orderedInsert_append r a l1 l2]
  simp only [List.cons_append, List.insertionSort]
  rw [orderedInsert_commute r a b h]

```

lemma insertionSort_insertionSort_append[\[source\]](#)

```

lemma insertionSort_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] : (l1 l2 : List α) →
  List.insertionSort r (List.insertionSort r l1 ++ l2) = List.insertionSort r (l1 ++ l2)
| [], l2 => by
  simp
| a :: l1, l2 => by
  conv_lhs => simp
  rw [insertionSort_orderedInsert_append]
  simp only [List.cons_append, List.insertionSort]
  rw [insertionSort_insertionSort_append r l1 l2]

```

lemma insertionSort_append_insertionSort_append[\[source\]](#)

```

lemma insertionSort_append_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] : (l1 l2 l3 : List α) →
  List.insertionSort r (l1 ++ List.insertionSort r l2 ++ l3) =
  List.insertionSort r (l1 ++ l2 ++ l3)
| [], l2, l3 => by
  simp only [List.nil_append]
  exact insertionSort_insertionSort_append r l2 l3
| a :: l1, l2, l3 => by
  simp only [List.cons_append, List.insertionSort]
  rw [insertionSort_append_insertionSort_append r l1 l2 l3]

```

lemma orderedInsert_length[\[source\]](#)

```

@[simp]
lemma orderedInsert_length {α : Type} (r : α → α → Prop) [DecidableRel r] (a : α) (l : List α)
  :
  (List.orderedInsert r a l).length = (a :: l).length

```

lemma takeWhile_orderedInsert[\[source\]](#)

```

lemma takeWhile_orderedInsert {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r]
  (a b : α) (hr : ¬ r a b) : (l : List α) →

```

```

(List.takeWhile (fun c => !decide (r a c)) (List.orderedInsert r b l)).length =
(List.takeWhile (fun c => !decide (r a c)) l).length + 1
| [] => by
  simp [hr]
| c :: l => by
  simp only [List.orderedInsert]
  by_cases h : r b c
  · simp only [h, ↓reduceIte]
    rw [List.takeWhile_cons_of_pos]
    simp only [List.length_cons]
    simp [hr]
  · simp only [h, ↓reduceIte]
    have hrba : r b a

```

lemma takeWhile_orderedInsert'[\[source\]](#)

```

lemma takeWhile_orderedInsert' {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r]
  (a b : α) (hr : ¬ r a b) : (l : List α) →
  (List.takeWhile (fun c => !decide (r b c)) (List.orderedInsert r a l)).length =
  (List.takeWhile (fun c => !decide (r b c)) l).length
| [] => by
  simp only [List.orderedInsert, List.takeWhile_nil, List.length_nil, List.length_eq_zero,
    List.takeWhile_eq_nil_iff, List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue,
    List.get_eq_getElem, Fin.val_eq_zero, List.getElem_cons_zero, Bool.not_eq_eq_eq_not,
    Bool.not_true, decide_eq_false_iff_not, Decidable.not_not, forall_const]
  have ht

```

lemma insertionSortEquiv_commute[\[source\]](#)

```

lemma insertionSortEquiv_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a b : α) (hr : ¬ r a b) (n : ℕ) : (l : List α) →
  (hn : n + 2 < (a :: b :: l).length) →
  insertionSortEquiv r (a :: b :: l) (n + 2, hn) = (finCongr (by simp))
  (insertionSortEquiv r (b :: a :: l) (n + 2, hn))

```

lemma insertionSortEquiv_orderedInsert_append[\[source\]](#)

```

lemma insertionSortEquiv_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a a2 : α) : (l1 l2 : List α) →
  (insertionSortEquiv r (List.orderedInsert r a l1 ++ a2 :: l2) (l1.length + 1, by
    simp))
  = (finCongr (by
    simp only [List.cons_append, List.insertionSort, orderedInsert_length, List.length_cons,
      List.length_insertionSort, List.length_append]
    omega))
    ((insertionSortEquiv r (a :: l1 ++ a2 :: l2) (l1.length + 1, by simp)))
| [], l2 => by
  simp
| b :: l1, l2 => by
  by_cases h : r a b
  · have h1 : (List.orderedInsert r a (b :: l1) ++ a2 :: l2) = (a :: b :: l1 ++ a2 :: l2)

```

lemma insertionSortEquiv_insertionSort_append[\[source\]](#)

```

lemma insertionSortEquiv_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
  (insertionSortEquiv r (List.insertionSort r l1 ++ a :: l2) (l1.length, by simp))
= finCongr (by simp) (insertionSortEquiv r (l1 ++ a :: l2) (l1.length, by simp))
| [], l2 => by
  simp only [List.insertionSort, List.nil_append, List.length_cons, List.length_nil, Fin.
    zero_eta,
    finCongr_refl, Equiv.refl_apply]
| b :: l1, l2 => by
  simp only [List.insertionSort, List.length_cons, List.cons_append, finCongr_apply]
have hl

```

lemma orderedInsert_filter_of_pos

[\[source\]](#)

```

lemma orderedInsert_filter_of_pos {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) (p : α → Prop) [DecidablePred p] (h : p a) : (l : List α) →
  (hl : l.Sorted r) →
  List.filter p (List.orderedInsert r a l) = List.orderedInsert r a (List.filter p l)
| [], hl => by
  simp only [List.orderedInsert, List.filter_nil, List.orderedInsert_nil, List.filter_eq_self,
    List.mem_singleton, decide_eq_true_eq, forall_eq]
  exact h
| b :: l, hl => by
  simp only [List.orderedInsert]
  by_cases hpb : p b <=> by_cases hab : r a b
  · simp only [hab, ↓reduceIte, hpb, decide_true, List.filter_cons_of_pos,
    List.orderedInsert.eq_2]
    rw [List.filter_cons_of_pos (by simp [h])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
  · simp only [hab, ↓reduceIte]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    simp only [List.orderedInsert, hab, ↓reduceIte, List.cons.injEq, true_and]
    simp only [List.sorted_cons] at hl
    exact orderedInsert_filter_of_pos r a p h l hl.2
  · simp only [hab, ↓reduceIte]
    rw [List.filter_cons_of_pos (by simp [h]),
      List.filter_cons_of_neg (by simp [hpb])]
    rw [List.orderedInsert_eq_take_drop]
    have hl : List.takeWhile (fun b => decide ¬r a b)
      (List.filter (fun b => decide (p b)) l) = []

```

lemma orderedInsert_filter_of_neg

[\[source\]](#)

```

lemma orderedInsert_filter_of_neg {α : Type} (r : α → α → Prop) [DecidableRel r]
  (a : α) (p : α → Prop) [DecidablePred p] (h : ¬ p a) (l : List α) :
  List.filter p (List.orderedInsert r a l) = (List.filter p l)

```

lemma insertionSort_filter

[\[source\]](#)

```

lemma insertionSort_filter {α : Type} (r : α → α → Prop) [DecidableRel r] [IsTotal α r]
  [IsTrans α r] (p : α → Prop) [DecidablePred p] : (l : List α) →
  List.insertionSort r (List.filter p l) =
  List.filter p (List.insertionSort r l)

```

```

| [] => by simp
| a :: l => by
  simp only [List.insertionSort]
  by_cases h : p a
  · rw [orderedInsert_filter_of_pos]
    rw [List.filter_cons_of_pos]
    simp only [List.insertionSort]
    rw [insertionSort_filter]
    simp_all only [decide_true]
    simp_all only
    exact List.sorted_insertionSort r l
  · rw [orderedInsert_filter_of_neg]
    rw [List.filter_cons_of_neg]
    rw [insertionSort_filter]
    simp_all only [decide_false, Bool.false_eq_true, not_false_eq_true]
    exact h

```

lemma takeWhile_sorted_eq_filter

[\[source\]](#)

```

lemma takeWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.takeWhile (fun c => ¬ r a c) l = List.filter (fun c => ¬ r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.takeWhile_cons_of_pos,
    List.filter_cons_of_pos, List.cons.injEq, true_and]
    simp using takeWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.takeWhile_cons_of_neg, List.filter_cons_of_neg, List.nil_eq,
    List.filter_eq_nil_iff, Bool.not_eq_eq_eq_not, decide_eq_false_iff_not]
    intro c hc
    apply IsTrans.trans a b c hb
    exact hl.1 c hc

```

lemma dropWhile_sorted_eq_filter

[\[source\]](#)

```

lemma dropWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.dropWhile (fun c => ¬ r a c) l = List.filter (fun c => r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.dropWhile_cons_of_pos,
    Bool.false_eq_true, not_false_eq_true, List.filter_cons_of_neg]
    simp using dropWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.dropWhile_cons_of_neg, List.filter_cons_of_pos, List.cons.injEq,
    true_and]
    symm
    rw [List.filter_eq_self]
    intro c hc
    simp only [decide_eq_true_eq]

```

```

    apply IsTrans.trans a b c hb
    exact hl.1 c hc

```

lemma dropWhile_sorted_eq_filter_filter[\[source\]](#)

```

lemma dropWhile_sorted_eq_filter_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.filter (fun c => r a c) l =
  List.filter (fun c => r a c ∧ r c a) l ++ List.filter (fun c => r a c ∧ ¬ r c a) l
| [], _ => by
  simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [hb, decide_false, Bool.false_eq_true, not_false_eq_true, List.
    filter_cons_of_neg,
    Bool.decide_and, Bool.false_and, decide_not]
    simp using dropWhile_sorted_eq_filter_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_true, List.filter_cons_of_pos, Bool.decide_and,
    decide_not]
    by_cases hba : r b a
    · simp only [hba, decide_true, Bool.not_true, Bool.and_false, Bool.false_eq_true,
      not_false_eq_true, List.filter_cons_of_neg]
      rw [List.filter_cons_of_pos]
      rw [dropWhile_sorted_eq_filter_filter]
      simp only [Bool.decide_and, decide_not, List.cons_append]
      exact hl.2
      simp_all
    · simp only [hba, decide_false, Bool.and_false, Bool.false_eq_true, not_false_eq_true,
      List.filter_cons_of_neg]
      have hl : List.filter (fun c => decide (r a c) && decide (r c a)) l = []

```

lemma filter_rel_eq_insertionSort[\[source\]](#)

```

lemma filter_rel_eq_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l : List α) →
  List.filter (fun c => r a c ∧ r c a) (l.insertionSort r) =
  List.filter (fun c => r a c ∧ r c a) l
| [] => by simp
| b :: l => by
  simp only [List.insertionSort]
  by_cases h : r a b ∧ r b a
  · have hl

```

lemma insertionSort_of_eq_list[\[source\]](#)

```

lemma insertionSort_of_eq_list {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l1 l l2 : List α)
  (h : ∀ b ∈ l, r a b ∧ r b a) :
  List.insertionSort r (l1 ++ l ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ l
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))

```


lemma insertionSort_of_takeWhile_filter[\[source\]](#)

```

lemma insertionSort_of_takeWhile_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l1 l2 : List α) :
  List.insertionSort r (l1 ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))

```

2.6 PhysLean.Mathematics.PiTensorProduct

[PhysLean/Mathematics/PiTensorProduct.lean](#) on GitHub.

lemma induction_tmul[\[source\]](#)

```

lemma induction_tmul {f g : ((⊗[R] i : ι1, s1 i) ⊗[R] ⊗[R] i : ι2, s2 i) →l[R] M}
  (h : ∀ p q, f (PiTensorProduct.tprod R p ⊗t[R] PiTensorProduct.tprod R q)
  = g (PiTensorProduct.tprod R p ⊗t[R] PiTensorProduct.tprod R q)) : f = g

```

lemma induction_assoc[\[source\]](#)

```

lemma induction_assoc
  {f g : ((⊗[R] i : ι1, s1 i) ⊗[R] (⊗[R] i : ι2, s2 i) ⊗[R] ⊗[R] i : ι3, s3 i) →l[R] M}
  (h : ∀ p q m, f (PiTensorProduct.tprod R p ⊗t[R]
  PiTensorProduct.tprod R q ⊗t[R] PiTensorProduct.tprod R m)
  = g (PiTensorProduct.tprod R p ⊗t[R] PiTensorProduct.tprod R q
  ⊗t[R] PiTensorProduct.tprod R m)) : f = g

```

lemma induction_assoc'[\[source\]](#)

```

lemma induction_assoc'
  {f g : (((⊗[R] i : ι1, s1 i) ⊗[R] (⊗[R] i : ι2, s2 i)) ⊗[R] ⊗[R] i : ι3, s3 i) →l[R] M}
  (h : ∀ p q m, f ((PiTensorProduct.tprod R p ⊗t[R] PiTensorProduct.tprod R q) ⊗t[R]
  PiTensorProduct.tprod R m) = g ((PiTensorProduct.tprod R p ⊗t[R] PiTensorProduct.tprod R q)
  ⊗t[R] PiTensorProduct.tprod R m)) : f = g

```

lemma induction_tmul_mod[\[source\]](#)

```

lemma induction_tmul_mod
  {f g : ((⊗[R] i : ι1, s1 i) ⊗[R] N) →l[R] M}
  (h : ∀ p m, f (PiTensorProduct.tprod R p ⊗t[R] m) = g (PiTensorProduct.tprod R p ⊗t[R] m))
  :
  f = g

```

lemma induction_mod_tmul[\[source\]](#)

```

lemma induction_mod_tmul
  {f g : (N ⊗[R] ⊗[R] i : ι1, s1 i) →l[R] M}
  (h : ∀ m p, f (m ⊗t[R] PiTensorProduct.tprod R p) = g (m ⊗t[R] PiTensorProduct.tprod R p))
  :
  f = g

```

instance (i : ι1 ⊕ ι2) → AddCommMonoid ((fun i => Sum.elim s1 s2 i) i) [\[source\]](#)

Given two maps s1 and s2 whose targets carry an instance of an additive commutative monoid, the target of the sum of these two maps also carry an instance thereof.

```
instance : (i : ι1 ⊕ ι2) → AddCommMonoid ((fun i => Sum.elim s1 s2 i) i)
```

instance (i : ι1 ⊕ ι2) → Module R ((fun i => Sum.elim s1 s2 i) i) [\[source\]](#)

Given two maps s1 and s2 whose targets carry an instance of a module over R, the target of the sum of these two maps also carry an instance thereof.

```
instance : (i : ι1 ⊕ ι2) → Module R ((fun i => Sum.elim s1 s2 i) i)
```

def pureInl [\[source\]](#)

Takes a map (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i to the underlying map (i : ι1) → s1 i .

```
def pureInl (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) : (i : ι1) → s1 i
```

def pureInr [\[source\]](#)

Takes a map (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i to the underlying map (i : ι2) → s2 i .

```
def pureInr (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) : (i : ι2) → s2 i
```

lemma pureInl_update_left [\[source\]](#)

```
lemma pureInl_update_left [DecidableEq ι1] (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι1)
  (v1 : s1 x) : pureInl (Function.update f (Sum.inl x) v1) =
  Function.update (pureInl f) x v1
```

lemma pureInr_update_left [\[source\]](#)

```
lemma pureInr_update_left (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι1)
  (v2 : s1 x) :
  pureInr (Function.update f (Sum.inl x) v2) = (pureInr f)
```

lemma pureInr_update_right [\[source\]](#)

```
lemma pureInr_update_right [DecidableEq ι2] (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι2)
  (v2 : s2 x) : pureInr (Function.update f (Sum.inr x) v2) =
  Function.update (pureInr f) x v2
```

lemma pureInl_update_right [\[source\]](#)

```
lemma pureInl_update_right (f : (i : ι1 ⊕ ι2) → Sum.elim s1 s2 i) (x : ι2)
  (v1 : s2 x) :
  pureInl (Function.update f (Sum.inr x) v1) = (pureInl f)
```

def domCoproduct[\[source\]](#)

The multilinear map from $(\text{Sum.elim } s1 \ s2)$ to $((\bigotimes_{[R]} i : \iota1, s1 \ i) \otimes_{[R]} \bigotimes_{[R]} i : \iota2, s2 \ i)$ defined by splitting elements of $(\text{Sum.elim } s1 \ s2)$ into two parts.

```
def domCoproduct :
  MultilinearMap R (Sum.elim s1 s2) (( $\bigotimes_{[R]} i : \iota1, s1 \ i$ )  $\otimes_{[R]}$   $\bigotimes_{[R]} i : \iota2, s2 \ i$ ) where
```

def tmulSymm[\[source\]](#)

Expand PiTensorProduct on sums into a TensorProduct of two factors.

```
def tmulSymm : ( $\bigotimes_{[R]} i : \iota1 \oplus \iota2, (\text{Sum.elim } s1 \ s2) \ i$ )  $\rightarrow_l [R]$ 
  (( $\bigotimes_{[R]} i : \iota1, s1 \ i$ )  $\otimes_{[R]}$   $\bigotimes_{[R]} i : \iota2, s2 \ i$ )
```

def elimPureTensor[\[source\]](#)

Produces a map $(i : \iota1 \oplus \iota2) \rightarrow \text{Sum.elim } s1 \ s2 \ i$ from a map $(i : \iota1) \rightarrow s1 \ i$ and a map $q : (i : \iota2) \rightarrow s2 \ i$.

```
def elimPureTensor (p : (i :  $\iota1$ )  $\rightarrow s1 \ i$ ) (q : (i :  $\iota2$ )  $\rightarrow s2 \ i$ ) : (i :  $\iota1 \oplus \iota2$ )  $\rightarrow \text{Sum.elim } s1 \ s2 \ i$ 
```

lemma elimPureTensor_update_right[\[source\]](#)

```
lemma elimPureTensor_update_right (p : (i :  $\iota1$ )  $\rightarrow s1 \ i$ ) (q : (i :  $\iota2$ )  $\rightarrow s2 \ i$ )
  (y :  $\iota2$ ) (r :  $s2 \ y$ ) : elimPureTensor p (Function.update q y r) =
  Function.update (elimPureTensor p q) (Sum.inr y) r
```

lemma elimPureTensor_update_left[\[source\]](#)

```
@[simp]
lemma elimPureTensor_update_left (p : (i :  $\iota1$ )  $\rightarrow s1 \ i$ ) (q : (i :  $\iota2$ )  $\rightarrow s2 \ i$ )
  (x :  $\iota1$ ) (r :  $s1 \ x$ ) : elimPureTensor (Function.update p x r) q =
  Function.update (elimPureTensor p q) (Sum.inl x) r
```

def elimPureTensorMullin[\[source\]](#)

The multilinear map valued in multilinear maps defined by combining $(i : \iota1) \rightarrow s1 \ i$ and $q : (i : \iota2) \rightarrow s2 \ i$ into a PiTensorProduct .

```
def elimPureTensorMullin : MultilinearMap R s1
  (MultilinearMap R s2 ( $\bigotimes_{[R]} i : \iota1 \oplus \iota2, (\text{Sum.elim } s1 \ s2) \ i$ )) where
```

def tmul[\[source\]](#)

Collapse a TensorProduct of PiTensorProduct into a PiTensorProduct .

```
def tmul : (( $\bigotimes_{[R]} i : \iota1, s1 \ i$ )  $\otimes_{[R]}$   $\bigotimes_{[R]} i : \iota2, s2 \ i$ )  $\rightarrow_l [R]$ 
   $\bigotimes_{[R]} i : \iota1 \oplus \iota2, (\text{Sum.elim } s1 \ s2) \ i$ 
```

def tmulEquiv[\[source\]](#)

The equivalence formed by combining a TensorProduct into a PiTensorProduct.

```
def tmulEquiv : (( $\bigotimes_{[R]}$  i :  $\iota_1$ , s1 i)  $\otimes_{[R]}$   $\bigotimes_{[R]}$  i :  $\iota_2$ , s2 i)  $\approx_{\iota[R]}$ 
 $\bigotimes_{[R]}$  i :  $\iota_1 \otimes \iota_2$ , (Sum.elim s1 s2) i
```

lemma tmulEquiv_tmul_tprod[\[source\]](#)

```
@[simp]
lemma tmulEquiv_tmul_tprod (p : (i :  $\iota_1$ )  $\rightarrow$  s1 i) (q : (i :  $\iota_2$ )  $\rightarrow$  s2 i) :
  tmulEquiv ((PiTensorProduct.tprod R) p  $\otimes_{\iota[R]}$  (PiTensorProduct.tprod R) q) =
  (PiTensorProduct.tprod R) (elimPureTensor p q)
```

2.7 PhysLean.Mathematics.RatComplexNum

[PhysLean/Mathematics/RatComplexNum.lean](#) on GitHub.

structure RatComplexNum[\[source\]](#)

Rational complex numbers. This type is mainly used when decidability is needed.

```
structure RatComplexNum where
  fst :  $\mathbb{Q}$ 
  snd :  $\mathbb{Q}$ 
```

lemma ext[\[source\]](#)

```
@[ext]
lemma ext {x y : RatComplexNum} (h1 : x.1 = y.1) (h2 : x.2 = y.2) : x = y
```

def equivToProd[\[source\]](#)

The equivalence as a type of RatComplexNum with $\mathbb{Q} \times \mathbb{Q}$.

```
def equivToProd : RatComplexNum  $\approx$   $\mathbb{Q} \times \mathbb{Q}$  where
```

instance DecidableEq RatComplexNum[\[source\]](#)

```
instance : DecidableEq RatComplexNum
```

instance AddCommGroup RatComplexNum[\[source\]](#)

```
instance : AddCommGroup RatComplexNum where
```

instance Ring RatComplexNum[\[source\]](#)

```
instance : Ring RatComplexNum where
```

lemma one_fst[\[source\]](#)

```
@[simp]
lemma one_fst : (1 : RatComplexNum).fst = 1
```

lemma one_snd[\[source\]](#)

```
@[simp]
lemma one_snd : (1 : RatComplexNum).snd = 0
```

lemma add_fst[\[source\]](#)

```
@[simp]
lemma add_fst (x y : RatComplexNum) : (x + y).fst = x.fst + y.fst
```

lemma add_snd[\[source\]](#)

```
@[simp]
lemma add_snd (x y : RatComplexNum) : (x + y).snd = x.snd + y.snd
```

lemma mul_fst[\[source\]](#)

```
@[simp]
lemma mul_fst (x y : RatComplexNum) : (x * y).fst = x.fst * y.fst - x.snd * y.snd
```

lemma mul_snd[\[source\]](#)

```
@[simp]
lemma mul_snd (x y : RatComplexNum) : (x * y).snd = x.fst * y.snd + x.snd * y.fst
```

lemma zero_fst[\[source\]](#)

```
@[simp]
lemma zero_fst : (0 : RatComplexNum).fst = 0
```

lemma zero_snd[\[source\]](#)

```
@[simp]
lemma zero_snd : (0 : RatComplexNum).snd = 0
```

def toComplexNum[\[source\]](#)

The inclusion of `RatComplexNum` into the complex numbers.

```
noncomputable def toComplexNum : RatComplexNum →+* ℂ where
```

lemma I_mul_toComplexNum[\[source\]](#)

```
@[simp]
lemma I_mul_toComplexNum (a : RatComplexNum) : I * toComplexNum a = toComplexNum ((0, 1) * a)
```

lemma ofNat_mul_toComplexNum[\[source\]](#)

```
lemma ofNat_mul_toComplexNum (n : ℕ) (a : RatComplexNum) :
  n * toComplexNum a = toComplexNum (n * a)
```

lemma toComplexNum_injective[\[source\]](#)

```
lemma toComplexNum_injective : Function.Injective toComplexNum
```

2.8 PhysLean.Mathematics.SpecialFunctions.PhysicistsHermite

[PhysLean/Mathematics/SpecialFunctions/PhysicistsHermite.lean](#) on GitHub.

def physHermite[\[source\]](#)

The Physicists Hermite polynomial are defined as polynomials over $\mathbb{Z}$ in X recursively with $\text{physHermite } 0 = 1$ and

$\text{physHermite } (n + 1) = 2 \cdot X \cdot \text{physHermite } n - \text{derivative } (\text{physHermite } n).$

This polynomial will often be cast as a function $\mathbb{R} \rightarrow \mathbb{R}$ by evaluating the polynomial at X .

```
noncomputable def physHermite : ℕ → Polynomial ℤ
| 0 => 1
| n + 1 => 2 * X * physHermite n - derivative (physHermite n)
```

lemma physHermite_succ[\[source\]](#)

```
lemma physHermite_succ (n : ℕ) :
  physHermite (n + 1) = 2 * X * physHermite n - derivative (physHermite n)
```

lemma physHermite_eq_iterate[\[source\]](#)

```
lemma physHermite_eq_iterate (n : ℕ) :
  physHermite n = (fun p => 2 * X * p - derivative p)^[n] 1
```

lemma physHermite_zero[\[source\]](#)

```
@[simp]
lemma physHermite_zero : physHermite 0 = C 1
```

lemma physHermite_one[\[source\]](#)

```
lemma physHermite_one : physHermite 1 = 2 * X
```

lemma derivative_physHermite_succ[\[source\]](#)

```
lemma derivative_physHermite_succ : (n : ℕ) →
  derivative (physHermite (n + 1)) = 2 * (n + 1) * physHermite n
| 0 => by
  simp [physHermite_one]
| n + 1 => by
  rw [physHermite_succ]
  simp only [nsmul_eq_mul, Nat.cast_ofNat, derivative_sub, derivative_mul, derivative_ofNat,
    zero_mul, derivative_X, mul_one, zero_add, Nat.cast_add, Nat.cast_one]
  rw [derivative_physHermite_succ]
  simp only [physHermite_succ, nsmul_eq_mul, Nat.cast_ofNat, Nat.cast_add, Nat.cast_one,
    derivative_mul, derivative_ofNat, zero_mul, derivative_add, derivative_natCast,
```

```

    derivative_one, add_zero, zero_add]
ring

```

lemma derivative_physHermite[\[source\]](#)

```

lemma derivative_physHermite : (n : ℕ) →
  derivative (physHermite n) = 2 * n • physHermite (n - 1)
| 0 => by
  simp
| n + 1 => by
  rw [derivative_physHermite_succ]
  simp

```

lemma physHermite_succ'[\[source\]](#)

```

lemma physHermite_succ' (n : ℕ) :
  physHermite (n + 1) = 2 • X * physHermite n - 2 * n • physHermite (n - 1)

```

lemma coeff_physHermite_succ_zero[\[source\]](#)

```

lemma coeff_physHermite_succ_zero (n : ℕ) :
  coeff (physHermite (n + 1)) 0 = - coeff (physHermite n) 1

```

lemma coeff_physHermite_succ_succ[\[source\]](#)

```

lemma coeff_physHermite_succ_succ (n k : ℕ) : coeff (physHermite (n + 1)) (k + 1) =
  2 * coeff (physHermite n) k - (k + 2) * coeff (physHermite n) (k + 2)

```

lemma coeff_physHermite_of_lt[\[source\]](#)

```

lemma coeff_physHermite_of_lt {n k : ℕ} (h : n < k) : coeff (physHermite n) k = 0

```

lemma coeff_physHermite_self_succ[\[source\]](#)

```

@[simp]
lemma coeff_physHermite_self_succ (n : ℕ) : coeff (physHermite n) n = 2 ^ n

```

lemma degree_physHermite[\[source\]](#)

```

@[simp]
lemma degree_physHermite (n : ℕ) : degree (physHermite n) = n

```

lemma natDegree_physHermite[\[source\]](#)

```

@[simp]
lemma natDegree_physHermite {n : ℕ} : (physHermite n).natDegree = n

```

lemma iterate_derivative_physHermite_of_gt[\[source\]](#)

```

lemma iterate_derivative_physHermite_of_gt {n m : ℕ} (h : n < m) :
  derivative^[m] (physHermite n) = 0

```

lemma iterate_derivative_physHermite_self[\[source\]](#)

```
@[simp]
lemma iterate_derivative_physHermite_self {n : ℕ} :
  derivative^[n] (physHermite n) = C ((n ! : ℤ) * 2 ^ n)
```

lemma physHermite_leadingCoeff[\[source\]](#)

```
@[simp]
lemma physHermite_leadingCoeff {n : ℕ} : (physHermite n).leadingCoeff = 2 ^ n
```

lemma physHermite_ne_zero[\[source\]](#)

```
@[simp]
lemma physHermite_ne_zero {n : ℕ} : physHermite n ≠ 0
```

instance CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ)[\[source\]](#)

```
instance : CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ) where
```

lemma physHermite_eq_aeval[\[source\]](#)

```
lemma physHermite_eq_aeval (n : ℕ) (x : ℝ) :
  physHermite n x = (physHermite n).aeval x
```

lemma physHermite_zero_apply[\[source\]](#)

```
lemma physHermite_zero_apply (x : ℝ) : physHermite 0 x = 1
```

lemma physHermite_pow[\[source\]](#)

```
lemma physHermite_pow (n m : ℕ) (x : ℝ) :
  physHermite n x ^ m = aeval x (physHermite n ^ m)
```

lemma physHermite_succ_fun[\[source\]](#)

```
lemma physHermite_succ_fun (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) = 2 • (fun x => x) *
  (physHermite n : ℝ → ℝ) - (2 * n : ℝ) • (physHermite (n - 1) : ℝ → ℝ)
```

lemma physHermite_succ_fun'[\[source\]](#)

```
lemma physHermite_succ_fun' (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) = fun x => 2 • x *
  physHermite n x -
  (2 * n : ℝ) • physHermite (n - 1) x
```

lemma iterated_deriv_physHermite_eq_aeval[\[source\]](#)

```
lemma iterated_deriv_physHermite_eq_aeval (n : ℕ) : (m : ℕ) →
  deriv^[m] (physHermite n) = fun x => (derivative^[m] (physHermite n)).aeval x
| 0 => by
```



```

    simp
| m + 1 => by
    simp only [Function.iterate_succ_apply', Function.comp_apply]
    rw [iterated_deriv_physHermite_eq_aeval n m]
    funext x
    rw [Polynomial.deriv_aeval]

```

lemma physHermite_differentiableAt[\[source\]](#)

```

@[fun_prop]
lemma physHermite_differentiableAt (n : ℕ) (x : ℝ) :
  DifferentiableAt ℝ (physHermite n) x

```

lemma deriv_physHermite_differentiableAt[\[source\]](#)

```

@[fun_prop]
lemma deriv_physHermite_differentiableAt (n m : ℕ) (x : ℝ) :
  DifferentiableAt ℝ (deriv^[m] (physHermite n)) x

```

lemma deriv_physHermite[\[source\]](#)

```

lemma deriv_physHermite (n : ℕ) :
  deriv (physHermite n) = 2 * n * (physHermite (n - 1))

```

lemma fderiv_physHermite[\[source\]](#)

```

lemma fderiv_physHermite
  {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] (x : E)
  (f : E → ℝ) (hf : DifferentiableAt ℝ f x) (n : ℕ) :
  fderiv ℝ (fun x => physHermite n (f x)) x
  = (2 * n * physHermite (n - 1) (f x)) • fderiv ℝ f x

```

lemma deriv_physHermite'[\[source\]](#)

```

@[simp]
lemma deriv_physHermite' (x : ℝ)
  (f : ℝ → ℝ) (hf : DifferentiableAt ℝ f x) (n : ℕ) :
  deriv (fun x => physHermite n (f x)) x
  = (2 * n * physHermite (n - 1) (f x)) * deriv f x

```

lemma physHermite_parity[\[source\]](#)

```

lemma physHermite_parity: (n : ℕ) → (x : ℝ) →
  physHermite n (-x) = (-1)^n * physHermite n x
| 0, x => by simp
| 1, x => by
  simp [physHermite_one, aeval]
| n + 2, x => by
  rw [physHermite_succ_fun']
  have h1

```

lemma deriv_gaussian_eq_physHermite_mul_gaussian[\[source\]](#)

```

lemma deriv_gaussian_eq_physHermite_mul_gaussian (n : ℕ) (x : ℝ) :
  deriv^[n] (fun y => Real.exp (- y ^ 2)) x =
  (-1 : ℝ) ^ n * physHermite n x * Real.exp (- x ^ 2)

```

lemma physHermite_eq_deriv_gaussian[\[source\]](#)

```

lemma physHermite_eq_deriv_gaussian (n : ℕ) (x : ℝ) :
  physHermite n x = (-1 : ℝ) ^ n * deriv^[n]
  (fun y => Real.exp (- y ^ 2)) x / Real.exp (- x ^ 2)

```

lemma physHermite_eq_deriv_gaussian'[\[source\]](#)

```

lemma physHermite_eq_deriv_gaussian' (n : ℕ) (x : ℝ) :
  physHermite n x = (-1 : ℝ) ^ n * deriv^[n] (fun y => Real.exp (- y ^ 2)) x *
  Real.exp (x ^ 2)

```

lemma gaussian_integrable_polynomial[\[source\]](#)

```

@[fun_prop]
lemma gaussian_integrable_polynomial {b : ℝ} (hb : 0 < b) (P : Polynomial ℤ) :
  MeasureTheory.Integrable fun x : ℝ => (P.aeval x) * Real.exp (-b * x ^ 2)

```

lemma gaussian_integrable_polynomial_cons[\[source\]](#)

```

@[fun_prop]
lemma gaussian_integrable_polynomial_cons {b c : ℝ} (hb : 0 < b) (P : Polynomial ℤ) :
  MeasureTheory.Integrable fun x : ℝ => (P.aeval (c * x)) * Real.exp (-b * x ^ 2)

```

lemma physHermite_gaussian_integrable[\[source\]](#)

```

@[fun_prop]
lemma physHermite_gaussian_integrable (n p m : ℕ) :
  MeasureTheory.Integrable (deriv^[m] (physHermite p) * deriv^[n] fun x => Real.exp (-x ^ 2))
  MeasureTheory.volume

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_exp[\[source\]](#)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_exp (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (-x ^ 2) =
  (-1 : ℝ) ^ m * ∫ x : ℝ, (physHermite n x * (deriv^[m] fun x => Real.exp (-x ^ 2)) x)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_inductive[\[source\]](#)

```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv_inductive (n m : ℕ) :
  (p : ℕ) → (hpm : p ≤ m) →
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =
  (-1 : ℝ) ^ (m - p) * ∫ x : ℝ, (deriv^[p] (physHermite n) x *
  (deriv^[m - p] fun x => Real.exp (-x ^ 2)) x)
| 0, h => by
  exact integral_physHermite_mul_physHermite_eq_integral_deriv_exp n m
| p + 1, h => by

```

```
rw [integral_physHermite_mul_physHermite_eq_integral_deriv_inductive n m p (by omega)]
have h1 : m - p = m - (p + 1) + 1
```

lemma integral_physHermite_mul_physHermite_eq_integral_deriv

[\[source\]](#)

```
lemma integral_physHermite_mul_physHermite_eq_integral_deriv (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =
  ∫ x : ℝ, (deriv^[m] (physHermite n) x * (Real.exp (-x ^ 2)))
```

lemma physHermite_orthogonal_lt

[\[source\]](#)

```
lemma physHermite_orthogonal_lt {n m : ℕ} (hnm : n < m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

theorem physHermite_orthogonal

[\[source\]](#)

```
theorem physHermite_orthogonal {n m : ℕ} (hnm : n ≠ m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

lemma physHermite_orthogonal_cons

[\[source\]](#)

```
lemma physHermite_orthogonal_cons {n m : ℕ} (hnm : n ≠ m) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite m (c * x)) *
  Real.exp (- c ^ 2 * x ^ 2) = 0
```

theorem physHermite_norm

[\[source\]](#)

```
theorem physHermite_norm (n : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite n x) * Real.exp (- x ^ 2) =
  ↑n ! * 2 ^ n * √Real.pi
```

lemma physHermite_norm_cons

[\[source\]](#)

```
lemma physHermite_norm_cons (n : ℕ) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite n (c * x)) * Real.exp (- c ^ 2 * x ^ 2) =
  |c-1| • (↑n ! * 2 ^ n * √Real.pi)
```

lemma polynomial_mem_physHermite_span_induction

[\[source\]](#)

```
lemma polynomial_mem_physHermite_span_induction (P : Polynomial ℤ) : (n : ℕ) →
  (hn : P.natDegree = n) →
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))
| 0, h => by
  rw [natDegree_eq_zero] at h
  obtain (x, rfl)
```

lemma polynomial_mem_physHermite_span

[\[source\]](#)

```
lemma polynomial_mem_physHermite_span (P : Polynomial ℤ) :
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))
```

lemma cos_mem_physHermite_span_topologicalClosure[\[source\]](#)

```
lemma cos_mem_physHermite_span_topologicalClosure (c : ℝ) :
  (fun (x : ℝ) => Real.cos (c * x)) ∈
  (Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))).topologicalClosure
```

2.9 PhysLean.Meta.Basic

[PhysLean/Meta/Basic.lean](#) on GitHub.

def flatSize[\[source\]](#)

The size of a flattened array of arrays.

```
def Array.flatSize {α} : Array (Array α) → Nat
```

def flatFilterSizeM[\[source\]](#)

The size of a flattened array of arrays after applying an element-wise filter.

```
def Array.flatFilterSizeM {α m} [Monad m] (p : α → m Bool) : Array (Array α) → m Nat
```

def allImports[\[source\]](#)

Gets all imports within PhysLean.

```
def PhysLean.allImports : IO (Array Import)
```

def noImports[\[source\]](#)

Number of files within PhysLean.

```
def PhysLean.noImports : IO Nat
```

def getConsts[\[source\]](#)

Gets all constants from an import.

```
def PhysLean.Imports.getConsts (imp : Import) : IO (Array ConstantInfo)
```

def getUserConsts[\[source\]](#)

Gets all user defined constants from an import.

```
def PhysLean.Imports.getUserConsts (imp : Import) : CoreM (Array ConstantInfo)
```

def toFilePath[\[source\]](#)

Turns a name into a system file path.

```
def Lean.Name.toFilePath (c : Name) : System.FilePath
```

def getLines [\[source\]](#)

Lines from import.

```
def PhysLean.Imports.getLines (imp : Import) : IO (Array String)
```

def toRelativeFilePath [\[source\]](#)

Turns a name into a Lean file.

```
def toRelativeFilePath (c : Name) : System.FilePath
```

def toGitHubLink [\[source\]](#)

Turns a name, which represents a module, into a link to github.

```
def toGitHubLink (c : Name) (line : Nat) : String
```

def lineNumber [\[source\]](#)

Given a name, returns the line number.

```
def lineNumber (c : Name) : m Nat
```

def fileName [\[source\]](#)

Given a name, returns the file name corresponding to that declaration.

```
def fileName (c : Name) : m Name
```

def location [\[source\]](#)

Returns the location of a name.

```
def location (c : Name) : m String
```

def hasPos [\[source\]](#)

Determines if a name has a location.

```
def hasPos (c : Name) : m Bool
```

def hasDocString [\[source\]](#)

Determines if a name has a doc string.

```
def hasDocString (c : Name) : CoreM Bool
```

def getDocString [\[source\]](#)

The doc string from a name.

```
def getDocString (c : Name) : CoreM String
```

def getDeclString [\[source\]](#)

Given a name, returns the source code defining that name, including doc strings.

```
def getDeclString (name : Name) : CoreM String
```

def getDeclStringNoDoc [\[source\]](#)

Given a name, returns the source code defining that name, starting with the def ... or lemma... etc.

```
def getDeclStringNoDoc (name : Name) : CoreM String
```

def noDefs [\[source\]](#)

Number of definitions.

```
def noDefs : CoreM Nat
```

def noLemmas [\[source\]](#)

Number of definitions.

```
def noLemmas : CoreM Nat
```

def allDocStrings [\[source\]](#)

All docstrings present in PhysLean.

```
def allDocStrings : CoreM (Array String)
```

def noDefsNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noDefsNoDocString : CoreM Nat
```

def noLemmasNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noLemmasNoDocString : CoreM Nat
```

def noLines [\[source\]](#)

The number of lines in PhysLean.

```
def noLines : IO Nat
```

def noTODOs [\[source\]](#)

The number of TODO items.

```
def noTODOs : IO Nat
```

def noFilesWithTODOs[\[source\]](#)*The number of files with a TODO item.*

```
def noFilesWithTODOs : IO Nat
```

2.10 PhysLean.Meta.Informal.Post[PhysLean/Meta/Informal/Post.lean](#) on GitHub.**def noInformalLemmas**[\[source\]](#)*The number of informal lemmas in PhysLean.*

```
def noInformalLemmas : CoreM Nat
```

def noInformalDefs[\[source\]](#)*The number of informal definitions in PhysLean.*

```
def noInformalDefs : CoreM Nat
```

2.11 PhysLean.Meta.Notes.Basic[PhysLean/Meta/Notes/Basic.lean](#) on GitHub.**structure NoteInfo**[\[source\]](#)*The information from a note ... command. To be used in a note file*

```
structure NoteInfo where
  content : String
  fileName : Name
  line : Nat
```

def elabNote[\[source\]](#)*Elaborator for the note ... command*

```
@[command_elab note_comment]
def elabNote : Lean.Elab.Command.CommandElab
```

2.12 PhysLean.Meta.Notes.HTMLNote[PhysLean/Meta/Notes/HTMLNote.lean](#) on GitHub.**structure HTMLNote**[\[source\]](#)*A HTMLNote is a structure containing the html information from individual contributions (commands, informal commands, note ..) etc. to a note file.*

```
structure HTMLNote where
  fileName : Name
  content : String
  line : Nat
```

def ofNodeInfo[\[source\]](#)*Converts a note info into a HTMLNote.*

```
def HTMLNote.ofNodeInfo (ni : NoteInfo) : HTMLNote
```

def ofFormal[\[source\]](#)*An formal definition or lemma to html for a note.*

```
def HTMLNote.ofFormal (name : Name) : CoreM HTMLNote
```

def ofInformal[\[source\]](#)*An informal definition or lemma to html for a note.*

```
unsafe def HTMLNote.ofInformal (name : Name) : CoreM HTMLNote
```

2.13 PhysLean.Meta.Notes.NoteFile[PhysLean/Meta/Notes/NoteFile.lean](#) on GitHub.**structure NoteFile**[\[source\]](#)*A note consists of a title and a list of Lean files which make up the note.*

```
structure NoteFile where
  title : String
  abstract : String
  authors : List String
  files : List Name
```

def imports[\[source\]](#)*All imports associated to a note.*

```
def imports : Array Import
```

2.14 PhysLean.Meta.Notes.ToHTML[PhysLean/Meta/Notes/ToHTML.lean](#) on GitHub.**def sortLE**[\[source\]](#)*Sorts NoteInfo's by file name then by line number.*

```
def sortLE (ni1 ni2 : HTMLNote) : Bool
```

def getNodeInfo[\[source\]](#)*Returns a sorted list of NodeInfos for a file system.*

```
unsafe def getNodeInfo : CoreM (List HTMLNote)
```


def codeBlockHTML [\[source\]](#)

The HTML code needed to have syntax highlighting.

```
def codeBlockHTML : String
```

def informalDefStyle [\[source\]](#)

The html styles for informal definitions.

```
def informalDefStyle : String
```

def codeButton [\[source\]](#)

The html styles for code button.

```
def codeButton : String
```

def mathJaxScript [\[source\]](#)

HTML allowing the use of mathjax.

```
def mathJaxScript : String
```

def headerHTML [\[source\]](#)

The header to the html code.

```
def headerHTML : String
```

def titleHTML [\[source\]](#)

The html code corresponding to the title, abstract and authors.

```
def titleHTML : String
```

def leanNote [\[source\]](#)

The html code corresponding to a note about Lean and its use.

```
def leanNote : String
```

def footerHTML [\[source\]](#)

The footer of the html file.

```
def footerHTML : String
```

def toHTMLString [\[source\]](#)

The html file associated to a NoteFile string.

```
unsafe def toHTMLString : MetaM String
```

2.15 PhysLean.Meta.Remark.Basic

[PhysLean/Meta/Remark/Basic.lean](#) on GitHub.

structure RemarkInfo

[\[source\]](#)

The information from a remark ... command. To be used in a note file

```
structure RemarkInfo where
  content : String
  fileName : Name
  line : Nat
  name : Name
  nameSpace : Name
```

def elabRemark

[\[source\]](#)

Elaborator for the note ... command

```
@[command_elab remark_syntax]
def elabRemark : Elab.Command.CommandElab
```

2.16 PhysLean.Meta.Remark.Properties

[PhysLean/Meta/Remark/Properties.lean](#) on GitHub.

def allRemarkInfo

[\[source\]](#)

All remarks in the environment.

```
def allRemarkInfo : m (Array RemarkInfo)
```

def toFullName

[\[source\]](#)

The full name of a remark (name and namespace).

```
def RemarkInfo.toFullName (r : RemarkInfo) : Name
```

def IsRemark

[\[source\]](#)

A Bool which is true if a name corresponds to a remark.

```
def RemarkInfo.IsRemark (n : Name) : m Bool
```

def getRemarkInfo

[\[source\]](#)

Gets the remarkInfo from a name corresponding to a remark..

```
def RemarkInfo.getRemarkInfo (n : Name) : m RemarkInfo
```

2.17 PhysLean.Meta.TODO.Basic

[PhysLean/Meta/TODO/Basic.lean](#) on GitHub.

structure todoInfo[\[source\]](#)

The information from a TODO ... command.

```
structure todoInfo where
  content : String
  fileName : Name
  line : Nat
```

def elabTODO[\[source\]](#)

Elaborator for the TODO ... command

```
@[command_elab todo_comment]
def elabTODO : Elab.Command.CommandElab
```

2.18 PhysLean.Particles.StandardModel.HiggsBoson.Basic

[PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

instance ContMDiffVectorBundle τ HiggsVec HiggsBundle SpaceTime.asSmoothManifold [\[source\]](#)

The instance of a smooth vector bundle with total space HiggsBundle and fiber HiggsVec.

```
instance : ContMDiffVectorBundle  $\tau$  HiggsVec HiggsBundle SpaceTime.asSmoothManifold
```

2.19 PhysLean.Particles.StandardModel.HiggsBoson.GaugeAction

[PhysLean/Particles/StandardModel/HiggsBoson/GaugeAction.lean](#) on GitHub.

def rotateGaugeGroup[\[source\]](#)

Given a Higgs vector, an element of the gauge group which puts the first component of the vector to zero, and the second component to a real number.

```
def rotateGaugeGroup { $\phi$  : HiggsVec} (h $\phi$  :  $\phi \neq 0$ ) : GaugeGroupI
```

lemma rotateGaugeGroup_apply[\[source\]](#)

Acting on a non-zero Higgs vector with its rotation matrix gives a vector which is zero in the first component and a positive real in the second component.

```
lemma rotateGaugeGroup_apply { $\phi$  : HiggsVec} (h $\phi$  :  $\phi \neq 0$ ) :
  rep (rotateGaugeGroup h $\phi$ )  $\phi$  = ![0, Complex.ofRealHom || $\phi$ ||]
```

2.20 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Basic

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/Basic.lean](#) on GitHub.

instance Setoid (FieldOpFreeAlgebra $\mathcal{F}$)[\[source\]](#)

The instance of a setoid on FieldOpFreeAlgebra from the ideal TwoSidedIdeal.

```
instance : Setoid (FieldOpFreeAlgebra  $\mathcal{F}$ )
```

lemma equiv_iff_exists_add[\[source\]](#)

```
lemma equiv_iff_exists_add (x y : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
  x ≈ y ↔ ∃ a, x = y + a ∧ a ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet
```

lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_bosonic_or_zero[\[source\]](#)

```
lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_bosonic_or_zero (φ ψ :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOpF φ, ofCrAnOpF ψ]sF ∈ statisticSubmodule bosonic v
  ι [ofCrAnOpF φ, ofCrAnOpF ψ]sF = 0
```

lemma ι_superCommuteF_ofCrAnOpF_superCommuteF_ofCrAnOpF_ofCrAnOpF[\[source\]](#)

```
@[simp]
lemma ι_superCommuteF_ofCrAnOpF_superCommuteF_ofCrAnOpF_ofCrAnOpF (φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp) :
  ι [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF = 0
```

lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnOpF[\[source\]](#)

```
lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnOpF (φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp) :
  ι [[ofCrAnOpF φ1, ofCrAnOpF φ2]sF, ofCrAnOpF φ3]sF = 0
```

lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnListF[\[source\]](#)

```
lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnListF (φ1 φ2 :  $\mathcal{F}$ .CrAnFieldOp)
  (ψs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ι [[ofCrAnOpF φ1, ofCrAnOpF φ2]sF, ofCrAnListF ψs]sF = 0
```

lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_fieldOpFreeAlgebra[\[source\]](#)

```
@[simp]
lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_fieldOpFreeAlgebra (φ1 φ2 :  $\mathcal{F}$ .
  CrAnFieldOp)
  (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) : ι [[ofCrAnOpF φ1, ofCrAnOpF φ2]sF, a]sF = 0
```

lemma ι_commute_fieldOpFreeAlgebra_superCommuteF_ofCrAnOpF_ofCrAnOpF[\[source\]](#)

```
lemma ι_commute_fieldOpFreeAlgebra_superCommuteF_ofCrAnOpF_ofCrAnOpF (φ1 φ2 :  $\mathcal{F}$ .CrAnFieldOp)
  (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) : ι a * ι [ofCrAnOpF φ1, ofCrAnOpF φ2]sF -
  ι [ofCrAnOpF φ1, ofCrAnOpF φ2]sF * ι a = 0
```

lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_mem_center[\[source\]](#)

```
lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_mem_center (φ ψ :  $\mathcal{F}$ .CrAnFieldOp) :
  ι [ofCrAnOpF φ, ofCrAnOpF ψ]sF ∈ Subalgebra.center  $\subset \mathcal{F}$ .FieldOpAlgebra
```

lemma bosonicProjF_mem_fieldOpIdealSet_or_zero[\[source\]](#)

```
lemma bosonicProjF_mem_fieldOpIdealSet_or_zero (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈  $\mathcal{F}$ .fieldOpIdealSet) :
  x.bosonicProjF.1 ∈  $\mathcal{F}$ .fieldOpIdealSet ∨ x.bosonicProjF = 0
```

lemma fermionicProjF_mem_fieldOpIdealSet_or_zero[\[source\]](#)

```

lemma fermionicProjF_mem_fieldOpIdealSet_or_zero (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈  $\mathcal{F}$ .fieldOpIdealSet) :
  x.fermionicProjF.1 ∈  $\mathcal{F}$ .fieldOpIdealSet ∨ x.fermionicProjF = 0

```

lemma bosonicProjF_mem_ideal[\[source\]](#)

```

lemma bosonicProjF_mem_ideal (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) :
  x.bosonicProjF.1 ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet

```

lemma fermionicProjF_mem_ideal[\[source\]](#)

```

lemma fermionicProjF_mem_ideal (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) :
  x.fermionicProjF.1 ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet

```

lemma ι_eq_zero_iff_ι_bosonicProjF_fermonicProj_zero[\[source\]](#)

```

lemma ι_eq_zero_iff_ι_bosonicProjF_fermonicProj_zero (x : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
  ι x = 0 ↔ ι x.bosonicProjF.1 = 0 ∧ ι x.fermionicProjF.1 = 0

```

lemma ofFieldOp_eq_ι_ofFieldOpF[\[source\]](#)

```

lemma ofFieldOp_eq_ι_ofFieldOpF (φ :  $\mathcal{F}$ .FieldOp) : ofFieldOp φ = ι (ofFieldOpF φ)

```

lemma ofFieldOpList_eq_ι_ofFieldOpListF[\[source\]](#)

```

lemma ofFieldOpList_eq_ι_ofFieldOpListF (φs : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpList φs = ι (ofFieldOpListF φs)

```

def ofCrAnOp[\[source\]](#)

For a field specification $\mathcal{F}$ and an element ϕ of $\mathcal{F}$.CrAnFieldOp, ofCrAnOp ϕ is defined as the element of $\mathcal{F}$.FieldOpAlgebra given by ι (ofCrAnOpF ϕ).

```

def ofCrAnOp (φ :  $\mathcal{F}$ .CrAnFieldOp) :  $\mathcal{F}$ .FieldOpAlgebra

```

lemma ofCrAnOp_eq_ι_ofCrAnOpF[\[source\]](#)

```

lemma ofCrAnOp_eq_ι_ofCrAnOpF (φ :  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnOp φ = ι (ofCrAnOpF φ)

```

def ofCrAnList[\[source\]](#)

For a field specification $\mathcal{F}$ and a list ϕs of $\mathcal{F}$.CrAnFieldOp, ofCrAnList ϕs is defined as the element of $\mathcal{F}$.FieldOpAlgebra given by ι (ofCrAnListF ϕ).

```

def ofCrAnList (φs : List  $\mathcal{F}$ .CrAnFieldOp) :  $\mathcal{F}$ .FieldOpAlgebra

```

lemma ofCrAnList_eq_ι_ofCrAnListF[\[source\]](#)

```
lemma ofCrAnList_eq_ι_ofCrAnListF (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList φs = ι (ofCrAnListF φs)
```

lemma ofCrAnList_append[\[source\]](#)

```
lemma ofCrAnList_append (φs ψs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList (φs ++ ψs) = ofCrAnList φs * ofCrAnList ψs
```

lemma ofCrAnList_singleton[\[source\]](#)

```
lemma ofCrAnList_singleton (φ :  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList [φ] = ofCrAnOp φ
```

2.21 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Grading

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/Grading.lean](#) on GitHub.

def statSubmodule[\[source\]](#)

The submodule of $\mathcal{F}$.FieldOpAlgebra spanned by lists of field statistic f .

```
def statSubmodule (f : FieldStatistic) : Submodule  $\mathbb{C}$   $\mathcal{F}$ .FieldOpAlgebra
```

lemma ofCrAnList_mem_statSubmodule_of_eq[\[source\]](#)

```
lemma ofCrAnList_mem_statSubmodule_of_eq (φs : List  $\mathcal{F}$ .CrAnFieldOp) (f : FieldStatistic)
  (h : ( $\mathcal{F}$  |>s φs) = f) : ofCrAnList φs ∈ statSubmodule f
```

lemma ofCrAnList_mem_statSubmodule[\[source\]](#)

```
lemma ofCrAnList_mem_statSubmodule (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList φs ∈ statSubmodule ( $\mathcal{F}$  |>s φs)
```

lemma mem_bosonic_of_mem_free_bosonic[\[source\]](#)

```
lemma mem_bosonic_of_mem_free_bosonic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule bosonic) : ι a ∈ statSubmodule .bosonic
```

lemma mem_fermionic_of_mem_free_fermionic[\[source\]](#)

```
lemma mem_fermionic_of_mem_free_fermionic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule fermionic) : ι a ∈ statSubmodule .fermionic
```

lemma mem_statSubmodule_of_mem_statisticSubmodule[\[source\]](#)

```
lemma mem_statSubmodule_of_mem_statisticSubmodule (f : FieldStatistic) (a :  $\mathcal{F}$ .
  FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule f) : ι a ∈ statSubmodule f
```

def ιStateSubmodule[\[source\]](#)

The projection of statisticSubmodule ($\mathcal{F} := \mathcal{F}$) f defined in the free algebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) f .

```
def ιStateSubmodule (f : FieldStatistic) :
  statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $f \rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $f$  where
```

def bosonicProjFree[\[source\]](#)

The projection of $\mathcal{F}.FieldOpFreeAlgebra$ to statSubmodule ($\mathcal{F} := \mathcal{F}$) bosonic.

```
def bosonicProjFree :  $\mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) bosonic
```

lemma bosonicProjFree_eq_ι_bosonicProjF[\[source\]](#)

```
lemma bosonicProjFree_eq_ι_bosonicProjF (a :  $\mathcal{F}.FieldOpFreeAlgebra$ ) :
  (bosonicProjFree a).1 = ι (bosonicProjF a)
```

lemma bosonicProjFree_zero_of_ι_zero[\[source\]](#)

```
lemma bosonicProjFree_zero_of_ι_zero (a :  $\mathcal{F}.FieldOpFreeAlgebra$ ) (h : ι a = 0) :
  bosonicProjFree a = 0
```

lemma bosonicProjFree_eq_of_equiv[\[source\]](#)

```
lemma bosonicProjFree_eq_of_equiv (a b :  $\mathcal{F}.FieldOpFreeAlgebra$ ) (h : a ≈ b) :
  bosonicProjFree a = bosonicProjFree b
```

def bosonicProj[\[source\]](#)

The projection of $\mathcal{F}.FieldOpAlgebra$ to statSubmodule ($\mathcal{F} := \mathcal{F}$) bosonic.

```
def bosonicProj :  $\mathcal{F}.FieldOpAlgebra \rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) bosonic where
```

lemma bosonicProj_eq_bosonicProjFree[\[source\]](#)

```
lemma bosonicProj_eq_bosonicProjFree (a :  $\mathcal{F}.FieldOpFreeAlgebra$ ) :
  bosonicProj (ι a) = bosonicProjFree a
```

def fermionicProjFree[\[source\]](#)

The projection of $\mathcal{F}.FieldOpFreeAlgebra$ to statSubmodule ($\mathcal{F} := \mathcal{F}$) fermionic.

```
def fermionicProjFree :  $\mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) fermionic
```

lemma fermionicProjFree_eq_ι_fermionicProjF[\[source\]](#)

```
lemma fermionicProjFree_eq_ι_fermionicProjF (a :  $\mathcal{F}.FieldOpFreeAlgebra$ ) :
  (fermionicProjFree a).1 = ι (fermionicProjF a)
```

lemma fermionicProjFree_zero_of_ι_zero[\[source\]](#)

```
lemma fermionicProjFree_zero_of_ι_zero (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h :  $\iota a = 0$ ) :
  fermionicProjFree a = 0
```

lemma fermionicProjFree_eq_of_equiv[\[source\]](#)

```
lemma fermionicProjFree_eq_of_equiv (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h :  $a \approx b$ ) :
  fermionicProjFree a = fermionicProjFree b
```

def fermionicProj[\[source\]](#)

The projection of $\mathcal{F}$.FieldOpAlgebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) fermionic.

```
def fermionicProj :  $\mathcal{F}$ .FieldOpAlgebra  $\rightarrow$   $[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) fermionic where
```

lemma fermionicProj_eq_fermionicProjFree[\[source\]](#)

```
lemma fermionicProj_eq_fermionicProjFree (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  fermionicProj ( $\iota a$ ) = fermionicProjFree a
```

lemma bosonicProj_add_fermionicProj[\[source\]](#)

```
lemma bosonicProj_add_fermionicProj (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  bosonicProj a + (fermionicProj a).1 = a
```

lemma bosonicProj_mem_bosonic[\[source\]](#)

```
lemma bosonicProj_mem_bosonic (a :  $\mathcal{F}$ .FieldOpAlgebra) (ha :  $a \in \text{statSubmodule .bosonic}$ ) :
  bosonicProj a =  $\langle a, \text{ha} \rangle$ 
```

lemma fermionicProj_mem_fermionic[\[source\]](#)

```
lemma fermionicProj_mem_fermionic (a :  $\mathcal{F}$ .FieldOpAlgebra) (ha :  $a \in \text{statSubmodule .fermionic}$ ) :
  fermionicProj a =  $\langle a, \text{ha} \rangle$ 
```

lemma bosonicProj_mem_fermionic[\[source\]](#)

```
lemma bosonicProj_mem_fermionic (a :  $\mathcal{F}$ .FieldOpAlgebra) (ha :  $a \in \text{statSubmodule .fermionic}$ ) :
  bosonicProj a = 0
```

lemma fermionicProj_mem_bosonic[\[source\]](#)

```
lemma fermionicProj_mem_bosonic (a :  $\mathcal{F}$ .FieldOpAlgebra) (ha :  $a \in \text{statSubmodule .bosonic}$ ) :
  fermionicProj a = 0
```

lemma mem_bosonic_iff_fermionicProj_eq_zero[\[source\]](#)

```
lemma mem_bosonic_iff_fermionicProj_eq_zero (a :  $\mathcal{F}$ .FieldOpAlgebra) :
   $a \in \text{statSubmodule bosonic} \leftrightarrow \text{fermionicProj } a = 0$ 
```


lemma mem_fermionic_iff_bosonicProj_eq_zero[\[source\]](#)

```
lemma mem_fermionic_iff_bosonicProj_eq_zero (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  a ∈ statSubmodule fermionic ↔ bosonicProj a = 0
```

lemma eq_zero_of_bosonic_and_fermionic[\[source\]](#)

```
lemma eq_zero_of_bosonic_and_fermionic {a :  $\mathcal{F}$ .FieldOpAlgebra}
  (hb : a ∈ statSubmodule bosonic) (hf : a ∈ statSubmodule fermionic) : a = 0
```

lemma bosonicProj_fermionicProj_eq_zero[\[source\]](#)

```
@[simp]
lemma bosonicProj_fermionicProj_eq_zero (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  bosonicProj (fermionicProj a).1 = 0
```

lemma fermionicProj_bosonicProj_eq_zero[\[source\]](#)

```
@[simp]
lemma fermionicProj_bosonicProj_eq_zero (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  fermionicProj (bosonicProj a).1 = 0
```

lemma bosonicProj_bosonicProj_eq_bosonicProj[\[source\]](#)

```
@[simp]
lemma bosonicProj_bosonicProj_eq_bosonicProj (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  bosonicProj (bosonicProj a).1 = bosonicProj a
```

lemma fermionicProj_fermionicProj_eq_fermionicProj[\[source\]](#)

```
@[simp]
lemma fermionicProj_fermionicProj_eq_fermionicProj (a :  $\mathcal{F}$ .FieldOpAlgebra) :
  fermionicProj (fermionicProj a).1 = fermionicProj a
```

lemma bosonicProj_of_bosonic_part[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_bosonic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProj (a bosonic).1 = (a bosonic)
```

lemma bosonicProj_of_fermionic_part[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProj (a fermionic).1 = 0
```

lemma fermionicProj_of_bosonic_part[\[source\]](#)

```
@[simp]
lemma fermionicProj_of_bosonic_part
```

```
(a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
fermionicProj (a bosonic).1 = 0
```

lemma fermionicProj_of_fermionic_part

[\[source\]](#)

@[simp]

```
lemma fermionicProj_of_fermionic_part
(a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
fermionicProj (a fermionic).1 = (a fermionic)
```

lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic

[\[source\]](#)

```
lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic
(a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
DirectSum.coeAddMonoidHom statSubmodule a = a.1 bosonic + a.1 fermionic
```

lemma directSum_eq_bosonic_plus_fermionic

[\[source\]](#)

```
lemma directSum_eq_bosonic_plus_fermionic
(a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
a = (DirectSum.of (fun i => 1(statSubmodule (F := F) i)) bosonic) (a bosonic) +
(DirectSum.of (fun i => 1(statSubmodule (F := F) i)) fermionic) (a fermionic)
```

instance fieldOpAlgebraGrade

[\[source\]](#)

For a field statistic $\mathcal{F}$, the algebra $\mathcal{F}.FieldOpAlgebra$ is graded by $FieldStatistic$. Those of $CrAnList$ ϕs for which ϕs has an overall bosonic statistic (i.e. $\mathcal{F} \mid_s \phi s = \text{bosonic}$) span bosonic submodule, whilst those of $CrAnList$ ϕs for which ϕs has an overall fermionic statistic (i.e. $\mathcal{F} \mid_s \phi s = \text{fermionic}$) span the fermionic submodule.

```
instance fieldOpAlgebraGrade : GradedAlgebra (A := F.FieldOpAlgebra) statSubmodule where
```

2.22 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.Basic

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/NormalOrder/Basic.lean](#) on GitHub.

lemma 1_normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero

[\[source\]](#)

```
lemma 1_normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero
(φa φa' : F.CrAnFieldOp) (φs φs' : List F.CrAnFieldOp) :
1 Nf(ofCrAnListF φs * [ofCrAnOpF φa, ofCrAnOpF φa']sF * ofCrAnListF φs') = 0
```

lemma 1_normalOrderF_superCommuteF_ofCrAnListF_eq_zero

[\[source\]](#)

```
lemma 1_normalOrderF_superCommuteF_ofCrAnListF_eq_zero
(φa φa' : F.CrAnFieldOp) (φs : List F.CrAnFieldOp)
(a : F.FieldOpFreeAlgebra) :
1 Nf(ofCrAnListF φs * [ofCrAnOpF φa, ofCrAnOpF φa']sF * a) = 0
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnOpF_eq_zero_mul}}$ [\[source\]](#)

```
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnOpF\_eq\_zero\_mul}}$  ( $\phi a \ \phi a' : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnOpF } \phi a, \text{ofCrAnOpF } \phi a']_{\mathcal{F}} * b) = 0$ 
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnOpF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnOpF\_ofCrAnListF\_eq\_zero\_mul}}$  ( $\phi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnOpF } \phi a, \text{ofCrAnListF } \phi s]_{\mathcal{F}} * b) = 0$ 
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_ofCrAnOpF_eq_zero_mul}}$ [\[source\]](#)

```
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_ofCrAnOpF\_eq\_zero\_mul}}$  ( $\phi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, \text{ofCrAnOpF } \phi a]_{\mathcal{F}} * b) = 0$ 
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_ofCrAnListF\_eq\_zero\_mul}}$ 
  ( $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, \text{ofCrAnListF } \phi s']_{\mathcal{F}} * b) = 0$ 
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_eq\_zero\_mul}}$ 
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $a \ b \ c : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, c]_{\mathcal{F}} * b) = 0$ 
```

2.23 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.Lemmas

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/NormalOrder/Lemmas.lean](#) on GitHub.

lemma $\text{normalOrder_ofCrAnList}$ [\[source\]](#)

```
lemma  $\text{normalOrder\_ofCrAnList}$  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) :
   $\mathcal{N}(\text{ofCrAnList } \phi s) = \text{normalOrderSign } \phi s \cdot \text{ofCrAnList } (\text{normalOrderList } \phi s)$ 
```

lemma $\text{normalOrder_ofFieldOpList_nil}$ [\[source\]](#)

```
@[simp]
lemma  $\text{normalOrder\_ofFieldOpList\_nil}$  :  $\text{normalOrder } (\mathcal{F} := \mathcal{F}) (\text{ofFieldOpList } []) = 1$ 
```

lemma $\text{normalOrder_ofCrAnList_nil}$ [\[source\]](#)

```
@[simp]
lemma  $\text{normalOrder\_ofCrAnList\_nil}$  :  $\text{normalOrder } (\mathcal{F} := \mathcal{F}) (\text{ofCrAnList } []) = 1$ 
```

lemma ofCrAnList_eq_normalOrder[\[source\]](#)

```
lemma ofCrAnList_eq_normalOrder (φs : List ℱ.CrAnFieldOp) :
  ofCrAnList (normalOrderList φs) = normalOrderSign φs • ℒ(ofCrAnList φs)
```

lemma normalOrder_normalOrder_mid[\[source\]](#)

```
lemma normalOrder_normalOrder_mid (a b c : ℱ.FieldOpAlgebra) :
  ℒ(a * b * c) = ℒ(a * ℒ(b) * c)
```

lemma normalOrder_normalOrder_left[\[source\]](#)

```
lemma normalOrder_normalOrder_left (a b : ℱ.FieldOpAlgebra) :
  ℒ(a * b) = ℒ(ℒ(a) * b)
```

lemma normalOrder_normalOrder_right[\[source\]](#)

```
lemma normalOrder_normalOrder_right (a b : ℱ.FieldOpAlgebra) :
  ℒ(a * b) = ℒ(a * ℒ(b))
```

lemma normalOrder_normalOrder[\[source\]](#)

```
lemma normalOrder_normalOrder (a : ℱ.FieldOpAlgebra) : ℒ(ℒ(a)) = ℒ(a)
```

lemma normalOrder_ofCrAnOp_ofCrAnList[\[source\]](#)

```
lemma normalOrder_ofCrAnOp_ofCrAnList (φ : ℱ.CrAnFieldOp)
  (φs : List ℱ.CrAnFieldOp) : ℒ(ofCrAnOp φ * ofCrAnList φs) =
  S(ℱ |>_s φ, ℱ |>_s φs) • ℒ(ofCrAnList φs * ofCrAnOp φ)
```

lemma normalOrder_ofCrAnOp_ofFieldOpList_swap[\[source\]](#)

```
lemma normalOrder_ofCrAnOp_ofFieldOpList_swap (φ : ℱ.CrAnFieldOp) (φ' : List ℱ.FieldOp) :
  ℒ(ofCrAnOp φ * ofFieldOpList φ') = S(ℱ |>_s φ, ℱ |>_s φ') •
  ℒ(ofFieldOpList φ' * ofCrAnOp φ)
```

lemma ofCrAnOp_superCommute_normalOrder_ofCrAnList_sum[\[source\]](#)

For a field specification $\mathcal{F}$, an element ϕ of $\mathcal{F}.CrAnFieldOp$, a list ϕs of $\mathcal{F}.CrAnFieldOp$, the following relation holds

$$[\phi, \mathcal{L}(\phi_{0...}\phi_n)]_s = \sum i, S(\phi, \phi_{0...}\phi_{i-1}) \bullet [\phi, \phi_i]_s * \mathcal{L}(\phi_{0...}\phi_{i-1}\phi_{i+1...}\phi_n).$$

The proof of this result ultimately goes as follows - The definition of `normalOrder` is used to rewrite $\mathcal{L}(\phi_{0...}\phi_n)$ as a scalar multiple of a `ofCrAnList` ϕsn where ϕsn is the normal ordering of $\phi_{0...}\phi_n$. - `superCommuteF_ofCrAnListF_ofCrAnListF_eq_sum` is used to rewrite the super commutator of ϕ (considered as a list with one element) with `ofCrAnList` ϕsn as a sum of super commutators, one for each element of ϕsn . - The fact that super-commutators are in the center of $\mathcal{F}.FieldOpAlgebra$ is used to rearrange terms. - Properties of ordered lists, and `normalOrderSign_eraseIdx` are then used to complete the proof.

```

lemma ofCrAnOp_superCommute_normalOrder_ofCrAnList_sum (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (φs : List  $\mathcal{F}$ .CrAnFieldOp) : [ofCrAnOp φ,  $\mathcal{N}$ (ofCrAnList φs)]s =  $\sum n : \text{Fin } \phi s.\text{length},$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} (\phi s.\text{take } n)) \cdot [\text{ofCrAnOp } \phi, \text{ofCrAnOp } \phi s[n]]_s$ 
  *  $\mathcal{N}(\text{ofCrAnList } (\phi s.\text{eraseIdx } n))$ 

```

lemma ofCrAnOp_superCommute_normalOrder_ofFieldOpList_sum

[\[source\]](#)

```

lemma ofCrAnOp_superCommute_normalOrder_ofFieldOpList_sum (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (φs : List  $\mathcal{F}$ .FieldOp) :
  [ofCrAnOp φ,  $\mathcal{N}$ (ofFieldOpList φs)]s =  $\sum n : \text{Fin } \phi s.\text{length}, \mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} (\phi s.\text{take } n))$ 
  *
  [ofCrAnOp φ, ofFieldOp φs[n]]s *  $\mathcal{N}(\text{ofFieldOpList } (\phi s.\text{eraseIdx } n))$ 

```

2.24 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.NormalOrder.WickContractions

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/NormalOrder/WickContractions.lean](#) on GitHub.

lemma normalOrder_uncontracted_none

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}$.FieldOp, and a $i \leq \phi s.\text{length}$, then the following relation holds:

$$\mathcal{N}([\phi s \Lambda \Leftarrow \phi \text{ i none}]^{uc}) = s \cdot \mathcal{N}(\phi :: [\phi s \Lambda]^{uc})$$

where s is the exchange sign for ϕ and the uncontracted fields in $\phi_0 \dots \phi_{i-1}$.

The proof of this result ultimately is a consequence of `normalOrder_superCommute_eq_zero`.

```

lemma normalOrder_uncontracted_none (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (i : Fin φs.length.succ) (φsΛ : WickContraction φs.length) :
   $\mathcal{N}(\text{ofFieldOpList } [\phi s \Lambda \Leftarrow \phi \text{ i none}]^{uc})$ 
  =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} (\phi s.\text{get } i, \phi s \Lambda.\text{uncontracted}.\text{filter } (\lambda x \Rightarrow i.\text{succAbove } x < i)))$ 
  *  $\mathcal{N}(\text{ofFieldOpList } (\phi :: [\phi s \Lambda]^{uc}))$ 

```

lemma normalOrder_uncontracted_some

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}$.FieldOp, a $i \leq \phi s.\text{length}$ and a k in $\phi s \Lambda.\text{uncontracted}$, then $\mathcal{N}([\phi s \Lambda \Leftarrow \phi \text{ i (some } k)]^{uc})$ is equal to the normal ordering of $[\phi s \Lambda]^{uc}$ with the $\mathcal{F}$.FieldOp corresponding to k removed.

The proof of this result ultimately is a consequence of definitions.

```

lemma normalOrder_uncontracted_some (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (i : Fin φs.length.succ) (φsΛ : WickContraction φs.length) (k : φsΛ.uncontracted) :
   $\mathcal{N}(\text{ofFieldOpList } [\phi s \Lambda \Leftarrow \phi \text{ i (some } k)]^{uc})$ 
  =  $\mathcal{N}(\text{ofFieldOpList } (\text{optionEraseZ } [\phi s \Lambda]^{uc} \phi ((\text{uncontractedFieldOpEquiv } \phi s \phi s \Lambda) k)))$ 

```

2.25 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.StaticWickTerm

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/StaticWickTerm.lean](#) on GitHub.

def staticWickTerm

[\[source\]](#)

For a list φs of $\mathcal{F}.FieldOp$, and a Wick contraction $\varphi s \Lambda$ of φs , the element of $\mathcal{F}.FieldOpAlgebra$, $\varphi s \Lambda.staticWickTerm$ is defined as

$$\varphi s \Lambda.sign \cdot \varphi s \Lambda.staticContract * \mathcal{N}([\varphi s \Lambda]^{uc}).$$

This is a term which appears in the static version Wick's theorem.

```
def staticWickTerm {φs : List F.FieldOp} (φsΛ : WickContraction φs.length) : F.FieldOpAlgebra
```

lemma staticWickTerm_empty_nil

[\[source\]](#)

For the empty list $[]$ of $\mathcal{F}.FieldOp$, the $staticWickTerm$ of the Wick contraction corresponding to the empty set $\emptyset$ (the only Wick contraction of $[]$) is 1.

@[simp]

lemma staticWickTerm_empty_nil :

staticWickTerm (empty (n := ([] : List F.FieldOp).length)) = 1

lemma staticWickTerm_insert_zero_none

[\[source\]](#)

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\varphi s \Lambda$ of φs , and an element φ of $\mathcal{F}.FieldOp$, then $(\varphi s \Lambda \hookrightarrow \Lambda \varphi \emptyset none).staticWickTerm$ is equal to

$$\varphi s \Lambda.sign \cdot \varphi s \Lambda.staticWickTerm * \mathcal{N}(\varphi :: [\varphi s \Lambda]^{uc})$$

The proof of this result relies on - `staticContract_insert_none` to rewrite the static contract. - `sign_insert_none_zero` to rewrite the sign.

```
lemma staticWickTerm_insert_zero_none (φ : F.FieldOp) (φs : List F.FieldOp)
(φsΛ : WickContraction φs.length) :
(φsΛ ↪ Λ φ ∅ none).staticWickTerm =
φsΛ.sign • φsΛ.staticContract * N(ofFieldOpList (φ :: [φsΛ]^{uc}))
```

lemma staticWickTerm_insert_zero_some

[\[source\]](#)

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\varphi s \Lambda$ of φs , an element φ of $\mathcal{F}.FieldOp$, and a k in $\varphi s \Lambda.uncontracted$, $(\varphi s \Lambda \hookrightarrow \Lambda \varphi \emptyset (some\ k)).wickTerm$ is equal to the product of - the sign $S(\varphi, \varphi_0 \dots \varphi_{i-1})$ - the sign $\varphi s \Lambda.sign$ - $\varphi s \Lambda.staticContract$ - $s \cdot [anPart\ \varphi, ofFieldOp\ \varphi s[k]]_s$ where s is the sign associated with moving φ through uncontracted fields in $\varphi_0 \dots \varphi_{k-1}$ - the normal ordering of $[\varphi s \Lambda]^{uc}$ with the field operator $\varphi s[k]$ removed.

The proof of this result ultimately relies on - `staticContract_insert_some` to rewrite static contractions. - `normalOrder_uncontracted_some` to rewrite normal orderings. - `sign_insert_some_zero` to rewrite signs.

```
lemma staticWickTerm_insert_zero_some (φ : F.FieldOp) (φs : List F.FieldOp)
(φsΛ : WickContraction φs.length) (k : { x // x ∈ φsΛ.uncontracted }) :
(φsΛ ↪ Λ φ ∅ k).staticWickTerm =
sign φs φsΛ • (↑φsΛ.staticContract *
(contractStateAtIndex φ [φsΛ]^{uc} ((uncontractedFieldOpEquiv φs φsΛ) (some k)) *
N(ofFieldOpList (optionEraseZ [φsΛ]^{uc} φ (uncontractedFieldOpEquiv φs φsΛ k))))))
```

lemma mul_staticWickTerm_eq_sum

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , the following relation holds

$$\phi * \phi s \Lambda.staticWickTerm = \sum k, (\phi s \Lambda \leftarrow \Lambda \phi \ 0 \ k).staticWickTerm$$

where the sum is over all k in $Option \ \phi s \Lambda.uncontracted$, so k is either none or some k .

The proof proceeds as follows: - `ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum` is used to expand $\phi \ \mathcal{N}([\phi s \Lambda]^{uc})$ as a sum over k in $Option \ \phi s \Lambda.uncontracted$ of terms involving $[anPart \ \phi, \ \phi s[k]]_s$. - Then `staticWickTerm_insert_zero_none` and `staticWickTerm_insert_zero_some` are used to equate terms.

```
lemma mul_staticWickTerm_eq_sum (ϕ : F.FieldOp) (ϕs : List F.FieldOp)
  (ϕsΛ : WickContraction ϕs.length) :
  ofFieldOp ϕ * ϕsΛ.staticWickTerm =
  ∑ (k : Option ϕsΛ.uncontracted), (ϕsΛ ←Λ ϕ 0 k).staticWickTerm
```

2.26 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.SuperCommute

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/SuperCommute.lean](#) on GitHub.

lemma superCommute_ofCrAnOp_ofFieldOp_diff_stat_zero

[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_diff_stat_zero (ϕ : F.CrAnFieldOp) (ψ : F.FieldOp)
  (h : (F |>_s ϕ) ≠ (F |>_s ψ)) : [ofCrAnOp ϕ, ofFieldOp ψ]_s = 0
```

lemma superCommute_anPart_ofFieldOpF_diff_grade_zero

[\[source\]](#)

```
lemma superCommute_anPart_ofFieldOpF_diff_grade_zero (ϕ ψ : F.FieldOp)
  (h : (F |>_s ϕ) ≠ (F |>_s ψ)) : [anPart ϕ, ofFieldOp ψ]_s = 0
```

lemma superCommute_ofCrAnOp_ofCrAnOp_mem_center

[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofCrAnOp_mem_center (ϕ ϕ' : F.CrAnFieldOp) :
  [ofCrAnOp ϕ, ofCrAnOp ϕ']_s ∈ Subalgebra.center C (FieldOpAlgebra F)
```

lemma superCommute_ofCrAnOp_ofCrAnOp_commute

[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofCrAnOp_commute (ϕ ϕ' : F.CrAnFieldOp)
  (a : FieldOpAlgebra F) :
  a * [ofCrAnOp ϕ, ofCrAnOp ϕ']_s = [ofCrAnOp ϕ, ofCrAnOp ϕ']_s * a
```

lemma superCommute_ofCrAnOp_ofFieldOp_mem_center

[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_mem_center (ϕ : F.CrAnFieldOp) (ϕ' : F.FieldOp) :
  [ofCrAnOp ϕ, ofFieldOp ϕ']_s ∈ Subalgebra.center C (FieldOpAlgebra F)
```

lemma superCommute_ofCrAnOp_ofFieldOp_commute

[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_commute (ϕ : F.CrAnFieldOp) (ϕ' : F.FieldOp)
  (a : FieldOpAlgebra F) : a * [ofCrAnOp ϕ, ofFieldOp ϕ']_s =
  [ofCrAnOp ϕ, ofFieldOp ϕ']_s * a
```

lemma superCommute_ofCrAnList_ofCrAnList[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofCrAnList (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
  ofCrAnList (φs ++ φs') -  $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$  • ofCrAnList (φs' ++ φs)

```

lemma superCommute_ofCrAnOp_ofCrAnOp[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofCrAnOp (φ φ' :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOp φ, ofCrAnOp φ']s = ofCrAnOp φ * ofCrAnOp φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • ofCrAnOp φ' * ofCrAnOp φ

```

lemma superCommute_ofCrAnList_ofFieldOpList[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofFieldOpList (φcas : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs : List  $\mathcal{F}$ .FieldOp) :
  [ofCrAnList φcas, ofFieldOpList φs]s = ofCrAnList φcas * ofFieldOpList φs -
   $\mathcal{S}(\mathcal{F} \mid>_s \phi cas, \mathcal{F} \mid>_s \phi s)$  • ofFieldOpList φs * ofCrAnList φcas

```

lemma ofCrAnList_mul_ofCrAnList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnList_mul_ofCrAnList_eq_superCommute (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList φs * ofCrAnList φs' =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$  • ofCrAnList φs' * ofCrAnList φs
  + [ofCrAnList φs, ofCrAnList φs']s

```

lemma ofCrAnOp_mul_ofCrAnList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnOp_mul_ofCrAnList_eq_superCommute (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .CrAnFieldOp) : ofCrAnOp φ * ofCrAnList φs' =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s')$  • ofCrAnList φs' * ofCrAnOp φ
  + [ofCrAnOp φ, ofCrAnList φs']s

```

lemma ofCrAnList_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnList_mul_ofFieldOpList_eq_superCommute (φs : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .FieldOp) : ofCrAnList φs * ofFieldOpList φs' =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$  • ofFieldOpList φs' * ofCrAnList φs
  + [ofCrAnList φs, ofFieldOpList φs']s

```

lemma superCommute_ofCrAnList_ofCrAnList_symm[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofCrAnList_symm (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
  (-  $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$ ) • [ofCrAnList φs', ofCrAnList φs]s

```

lemma superCommute_ofCrAnOp_ofCrAnOp_symm[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofCrAnOp_symm (φ φ' :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOp φ, ofCrAnOp φ']s =
  (-  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$ ) • [ofCrAnOp φ', ofCrAnOp φ]s

```


lemma superCommute_ofCrAnList_ofCrAnList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofCrAnList_eq_sum (φs φs' : List ℱ.CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
  ∑ (n : Fin φs'.length), S(ℱ |>s φs, ℱ |>s φs'.take n) •
  ofCrAnList (φs'.take n) * [ofCrAnList φs, ofCrAnOp (φs'.get n)]s *
  ofCrAnList (φs'.drop (n + 1))

```

lemma superCommute_ofCrAnOp_ofCrAnList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofCrAnList_eq_sum (φ : ℱ.CrAnFieldOp)
  (φs' : List ℱ.CrAnFieldOp) : [ofCrAnOp φ, ofCrAnList φs']s =
  ∑ (n : Fin φs'.length), S(ℱ |>s φ, ℱ |>s φs'.take n) •
  [ofCrAnOp φ, ofCrAnOp (φs'.get n)]s * ofCrAnList (φs'.eraseIdx n)

```

lemma superCommute_ofCrAnList_ofFieldOpList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofFieldOpList_eq_sum (φs : List ℱ.CrAnFieldOp)
  (φs' : List ℱ.FieldOp) : [ofCrAnList φs, ofFieldOpList φs']s =
  ∑ (n : Fin φs'.length), S(ℱ |>s φs, ℱ |>s φs'.take n) •
  ofFieldOpList (φs'.take n) * [ofCrAnList φs, ofFieldOp (φs'.get n)]s *
  ofFieldOpList (φs'.drop (n + 1))

```

lemma superCommute_ofCrAnOp_ofFieldOpList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofFieldOpList_eq_sum (φ : ℱ.CrAnFieldOp) (φs' : List ℱ.FieldOp) :
  [ofCrAnOp φ, ofFieldOpList φs']s =
  ∑ (n : Fin φs'.length), S(ℱ |>s φ, ℱ |>s φs'.take n) •
  [ofCrAnOp φ, ofFieldOp (φs'.get n)]s * ofFieldOpList (φs'.eraseIdx n)

```

2.27 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.TimeContraction

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/TimeContraction.lean](#) on GitHub.

lemma timeContract_of_not_timeOrderRel_expand[\[source\]](#)

For a field specification $\mathcal{F}$, and ϕ and ψ elements of $\mathcal{F}.FieldOp$, if ϕ and ψ are not time-ordered then

$$timeContract \phi \psi = S(\mathcal{F} |>_s \phi, \mathcal{F} |>_s \psi) \cdot [anPart \psi, ofFieldOp \phi]_s.$$

```

lemma timeContract_of_not_timeOrderRel_expand (φ ψ : ℱ.FieldOp) (h : ¬ timeOrderRel φ ψ) :
  timeContract φ ψ = S(ℱ |>s φ, ℱ |>s ψ) • [anPart ψ, ofFieldOp φ]s

```

lemma normalOrder_timeContract[\[source\]](#)

```

lemma normalOrder_timeContract (φ ψ : ℱ.FieldOp) :
  ℒ(timeContract φ ψ) = 0

```

lemma timeOrder_timeContract_eq_time_mid[\[source\]](#)

```

lemma timeOrder_timeContract_eq_time_mid {φ ψ : ℱ.FieldOp}
  (h1 : timeOrderRel φ ψ) (h2 : timeOrderRel ψ φ) (a b : ℱ.FieldOpAlgebra) :
  ℒ(a * timeContract φ ψ * b) = timeContract φ ψ * ℒ(a * b)

```

lemma timeOrder_timeContract_eq_time_left[\[source\]](#)

```

lemma timeOrder_timeContract_eq_time_left { $\phi \psi : \mathcal{F}.FieldOp$ }
  (h1 : timeOrderRel  $\phi \psi$ ) (h2 : timeOrderRel  $\psi \phi$ ) (b :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(\text{timeContract } \phi \psi * b) = \text{timeContract } \phi \psi * \mathcal{T}(b)$ 

```

lemma timeOrder_timeContract_neq_time[\[source\]](#)

```

lemma timeOrder_timeContract_neq_time { $\phi \psi : \mathcal{F}.FieldOp$ }
  (h1 :  $\neg (\text{timeOrderRel } \phi \psi \wedge \text{timeOrderRel } \psi \phi)$ ) :
   $\mathcal{T}(\text{timeContract } \phi \psi) = 0$ 

```

2.28 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.TimeOrder

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/TimeOrder.lean](#) on GitHub.

lemma ι _timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnListF[\[source\]](#)

```

lemma  $\iota$ _timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnListF { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.CrAnFieldOp$ }
  ( $\phi s_1 \phi s_2 : \text{List } \mathcal{F}.CrAnFieldOp$ ) (h :
    crAnTimeOrderRel  $\phi_1 \phi_2 \wedge \text{crAnTimeOrderRel } \phi_1 \phi_3 \wedge$ 
    crAnTimeOrderRel  $\phi_2 \phi_1 \wedge \text{crAnTimeOrderRel } \phi_2 \phi_3 \wedge$ 
    crAnTimeOrderRel  $\phi_3 \phi_1 \wedge \text{crAnTimeOrderRel } \phi_3 \phi_2$ ) :
   $\iota \mathcal{T}^f(\text{ofCrAnListF } \phi s_1 * [\text{ofCrAnOpF } \phi_1, [\text{ofCrAnOpF } \phi_2, \text{ofCrAnOpF } \phi_3]_s]_s)_s \mathcal{F} * \text{ofCrAnListF } \phi s_2) = 0$ 

```

lemma ι _timeOrderF_superCommuteF_superCommuteF_ofCrAnListF[\[source\]](#)

```

lemma  $\iota$ _timeOrderF_superCommuteF_superCommuteF_ofCrAnListF { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.CrAnFieldOp$ }
  ( $\phi s_1 \phi s_2 : \text{List } \mathcal{F}.CrAnFieldOp$ ) :
   $\iota \mathcal{T}^f(\text{ofCrAnListF } \phi s_1 * [\text{ofCrAnOpF } \phi_1, [\text{ofCrAnOpF } \phi_2, \text{ofCrAnOpF } \phi_3]_s]_s \mathcal{F} * \text{ofCrAnListF } \phi s_2) = 0$ 

```

lemma timeOrder_superCommute_eq_time_mid[\[source\]](#)

```

lemma timeOrder_superCommute_eq_time_mid { $\phi \psi : \mathcal{F}.CrAnFieldOp$ }
  (h $\phi\psi$  : crAnTimeOrderRel  $\phi \psi$ ) (h $\psi\phi$  : crAnTimeOrderRel  $\psi \phi$ ) (a b :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(a * [\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s * b) =$ 
   $[\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s * \mathcal{T}(a * b)$ 

```

lemma timeOrder_superCommute_eq_time_left[\[source\]](#)

```

lemma timeOrder_superCommute_eq_time_left { $\phi \psi : \mathcal{F}.CrAnFieldOp$ }
  (h $\phi\psi$  : crAnTimeOrderRel  $\phi \psi$ ) (h $\psi\phi$  : crAnTimeOrderRel  $\psi \phi$ ) (b :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}([\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s * b) =$ 
   $[\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s * \mathcal{T}(b)$ 

```

lemma timeOrder_superCommute_neq_time[\[source\]](#)

```

lemma timeOrder_superCommute_neq_time { $\phi \psi : \mathcal{F}.CrAnFieldOp$ }
  (h $\phi\psi$  :  $\neg (\text{crAnTimeOrderRel } \phi \psi \wedge \text{crAnTimeOrderRel } \psi \phi)$ ) :
   $\mathcal{T}([\text{ofCrAnOp } \phi, \text{ofCrAnOp } \psi]_s) = 0$ 

```

lemma timeOrder_superCommute_anPart_ofFieldOp_neq_time[\[source\]](#)

```
lemma timeOrder_superCommute_anPart_ofFieldOp_neq_time { $\phi \psi : \mathcal{F}.FieldOp$ }
  (h $\phi\psi : \neg (\text{timeOrderRel } \phi \psi \wedge \text{timeOrderRel } \psi \phi) :$ 
   $\mathcal{T}([\text{anPart } \phi, \text{ofFieldOp } \psi]_s) = 0$ 
```

lemma timeOrder_timeOrder_mid[\[source\]](#)

*For a field specification $\mathcal{F}$, and a, b, c in $\mathcal{F}.FieldOpAlgebra$, then $\mathcal{T}(a * b * c) = \mathcal{T}(a * \mathcal{T}(b) * c)$.*

```
lemma timeOrder_timeOrder_mid (a b c :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(a * b * c) = \mathcal{T}(a * \mathcal{T}(b) * c)$ 
```

lemma timeOrder_timeOrder_left[\[source\]](#)

```
lemma timeOrder_timeOrder_left (b c :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(b * c) = \mathcal{T}(\mathcal{T}(b) * c)$ 
```

lemma timeOrder_timeOrder_right[\[source\]](#)

```
lemma timeOrder_timeOrder_right (a b :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(a * b) = \mathcal{T}(a * \mathcal{T}(b))$ 
```

lemma timeOrder_timeOrder[\[source\]](#)

Time ordering is a projection.

```
lemma timeOrder_timeOrder (a :  $\mathcal{F}.FieldOpAlgebra$ ) :
   $\mathcal{T}(\mathcal{T}(a)) = \mathcal{T}(a)$ 
```

2.29 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.Universality

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/Universality.lean](#) on GitHub.

def universalLiftMap[\[source\]](#)

For a field specification, $\mathcal{F}$, given an algebra A and a function $f : \mathcal{F}.CrAnFieldOp \rightarrow A$ such that the lift of f to $FreeAlgebra.lift \mathbb{C} f : FreeAlgebra \mathbb{C} \mathcal{F}.CrAnFieldOp \rightarrow A$ is zero on the ideal defining $\mathcal{F}.FieldOpAlgebra$, the corresponding map $\mathcal{F}.FieldOpAlgebra \rightarrow A$.

```
def universalLiftMap {A : Type} [Semiring A] [Algebra  $\mathbb{C} A$ ] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in \text{TwoSidedIdeal.span } \mathcal{F}.fieldOpIdealSet, \text{FreeAlgebra.lift } \mathbb{C} f a = 0$ ) :
   $\text{FieldOpAlgebra } \mathcal{F} \rightarrow A$ 
```

lemma universalLiftMap_1[\[source\]](#)

```
@[simp]
lemma universalLiftMap_1 {A : Type} [Semiring A] [Algebra  $\mathbb{C} A$ ] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in \text{TwoSidedIdeal.span } \mathcal{F}.fieldOpIdealSet, \text{FreeAlgebra.lift } \mathbb{C} f a = 0$ ) :
   $\text{universalLiftMap } f \text{ h1 } (1 a) = \text{FreeAlgebra.lift } \mathbb{C} f a$ 
```

def universalLift[\[source\]](#)

For a field specification, $\mathcal{F}$, given an algebra A and a function $f : \mathcal{F}.CrAnFieldOp \rightarrow A$ such that the lift of f to $FreeAlgebra.lift \mathbb{C} f : FreeAlgebra \mathbb{C} \mathcal{F}.CrAnFieldOp \rightarrow A$ is zero on the ideal defining $\mathcal{F}.FieldOpAlgebra$, the corresponding algebra map $\mathcal{F}.FieldOpAlgebra \rightarrow A$.

```
def universalLift {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
(h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
FieldOpAlgebra  $\mathcal{F} \rightarrow_a[\mathbb{C}] A$  where
```

lemma universalLift_ι[\[source\]](#)

```
@[simp]
lemma universalLift_ι {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
(h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
universalLift f h1 (ι a) = FreeAlgebra.lift  $\mathbb{C}$  f a
```

lemma universality[\[source\]](#)

For a field specification, $\mathcal{F}$, the algebra $\mathcal{F}.FieldOpAlgebra$ satisfies the following universal property. Let $f : \mathcal{F}.CrAnFieldOp \rightarrow A$ be a function and $g : \mathcal{F}.FieldOpFreeAlgebra \rightarrow_a[\mathbb{C}] A$ the universal lift of that function associated with the free algebra $\mathcal{F}.FieldOpFreeAlgebra$. If g is zero on the ideal defining $\mathcal{F}.FieldOpAlgebra$, then there exists algebra map $g' : FieldOpAlgebra \mathcal{F} \rightarrow_a[\mathbb{C}] A$ such that $g' \circ \iota = g$, and furthermore this algebra map is unique.

```
lemma universality {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
(h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
 $\exists! g : FieldOpAlgebra \mathcal{F} \rightarrow_a[\mathbb{C}] A, g \circ \iota = FreeAlgebra.lift \mathbb{C} f$ 
```

2.30 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.WicksTheoremNormal

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/WicksTheoremNormal.lean](#) on GitHub.

lemma timeOrder_ofFieldOpList_eqTimeOnly[\[source\]](#)

For a list ϕs of $\mathcal{F}.FieldOp$, then

$$\mathcal{T}(\phi s) = \sum \phi s \Lambda, \phi s \Lambda.sign \cdot \phi s \Lambda.timeContract * \mathcal{T}(\mathcal{N}([\phi s \Lambda]^{uc}))$$

where the sum is over all Wick contraction $\phi s \Lambda$ which only have equal time contractions.

This result follows from - `static_wick_theorem` to rewrite $\mathcal{T}(\phi s)$ on the left hand side as a sum of $\mathcal{T}(\phi s \Lambda.staticWickTerm)$. - `EqTimeOnly.timeOrder_staticContract_of_not_mem` and `timeOrder_timeOrder_mid` to set to those $\mathcal{T}(\phi s \Lambda.staticWickTerm)$ for which $\phi s \Lambda$ has a contracted pair which are not equal time to zero. - `staticContract_eq_timeContract_of_eqTimeOnly` to rewrite the static contract in the remaining $\mathcal{T}(\phi s \Lambda.staticWickTerm)$ as a time contract. - `timeOrder_timeContract_mul_of_eqTimeOnly_left` to move the time contracts out of the time ordering.

```
lemma timeOrder_ofFieldOpList_eqTimeOnly (phi s : List  $\mathcal{F}.FieldOp$ ) :
 $\mathcal{T}(ofFieldOpList \phi s) = \sum (\phi s \Lambda : \{\phi s \Lambda // \phi s \Lambda.EqTimeOnly (\phi s := \phi s)\}),$ 
 $\phi s \Lambda.1.sign \cdot \phi s \Lambda.1.timeContract.1 * \mathcal{T}(\mathcal{N}(ofFieldOpList [\phi s \Lambda.1]^{uc}))$ 
```

lemma timeOrder_ofFieldOpList_eq_eqTimeOnly_empty[\[source\]](#)

```

lemma timeOrder_ofFieldOpList_eq_eqTimeOnly_empty (φs : List ℱ.FieldOp) :
  T(ofFieldOpList φs) = T(N(ofFieldOpList φs)) +
  ∑ (φsΛ : {φsΛ // φsΛ.EqTimeOnly (φs := φs) ∧ φsΛ ≠ empty}),
  φsΛ.1.sign • φsΛ.1.timeContract.1 * T(N(ofFieldOpList [φsΛ.1]uc))

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty[\[source\]](#)

For a list ϕs of $\mathcal{F}.FieldOp$, then

$$T(N(\phi s)) = T(\phi s) - \sum \phi s\Lambda, \phi s\Lambda.sign \cdot \phi s\Lambda.timeContract.1 * T(N([\phi s\Lambda]^{uc}))$$

where the sum is over all **non-empty** Wick contraction $\phi s\Lambda$ which only have equal time contractions.

This result follows directly from - timeOrder_ofFieldOpList_eqTimeOnly

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty (φs : List ℱ.FieldOp) :
  T(N(ofFieldOpList φs)) = T(ofFieldOpList φs) -
  ∑ (φsΛ : {φsΛ // φsΛ.EqTimeOnly (φs := φs) ∧ φsΛ ≠ empty}),
  φsΛ.1.sign • φsΛ.1.timeContract.1 * T(N(ofFieldOpList [φsΛ.1]uc))

```

lemma timeOrder_haveEqTime_split[\[source\]](#)

For a list ϕs of $\mathcal{F}.FieldOp$, then $T(\phi s)$ is equal to the sum of

- $\sum \phi s\Lambda, \phi s\Lambda.wickTerm$ where the sum is over all Wick contraction $\phi s\Lambda$ which have no contractions of equal time. - $\sum \phi s\Lambda, \phi s\Lambda.sign \cdot \phi s\Lambda.timeContract * (\sum \phi ssuc\Lambda, \phi ssuc\Lambda.wickTerm)$, where the first sum is over all Wick contraction $\phi s\Lambda$ which only have equal time contractions and the second sum is over all Wick contraction $\phi ssuc\Lambda$ of the uncontracted elements of $\phi s\Lambda$ which do not have any equal time contractions.

The proof proceeds as follows - wicks_theorem is used to rewrite $T(\phi s)$ as a sum over all Wick contractions. - The sum over all Wick contractions is then split additively into two parts based on having or not having an equal time contractions. - Using join, the sum $\sum \phi s\Lambda, _$ over Wick contractions which do have equal time contractions is split into two sums $\sum \phi s\Lambda, \sum \phi suc\Lambda, _$, the first over non-zero elements which only have equal time contractions and the second over Wick contractions $\phi suc\Lambda$ of $[\phi s\Lambda]^{uc}$ which do not have equal time contractions. - join_sign_timeContract is then used to equate terms.

```

lemma timeOrder_haveEqTime_split (φs : List ℱ.FieldOp) :
  T(ofFieldOpList φs) = (∑ (φsΛ : {φsΛ : WickContraction φs.length // ¬ HaveEqTime φsΛ}),
  φsΛ.1.sign • φsΛ.1.timeContract.1 * N(ofFieldOpList [φsΛ.1]uc))
+ ∑ (φsΛ : {φsΛ // φsΛ.EqTimeOnly (φs := φs) ∧ φsΛ ≠ empty}), φsΛ.1.sign • φsΛ.1.
timeContract *
(∑ φssucΛ : { φssucΛ : WickContraction [φsΛ.1]uc.length // ¬ φssucΛ.HaveEqTime },
φssucΛ.1.wickTerm)

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive[\[source\]](#)

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive (φs : List ℱ.FieldOp) :
  T(N(ofFieldOpList φs)) =
  (∑ (φsΛ : {φsΛ : WickContraction φs.length // ¬ HaveEqTime φsΛ}), φsΛ.1.wickTerm)
+ ∑ (φsΛ : {φsΛ // φsΛ.EqTimeOnly (φs := φs) ∧ φsΛ ≠ empty}),

```

```
sign  $\varphi s \uparrow \varphi s \Lambda \bullet (\varphi s \Lambda.1).timeContract *$ 
 $(\sum \varphi ssuc \Lambda : \{ \varphi ssuc \Lambda : WickContraction [\varphi s \Lambda.1]^{uc}.length // \neg \varphi ssuc \Lambda.HaveEqTime \},$ 
 $\varphi ssuc \Lambda.1.wickTerm - \mathcal{T}(\mathcal{N}(ofFieldOpList [\varphi s \Lambda.1]^{uc})))$ 
```

lemma wicks_theorem_normal_order_empty[\[source\]](#)

```
lemma wicks_theorem_normal_order_empty :  $\mathcal{T}(\mathcal{N}(ofFieldOpList [])) =$ 
 $\sum (\varphi s \Lambda : \{ \varphi s \Lambda : WickContraction ([] : List \mathcal{F}.FieldOp).length // \neg HaveEqTime \varphi s \Lambda \}),$ 
 $\varphi s \Lambda.1.wickTerm$ 
```

theorem wicks_theorem_normal_order[\[source\]](#)

For a list φs of $\mathcal{F}.FieldOp$, the normal-ordered version of Wick's theorem states that

$$\mathcal{T}(\mathcal{N}(\varphi s)) = \sum \varphi s \Lambda, \varphi s \Lambda.wickTerm$$

where the sum is over all Wick contraction $\varphi s \Lambda$ in which no two contracted elements have the same time.

The proof proceeds by induction on φs , with the base case $[]$ holding by following through definitions. and the inductive case holding as a result of - `timeOrder_haveEqTime_split` - `normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty` - and the induction hypothesis on $\mathcal{T}(\mathcal{N}([\varphi s \Lambda.1]^{uc}))$ for contractions $\varphi s \Lambda$ of φs which only have equal time contractions and are non-empty.

```
theorem wicks_theorem_normal_order : ( $\varphi s : List \mathcal{F}.FieldOp$ )  $\rightarrow$ 
 $\mathcal{T}(\mathcal{N}(ofFieldOpList \varphi s)) =$ 
 $\sum (\varphi s \Lambda : \{ \varphi s \Lambda : WickContraction \varphi s.length // \neg HaveEqTime \varphi s \Lambda \}), \varphi s \Lambda.1.wickTerm$ 
| [] => wicks_theorem_normal_order_empty
|  $\varphi :: \varphi s$  => by
  rw [normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive]
  simp only [Algebra.smul_mul_assoc, ne_eq, add_right_eq_self]
  apply Finset.sum_eq_zero
  intro  $\varphi s \Lambda$   $h\varphi s \Lambda$ 
  simp only [smul_eq_zero]
  right
  have ih
```

2.31 PhysLean.QFT.PerturbationTheory.FieldOpAlgebra.WickTerm

[PhysLean/QFT/PerturbationTheory/FieldOpAlgebra/WickTerm.lean](#) on GitHub.

def wickTerm[\[source\]](#)

For a list φs of $\mathcal{F}.FieldOp$, and a Wick contraction $\varphi s \Lambda$ of φs , the element of $\mathcal{F}.FieldOpAlgebra$, $\varphi s \Lambda.wickTerm$ is defined as

$$\varphi s \Lambda.sign \bullet \varphi s \Lambda.timeContract * \mathcal{N}([\varphi s \Lambda]^{uc}).$$

This is a term which appears in the Wick's theorem.

```
def wickTerm { $\varphi s : List \mathcal{F}.FieldOp$ } ( $\varphi s \Lambda : WickContraction \varphi s.length$ ) :  $\mathcal{F}.FieldOpAlgebra$ 
```

lemma wickTerm_empty_nil[\[source\]](#)

For the empty list $[]$ of $\mathcal{F}.FieldOp$, the `wickTerm` of the Wick contraction corresponding to the empty set $\emptyset$ (the only Wick contraction of $[]$) is 1.

@[simp]

lemma wickTerm_empty_nil :

wickTerm (empty (n := ([] : List $\mathcal{F}.FieldOp$)).length)) = 1

lemma wickTerm_insert_none

[source]

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\varphi s \Lambda$ of φs , an element φ of $\mathcal{F}.FieldOp$, and $i \leq \varphi s.length$, then $(\varphi s \Lambda \dashv \Lambda \varphi \ i \ none).wickTerm$ is equal to

$$S(\varphi, \varphi_0 \dots \varphi_{i-1}) \varphi s \Lambda.sign \cdot \varphi s \Lambda.timeContract * \mathcal{N}(\varphi :: [\varphi s \Lambda]^{uc})$$

The proof of this result relies on - `normalOrder_uncontracted_none` to rewrite normal orderings. - `timeContract_insert_none` to rewrite the time contract. - `sign_insert_none` to rewrite the sign.

lemma wickTerm_insert_none ($\varphi : \mathcal{F}.FieldOp$) ($\varphi s : List \mathcal{F}.FieldOp$)

($i : Fin \varphi s.length.succ$) ($\varphi s \Lambda : WickContraction \varphi s.length$) :

$(\varphi s \Lambda \dashv \Lambda \varphi \ i \ none).wickTerm =$

$S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} (\varphi s.get, (Finset.univ.filter (\text{fun } k \Rightarrow i.succAbove \ k < i))))$

$\cdot (\varphi s \Lambda.sign \cdot \varphi s \Lambda.timeContract * \mathcal{N}(\text{ofFieldOpList } (\varphi :: [\varphi s \Lambda]^{uc})))$

lemma wickTerm_insert_some

[source]

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\varphi s \Lambda$ of φs , an element φ of $\mathcal{F}.FieldOp$, $i \leq \varphi s.length$ and a k in $\varphi s \Lambda.uncontracted$, such that all $\mathcal{F}.FieldOp$ in $\varphi_0 \dots \varphi_{i-1}$ have time strictly less than φ and φ has a time greater than or equal to all $\mathcal{F}.FieldOp$ in $\varphi_0 \dots \varphi_n$, then $(\varphi s \Lambda \dashv \Lambda \varphi \ i \ (some \ k)).staticWickTerm$ is equal to the product of - the sign $S(\varphi, \varphi_0 \dots \varphi_{i-1})$ - the sign $\varphi s \Lambda.sign$ - $\varphi s \Lambda.timeContract$ - $s \cdot [anPart \ \varphi, \text{ofFieldOp } \varphi s[k]]_s$ where s is the sign associated with moving φ through uncontracted fields in $\varphi_0 \dots \varphi_{k-1}$ - the normal ordering $[\varphi s \Lambda]^{uc}$ with the field corresponding to k removed.

The proof of this result relies on - `timeContract_insert_some_of_not_lt` and `timeContract_insert_some_of` to rewrite time contractions. - `normalOrder_uncontracted_some` to rewrite normal orderings. - `sign_insert_some_of_not_lt` and `sign_insert_some_of_lt` to rewrite signs.

lemma wickTerm_insert_some ($\varphi : \mathcal{F}.FieldOp$) ($\varphi s : List \mathcal{F}.FieldOp$)

($i : Fin \varphi s.length.succ$) ($\varphi s \Lambda : WickContraction \varphi s.length$) ($k : \varphi s \Lambda.uncontracted$)

($hlt : \forall (k : Fin \varphi s.length), \text{timeOrderRel } \varphi \ \varphi s[k]$)

($hn : \forall (k : Fin \varphi s.length), i.succAbove \ k < i \rightarrow \neg \text{timeOrderRel } \varphi s[k] \ \varphi$) :

$(\varphi s \Lambda \dashv \Lambda \varphi \ i \ (some \ k)).wickTerm =$

$S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} (\varphi s.get, (Finset.univ.filter (\text{fun } x \Rightarrow i.succAbove \ x < i))))$

$\cdot (\varphi s \Lambda.sign \cdot (\text{contractStateAtIndex } \varphi \ [\varphi s \Lambda]^{uc}$

$((\text{uncontractedFieldOpEquiv } \varphi s \ \varphi s \Lambda) \ (some \ k)) * \varphi s \Lambda.timeContract)$

$* \mathcal{N}(\text{ofFieldOpList } (\text{optionEraseZ } [\varphi s \Lambda]^{uc} \ \varphi \ (\text{uncontractedFieldOpEquiv } \varphi s \ \varphi s \Lambda \ k))))$

lemma mul_wickTerm_eq_sum

[source]

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\varphi s \Lambda$ of φs , an element φ of $\mathcal{F}.FieldOp$, and $i \leq \varphi s.length$ such that all $\mathcal{F}.FieldOp$ in $\varphi_0 \dots \varphi_{i-1}$ have time strictly less than φ and φ has a time greater than or equal to all $\mathcal{F}.FieldOp$ in $\varphi_0 \dots \varphi_n$, then

$$\varphi * \varphi s \Lambda.wickTerm = S(\varphi, \varphi_0 \dots \varphi_{i-1}) \cdot \sum k, (\varphi s \Lambda \dashv \Lambda \varphi \ i \ k).wickTerm$$

where the sum is over all k in $\text{Option } \varphi\Lambda.\text{uncontracted}$, so k is either none or some k .

The proof proceeds as follows: - `ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum` is used to expand $\varphi \mathcal{N}([\varphi\Lambda]^{uc})$ as a sum over k in $\text{Option } \varphi\Lambda.\text{uncontracted}$ of terms involving $[\text{anPart } \varphi, \varphi[k]]_s$. - Then `wickTerm_insert_none` and `wickTerm_insert_some` are used to equate terms.

```
lemma mul_wickTerm_eq_sum (ϕ : F.FieldOp) (ϕs : List F.FieldOp) (i : Fin ϕs.length.succ)
(ϕsΛ : WickContraction ϕs.length) (hlt : ∀ (k : Fin ϕs.length), timeOrderRel ϕ ϕs[k])
(hn : ∀ (k : Fin ϕs.length), i.succAbove k < i → ¬timeOrderRel ϕs[k] ϕ) :
ofFieldOp ϕ * ϕsΛ.wickTerm =
S(F |>_s ϕ, F |>_s (ϕs.get, (Finset.univ.filter (fun x => i.succAbove x < i)))) •
∑ (k : Option ϕsΛ.uncontracted), (ϕsΛ ↦Λ ϕ i k).wickTerm
```

2.32 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean](#) on GitHub.

abbrev FieldOpFreeAlgebra

[\[source\]](#)

For a field specification $\mathcal{F}$, the algebra $\mathcal{F}.\text{FieldOpFreeAlgebra}$ is the free algebra generated by $\mathcal{F}.\text{CrAnFieldOp}$.

```
abbrev FieldOpFreeAlgebra (F : FieldSpecification) : Type
```

def ofCrAnOpF

[\[source\]](#)

For a field specification $\mathcal{F}$, and a element φ of $\mathcal{F}.\text{CrAnFieldOp}$, `ofCrAnOpF ϕ` is defined as the element of $\mathcal{F}.\text{FieldOpFreeAlgebra}$ formed by φ .

```
def ofCrAnOpF (ϕ : F.CrAnFieldOp) : FieldOpFreeAlgebra F
```

lemma universality

[\[source\]](#)

The algebra $\mathcal{F}.\text{FieldOpFreeAlgebra}$ satisfies the universal property that for any other algebra A (e.g. the operator algebra of the theory) with a map $f : \mathcal{F}.\text{CrAnFieldOp} \rightarrow A$ (e.g. the inclusion of the creation and annihilation parts of field operators into the operator algebra) there is a unique algebra map $g : \mathcal{F}.\text{FieldOpFreeAlgebra} \rightarrow A$ such that $g \circ \text{ofCrAnOpF} = f$.

The unique g is given by `FreeAlgebra.lift C f`.

```
lemma universality {A : Type} [Semiring A] [Algebra C A] (f : F.CrAnFieldOp → A) :
∃! g : FieldOpFreeAlgebra F →*[C] A, g ∘ ofCrAnOpF = f
```

def ofCrAnListF

[\[source\]](#)

For a field specification $\mathcal{F}$, and a list φs of $\mathcal{F}.\text{CrAnFieldOp}$, `ofCrAnListF ϕs` is defined as the element of $\mathcal{F}.\text{FieldOpFreeAlgebra}$ obtained by the product of `ofCrAnListF ϕ` for each φ in φs . For example `ofCrAnListF [ϕ1, ϕ2, ϕ3] = ofCrAnOpF ϕ1 * ofCrAnOpF ϕ2 * ofCrAnOpF ϕ3`. The set of all `ofCrAnListF ϕs` forms a basis of $\text{FieldOpFreeAlgebra } \mathcal{F}$.

```
def ofCrAnListF (ϕs : List F.CrAnFieldOp) : FieldOpFreeAlgebra F
```


lemma ofCrAnListF_nil[\[source\]](#)

@[simp]

lemma ofCrAnListF_nil : ofCrAnListF ([]) : List $\mathcal{F}$.CrAnFieldOp = 1**lemma ofCrAnListF_cons**[\[source\]](#)**lemma** ofCrAnListF_cons (ϕ : $\mathcal{F}$.CrAnFieldOp) (ϕ s : List $\mathcal{F}$.CrAnFieldOp) :
ofCrAnListF (ϕ :: ϕ s) = ofCrAnOpF ϕ * ofCrAnListF ϕ s**lemma ofCrAnListF_append**[\[source\]](#)**lemma** ofCrAnListF_append (ϕ s ϕ s' : List $\mathcal{F}$.CrAnFieldOp) :
ofCrAnListF (ϕ s ++ ϕ s') = ofCrAnListF ϕ s * ofCrAnListF ϕ s'**lemma ofCrAnListF_singleton**[\[source\]](#)**lemma** ofCrAnListF_singleton (ϕ : $\mathcal{F}$.CrAnFieldOp) :
ofCrAnListF [ϕ] = ofCrAnOpF ϕ **def ofFieldOpF**[\[source\]](#)

For a field specification $\mathcal{F}$, and an element ϕ of $\mathcal{F}$.FieldOp, ofFieldOpF ϕ is the element of $\mathcal{F}$.FieldOpFreeAlgebra formed by summing over ofCrAnOpF of the creation and annihilation parts of ϕ .

For example, for ϕ an incoming asymptotic field operator we get ofCrAnOpF (ϕ , ()), and for ϕ a position field operator we get ofCrAnOpF (ϕ , .create) + ofCrAnOpF (ϕ , .annihilate).

def ofFieldOpF (ϕ : $\mathcal{F}$.FieldOp) : FieldOpFreeAlgebra $\mathcal{F}$ **def ofFieldOpListF**[\[source\]](#)

For a field specification $\mathcal{F}$, and a list ϕ s of $\mathcal{F}$.FieldOp, $\mathcal{F}$.ofFieldOpListF ϕ s is defined as the element of $\mathcal{F}$.FieldOpFreeAlgebra obtained by the product of ofFieldOpF ϕ for each ϕ in ϕ s. For example ofFieldOpListF [ϕ_1 , ϕ_2 , ϕ_3] = ofFieldOpF ϕ_1 * ofFieldOpF ϕ_2 * ofFieldOpF ϕ_3 .

def ofFieldOpListF (ϕ s : List $\mathcal{F}$.FieldOp) : FieldOpFreeAlgebra $\mathcal{F}$ **instance Coe (List $\mathcal{F}$.FieldOp) (FieldOpFreeAlgebra $\mathcal{F}$)**[\[source\]](#)

Coercion from List $\mathcal{F}$.FieldOp to FieldOpFreeAlgebra $\mathcal{F}$ through ofFieldOpListF.

instance : Coe (List $\mathcal{F}$.FieldOp) (FieldOpFreeAlgebra $\mathcal{F}$)**lemma ofFieldOpListF_nil**[\[source\]](#)

@[simp]

lemma ofFieldOpListF_nil : ofFieldOpListF ([]) : List $\mathcal{F}$.FieldOp = 1

lemma ofFieldOpListF_cons[\[source\]](#)

```
lemma ofFieldOpListF_cons (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpListF (φ :: φs) = ofFieldOpF φ * ofFieldOpListF φs
```

lemma ofFieldOpListF_singleton[\[source\]](#)

```
lemma ofFieldOpListF_singleton (φ :  $\mathcal{F}$ .FieldOp) :
  ofFieldOpListF [φ] = ofFieldOpF φ
```

lemma ofFieldOpListF_append[\[source\]](#)

```
lemma ofFieldOpListF_append (φs φs' : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpListF (φs ++ φs') = ofFieldOpListF φs * ofFieldOpListF φs'
```

lemma ofFieldOpListF_sum[\[source\]](#)

```
lemma ofFieldOpListF_sum (φs : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpListF φs =  $\sum$  (s : CrAnSection φs), ofCrAnListF s.1
```

def crPartF[\[source\]](#)

The algebra map taking an element of the free-state algebra to the part of it in the creation and annihilation free algebra spanned by creation operators.

```
def crPartF :  $\mathcal{F}$ .FieldOp →  $\mathcal{F}$ .FieldOpFreeAlgebra
```

lemma crPartF_negAsymp[\[source\]](#)

```
@[simp]
lemma crPartF_negAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × (Fin 3 → ℝ)) :
  crPartF (FieldOp.inAsymp φ) = ofCrAnOpF (FieldOp.inAsymp φ, ())
```

lemma crPartF_position[\[source\]](#)

```
@[simp]
lemma crPartF_position (φ : (Σ f,  $\mathcal{F}$ .PositionLabel f) × SpaceTime) :
  crPartF (FieldOp.position φ) =
  ofCrAnOpF (FieldOp.position φ, CreateAnnihilate.create)
```

lemma crPartF_posAsymp[\[source\]](#)

```
@[simp]
lemma crPartF_posAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × (Fin 3 → ℝ)) :
  crPartF (FieldOp.outAsymp φ) = 0
```

def anPartF[\[source\]](#)

The algebra map taking an element of the free-state algebra to the part of it in the creation and annihilation free algebra spanned by annihilation operators.

```
def anPartF :  $\mathcal{F}$ .FieldOp →  $\mathcal{F}$ .FieldOpFreeAlgebra
```

lemma anPartF_negAsymp[\[source\]](#)

@[simp]

```
lemma anPartF_negAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × (Fin 3 → ℝ)) :
  anPartF (FieldOp.inAsymp φ) = 0
```

lemma anPartF_position[\[source\]](#)

@[simp]

```
lemma anPartF_position (φ : (Σ f,  $\mathcal{F}$ .PositionLabel f) × SpaceTime) :
  anPartF (FieldOp.position φ) =
  ofCrAnOpF (FieldOp.position φ, CreateAnnihilate.annihilate)
```

lemma anPartF_posAsymp[\[source\]](#)

@[simp]

```
lemma anPartF_posAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × (Fin 3 → ℝ)) :
  anPartF (FieldOp.outAsymp φ) = ofCrAnOpF (FieldOp.outAsymp φ, ())
```

lemma ofFieldOpF_eq_crPartF_add_anPartF[\[source\]](#)

```
lemma ofFieldOpF_eq_crPartF_add_anPartF (φ :  $\mathcal{F}$ .FieldOp) :
  ofFieldOpF φ = crPartF φ + anPartF φ
```

def ofCrAnListFBasis[\[source\]](#)

The basis of the free creation and annihilation algebra formed by lists of CrAnFieldOp.

```
noncomputable def ofCrAnListFBasis : Basis (List  $\mathcal{F}$ .CrAnFieldOp)  $\mathbb{C}$  (FieldOpFreeAlgebra  $\mathcal{F}$ ) where
```

lemma ofListBasis_eq_ofList[\[source\]](#)

@[simp]

```
lemma ofListBasis_eq_ofList (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnListFBasis φs = ofCrAnListF φs
```

def mulLinearMap[\[source\]](#)

The bi-linear map associated with multiplication in FieldOpFreeAlgebra.

```
noncomputable def mulLinearMap : FieldOpFreeAlgebra  $\mathcal{F}$  →l  $\mathbb{C}$  FieldOpFreeAlgebra  $\mathcal{F}$  →l  $\mathbb{C}$ 
  FieldOpFreeAlgebra  $\mathcal{F}$  where
```

lemma mulLinearMap_apply[\[source\]](#)

```
lemma mulLinearMap_apply (a b : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
  mulLinearMap a b = a * b
```

def smulLinearMap[\[source\]](#)

The linear map associated with scalar-multiplication in FieldOpFreeAlgebra.

```
noncomputable def smulLinearMap (c : ℂ) : FieldOpFreeAlgebra  $\mathcal{F}$   $\rightarrow_l$  [ℂ] FieldOpFreeAlgebra  $\mathcal{F}$ 
  where
```

2.33 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Grading

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Grading.lean](#) on GitHub.

def statisticSubmodule [source]

The submodule of FieldOpFreeAlgebra spanned by lists of field statistic f.

```
def statisticSubmodule (f : FieldStatistic) : Submodule ℂ  $\mathcal{F}$ .FieldOpFreeAlgebra
```

lemma ofCrAnListF_mem_statisticSubmodule_of [source]

```
lemma ofCrAnListF_mem_statisticSubmodule_of (φs : List  $\mathcal{F}$ .CrAnFieldOp) (f : FieldStatistic)
  (h : ( $\mathcal{F}$  |>s φs) = f) :
  ofCrAnListF φs ∈ statisticSubmodule f
```

lemma ofCrAnListF_bosonic_or_fermionic [source]

```
lemma ofCrAnListF_bosonic_or_fermionic (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnListF φs ∈ statisticSubmodule bosonic v
  ofCrAnListF φs ∈ statisticSubmodule fermionic
```

lemma ofCrAnOpF_bosonic_or_fermionic [source]

```
lemma ofCrAnOpF_bosonic_or_fermionic (φ :  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnOpF φ ∈ statisticSubmodule bosonic v ofCrAnOpF φ ∈ statisticSubmodule fermionic
```

def bosonicProjF [source]

The projection of an element of FieldOpFreeAlgebra onto it's bosonic part.

```
def bosonicProjF :  $\mathcal{F}$ .FieldOpFreeAlgebra  $\rightarrow_l$  [ℂ] statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) bosonic
```

lemma bosonicProjF_ofCrAnListF [source]

```
lemma bosonicProjF_ofCrAnListF (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  bosonicProjF (ofCrAnListF φs) = if h : ( $\mathcal{F}$  |>s φs) = bosonic then
    (ofCrAnListF φs, Submodule.mem_span.mpr fun _ a => a (φs, (rfl, h))) else 0
```

lemma bosonicProjF_of_mem_bosonic [source]

```
lemma bosonicProjF_of_mem_bosonic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h : a ∈ statisticSubmodule
  bosonic) :
  bosonicProjF a = {a, h}
```

lemma bosonicProjF_of_mem_fermionic[\[source\]](#)

```

lemma bosonicProjF_of_mem_fermionic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule fermionic) :
  bosonicProjF a = 0

```

lemma bosonicProjF_of_bononic_part[\[source\]](#)

```

@[simp]
lemma bosonicProjF_of_bononic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProjF (a bosonic) = a bosonic

```

lemma bosonicProjF_of_fermionic_part[\[source\]](#)

```

@[simp]
lemma bosonicProjF_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProjF (a fermionic).1 = 0

```

def fermionicProjF[\[source\]](#)

The projection of an element of FieldOpFreeAlgebra onto it's fermionic part.

```

def fermionicProjF :  $\mathcal{F}$ .FieldOpFreeAlgebra →l[ $\mathbb{C}$ ] statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) fermionic

```

lemma fermionicProjF_ofCrAnListF[\[source\]](#)

```

lemma fermionicProjF_ofCrAnListF (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  fermionicProjF (ofCrAnListF φs) = if h : ( $\mathcal{F}$  |>s φs) = fermionic then
    (ofCrAnListF φs, Submodule.mem_span.mpr fun _ a => a ⟨φs, ⟨rfl, h⟩⟩) else 0

```

lemma fermionicProjF_ofCrAnListF_if_bosonic[\[source\]](#)

```

lemma fermionicProjF_ofCrAnListF_if_bosonic (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  fermionicProjF (ofCrAnListF φs) = if h : ( $\mathcal{F}$  |>s φs) = bosonic then
    0 else (ofCrAnListF φs, Submodule.mem_span.mpr fun _ a => a ⟨φs, ⟨rfl,
      by simp using h⟩⟩)

```

lemma fermionicProjF_of_mem_fermionic[\[source\]](#)

```

lemma fermionicProjF_of_mem_fermionic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule fermionic) :
  fermionicProjF a = ⟨a, h⟩

```

lemma fermionicProjF_of_mem_bosonic[\[source\]](#)

```

lemma fermionicProjF_of_mem_bosonic (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule bosonic) : fermionicProjF a = 0

```

lemma fermionicProjF_of_bosonic_part[\[source\]](#)

```
@[simp]
lemma fermionicProjF_of_bosonic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule (F := F) i))) :
  fermionicProjF (a bosonic).1 = 0
```

lemma fermionicProjF_of_fermionic_part[\[source\]](#)

```
@[simp]
lemma fermionicProjF_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule (F := F) i))) :
  fermionicProjF (a fermionic) = a fermionic
```

lemma bosonicProjF_add_fermionicProjF[\[source\]](#)

```
lemma bosonicProjF_add_fermionicProjF (a : F.FieldOpFreeAlgebra) :
  a.bosonicProjF + (a.fermionicProjF).1 = a
```

lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic[\[source\]](#)

```
lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule (F := F) i))) :
  DirectSum.coeAddMonoidHom statisticSubmodule a = a.1 bosonic + a.1 fermionic
```

lemma directSum_eq_bosonic_plus_fermionic[\[source\]](#)

```
lemma directSum_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule (F := F) i))) :
  a = (DirectSum.of (fun i => 1(statisticSubmodule i)) bosonic) (a bosonic) +
  (DirectSum.of (fun i => 1(statisticSubmodule i)) fermionic) (a fermionic)
```

instance fieldOpFreeAlgebraGrade[\[source\]](#)

For a field specification $\mathcal{F}$, the algebra $\mathcal{F}.FieldOpFreeAlgebra$ is graded by $FieldStatistic$. Those of $CrAnListF$ ϕs for which ϕs has an overall bosonic statistic (i.e. $\mathcal{F} \mid_s \phi s = \text{bosonic}$) span bosonic submodule, whilst those of $CrAnListF$ ϕs for which ϕs has an overall fermionic statistic (i.e. $\mathcal{F} \mid_s \phi s = \text{fermionic}$) span the fermionic submodule.

```
instance fieldOpFreeAlgebraGrade :
  GradedAlgebra (A := F.FieldOpFreeAlgebra) statisticSubmodule where
```

lemma eq_zero_of_bosonic_and_fermionic[\[source\]](#)

```
lemma eq_zero_of_bosonic_and_fermionic {a : F.FieldOpFreeAlgebra}
  (hb : a ∈ statisticSubmodule bosonic) (hf : a ∈ statisticSubmodule fermionic) : a = 0
```

lemma bosonicProjF_mul[\[source\]](#)

```
lemma bosonicProjF_mul (a b : F.FieldOpFreeAlgebra) :
  (a * b).bosonicProjF.1 = a.bosonicProjF.1 * b.bosonicProjF.1
  + a.fermionicProjF.1 * b.fermionicProjF.1
```

lemma fermionicProjF_mul[\[source\]](#)

```

lemma fermionicProjF_mul (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  (a * b).fermionicProjF.1 = a.bosonicProjF.1 * b.fermionicProjF.1
  + a.fermionicProjF.1 * b.bosonicProjF.1

```

2.34 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormalOrder

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormalOrder.lean](#) on GitHub.

def normalOrderF[\[source\]](#)

For a field specification $\mathcal{F}$, `normalOrderF` is the linear map

$\text{FieldOpFreeAlgebra } \mathcal{F} \rightarrow_l[\mathbb{C}] \text{FieldOpFreeAlgebra } \mathcal{F}$

defined by its action on the basis of `CrAnListF` ϕ s, taking of `CrAnListF` ϕ s to

$\text{normalOrderSign } \phi \cdot \text{ofCrAnListF } (\text{normalOrderList } \phi)$.

That is, `normalOrderF` normal-orders the field operators and multiplies by the sign of the normal order.

The notation $\mathcal{N}^f(a)$ is used for `normalOrderF a` for a an element of `FieldOpFreeAlgebra` $\mathcal{F}$.

```

def normalOrderF : FieldOpFreeAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}] \text{FieldOpFreeAlgebra } \mathcal{F}$ 
```

lemma normalOrderF_ofCrAnListF[\[source\]](#)

```

lemma normalOrderF_ofCrAnListF ( $\phi$ s : List  $\mathcal{F}$ .CrAnFieldOp) :
   $\mathcal{N}^f(\text{ofCrAnListF } \phi \text{s}) = \text{normalOrderSign } \phi \text{s} \cdot \text{ofCrAnListF } (\text{normalOrderList } \phi \text{s})$ 
```

lemma ofCrAnListF_eq_normalOrderF[\[source\]](#)

```

lemma ofCrAnListF_eq_normalOrderF ( $\phi$ s : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnListF (normalOrderList  $\phi$ s) = normalOrderSign  $\phi$ s *  $\mathcal{N}^f(\text{ofCrAnListF } \phi \text{s})$ 
```

lemma normalOrderF_one[\[source\]](#)

```

lemma normalOrderF_one : normalOrderF ( $\mathcal{F} := \mathcal{F}$ ) 1 = 1

```

lemma normalOrderF_normalOrderF_mid[\[source\]](#)

```

lemma normalOrderF_normalOrderF_mid (a b c :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
   $\mathcal{N}^f(a * b * c) = \mathcal{N}^f(a * \mathcal{N}^f(b) * c)$ 
```

lemma normalOrderF_normalOrderF_right[\[source\]](#)

```

lemma normalOrderF_normalOrderF_right (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
   $\mathcal{N}^f(a * b) = \mathcal{N}^f(a * \mathcal{N}^f(b))$ 
```

lemma normalOrderF_normalOrderF_left[\[source\]](#)

lemma normalOrderF_normalOrderF_left (a b : $\mathcal{F}$.FieldOpFreeAlgebra) :
 $\mathcal{N}^f(a * b) = \mathcal{N}^f(\mathcal{N}^f(a) * b)$

lemma normalOrderF_ofCrAnListF_cons_create[\[source\]](#)

lemma normalOrderF_ofCrAnListF_cons_create (ϕ : $\mathcal{F}$.CrAnFieldOp)
 ($h\phi$: $\mathcal{F} \mid >^c \phi = \text{CreateAnnihilate.create}$) (ϕs : List $\mathcal{F}$.CrAnFieldOp) :
 $\mathcal{N}^f(\text{ofCrAnListF } (\phi :: \phi s)) = \text{ofCrAnOpF } \phi * \mathcal{N}^f(\text{ofCrAnListF } \phi s)$

lemma normalOrderF_create_mul[\[source\]](#)

lemma normalOrderF_create_mul (ϕ : $\mathcal{F}$.CrAnFieldOp)
 ($h\phi$: $\mathcal{F} \mid >^c \phi = \text{CreateAnnihilate.create}$) (a : FieldOpFreeAlgebra $\mathcal{F}$) :
 $\mathcal{N}^f(\text{ofCrAnOpF } \phi * a) = \text{ofCrAnOpF } \phi * \mathcal{N}^f(a)$

lemma normalOrderF_ofCrAnListF_append_annihilate[\[source\]](#)

lemma normalOrderF_ofCrAnListF_append_annihilate (ϕ : $\mathcal{F}$.CrAnFieldOp)
 ($h\phi$: $\mathcal{F} \mid >^c \phi = \text{CreateAnnihilate.annihilate}$) (ϕs : List $\mathcal{F}$.CrAnFieldOp) :
 $\mathcal{N}^f(\text{ofCrAnListF } (\phi s ++ [\phi])) = \mathcal{N}^f(\text{ofCrAnListF } \phi s) * \text{ofCrAnOpF } \phi$

lemma normalOrderF_mul_annihilate[\[source\]](#)

lemma normalOrderF_mul_annihilate (ϕ : $\mathcal{F}$.CrAnFieldOp)
 ($h\phi$: $\mathcal{F} \mid >^c \phi = \text{CreateAnnihilate.annihilate}$)
 (a : FieldOpFreeAlgebra $\mathcal{F}$) : $\mathcal{N}^f(a * \text{ofCrAnOpF } \phi) = \mathcal{N}^f(a) * \text{ofCrAnOpF } \phi$

lemma normalOrderF_crPartF_mul[\[source\]](#)

lemma normalOrderF_crPartF_mul (ϕ : $\mathcal{F}$.FieldOp) (a : FieldOpFreeAlgebra $\mathcal{F}$) :
 $\mathcal{N}^f(\text{crPartF } \phi * a) =$
 $\text{crPartF } \phi * \mathcal{N}^f(a)$

lemma normalOrderF_mul_anPartF[\[source\]](#)

lemma normalOrderF_mul_anPartF (ϕ : $\mathcal{F}$.FieldOp) (a : FieldOpFreeAlgebra $\mathcal{F}$) :
 $\mathcal{N}^f(a * \text{anPartF } \phi) =$
 $\mathcal{N}^f(a) * \text{anPartF } \phi$

lemma normalOrderF_swap_create_annihilate_ofCrAnListF_ofCrAnListF[\[source\]](#)

lemma normalOrderF_swap_create_annihilate_ofCrAnListF_ofCrAnListF ($\phi c \phi a$: $\mathcal{F}$.CrAnFieldOp)
 ($h\phi c$: $\mathcal{F} \mid >^c \phi c = \text{CreateAnnihilate.create}$) ($h\phi a$: $\mathcal{F} \mid >^c \phi a = \text{CreateAnnihilate.annihilate}$)
 ($\phi s \phi s'$: List $\mathcal{F}$.CrAnFieldOp) :
 $\mathcal{N}^f(\text{ofCrAnListF } \phi s' * \text{ofCrAnOpF } \phi c * \text{ofCrAnOpF } \phi a * \text{ofCrAnListF } \phi s) = S(\mathcal{F} \mid >_s \phi c, \mathcal{F} \mid >_s$
 $\phi a) \cdot$
 $\mathcal{N}^f(\text{ofCrAnListF } \phi s' * \text{ofCrAnOpF } \phi a * \text{ofCrAnOpF } \phi c * \text{ofCrAnListF } \phi s)$

lemma normalOrderF_swap_create_annihilate_ofCrAnListF[\[source\]](#)

```

lemma normalOrderF_swap_create_annihilate_ofCrAnListF (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi_c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid^{>c} \varphi_a = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnFieldOp) (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
 $\mathcal{N}^f(\text{ofCrAnListF } \varphi_s * \text{ofCrAnOpF } \varphi_c * \text{ofCrAnOpF } \varphi_a * a) = S(\mathcal{F} \mid^{>s} \varphi_c, \mathcal{F} \mid^{>s} \varphi_a) \cdot$ 
 $\mathcal{N}^f(\text{ofCrAnListF } \varphi_s * \text{ofCrAnOpF } \varphi_a * \text{ofCrAnOpF } \varphi_c * a)$ 

```

lemma normalOrderF_swap_create_annihilate[\[source\]](#)

```

lemma normalOrderF_swap_create_annihilate (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi_c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid^{>c} \varphi_a = \text{CreateAnnihilate.annihilate}$ )
  (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
 $\mathcal{N}^f(a * \text{ofCrAnOpF } \varphi_c * \text{ofCrAnOpF } \varphi_a * b) = S(\mathcal{F} \mid^{>s} \varphi_c, \mathcal{F} \mid^{>s} \varphi_a) \cdot$ 
 $\mathcal{N}^f(a * \text{ofCrAnOpF } \varphi_a * \text{ofCrAnOpF } \varphi_c * b)$ 

```

lemma normalOrderF_superCommuteF_create_annihilate[\[source\]](#)

```

lemma normalOrderF_superCommuteF_create_annihilate (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi_c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid^{>c} \varphi_a = \text{CreateAnnihilate.annihilate}$ )
  (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
 $\mathcal{N}^f(a * [\text{ofCrAnOpF } \varphi_c, \text{ofCrAnOpF } \varphi_a]_sF * b) = 0$ 

```

lemma normalOrderF_superCommuteF_annihilate_create[\[source\]](#)

```

lemma normalOrderF_superCommuteF_annihilate_create (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi_c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid^{>c} \varphi_a = \text{CreateAnnihilate.annihilate}$ )
  (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
 $\mathcal{N}^f(a * [\text{ofCrAnOpF } \varphi_a, \text{ofCrAnOpF } \varphi_c]_sF * b) = 0$ 

```

lemma normalOrderF_swap_crPartF_anPartF[\[source\]](#)

```

lemma normalOrderF_swap_crPartF_anPartF (φ φ' :  $\mathcal{F}$ .FieldOp) (a b : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * (\text{crPartF } \varphi) * (\text{anPartF } \varphi') * b) =$ 
 $S(\mathcal{F} \mid^{>s} \varphi, \mathcal{F} \mid^{>s} \varphi') \cdot$ 
 $\mathcal{N}^f(a * (\text{anPartF } \varphi') * (\text{crPartF } \varphi) * b)$ 

```

lemma normalOrderF_swap_anPartF_crPartF[\[source\]](#)

```

lemma normalOrderF_swap_anPartF_crPartF (φ φ' :  $\mathcal{F}$ .FieldOp) (a b : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * (\text{anPartF } \varphi) * (\text{crPartF } \varphi') * b) =$ 
 $S(\mathcal{F} \mid^{>s} \varphi, \mathcal{F} \mid^{>s} \varphi') \cdot \mathcal{N}^f(a * (\text{crPartF } \varphi') * (\text{anPartF } \varphi) * b)$ 

```

lemma normalOrderF_superCommuteF_crPartF_anPartF[\[source\]](#)

```

lemma normalOrderF_superCommuteF_crPartF_anPartF (φ φ' :  $\mathcal{F}$ .FieldOp) (a b : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * \text{superCommuteF}$ 
   $(\text{crPartF } \varphi) (\text{anPartF } \varphi') * b) = 0$ 

```

lemma normalOrderF_superCommuteF_anPartF_crPartF[\[source\]](#)

```

lemma normalOrderF_superCommuteF_anPartF_crPartF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) (a b : FieldOpFreeAlgebra
 $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * \text{superCommuteF}$ 
 $(\text{anPartF } \phi) (\text{crPartF } \phi') * b) = 0$ 

```

lemma normalOrderF_crPartF_mul_crPartF[\[source\]](#)

```

@[simp]
lemma normalOrderF_crPartF_mul_crPartF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) :
 $\mathcal{N}^f(\text{crPartF } \phi * \text{crPartF } \phi') =$ 
 $\text{crPartF } \phi * \text{crPartF } \phi'$ 

```

lemma normalOrderF_anPartF_mul_anPartF[\[source\]](#)

```

@[simp]
lemma normalOrderF_anPartF_mul_anPartF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) :
 $\mathcal{N}^f(\text{anPartF } \phi * \text{anPartF } \phi') =$ 
 $\text{anPartF } \phi * \text{anPartF } \phi'$ 

```

lemma normalOrderF_crPartF_mul_anPartF[\[source\]](#)

```

@[simp]
lemma normalOrderF_crPartF_mul_anPartF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) :
 $\mathcal{N}^f(\text{crPartF } \phi * \text{anPartF } \phi') =$ 
 $\text{crPartF } \phi * \text{anPartF } \phi'$ 

```

lemma normalOrderF_anPartF_mul_crPartF[\[source\]](#)

```

@[simp]
lemma normalOrderF_anPartF_mul_crPartF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) :
 $\mathcal{N}^f(\text{anPartF } \phi * \text{crPartF } \phi') =$ 
 $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \cdot$ 
 $(\text{crPartF } \phi' * \text{anPartF } \phi)$ 

```

lemma normalOrderF_ofFieldOpF_mul_ofFieldOpF[\[source\]](#)

```

lemma normalOrderF_ofFieldOpF_mul_ofFieldOpF ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp) :
 $\mathcal{N}^f(\text{ofFieldOpF } \phi * \text{ofFieldOpF } \phi') =$ 
 $\text{crPartF } \phi * \text{crPartF } \phi' +$ 
 $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \cdot$ 
 $(\text{crPartF } \phi' * \text{anPartF } \phi) +$ 
 $\text{crPartF } \phi * \text{anPartF } \phi' +$ 
 $\text{anPartF } \phi * \text{anPartF } \phi'$ 

```

lemma normalOrderF_superCommuteF_ofCrAnListF_create_create_ofCrAnListF[\[source\]](#)

```

lemma normalOrderF_superCommuteF_ofCrAnListF_create_create_ofCrAnListF
( $\phi c \phi'$  :  $\mathcal{F}$ .CrAnFieldOp) ( $h\phi c$  :  $\mathcal{F} \mid_{>c} \phi c = \text{CreateAnnihilate.create}$ )
( $h\phi c'$  :  $\mathcal{F} \mid_{>c} \phi c' = \text{CreateAnnihilate.create}$ ) ( $\phi s \phi s'$  : List  $\mathcal{F}$ .CrAnFieldOp) :
 $(\mathcal{N}^f(\text{ofCrAnListF } \phi s * [\text{ofCrAnOpF } \phi c, \text{ofCrAnOpF } \phi c']_s * \text{ofCrAnListF } \phi s')) =$ 
 $\text{normalOrderSign } (\phi s ++ \phi c' :: \phi c :: \phi s') \cdot$ 
 $(\text{ofCrAnListF } (\text{createFilter } \phi s) * [\text{ofCrAnOpF } \phi c, \text{ofCrAnOpF } \phi c']_s * \text{ofCrAnListF } \phi s')$ 

```

```
ofCrAnListF (createFilter  $\phi s'$ ) * ofCrAnListF (annihilateFilter ( $\phi s ++ \phi s'$ )))
```

lemma normalOrderF_superCommuteF_ofCrAnListF_annihilate_annihilate_ofCrAnListF [\[source\]](#)

```
lemma normalOrderF_superCommuteF_ofCrAnListF_annihilate_annihilate_ofCrAnListF
  ( $\phi a \phi a' : \mathcal{F}.CrAnFieldOp$ )
  ( $h\phi a : \mathcal{F} \mid >^c \phi a = CreateAnnihilate.annihilate$ )
  ( $h\phi a' : \mathcal{F} \mid >^c \phi a' = CreateAnnihilate.annihilate$ )
  ( $\phi s \phi s' : List \mathcal{F}.CrAnFieldOp$ ) :
 $\mathcal{N}^f(ofCrAnListF \phi s * [ofCrAnOpF \phi a, ofCrAnOpF \phi a']_sF * ofCrAnListF \phi s') =$ 
  normalOrderSign ( $\phi s ++ \phi a' :: \phi a :: \phi s'$ ) •
  ( $ofCrAnListF (createFilter (\phi s ++ \phi s'))$ 
    *  $ofCrAnListF (annihilateFilter \phi s) * [ofCrAnOpF \phi a, ofCrAnOpF \phi a']_sF$ 
    *  $ofCrAnListF (annihilateFilter \phi s')$ )
```

lemma ofCrAnListF_superCommuteF_normalOrderF_ofCrAnListF [\[source\]](#)

```
lemma ofCrAnListF_superCommuteF_normalOrderF_ofCrAnListF ( $\phi s \phi s' : List \mathcal{F}.CrAnFieldOp$ ) :
  [ $ofCrAnListF \phi s, \mathcal{N}^f(ofCrAnListF \phi s')$ ] $_sF =$ 
   $ofCrAnListF \phi s * \mathcal{N}^f(ofCrAnListF \phi s') -$ 
   $S(\mathcal{F} \mid >_s \phi s, \mathcal{F} \mid >_s \phi s') \cdot \mathcal{N}^f(ofCrAnListF \phi s') * ofCrAnListF \phi s$ 
```

lemma ofCrAnListF_superCommuteF_normalOrderF_ofFieldOpListF [\[source\]](#)

```
lemma ofCrAnListF_superCommuteF_normalOrderF_ofFieldOpListF ( $\phi s : List \mathcal{F}.CrAnFieldOp$ )
  ( $\phi s' : List \mathcal{F}.FieldOp$ ) : [ $ofCrAnListF \phi s, \mathcal{N}^f(ofFieldOpListF \phi s')$ ] $_sF =$ 
   $ofCrAnListF \phi s * \mathcal{N}^f(ofFieldOpListF \phi s') -$ 
   $S(\mathcal{F} \mid >_s \phi s, \mathcal{F} \mid >_s \phi s') \cdot \mathcal{N}^f(ofFieldOpListF \phi s') * ofCrAnListF \phi s$ 
```

lemma ofCrAnListF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF [\[source\]](#)

```
lemma ofCrAnListF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF ( $\phi s : List \mathcal{F}.CrAnFieldOp$ )
  ( $\phi s' : List \mathcal{F}.FieldOp$ ) :
   $ofCrAnListF \phi s * \mathcal{N}^f(ofFieldOpListF \phi s') =$ 
   $S(\mathcal{F} \mid >_s \phi s, \mathcal{F} \mid >_s \phi s') \cdot \mathcal{N}^f(ofFieldOpListF \phi s') * ofCrAnListF \phi s$ 
  + [ $ofCrAnListF \phi s, \mathcal{N}^f(ofFieldOpListF \phi s')$ ] $_sF$ 
```

lemma ofCrAnOpF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF [\[source\]](#)

```
lemma ofCrAnOpF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF ( $\phi : \mathcal{F}.CrAnFieldOp$ )
  ( $\phi s' : List \mathcal{F}.FieldOp$ ) :  $ofCrAnOpF \phi * \mathcal{N}^f(ofFieldOpListF \phi s') =$ 
   $S(\mathcal{F} \mid >_s \phi, \mathcal{F} \mid >_s \phi s') \cdot \mathcal{N}^f(ofFieldOpListF \phi s') * ofCrAnOpF \phi$ 
  + [ $ofCrAnOpF \phi, \mathcal{N}^f(ofFieldOpListF \phi s')$ ] $_sF$ 
```

lemma anPartF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF [\[source\]](#)

```
lemma anPartF_mul_normalOrderF_ofFieldOpListF_eq_superCommuteF ( $\phi : \mathcal{F}.FieldOp$ )
  ( $\phi s' : List \mathcal{F}.FieldOp$ ) :
   $anPartF \phi * \mathcal{N}^f(ofFieldOpListF \phi s') =$ 
   $S(\mathcal{F} \mid >_s \phi, \mathcal{F} \mid >_s \phi s') \cdot \mathcal{N}^f(ofFieldOpListF \phi s') * anPartF \phi$ 
  + [ $anPartF \phi, \mathcal{N}^f(ofFieldOpListF \phi s')$ ] $_sF$ 
```

2.35 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.NormTimeOrder

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/NormTimeOrder.lean](#) on GitHub.

def normTimeOrder

[\[source\]](#)

The normal-time ordering on FieldOpFreeAlgebra.

```
def normTimeOrder : FieldOpFreeAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}]$  FieldOpFreeAlgebra  $\mathcal{F}$ 
```

lemma normTimeOrder_ofCrAnListF

[\[source\]](#)

```
lemma normTimeOrder_ofCrAnListF ( $\phi s$  : List  $\mathcal{F}.CrAnFieldOp$ ) :  
   $\mathcal{TN}^f(\text{ofCrAnListF } \phi s) = \text{normTimeOrderSign } \phi s \cdot \text{ofCrAnListF } (\text{normTimeOrderList } \phi s)$ 
```

2.36 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.SuperCommute

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.lean](#) on GitHub.

def superCommuteF

[\[source\]](#)

For a field specification $\mathcal{F}$, the super commutator superCommuteF is defined as the linear map $\mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}] \mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}] \mathcal{F}.FieldOpFreeAlgebra$ which on the lists ϕs and $\phi s'$ of $\mathcal{F}.CrAnFieldOp$ gives

$$\text{superCommuteF } \phi s \ \phi s' = \phi s * \phi s' - S(\phi s, \phi s') \cdot \phi s' * \phi s.$$

The notation $[a, b]_s \mathcal{F}$ can be used for $\text{superCommuteF } a \ b$.

```
noncomputable def superCommuteF :  $\mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}] \mathcal{F}.FieldOpFreeAlgebra \rightarrow_l[\mathbb{C}]$   
   $\mathcal{F}.FieldOpFreeAlgebra$ 
```

lemma superCommuteF_ofCrAnListF_ofCrAnListF

[\[source\]](#)

```
lemma superCommuteF_ofCrAnListF_ofCrAnListF ( $\phi s \ \phi s'$  : List  $\mathcal{F}.CrAnFieldOp$ ) :  
   $[\text{ofCrAnListF } \phi s, \text{ofCrAnListF } \phi s']_s \mathcal{F} =$   
   $\text{ofCrAnListF } (\phi s ++ \phi s') - S(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s') \cdot \text{ofCrAnListF } (\phi s' ++ \phi s)$ 
```

lemma superCommuteF_ofCrAnOpF_ofCrAnOpF

[\[source\]](#)

```
lemma superCommuteF_ofCrAnOpF_ofCrAnOpF ( $\phi \ \phi'$  :  $\mathcal{F}.CrAnFieldOp$ ) :  
   $[\text{ofCrAnOpF } \phi, \text{ofCrAnOpF } \phi']_s \mathcal{F} =$   
   $\text{ofCrAnOpF } \phi * \text{ofCrAnOpF } \phi' - S(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot \text{ofCrAnOpF } \phi' * \text{ofCrAnOpF } \phi$ 
```

lemma superCommuteF_ofCrAnListF_ofFieldOpFsList

[\[source\]](#)

```
lemma superCommuteF_ofCrAnListF_ofFieldOpFsList ( $\phi cas$  : List  $\mathcal{F}.CrAnFieldOp$ ) ( $\phi s$  : List  $\mathcal{F}.FieldOp$ ) :  
   $[\text{ofCrAnListF } \phi cas, \text{ofFieldOpListF } \phi s]_s \mathcal{F} = \text{ofCrAnListF } \phi cas * \text{ofFieldOpListF } \phi s -$   
   $S(\mathcal{F} \mid>_s \phi cas, \mathcal{F} \mid>_s \phi s) \cdot \text{ofFieldOpListF } \phi s * \text{ofCrAnListF } \phi cas$ 
```

lemma superCommuteF_ofFieldOpListF_ofFieldOpFsList[\[source\]](#)

```

lemma superCommuteF_ofFieldOpListF_ofFieldOpFsList (φ : List  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
  :
  [ofFieldOpListF φ, ofFieldOpListF ψs]sF = ofFieldOpListF φ * ofFieldOpListF ψs -
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s ψs) • ofFieldOpListF ψs * ofFieldOpListF φ

```

lemma superCommuteF_ofFieldOpF_ofFieldOpFsList[\[source\]](#)

```

lemma superCommuteF_ofFieldOpF_ofFieldOpFsList (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
  [ofFieldOpF φ, ofFieldOpListF ψs]sF = ofFieldOpF φ * ofFieldOpListF ψs -
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s ψs) • ofFieldOpListF ψs * ofFieldOpF φ

```

lemma superCommuteF_ofFieldOpListF_ofFieldOpF[\[source\]](#)

```

lemma superCommuteF_ofFieldOpListF_ofFieldOpF (ψs : List  $\mathcal{F}$ .FieldOp) (φ :  $\mathcal{F}$ .FieldOp) :
  [ofFieldOpListF ψs, ofFieldOpF φ]sF = ofFieldOpListF ψs * ofFieldOpF φ -
  S( $\mathcal{F}$  |>s ψs,  $\mathcal{F}$  |>s φ) • ofFieldOpF φ * ofFieldOpListF ψs

```

lemma superCommuteF_anPartF_crPartF[\[source\]](#)

```

lemma superCommuteF_anPartF_crPartF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [anPartF φ, crPartF φ']sF = anPartF φ * crPartF φ' -
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s φ') • crPartF φ' * anPartF φ

```

lemma superCommuteF_crPartF_anPartF[\[source\]](#)

```

lemma superCommuteF_crPartF_anPartF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [crPartF φ, anPartF φ']sF = crPartF φ * anPartF φ' -
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s φ') • anPartF φ' * crPartF φ

```

lemma superCommuteF_crPartF_crPartF[\[source\]](#)

```

lemma superCommuteF_crPartF_crPartF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [crPartF φ, crPartF φ']sF = crPartF φ * crPartF φ' -
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s φ') • crPartF φ' * crPartF φ

```

lemma superCommuteF_anPartF_anPartF[\[source\]](#)

```

lemma superCommuteF_anPartF_anPartF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [anPartF φ, anPartF φ']sF =
  anPartF φ * anPartF φ' - S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s φ') • anPartF φ' * anPartF φ

```

lemma superCommuteF_crPartF_ofFieldOpListF[\[source\]](#)

```

lemma superCommuteF_crPartF_ofFieldOpListF (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
  [crPartF φ, ofFieldOpListF ψs]sF =
  crPartF φ * ofFieldOpListF ψs - S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s ψs) • ofFieldOpListF ψs *
  crPartF φ

```

lemma superCommuteF_anPartF_ofFieldOpListF[\[source\]](#)

```

lemma superCommuteF_anPartF_ofFieldOpListF (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
  [anPartF φ, ofFieldOpListF ψs]sF =
  anPartF φ * ofFieldOpListF ψs -  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi s) \cdot$ 
  ofFieldOpListF ψs * anPartF φ

```

lemma superCommuteF_crPartF_ofFieldOpF[\[source\]](#)

```

lemma superCommuteF_crPartF_ofFieldOpF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [crPartF φ, ofFieldOpF φ']sF =
  crPartF φ * ofFieldOpF φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot$  ofFieldOpF φ' * crPartF φ

```

lemma superCommuteF_anPartF_ofFieldOpF[\[source\]](#)

```

lemma superCommuteF_anPartF_ofFieldOpF (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [anPartF φ, ofFieldOpF φ']sF =
  anPartF φ * ofFieldOpF φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot$  ofFieldOpF φ' * anPartF φ

```

lemma ofCrAnListF_mul_ofCrAnListF_eq_superCommuteF[\[source\]](#)

```

lemma ofCrAnListF_mul_ofCrAnListF_eq_superCommuteF (ψs ψs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnListF ψs * ofCrAnListF ψs' =  $\mathcal{S}(\mathcal{F} \mid>_s \psi s, \mathcal{F} \mid>_s \psi s') \cdot$  ofCrAnListF ψs' * ofCrAnListF
  ψs
  + [ofCrAnListF ψs, ofCrAnListF ψs']sF

```

lemma ofCrAnOpF_mul_ofCrAnListF_eq_superCommuteF[\[source\]](#)

```

lemma ofCrAnOpF_mul_ofCrAnListF_eq_superCommuteF (φ :  $\mathcal{F}$ .CrAnFieldOp) (ψs' : List  $\mathcal{F}$ .CrAnFieldOp
) :
  ofCrAnOpF φ * ofCrAnListF ψs' =  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi s') \cdot$  ofCrAnListF ψs' * ofCrAnOpF φ
  + [ofCrAnOpF φ, ofCrAnListF ψs']sF

```

lemma ofFieldOpListF_mul_ofFieldOpListF_eq_superCommuteF[\[source\]](#)

```

lemma ofFieldOpListF_mul_ofFieldOpListF_eq_superCommuteF (ψs ψs' : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpListF ψs * ofFieldOpListF ψs' =
   $\mathcal{S}(\mathcal{F} \mid>_s \psi s, \mathcal{F} \mid>_s \psi s') \cdot$  ofFieldOpListF ψs' * ofFieldOpListF ψs
  + [ofFieldOpListF ψs, ofFieldOpListF ψs']sF

```

lemma ofFieldOpF_mul_ofFieldOpListF_eq_superCommuteF[\[source\]](#)

```

lemma ofFieldOpF_mul_ofFieldOpListF_eq_superCommuteF (φ :  $\mathcal{F}$ .FieldOp) (ψs' : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpF φ * ofFieldOpListF ψs' =  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi s') \cdot$  ofFieldOpListF ψs' *
  ofFieldOpF φ
  + [ofFieldOpF φ, ofFieldOpListF ψs']sF

```

lemma ofFieldOpListF_mul_ofFieldOpF_eq_superCommuteF[\[source\]](#)

```

lemma ofFieldOpListF_mul_ofFieldOpF_eq_superCommuteF (ψs : List  $\mathcal{F}$ .FieldOp) (φ :  $\mathcal{F}$ .FieldOp) :

```

$$\begin{aligned} & \text{ofFieldOpListF } \varphi s * \text{ofFieldOpF } \varphi = S(\mathcal{F} \mid_{>s} \varphi s, \mathcal{F} \mid_{>s} \varphi) \cdot \text{ofFieldOpF } \varphi * \text{ofFieldOpListF } \\ & \varphi s \\ & + [\text{ofFieldOpListF } \varphi s, \text{ofFieldOpF } \varphi]_s F \end{aligned}$$
lemma crPartF_mul_anPartF_eq_superCommuteF[\[source\]](#)

lemma crPartF_mul_anPartF_eq_superCommuteF ($\varphi \varphi' : \mathcal{F}.\text{FieldOp}$) :

$$\begin{aligned} & \text{crPartF } \varphi * \text{anPartF } \varphi' = \\ & S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \text{anPartF } \varphi' * \text{crPartF } \varphi + \\ & [\text{crPartF } \varphi, \text{anPartF } \varphi']_s F \end{aligned}$$
lemma anPartF_mul_crPartF_eq_superCommuteF[\[source\]](#)

lemma anPartF_mul_crPartF_eq_superCommuteF ($\varphi \varphi' : \mathcal{F}.\text{FieldOp}$) :

$$\begin{aligned} & \text{anPartF } \varphi * \text{crPartF } \varphi' = \\ & S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \text{crPartF } \varphi' * \text{anPartF } \varphi + \\ & [\text{anPartF } \varphi, \text{crPartF } \varphi']_s F \end{aligned}$$
lemma crPartF_mul_crPartF_eq_superCommuteF[\[source\]](#)

lemma crPartF_mul_crPartF_eq_superCommuteF ($\varphi \varphi' : \mathcal{F}.\text{FieldOp}$) :

$$\begin{aligned} & \text{crPartF } \varphi * \text{crPartF } \varphi' = \\ & S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \text{crPartF } \varphi' * \text{crPartF } \varphi + \\ & [\text{crPartF } \varphi, \text{crPartF } \varphi']_s F \end{aligned}$$
lemma anPartF_mul_anPartF_eq_superCommuteF[\[source\]](#)

lemma anPartF_mul_anPartF_eq_superCommuteF ($\varphi \varphi' : \mathcal{F}.\text{FieldOp}$) :

$$\begin{aligned} & \text{anPartF } \varphi * \text{anPartF } \varphi' = S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \text{anPartF } \varphi' * \text{anPartF } \varphi + \\ & [\text{anPartF } \varphi, \text{anPartF } \varphi']_s F \end{aligned}$$
lemma ofCrAnListF_mul_ofFieldOpListF_eq_superCommuteF[\[source\]](#)

lemma ofCrAnListF_mul_ofFieldOpListF_eq_superCommuteF ($\varphi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$)
($\varphi s' : \text{List } \mathcal{F}.\text{FieldOp}$) : $\text{ofCrAnListF } \varphi s * \text{ofFieldOpListF } \varphi s' =$

$$\begin{aligned} & S(\mathcal{F} \mid_{>s} \varphi s, \mathcal{F} \mid_{>s} \varphi s') \cdot \text{ofFieldOpListF } \varphi s' * \text{ofCrAnListF } \varphi s \\ & + [\text{ofCrAnListF } \varphi s, \text{ofFieldOpListF } \varphi s']_s F \end{aligned}$$

lemma superCommuteF_ofCrAnListF_ofCrAnListF_symm[\[source\]](#)

lemma superCommuteF_ofCrAnListF_ofCrAnListF_symm ($\varphi s \varphi s' : \text{List } \mathcal{F}.\text{CrAnFieldOp}$) :

$$\begin{aligned} & [\text{ofCrAnListF } \varphi s, \text{ofCrAnListF } \varphi s']_s F = \\ & (- S(\mathcal{F} \mid_{>s} \varphi s, \mathcal{F} \mid_{>s} \varphi s')) \cdot [\text{ofCrAnListF } \varphi s', \text{ofCrAnListF } \varphi s]_s F \end{aligned}$$
lemma superCommuteF_ofCrAnOpF_ofCrAnOpF_symm[\[source\]](#)

lemma superCommuteF_ofCrAnOpF_ofCrAnOpF_symm ($\varphi \varphi' : \mathcal{F}.\text{CrAnFieldOp}$) :

$$\begin{aligned} & [\text{ofCrAnOpF } \varphi, \text{ofCrAnOpF } \varphi']_s F = \\ & (- S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi')) \cdot [\text{ofCrAnOpF } \varphi', \text{ofCrAnOpF } \varphi]_s F \end{aligned}$$

lemma superCommuteF_ofCrAnListF_ofCrAnListF_cons[\[source\]](#)

```

lemma superCommuteF_ofCrAnListF_ofCrAnListF_cons (φ :  $\mathcal{F}$ .CrAnFieldOp) (φs φs' : List  $\mathcal{F}$ .
  CrAnFieldOp) :
  [ofCrAnListF φs, ofCrAnListF (φ :: φs')]_sF =
  [ofCrAnListF φs, ofCrAnOpF φ]_sF * ofCrAnListF φs' +
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi_s, \mathcal{F} \mid_{>s} \varphi)$ 
  • ofCrAnOpF φ * [ofCrAnListF φs, ofCrAnListF φs']_sF

```

lemma superCommuteF_ofCrAnListF_ofFieldOpListF_cons[\[source\]](#)

```

lemma superCommuteF_ofCrAnListF_ofFieldOpListF_cons (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .FieldOp) : [ofCrAnListF φs, ofFieldOpListF (φ :: φs')]_sF =
  [ofCrAnListF φs, ofFieldOpF φ]_sF * ofFieldOpListF φs' +
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi_s, \mathcal{F} \mid_{>s} \varphi)$  • ofFieldOpF φ * [ofCrAnListF φs, ofFieldOpListF φs']_sF

```

lemma superCommuteF_ofCrAnListF_ofCrAnListF_eq_sum[\[source\]](#)

For a field specification $\mathcal{F}$, and two lists $\varphi_s = \varphi_0 \dots \varphi_n$ and φ_s' of $\mathcal{F}$.CrAnFieldOp the following super commutation relation holds:

$$[\varphi_s', \varphi_0 \dots \varphi_n]_sF = \sum_i \mathcal{S}(\varphi_s', \varphi_0 \dots \varphi_{i-1}) \cdot \varphi_0 \dots \varphi_{i-1} * [\varphi_s', \varphi_i]_sF * \varphi_{i+1} \dots \varphi_n$$

The proof of this relation is via induction on the length of φ_s .

```

lemma superCommuteF_ofCrAnListF_ofCrAnListF_eq_sum (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  (φs' : List  $\mathcal{F}$ .CrAnFieldOp) → [ofCrAnListF φs, ofCrAnListF φs']_sF =
   $\sum (n : \text{Fin } \varphi_s'.\text{length}), \mathcal{S}(\mathcal{F} \mid_{>s} \varphi_s, \mathcal{F} \mid_{>s} \varphi_s'.\text{take } n) \cdot$ 
  ofCrAnListF (φs'.take n) * [ofCrAnListF φs, ofCrAnOpF (φs'.get n)]_sF *
  ofCrAnListF (φs'.drop (n + 1))
| [] => by
  simp [← ofCrAnListF_nil, superCommuteF_ofCrAnListF_ofCrAnListF]
| φ :: φs' => by
  rw [superCommuteF_ofCrAnListF_ofCrAnListF_cons,
    superCommuteF_ofCrAnListF_ofCrAnListF_eq_sum φs φs']
  conv_rhs => erw [Fin.sum_univ_succ]
  congr 1
  • simp
  • simp [Finset.mul_sum, smul_smul, ofCrAnListF_cons, mul_assoc,
    FieldStatistic.ofList_cons_eq_mul, mul_comm]

```

lemma superCommuteF_ofCrAnListF_ofFieldOpListF_eq_sum[\[source\]](#)

```

lemma superCommuteF_ofCrAnListF_ofFieldOpListF_eq_sum (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  (φs' : List  $\mathcal{F}$ .FieldOp) →
  [ofCrAnListF φs, ofFieldOpListF φs']_sF =
   $\sum (n : \text{Fin } \varphi_s'.\text{length}), \mathcal{S}(\mathcal{F} \mid_{>s} \varphi_s, \mathcal{F} \mid_{>s} \varphi_s'.\text{take } n) \cdot$ 
  ofFieldOpListF (φs'.take n) * [ofCrAnListF φs, ofFieldOpF (φs'.get n)]_sF *
  ofFieldOpListF (φs'.drop (n + 1))
| [] => by
  simp only [superCommuteF_ofCrAnListF_ofFieldOpFsList, instCommGroup, ofList_empty,
    exchangeSign_bosonic, one_smul, List.length_nil, Finset.univ_eq_empty, List.take_nil,
    List.get_eq_getElem, List.drop_nil, Finset.sum_empty]
  simp
| φ :: φs' => by
  rw [superCommuteF_ofCrAnListF_ofFieldOpListF_cons,
    superCommuteF_ofCrAnListF_ofFieldOpListF_eq_sum φs φs']

```



```

conv_rhs => erw [Fin.sum_univ_succ]
congr 1
· simp
· simp [Finset.mul_sum, smul_smul, ofFieldOpListF_cons, mul_assoc,
  FieldStatistic.ofList_cons_eq_mul, mul_comm]

```

lemma summerCommute_jacobi_ofCrAnListF[\[source\]](#)

```

lemma summerCommute_jacobi_ofCrAnListF (φs1 φs2 φs3 : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnListF φs1, [ofCrAnListF φs2, ofCrAnListF φs3]sF]sF =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi s1, \mathcal{F} \mid_{>s} \phi s3) \cdot$ 
   $(-\mathcal{S}(\mathcal{F} \mid_{>s} \phi s2, \mathcal{F} \mid_{>s} \phi s3) \cdot [ofCrAnListF \phi s3, [ofCrAnListF \phi s1, ofCrAnListF \phi s2]_s F]_s F -$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi s1, \mathcal{F} \mid_{>s} \phi s2) \cdot [ofCrAnListF \phi s2, [ofCrAnListF \phi s3, ofCrAnListF \phi s1]_s F]_s F)$ 

```

lemma superCommuteF_grade[\[source\]](#)

```

lemma superCommuteF_grade {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra} {f1 f2 : FieldStatistic}
  (ha : a ∈ statisticSubmodule f1) (hb : b ∈ statisticSubmodule f2) :
  [a, b]sF ∈ statisticSubmodule (f1 + f2)

```

lemma superCommuteF_bosonic_bosonic[\[source\]](#)

```

lemma superCommuteF_bosonic_bosonic {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule bosonic) (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sF = a * b - b * a

```

lemma superCommuteF_bosonic_fermionic[\[source\]](#)

```

lemma superCommuteF_bosonic_fermionic {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule bosonic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sF = a * b - b * a

```

lemma superCommuteF_fermionic_bonsonic[\[source\]](#)

```

lemma superCommuteF_fermionic_bonsonic {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sF = a * b - b * a

```

lemma superCommuteF_bonsonic[\[source\]](#)

```

lemma superCommuteF_bonsonic {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra} (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sF = a * b - b * a

```

lemma bosonic_superCommuteF[\[source\]](#)

```

lemma bosonic_superCommuteF {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra} (ha : a ∈ statisticSubmodule bosonic) :
  [a, b]sF = a * b - b * a

```

lemma superCommuteF_bonsonic_symm[\[source\]](#)

```
lemma superCommuteF_bonsonic_symm {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sF = - [b, a]sF
```

lemma bonsonic_superCommuteF_symm[\[source\]](#)

```
lemma bonsonic_superCommuteF_symm {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule bosonic) :
  [a, b]sF = - [b, a]sF
```

lemma superCommuteF_fermionic_fermionic[\[source\]](#)

```
lemma superCommuteF_fermionic_fermionic {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sF = a * b + b * a
```

lemma superCommuteF_fermionic_fermionic_symm[\[source\]](#)

```
lemma superCommuteF_fermionic_fermionic_symm {a b :  $\mathcal{F}$ .FieldOpFreeAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sF = [b, a]sF
```

lemma superCommuteF_expand_bosonicProjF_fermionicProjF[\[source\]](#)

```
lemma superCommuteF_expand_bosonicProjF_fermionicProjF (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  [a, b]sF = bosonicProjF a * bosonicProjF b - bosonicProjF b * bosonicProjF a +
  bosonicProjF a * fermionicProjF b - fermionicProjF b * bosonicProjF a +
  fermionicProjF a * bosonicProjF b - bosonicProjF b * fermionicProjF a +
  fermionicProjF a * fermionicProjF b + fermionicProjF b * fermionicProjF a
```

lemma superCommuteF_ofCrAnListF_ofCrAnListF_bosonic_or_fermionic[\[source\]](#)

```
lemma superCommuteF_ofCrAnListF_ofCrAnListF_bosonic_or_fermionic (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp)
  :
  [ofCrAnListF φs, ofCrAnListF φs']sF ∈ statisticSubmodule bosonic v
  [ofCrAnListF φs, ofCrAnListF φs']sF ∈ statisticSubmodule fermionic
```

lemma superCommuteF_ofCrAnOpF_ofCrAnOpF_bosonic_or_fermionic[\[source\]](#)

```
lemma superCommuteF_ofCrAnOpF_ofCrAnOpF_bosonic_or_fermionic (φ φ' :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOpF φ, ofCrAnOpF φ']sF ∈ statisticSubmodule bosonic v
  [ofCrAnOpF φ, ofCrAnOpF φ']sF ∈ statisticSubmodule fermionic
```

lemma superCommuteF_superCommuteF_ofCrAnOpF_bosonic_or_fermionic[\[source\]](#)

```
lemma superCommuteF_superCommuteF_ofCrAnOpF_bosonic_or_fermionic (φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF ∈ statisticSubmodule bosonic v
  [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF ∈ statisticSubmodule fermionic
```

lemma superCommuteF_bosonic_ofCrAnListF_eq_sum[\[source\]](#)

```

lemma superCommuteF_bosonic_ofCrAnListF_eq_sum (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) ( $\varphi$ s : List  $\mathcal{F}$ .CrAnFieldOp)
  (ha : a  $\in$  statisticSubmodule bosonic) :
  [a, ofCrAnListF  $\varphi$ s] $_s$ F =  $\sum$  (n : Fin  $\varphi$ s.length),
    ofCrAnListF ( $\varphi$ s.take n) * [a, ofCrAnOpF ( $\varphi$ s.get n)] $_s$ F *
    ofCrAnListF ( $\varphi$ s.drop (n + 1))

```

lemma superCommuteF_fermionic_ofCrAnListF_eq_sum[\[source\]](#)

```

lemma superCommuteF_fermionic_ofCrAnListF_eq_sum (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
  ( $\varphi$ s : List  $\mathcal{F}$ .CrAnFieldOp) (ha : a  $\in$  statisticSubmodule fermionic) :
  [a, ofCrAnListF  $\varphi$ s] $_s$ F =  $\sum$  (n : Fin  $\varphi$ s.length),  $S$ (fermionic,  $\mathcal{F} \mid>_s \varphi$ s.take n) •
    ofCrAnListF ( $\varphi$ s.take n) * [a, ofCrAnOpF ( $\varphi$ s.get n)] $_s$ F *
    ofCrAnListF ( $\varphi$ s.drop (n + 1))

```

lemma statistic_neq_of_superCommuteF_fermionic[\[source\]](#)

```

lemma statistic_neq_of_superCommuteF_fermionic { $\varphi$ s  $\varphi$ s' : List  $\mathcal{F}$ .CrAnFieldOp}
  (h : [ofCrAnListF  $\varphi$ s, ofCrAnListF  $\varphi$ s'] $_s$ F  $\in$  statisticSubmodule fermionic) :
  ( $\mathcal{F} \mid>_s \varphi$ s)  $\neq$  ( $\mathcal{F} \mid>_s \varphi$ s')  $\vee$  [ofCrAnListF  $\varphi$ s, ofCrAnListF  $\varphi$ s'] $_s$ F = 0

```

2.37 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.TimeOrder

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrder.lean](#) on GitHub.

def timeOrderF[\[source\]](#)

For a field specification $\mathcal{F}$, *timeOrderF* is the linear map

$\text{FieldOpFreeAlgebra } \mathcal{F} \rightarrow_l[\mathbb{C}] \text{FieldOpFreeAlgebra } \mathcal{F}$

defined by its action on the basis ofCrAnListF φ s, taking ofCrAnListF φ s to

$\text{crAnTimeOrderSign } \varphi$ s • ofCrAnListF (crAnTimeOrderList φ s).

That is, *timeOrderF* time-orders the field operators and multiplies by the sign of the time order.

The notation $\mathcal{T}^f(a)$ is used for *timeOrderF* a

```

def timeOrderF : FieldOpFreeAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}] \text{FieldOpFreeAlgebra } \mathcal{F}$ 
```

lemma timeOrderF_ofCrAnListF[\[source\]](#)

```

lemma timeOrderF_ofCrAnListF ( $\varphi$ s : List  $\mathcal{F}$ .CrAnFieldOp) :
   $\mathcal{T}^f$ (ofCrAnListF  $\varphi$ s) = crAnTimeOrderSign  $\varphi$ s • ofCrAnListF (crAnTimeOrderList  $\varphi$ s)

```

lemma timeOrderF_timeOrderF_mid[\[source\]](#)

```

lemma timeOrderF_timeOrderF_mid (a b c :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
   $\mathcal{T}^f(a * b * c) = \mathcal{T}^f(a * \mathcal{T}^f(b) * c)$ 
```

lemma timeOrderF_timeOrderF_right[\[source\]](#)

lemma timeOrderF_timeOrderF_right (a b : $\mathcal{F}$.FieldOpFreeAlgebra) : $\mathcal{T}^f(a * b) = \mathcal{T}^f(a * \mathcal{T}^f(b))$

lemma timeOrderF_timeOrderF_left[\[source\]](#)

lemma timeOrderF_timeOrderF_left (a b : $\mathcal{F}$.FieldOpFreeAlgebra) : $\mathcal{T}^f(a * b) = \mathcal{T}^f(\mathcal{T}^f(a) * b)$

lemma timeOrderF_ofFieldOpListF[\[source\]](#)

lemma timeOrderF_ofFieldOpListF (ϕ s : List $\mathcal{F}$.FieldOp) :
 $\mathcal{T}^f(\text{ofFieldOpListF } \phi\text{s}) = \text{timeOrderSign } \phi\text{s} \cdot \text{ofFieldOpListF } (\text{timeOrderList } \phi\text{s})$

lemma timeOrderF_ofFieldOpListF_nil[\[source\]](#)

lemma timeOrderF_ofFieldOpListF_nil : timeOrderF ($\mathcal{F} := \mathcal{F}$) (ofFieldOpListF []) = 1

lemma timeOrderF_ofFieldOpListF_singleton[\[source\]](#)

@[simp]
lemma timeOrderF_ofFieldOpListF_singleton (ϕ : $\mathcal{F}$.FieldOp) :
 $\mathcal{T}^f(\text{ofFieldOpListF } [\phi]) = \text{ofFieldOpListF } [\phi]$

lemma timeOrderF_ofFieldOpF_ofFieldOpF_ordered[\[source\]](#)

lemma timeOrderF_ofFieldOpF_ofFieldOpF_ordered { $\phi \psi$: $\mathcal{F}$.FieldOp} (h : timeOrderRel $\phi \psi$) :
 $\mathcal{T}^f(\text{ofFieldOpF } \phi * \text{ofFieldOpF } \psi) = \text{ofFieldOpF } \phi * \text{ofFieldOpF } \psi$

lemma timeOrderF_ofFieldOpF_ofFieldOpF_not_ordered[\[source\]](#)

lemma timeOrderF_ofFieldOpF_ofFieldOpF_not_ordered { $\phi \psi$: $\mathcal{F}$.FieldOp} (h : $\neg \text{timeOrderRel } \phi \psi$) :
 $\mathcal{T}^f(\text{ofFieldOpF } \phi * \text{ofFieldOpF } \psi) = S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \psi) \cdot \text{ofFieldOpF } \psi * \text{ofFieldOpF } \phi$

lemma timeOrderF_ofFieldOpF_ofFieldOpF_not_ordered_eq_timeOrderF[\[source\]](#)

lemma timeOrderF_ofFieldOpF_ofFieldOpF_not_ordered_eq_timeOrderF { $\phi \psi$: $\mathcal{F}$.FieldOp}
(h : $\neg \text{timeOrderRel } \phi \psi$) :
 $\mathcal{T}^f(\text{ofFieldOpF } \phi * \text{ofFieldOpF } \psi) = S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \psi) \cdot \mathcal{T}^f(\text{ofFieldOpF } \psi * \text{ofFieldOpF } \phi)$

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel[\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel
{ $\phi \psi$: $\mathcal{F}$.CrAnFieldOp} (h : $\neg \text{crAnTimeOrderRel } \phi \psi$) :
 $\mathcal{T}^f([\text{ofCrAnOpF } \phi, \text{ofCrAnOpF } \psi]_s \mathcal{F}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_right[\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_right
{ $\phi \psi$: $\mathcal{F}$.CrAnFieldOp} (h : $\neg \text{crAnTimeOrderRel } \phi \psi$) (a : $\mathcal{F}$.FieldOpFreeAlgebra) :
 $\mathcal{T}^f(a * [\text{ofCrAnOpF } \phi, \text{ofCrAnOpF } \psi]_s \mathcal{F}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_left [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_left
 $\{\varphi \psi : \mathcal{F}.\text{CrAnFieldOp}\} (h : \neg \text{crAnTimeOrderRel } \varphi \psi) (a : \mathcal{F}.\text{FieldOpFreeAlgebra}) :$
 $\mathcal{T}^f([\text{ofCrAnOpF } \varphi, \text{ofCrAnOpF } \psi]_s F * a) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_mid [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_not_crAnTimeOrderRel_mid
 $\{\varphi \psi : \mathcal{F}.\text{CrAnFieldOp}\} (h : \neg \text{crAnTimeOrderRel } \varphi \psi) (a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}) :$
 $\mathcal{T}^f(a * [\text{ofCrAnOpF } \varphi, \text{ofCrAnOpF } \psi]_s F * b) = 0$

lemma timeOrderF_superCommuteF_superCommuteF_ofCrAnOpF_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_superCommuteF_ofCrAnOpF_not_crAnTimeOrderRel
 $\{\varphi_1 \varphi_2 : \mathcal{F}.\text{CrAnFieldOp}\} (h : \neg \text{crAnTimeOrderRel } \varphi_1 \varphi_2) (a : \mathcal{F}.\text{FieldOpFreeAlgebra}) :$
 $\mathcal{T}^f([a, [\text{ofCrAnOpF } \varphi_1, \text{ofCrAnOpF } \varphi_2]_s F]_s F) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_not_crAnTimeOrderRel
 $\{\varphi_1 \varphi_2 \varphi_3 : \mathcal{F}.\text{CrAnFieldOp}\} (h12 : \neg \text{crAnTimeOrderRel } \varphi_1 \varphi_2)$
 $(h13 : \neg \text{crAnTimeOrderRel } \varphi_1 \varphi_3) :$
 $\mathcal{T}^f([\text{ofCrAnOpF } \varphi_1, [\text{ofCrAnOpF } \varphi_2, \text{ofCrAnOpF } \varphi_3]_s F]_s F) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_not_crAnTimeOrderRel' [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_not_crAnTimeOrderRel'
 $\{\varphi_1 \varphi_2 \varphi_3 : \mathcal{F}.\text{CrAnFieldOp}\} (h12 : \neg \text{crAnTimeOrderRel } \varphi_2 \varphi_1)$
 $(h13 : \neg \text{crAnTimeOrderRel } \varphi_3 \varphi_1) :$
 $\mathcal{T}^f([\text{ofCrAnOpF } \varphi_1, [\text{ofCrAnOpF } \varphi_2, \text{ofCrAnOpF } \varphi_3]_s F]_s F) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_all_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_superCommuteF_all_not_crAnTimeOrderRel
 $(\varphi_1 \varphi_2 \varphi_3 : \mathcal{F}.\text{CrAnFieldOp}) (h : \neg$
 $(\text{crAnTimeOrderRel } \varphi_1 \varphi_2 \wedge \text{crAnTimeOrderRel } \varphi_1 \varphi_3 \wedge$
 $\text{crAnTimeOrderRel } \varphi_2 \varphi_1 \wedge \text{crAnTimeOrderRel } \varphi_2 \varphi_3 \wedge$
 $\text{crAnTimeOrderRel } \varphi_3 \varphi_1 \wedge \text{crAnTimeOrderRel } \varphi_3 \varphi_2)) :$
 $\mathcal{T}^f([\text{ofCrAnOpF } \varphi_1, [\text{ofCrAnOpF } \varphi_2, \text{ofCrAnOpF } \varphi_3]_s F]_s F) = 0$

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_eq_time [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnOpF_ofCrAnOpF_eq_time
 $\{\varphi \psi : \mathcal{F}.\text{CrAnFieldOp}\} (h1 : \text{crAnTimeOrderRel } \varphi \psi) (h2 : \text{crAnTimeOrderRel } \psi \varphi) :$
 $\mathcal{T}^f([\text{ofCrAnOpF } \varphi, \text{ofCrAnOpF } \psi]_s F) = [\text{ofCrAnOpF } \varphi, \text{ofCrAnOpF } \psi]_s F$

lemma timeOrderF_eq_maxTimeField_mul [\[source\]](#)

*In the state algebra time, ordering obeys $T(\varphi \circ \varphi_1 \dots \varphi_n) = s * \varphi_i * T(\varphi \circ \varphi_1 \dots \varphi_{i-1} \varphi_{i+1} \dots \varphi_n)$ where φ_i is the state which has maximum time and s is the exchange sign of φ_i and $\varphi \circ \varphi_1 \dots \varphi_{i-1}$.*

```

lemma timeOrderF_eq_maxTimeField_mul (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{T}^f(\text{ofFieldOpListF } (\phi :: \psis)) =$ 
   $S(\mathcal{F} \mid_{>s} \text{maxTimeField } \phi \ \psis, \mathcal{F} \mid_{>s} (\phi :: \psis).take \ (\text{maxTimeFieldPos } \phi \ \psis)) \cdot$ 
   $\text{ofFieldOpF } (\text{maxTimeField } \phi \ \psis) * \mathcal{T}^f(\text{ofFieldOpListF } (\text{eraseMaxTimeField } \phi \ \psis))$ 

```

lemma timeOrderF_eq_maxTimeField_mul_finset

[source]

*In the state algebra time, ordering obeys $T(\phi \circ \phi_1 \dots \phi_n) = s * \phi_i * T(\phi \circ \phi_1 \dots \phi_{i-1} \phi_{i+1} \dots \phi_n)$ where ϕ_i is the state which has maximum time and s is the exchange sign of ϕ_i and $\phi \circ \phi_1 \dots \phi_{i-1}$. Here s is written using finite sets.*

```

lemma timeOrderF_eq_maxTimeField_mul_finset (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{T}^f(\text{ofFieldOpListF } (\phi :: \psis)) = S(\mathcal{F} \mid_{>s} \text{maxTimeField } \phi \ \psis, \mathcal{F} \mid_{>s} ((\text{eraseMaxTimeField } \phi \ \psis)$ 
   $\cdot \text{get},$ 
   $(\text{Finset.filter } (\text{fun } x \Rightarrow$ 
     $(\text{maxTimeFieldPosFin } \phi \ \psis).succAbove \ x < \text{maxTimeFieldPosFin } \phi \ \psis) \text{ Finset.univ}))) \cdot$ 
   $\text{ofFieldOpF } (\text{maxTimeField } \phi \ \psis) * \mathcal{T}^f(\text{ofFieldOpListF } (\text{eraseMaxTimeField } \phi \ \psis))$ 

```

2.38 PhysLean.QFT.PerturbationTheory.FieldSpecification.Basic

PhysLean/QFT/PerturbationTheory/FieldSpecification/Basic.lean on GitHub.

inductive FieldOp

[source]

For a field specification $\mathcal{F}$, the inductive type $\mathcal{F}$.FieldOp is defined to contain the following elements:

- *For every f in $\mathcal{F}$.Field, element of e of AsymptoticLabel f and 3-momentum p , an element labelled $\text{inAsymp } f \ e \ p$ corresponding to an incoming asymptotic field operator of the field f , of label e (e.g. specifying the spin), and momentum p .*
- *For every f in $\mathcal{F}$.Field, element of e of PositionLabel f and space-time position x , an element labelled $\text{position } f \ e \ x$ corresponding to a position field operator of the field f , of label e (e.g. specifying the Lorentz index), and position x .*
- *For every f in $\mathcal{F}$.Field, element of e of AsymptoticLabel f and 3-momentum p , an element labelled $\text{outAsymp } f \ e \ p$ corresponding to an outgoing asymptotic field operator of the field f , of label e (e.g. specifying the spin), and momentum p .*

As an example, if f corresponds to a Weyl-fermion field, then

- *For $\text{inAsymp } f \ e \ p$, e would correspond to a spin s , and $\text{inAsymp } f \ e \ p$ would, once represented in the operator algebra, be proportional to the creation operator $a(p, s)$.*
- *position $f \ e \ x$, e would correspond to a Lorentz index a , and position $f \ e \ x$ would, once represented in the operator algebra, be proportional to the operator*

$$\sum s, \int d^3p / (\dots) (x_a(p, s) a(p, s) e^{(-i p x)} + y_a(p, s) a^\dagger(p, s) e^{(-i p x)}).$$

- *outAsymp $f \ e \ p$, e would correspond to a spin s , and $\text{outAsymp } f \ e \ p$ would, once represented in the operator algebra, be proportional to the annihilation operator $a^\dagger(p, s)$.*

```

inductive FieldOp ( $\mathcal{F}$  : FieldSpecification) where
  | inAsymp : ( $\Sigma f, \mathcal{F}$ .AsymptoticLabel  $f$ )  $\times$  ( $\text{Fin } 3 \rightarrow \mathbb{R}$ )  $\rightarrow \mathcal{F}$ .FieldOp
  | position : ( $\Sigma f, \mathcal{F}$ .PositionLabel  $f$ )  $\times$  SpaceTime  $\rightarrow \mathcal{F}$ .FieldOp
  | outAsymp : ( $\Sigma f, \mathcal{F}$ .AsymptoticLabel  $f$ )  $\times$  ( $\text{Fin } 3 \rightarrow \mathbb{R}$ )  $\rightarrow \mathcal{F}$ .FieldOp

```

def fieldOpToField

[source]

For a field specification $\mathcal{F}$, $\mathcal{F}.fieldOpToField$ is defined to take field operators to their underlying field.

```
def fieldOpToField :  $\mathcal{F}.FieldOp \rightarrow \mathcal{F}.Field$ 
| FieldOp.inAsymp (f, _) => f.1
| FieldOp.position (f, _) => f.1
| FieldOp.outAsymp (f, _) => f.1
```

def fieldOpStatistic

[\[source\]](#)

For a field specification $\mathcal{F}$, and an element ϕ of $\mathcal{F}.FieldOp$. The field statistic $fieldOpStatistic \phi$ is defined to be the statistic associated with the field underlying ϕ .

The following notation is used in relation to $fieldOpStatistic$: - For ϕ an element of $\mathcal{F}.FieldOp$, $\mathcal{F} \mid_{>s} \phi$ is $fieldOpStatistic \phi$. - For ϕs a list of $\mathcal{F}.FieldOp$, $\mathcal{F} \mid_{>s} \phi s$ is the product of $fieldOpStatistic \phi$ over the list ϕs . - For a function $f : Fin\ n \rightarrow \mathcal{F}.FieldOp$ and a finite set a of $Fin\ n$, $\mathcal{F} \mid_{>s} (f, a)$ is the product of $fieldOpStatistic (f\ i)$ for all $i \in a$.

```
def fieldOpStatistic :  $\mathcal{F}.FieldOp \rightarrow FieldStatistic$ 
```

2.39 PhysLean.QFT.PerturbationTheory.FieldSpecification.CrAnFieldOp

[PhysLean/QFT/PerturbationTheory/FieldSpecification/CrAnFieldOp.lean](#) on GitHub.

def fieldOpToCrAnType

[\[source\]](#)

To each field operator the specification of the type of creation and annihilation parts. For asymptotic states there is only one allowed part, whilst for position field operator there is two.

```
def fieldOpToCrAnType :  $\mathcal{F}.FieldOp \rightarrow Type$ 
| FieldOp.inAsymp _ => Unit
| FieldOp.position _ => CreateAnnihilate
| FieldOp.outAsymp _ => Unit
```

instance $\forall i$, Fintype ($\mathcal{F}.fieldOpToCrAnType\ i$)

[\[source\]](#)

The instance of a finite type on $\mathcal{F}.fieldOpToCreateAnnihilateType\ i$.

```
instance :  $\forall i$ , Fintype ( $\mathcal{F}.fieldOpToCrAnType\ i$ )
```

instance $\forall i$, DecidableEq ($\mathcal{F}.fieldOpToCrAnType\ i$)

[\[source\]](#)

The instance of a decidable equality on $\mathcal{F}.fieldOpToCreateAnnihilateType\ i$.

```
instance :  $\forall i$ , DecidableEq ( $\mathcal{F}.fieldOpToCrAnType\ i$ )
```

def fieldOpToCreateAnnihilateTypeCongr

[\[source\]](#)

The equivalence between $\mathcal{F}.fieldOpToCreateAnnihilateType\ i$ and $\mathcal{F}.fieldOpToCreateAnnihilateType\ j$ from an equality $i = j$.

```
def fieldOpToCreateAnnihilateTypeCongr : {i j :  $\mathcal{F}$ .FieldOp} → i = j →
   $\mathcal{F}$ .fieldOpToCrAnType i ≈  $\mathcal{F}$ .fieldOpToCrAnType j
| _, _, rfl => Equiv.refl _
```

def CrAnFieldOp

[\[source\]](#)

For a field specification $\mathcal{F}$, the (sigma) type $\mathcal{F}$.CrAnFieldOp corresponds to the type of creation and annihilation parts of field operators. It formally defined to consist of the following elements: - For each incoming asymptotic field operator ϕ in $\mathcal{F}$.FieldOp an element written as $\langle \phi, () \rangle$ in $\mathcal{F}$.CrAnFieldOp, corresponding to the creation part of ϕ . Here ϕ has no annihilation part. (Here $()$ is the unique element of Unit.) - For each position field operator ϕ in $\mathcal{F}$.FieldOp an element of $\mathcal{F}$.CrAnFieldOp written as $\langle \phi, .create \rangle$, corresponding to the creation part of ϕ . - For each position field operator ϕ in $\mathcal{F}$.FieldOp an element of $\mathcal{F}$.CrAnFieldOp written as $\langle \phi, .annihilate \rangle$, corresponding to the annihilation part of ϕ . - For each outgoing asymptotic field operator ϕ in $\mathcal{F}$.FieldOp an element written as $\langle \phi, () \rangle$ in $\mathcal{F}$.CrAnFieldOp, corresponding to the annihilation part of ϕ . Here ϕ has no creation part. (Here $()$ is the unique element of Unit.)

As an example, if f corresponds to a Weyl-fermion field, it would contribute the following elements to $\mathcal{F}$.CrAnFieldOp - For each spin s , an element corresponding to an incoming asymptotic operator: $a(p, s)$. - For each Lorentz index a , an element corresponding to the creation part of a position operator:

$\sum s, \int d^3p/(...) (x_a(p, s) a(p, s) e^{(-i p x)})$. - For each Lorentz index a , an element corresponding to annihilation part of a position operator:

$\sum s, \int d^3p/(...) (y_a(p, s) a^\dagger(p, s) e^{(-i p x)})$. - For each spin s , element corresponding to an outgoing asymptotic operator: $a^\dagger(p, s)$.

```
def CrAnFieldOp : Type
```

def crAnFieldOpToFieldOp

[\[source\]](#)

The map from creation and annihilation field operator to their underlying states.

```
def crAnFieldOpToFieldOp :  $\mathcal{F}$ .CrAnFieldOp →  $\mathcal{F}$ .FieldOp
```

lemma crAnFieldOpToFieldOp_prod

[\[source\]](#)

```
@[simp]
```

```
lemma crAnFieldOpToFieldOp_prod (s :  $\mathcal{F}$ .FieldOp) (t :  $\mathcal{F}$ .fieldOpToCrAnType s) :
   $\mathcal{F}$ .crAnFieldOpToFieldOp {s, t} = s
```

def crAnFieldOpToCreateAnnihilate

[\[source\]](#)

For a field specification $\mathcal{F}$, $\mathcal{F}$.crAnFieldOpToCreateAnnihilate is the map from $\mathcal{F}$.CrAnFieldOp to CreateAnnihilate taking ϕ to create if ϕ corresponds to an incoming asymptotic field operator or the creation part of a position based field operator.

otherwise it takes ϕ to annihilate.

```
def crAnFieldOpToCreateAnnihilate :  $\mathcal{F}$ .CrAnFieldOp → CreateAnnihilate
| {FieldOp.inAsymp _, _} => CreateAnnihilate.create
```



```

| (FieldOp.position _, CreateAnnihilate.create) => CreateAnnihilate.create
| (FieldOp.position _, CreateAnnihilate.annihilate) => CreateAnnihilate.annihilate
| (FieldOp.outAsymp _, _) => CreateAnnihilate.annihilate

```

2.40 PhysLean.QFT.PerturbationTheory.FieldSpecification.NormalOrder

[PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean](#) on GitHub.

instance IsTotal $\mathcal{F}.\text{CrAnFieldOp } \mathcal{F}.\text{normalOrderRel}$

[\[source\]](#)

Normal ordering is total.

instance : IsTotal $\mathcal{F}.\text{CrAnFieldOp } \mathcal{F}.\text{normalOrderRel}$ **where**

instance IsTrans $\mathcal{F}.\text{CrAnFieldOp } \mathcal{F}.\text{normalOrderRel}$

[\[source\]](#)

Normal ordering is transitive.

instance : IsTrans $\mathcal{F}.\text{CrAnFieldOp } \mathcal{F}.\text{normalOrderRel}$ **where**

instance Decidable (normalOrderRel $\varphi \varphi'$)

[\[source\]](#)

A decidable instance on the normal ordering relation.

instance ($\varphi \varphi' : \mathcal{F}.\text{CrAnFieldOp}$) : Decidable (normalOrderRel $\varphi \varphi'$)

lemma normalOrderSign_append_annihilate

[\[source\]](#)

```

lemma normalOrderSign_append_annihilate ( $\varphi : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $h\varphi : \mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ ) :
  ( $\varphi s : \text{List } \mathcal{F}.\text{CrAnFieldOp} \rightarrow$ 
    normalOrderSign ( $\varphi s ++ [\varphi]$ ) = normalOrderSign  $\varphi s$ 
  | [] => by simp
  |  $\varphi' :: \varphi s \Rightarrow$  by
    dsimp only [List.cons_append, normalOrderSign, Wick.koszulSign]
    have hi

```

lemma normalOrderSign_swap_create_annihilate_fst

[\[source\]](#)

```

lemma normalOrderSign_swap_create_annihilate_fst ( $\varphi c \varphi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $h\varphi c : \mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ )
  ( $h\varphi a : \mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  ( $\varphi s : \text{List } \mathcal{F}.\text{CrAnFieldOp} :$ 
    normalOrderSign ( $\varphi c :: \varphi a :: \varphi s$ ) =
    FieldStatistic.exchangeSign ( $\mathcal{F}.\text{crAnStatistics } \varphi c$ ) ( $\mathcal{F}.\text{crAnStatistics } \varphi a$ ) *
    normalOrderSign ( $\varphi a :: \varphi c :: \varphi s$ )

```

lemma normalOrderSign_swap_create_annihilate

[\[source\]](#)

```

lemma normalOrderSign_swap_create_annihilate ( $\varphi c \varphi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $h\varphi c : \mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) ( $h\varphi a : \mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  :
  ( $\varphi s \varphi s' : \text{List } \mathcal{F}.\text{CrAnFieldOp} \rightarrow$  normalOrderSign ( $\varphi s ++ \varphi c :: \varphi a :: \varphi s'$ ) =
    FieldStatistic.exchangeSign ( $\mathcal{F}.\text{crAnStatistics } \varphi c$ ) ( $\mathcal{F}.\text{crAnStatistics } \varphi a$ ) *

```

```

normalOrderSign (φs ++ φa :: φc :: φs')
| [], φs' => normalOrderSign_swap_create_annihilate_fst φc φa hφc hφa φs'
| φ :: φs, φs' => by
  rw [normalOrderSign]
  dsimp only [List.cons_append, Wick.koszulSign, FieldStatistic.instCommGroup.eq_1]
  rw [← normalOrderSign, normalOrderSign_swap_create_annihilate φc φa hφc hφa φs φs']
  rw [← mul_assoc, mul_comm _ (FieldStatistic.exchangeSign _), mul_assoc]
  simp only [FieldStatistic.instCommGroup.eq_1, mul_eq_mul_left_iff]
  apply Or.inl
  conv_rhs => rw [normalOrderSign, Wick.koszulSign, ← normalOrderSign]
  simp only [mul_eq_mul_right_iff]
  left
  rw [koszulSignInsert_swap]

```

lemma normalOrder_swap_create_annihilate_fst

[\[source\]](#)

```

lemma normalOrder_swap_create_annihilate_fst (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>^c} \phi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>^c} \phi a = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  normalOrderList (φc :: φa :: φs) = normalOrderList (φa :: φc :: φs)

```

lemma normalOrderList_swap_create_annihilate

[\[source\]](#)

```

lemma normalOrderList_swap_create_annihilate (φc φa :  $\mathcal{F}$ .CrAnFieldOp)
  (hφc :  $\mathcal{F} \mid^{>^c} \phi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>^c} \phi a = \text{CreateAnnihilate.annihilate}$ ) :
  (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) →
  normalOrderList (φs ++ φc :: φa :: φs') = normalOrderList (φs ++ φa :: φc :: φs')
| [], φs' => normalOrder_swap_create_annihilate_fst φc φa hφc hφa φs'
| φ :: φs, φs' => by
  dsimp only [List.cons_append, normalOrderList, List.insertionSort]
  have hi

```

2.41 PhysLean.QFT.PerturbationTheory.FieldSpecification.TimeOrder

[PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean](#) on GitHub.

instance (φ φ' : $\mathcal{F}$.FieldOp) → Decidable (timeOrderRel φ φ') | FieldOp.outAsymp... [\[source\]](#)

The relation timeOrderRel is decidable, but not computable so due to Real.decidableLE.

```

noncomputable instance : (φ φ' :  $\mathcal{F}$ .FieldOp) → Decidable (timeOrderRel φ φ')
| FieldOp.outAsymp _, _ => isTrue True.intro
| FieldOp.position φ0, FieldOp.position φ1 => inferInstanceAs (Decidable (φ1.2 0 ≤ φ0.2 0))
| FieldOp.position _, FieldOp.inAsymp _ => isTrue True.intro
| FieldOp.position _, FieldOp.outAsymp _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.outAsymp _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.position _ => isFalse (fun a => a)
| FieldOp.inAsymp _, FieldOp.inAsymp _ => isTrue True.intro

```

instance IsTotal $\mathcal{F}$.FieldOp $\mathcal{F}$.timeOrderRel

[\[source\]](#)

Time ordering is total.

```
instance : IsTotal  $\mathcal{F}$ .FieldOp  $\mathcal{F}$ .timeOrderRel where
```

```
instance IsTrans  $\mathcal{F}$ .FieldOp  $\mathcal{F}$ .timeOrderRel
```

[\[source\]](#)

Time ordering is transitive.

```
instance : IsTrans  $\mathcal{F}$ .FieldOp  $\mathcal{F}$ .timeOrderRel where
```

```
instance Decidable (crAnTimeOrderRel  $\phi$   $\phi'$ )
```

[\[source\]](#)

The relation crAnTimeOrderRel is decidable, but not computable so due to Real.decidableLE.

```
noncomputable instance ( $\phi$   $\phi'$  :  $\mathcal{F}$ .CrAnFieldOp) : Decidable (crAnTimeOrderRel  $\phi$   $\phi'$ )
```

```
instance IsTotal  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .crAnTimeOrderRel
```

[\[source\]](#)

Time ordering of CrAnFieldOp is total.

```
instance : IsTotal  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .crAnTimeOrderRel where
```

```
instance IsTrans  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .crAnTimeOrderRel
```

[\[source\]](#)

Time ordering of CrAnFieldOp is transitive.

```
instance : IsTrans  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .crAnTimeOrderRel where
```

```
instance Decidable (normTimeOrderRel  $\phi$   $\phi'$ )
```

[\[source\]](#)

The relation normTimeOrderRel is decidable, but not computable so due to Real.decidableLE.

```
noncomputable instance ( $\phi$   $\phi'$  :  $\mathcal{F}$ .CrAnFieldOp) : Decidable (normTimeOrderRel  $\phi$   $\phi'$ )
```

```
instance IsTotal  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .normTimeOrderRel
```

[\[source\]](#)

Norm-Time ordering of CrAnFieldOp is total.

```
instance : IsTotal  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .normTimeOrderRel where
```

```
instance IsTrans  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .normTimeOrderRel
```

[\[source\]](#)

Norm-Time ordering of CrAnFieldOp is transitive.

```
instance : IsTrans  $\mathcal{F}$ .CrAnFieldOp  $\mathcal{F}$ .normTimeOrderRel where
```

2.42 PhysLean.QFT.PerturbationTheory.FieldStatistics.OfFinset

[PhysLean/QFT/PerturbationTheory/FieldStatistics/OfFinset.lean](#) on GitHub.

```
lemma ofFinset_filter_mul_neg
```

[\[source\]](#)

```
lemma ofFinset_filter_mul_neg (q :  $\mathcal{F}$  → FieldStatistic) ( $\phi$ s : List  $\mathcal{F}$ ) (a : Finset (Fin  $\phi$ s.length))
  (p : Fin  $\phi$ s.length → Prop) [DecidablePred p] :
```

```

offFinset q φs.get (Finset.filter p a) *
offFinset q φs.get (Finset.filter (fun i => ¬ p i) a) = offFinset q φs.get a

```

lemma offFinset_filter[\[source\]](#)

```

lemma offFinset_filter (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) (φs : List  $\mathcal{F}$ ) (a : Finset (Fin φs.length))
  (p : Fin φs.length → Prop) [DecidablePred p] :
  offFinset q φs.get (Finset.filter p a) = offFinset q φs.get (Finset.filter (fun i => ¬ p i) a) *
  offFinset q φs.get a

```

2.43 PhysLean.QFT.PerturbationTheory.WickContraction.Basic

[PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean](#) on GitHub.

instance DecidableEq (WickContraction n)[\[source\]](#)

Wick contractions are decidable.

```

instance : DecidableEq (WickContraction n)

```

lemma card_zero_iff_empty[\[source\]](#)

```

lemma card_zero_iff_empty (c : WickContraction n) : c.l.card = 0 ↔ c = empty

```

lemma exists_pair_of_not_eq_empty[\[source\]](#)

```

lemma exists_pair_of_not_eq_empty (c : WickContraction n) (h : c ≠ empty) :
  ∃ i j, {i, j} ∈ c.l

```

lemma card_congr[\[source\]](#)

```

@[simp]
lemma card_congr {n m : ℕ} (h : n = m) (c : WickContraction n) :
  (congr h c).l.card = c.l.card

```

lemma mem_congr_iff[\[source\]](#)

```

lemma mem_congr_iff {n m : ℕ} (h : n = m) {c : WickContraction n} {a : Finset (Fin m)} :
  a ∈ (congr h c).l ↔ Finset.map (finCongr h.symm).toEmbedding a ∈ c.l

```

def congrLiftInv[\[source\]](#)

Given a contracted pair in $c : \text{WickContraction } n$ the contracted pair in $\text{congr } h \ c$.

```

def congrLiftInv {n m : ℕ} (h : n = m) {c : WickContraction n} (a : (congr h c).l) : c.l

```

lemma congrLiftInv_refl[\[source\]](#)

```

lemma congrLiftInv_refl {n : ℕ} {c : WickContraction n} :
  c.congrLiftInv rfl = id

```

lemma gradingCompliant_congr[\[source\]](#)

```

lemma gradingCompliant_congr { $\phi s \phi s'$  : List  $\mathcal{F}$ .FieldOp} (h :  $\phi s = \phi s'$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  GradingCompliant  $\phi s \phi s \Lambda \leftrightarrow$  GradingCompliant  $\phi s'$  (congr (by simp [h])  $\phi s \Lambda$ )

```

2.44 PhysLean.QFT.PerturbationTheory.WickContraction.Card

[PhysLean/QFT/PerturbationTheory/WickContraction/Card.lean](#) on GitHub.

lemma wickContraction_card_eq_sum_zero_none_isSome[\[source\]](#)

```

lemma wickContraction_card_eq_sum_zero_none_isSome : Fintype.card (WickContraction n.succ)
  = Fintype.card {c : WickContraction n.succ //  $\neg$  (c.getDual? 0).isSome} +
  Fintype.card {c : WickContraction n.succ // (c.getDual? 0).isSome}

```

lemma wickContraction_zero_none_card[\[source\]](#)

```

lemma wickContraction_zero_none_card :
  Fintype.card {c : WickContraction n.succ //  $\neg$  (c.getDual? 0).isSome} =
  Fintype.card (WickContraction n)

```

lemma wickContraction_zero_some_eq_sum[\[source\]](#)

```

lemma wickContraction_zero_some_eq_sum :
  Fintype.card {c : WickContraction n.succ // (c.getDual? 0).isSome} =
   $\sum i$ , Fintype.card {c : WickContraction n.succ // (c.getDual? 0).isSome  $\wedge$ 
     $\forall$  (h : (c.getDual? 0).isSome), (c.getDual? 0).get h = Fin.succ i}

```

lemma finset_succAbove_succ_disjoint[\[source\]](#)

```

lemma finset_succAbove_succ_disjoint (a : Finset (Fin n)) (i : Fin n.succ) :
  Disjoint ((Finset.map (Fin.succEmb (n + 1))) ((Finset.map i.succAboveEmb) a)) {0, i.succ}

```

def consAddContract[\[source\]](#)

The Wick contraction in $WickContraction\ n.succ.succ$ formed by a Wick contraction $WickContraction\ n$ by inserting at the 0 and $i.succ$ and contracting these two.

```

def consAddContract (i : Fin n.succ) (c : WickContraction n) :
  WickContraction n.succ.succ

```

lemma consAddContract_getDual?_zero[\[source\]](#)

```

@[simp]
lemma consAddContract_getDual?_zero (i : Fin n.succ) (c : WickContraction n) :
  (consAddContract i c).getDual? 0 = some i.succ

```

lemma consAddContract_getDual?_self_succ[\[source\]](#)

```

@[simp]
lemma consAddContract_getDual?_self_succ (i : Fin n.succ) (c : WickContraction n) :
  (consAddContract i c).getDual? i.succ = some 0

```

lemma mem_consAddContract_of_mem_iff[\[source\]](#)

```
lemma mem_consAddContract_of_mem_iff (i : Fin n.succ) (c : WickContraction n) (a : Finset (Fin n)) :
  a ∈ c.1 ↔ (a.map i.succAboveEmb).map (Fin.succEmb n.succ) ∈ (consAddContract i c).1
```

lemma consAddContract_injective[\[source\]](#)

```
lemma consAddContract_injective (i : Fin n.succ) : Function.Injective (consAddContract i)
```

lemma consAddContract_surjective_on_zero_contract[\[source\]](#)

```
lemma consAddContract_surjective_on_zero_contract (i : Fin n.succ)
  (c : WickContraction n.succ.succ)
  (h : (c.getDual? 0).isSome) (h2 : (c.getDual? 0).get h = i.succ) :
  ∃ c', consAddContract i c' = c
```

lemma consAddContract_bijection[\[source\]](#)

```
lemma consAddContract_bijection (i : Fin n.succ) :
  Function.Bijective (fun c => ((consAddContract i c), by simp) :
    {c : WickContraction n.succ.succ // (c.getDual? 0).isSome} →
    {c : WickContraction n.succ.succ // (c.getDual? 0).isSome})
```

lemma wickContraction_zero_some_eq_mul[\[source\]](#)

```
lemma wickContraction_zero_some_eq_mul :
  Fintype.card {c : WickContraction n.succ.succ // (c.getDual? 0).isSome} =
  (n + 1) * Fintype.card (WickContraction n)
```

def cardFun[\[source\]](#)

The cardinality of Wick's contractions as a recursive formula. This corresponds to OEIS:A000085.

```
def cardFun : ℕ → ℕ
| 0 => 1
| 1 => 1
| Nat.succ (Nat.succ n) => cardFun (Nat.succ n) + (n + 1) * cardFun n
```

theorem card_eq_cardFun[\[source\]](#)

The number of Wick contractions in WickContraction n is equal to the terms in Online Encyclopedia of Integer Sequences (OEIS) A000085. That is: 1, 1, 2, 4, 10, 26, 76, 232, 764, 2620, 9496, ...

```
theorem card_eq_cardFun : (n : ℕ) → Fintype.card (WickContraction n) = cardFun n
| 0 => by decide
| 1 => by decide
| Nat.succ (Nat.succ n) => by
  rw [wickContraction_card_eq_sum_zero_none_isSome, wickContraction_zero_none_card,
    wickContraction_zero_some_eq_mul]
  simp only [cardFun, succ_eq_add_one]
  rw [← card_eq_cardFun n, ← card_eq_cardFun (n + 1)]
```

2.45 PhysLean.QFT.PerturbationTheory.WickContraction.ExtractEquiv

[PhysLean/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean](#) on GitHub.

lemma mem_three

[\[source\]](#)

For $n = 3$ there are 4 possible Wick contractions:

- $\emptyset$, corresponding to the case where no fields are contracted.
- $\{\{0, 1\}\}$, corresponding to the case where the field at position 0 and 1 are contracted.
- $\{\{0, 2\}\}$, corresponding to the case where the field at position 0 and 2 are contracted.
- $\{\{1, 2\}\}$, corresponding to the case where the field at position 1 and 2 are contracted.

The proof of this result uses the fact that Lean is an executable programming language and can calculate all Wick contractions for a given n .

```
lemma mem_three (c : WickContraction 3) : c.l ∈ ({ $\emptyset$ ,  $\{\{0, 1\}\}$ ,  $\{\{0, 2\}\}$ ,  $\{\{1, 2\}\}$ } :
  Finset (Finset (Finset (Fin 3))))
```

lemma mem_four

[\[source\]](#)

For $n = 4$ there are 10 possible Wick contractions including e.g.

- $\emptyset$, corresponding to the case where no fields are contracted.
- $\{\{0, 1\}, \{2, 3\}\}$, corresponding to the case where the fields at position 0 and 1 are contracted, and the fields at position 2 and 3 are contracted.
- $\{\{0, 2\}, \{1, 3\}\}$, corresponding to the case where the fields at position 0 and 2 are contracted, and the fields at position 1 and 3 are contracted.

The proof of this result uses the fact that Lean is an executable programming language and can calculate all Wick contractions for a given n .

```
lemma mem_four (c : WickContraction 4) : c.l ∈ ({ $\emptyset$ ,
   $\{\{0, 1\}\}$ ,  $\{\{0, 2\}\}$ ,  $\{\{0, 3\}\}$ ,  $\{\{1, 2\}\}$ ,  $\{\{1, 3\}\}$ ,  $\{\{2, 3\}\}$ ,
   $\{\{0, 1\}, \{2, 3\}\}$ ,  $\{\{0, 2\}, \{1, 3\}\}$ ,  $\{\{0, 3\}, \{1, 2\}\}$ } :
  Finset (Finset (Finset (Fin 4))))
```

2.46 PhysLean.QFT.PerturbationTheory.WickContraction.InsertAndContractNat

[PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.lean](#) on GitHub.

lemma insertAndContractNat_none_getDual?_eq_none

[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_none_getDual?_eq_none (c : WickContraction n) (i : Fin n.succ) :
  (insertAndContractNat c i none).getDual? i = none
```

lemma insertAndContractNat_injective

[\[source\]](#)

```
lemma insertAndContractNat_injective (i : Fin n.succ) :
  Function.Injective (fun c => insertAndContractNat c i none)
```

lemma insertAndContractNat_surjective_on_nodual

[\[source\]](#)

```

lemma insertAndContractNat_surjective_on_nodual (i : Fin n.succ)
  (c : WickContraction n.succ) (hc : c.getDual? i = none) :
  ∃ c', insertAndContractNat c' i none = c

```

lemma insertAndContractNat_bijective

[\[source\]](#)

```

lemma insertAndContractNat_bijective (i : Fin n.succ) :
  Function.Bijective (fun c => ({insertAndContractNat c i none, by simp} :
    {c : WickContraction n.succ // c.getDual? i = none}))

```

2.47 PhysLean.QFT.PerturbationTheory.WickContraction.Join

[PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean](#) on GitHub.

def join

[\[source\]](#)

Given a list ϕs of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s\Lambda$ of ϕs and a Wick contraction $\phi suc\Lambda$ of $[\phi s\Lambda]^{uc}$, $join \ \phi s\Lambda \ \phi suc\Lambda$ is defined as the Wick contraction of ϕs consisting of the contractions in $\phi s\Lambda$ and those in $\phi suc\Lambda$.

As an example, for $\phi s = [\phi 1, \phi 2, \phi 3, \phi 4]$, $\phi s\Lambda = \{\{0, 1\}\}$ corresponding to the contraction of $\phi 1$ and $\phi 2$ in ϕs and $\phi suc\Lambda = \{\{0, 1\}\}$ corresponding to the contraction of $\phi 3$ and $\phi 4$ in $[\phi s\Lambda]^{uc} = [\phi 3, \phi 4]$, then $join \ \phi s\Lambda \ \phi suc\Lambda$ is the contraction $\{\{0, 1\}, \{2, 3\}\}$ of ϕs .

```

def join { $\phi s$  : List  $\mathcal{F}.FieldOp$ } ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ )
  ( $\phi suc\Lambda$  : WickContraction  $[\phi s\Lambda]^{uc}.length$ ) : WickContraction  $\phi s.length$ 

```

lemma join_congr

[\[source\]](#)

```

lemma join_congr { $\phi s$  : List  $\mathcal{F}.FieldOp$ } { $\phi s\Lambda$  : WickContraction  $\phi s.length$ }
  { $\phi suc\Lambda$  : WickContraction  $[\phi s\Lambda]^{uc}.length$ } { $\phi s\Lambda'$  : WickContraction  $\phi s.length$ }
  (h1 :  $\phi s\Lambda = \phi s\Lambda'$ ) :
  join  $\phi s\Lambda \ \phi suc\Lambda = join \ \phi s\Lambda' \ (congr \ (by \ simp \ [h1]) \ \phi suc\Lambda)$ 

```

def joinLiftLeft

[\[source\]](#)

Given a contracting pair within $\phi s\Lambda$ the corresponding contracting pair within $(join \ \phi s\Lambda \ \phi suc\Lambda)$.

```

def joinLiftLeft { $\phi s$  : List  $\mathcal{F}.FieldOp$ } { $\phi s\Lambda$  : WickContraction  $\phi s.length$ }
  { $\phi suc\Lambda$  : WickContraction  $[\phi s\Lambda]^{uc}.length$ } :  $\phi s\Lambda.1 \rightarrow (join \ \phi s\Lambda \ \phi suc\Lambda).1$ 

```

lemma jointLiftLeft_injective

[\[source\]](#)

```

lemma jointLiftLeft_injective { $\phi s$  : List  $\mathcal{F}.FieldOp$ } { $\phi s\Lambda$  : WickContraction  $\phi s.length$ }
  { $\phi suc\Lambda$  : WickContraction  $[\phi s\Lambda]^{uc}.length$ } :
  Function.Injective (@joinLiftLeft _ _  $\phi s\Lambda \ \phi suc\Lambda$ )

```

def joinLiftRight

[\[source\]](#)

Given a contracting pair within $\phi suc\Lambda$ the corresponding contracting pair within $(join \ \phi s\Lambda \ \phi suc\Lambda)$.


```
def joinLiftRight {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} : φsucΛ.1 → (join φsΛ φsucΛ).1
```

lemma joinLiftRight_injective

[\[source\]](#)

```
lemma joinLiftRight_injective {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} :
  Function.Injective (@joinLiftRight _ _ φsΛ φsucΛ)
```

lemma jointLiftLeft_disjoint_joinLiftRight

[\[source\]](#)

```
lemma jointLiftLeft_disjoint_joinLiftRight {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.
  length}
  {φsucΛ : WickContraction [φsΛ]uc.length} (a : φsΛ.1) (b : φsucΛ.1) :
  Disjoint (@joinLiftLeft _ _ _ φsucΛ a).1 (joinLiftRight b).1
```

lemma jointLiftLeft_neq_joinLiftRight

[\[source\]](#)

```
lemma jointLiftLeft_neq_joinLiftRight {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} (a : φsΛ.1) (b : φsucΛ.1) :
  joinLiftLeft a ≠ joinLiftRight b
```

def joinLift

[\[source\]](#)

The map from contracted pairs of $\phi s\Lambda$ and $\phi suc\Lambda$ to contracted pairs in $(\text{join } \phi s\Lambda \phi suc\Lambda)$.

```
def joinLift {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} : φsΛ.1 ⊕ φsucΛ.1 → (join φsΛ φsucΛ).1
```

lemma joinLift_injective

[\[source\]](#)

```
lemma joinLift_injective {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} : Function.Injective (@joinLift _ _ φsΛ φsucΛ)
```

lemma joinLift_surjective

[\[source\]](#)

```
lemma joinLift_surjective {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} : Function.Surjective (@joinLift _ _ φsΛ φsucΛ)
```

lemma joinLift_bijective

[\[source\]](#)

```
lemma joinLift_bijective {φs : List  $\mathcal{F}$ .FieldOp} {φsΛ : WickContraction φs.length}
  {φsucΛ : WickContraction [φsΛ]uc.length} : Function.Bijective (@joinLift _ _ φsΛ φsucΛ)
```

lemma prod_join

[\[source\]](#)

```
lemma prod_join {φs : List  $\mathcal{F}$ .FieldOp} (φsΛ : WickContraction φs.length)
  (φsucΛ : WickContraction [φsΛ]uc.length) (f : (join φsΛ φsucΛ).1 → M) [CommMonoid M]:
  ∏ (a : (join φsΛ φsucΛ).1), f a = (∏ (a : φsΛ.1), f (joinLiftLeft a)) *
  ∏ (a : φsucΛ.1), f (joinLiftRight a)
```

lemma joinLiftLeft_or_joinLiftRight_of_mem_join[\[source\]](#)

```

lemma joinLiftLeft_or_joinLiftRight_of_mem_join { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) {a : Finset (Fin  $\phi$ s.length)}
  (ha : a  $\in$  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).1) :
  ( $\exists$  b, a = (joinLiftLeft ( $\phi$ suc $\Lambda$  :=  $\phi$ suc $\Lambda$ ) b).1)  $\vee$ 
  ( $\exists$  b, a = (joinLiftRight ( $\phi$ suc $\Lambda$  :=  $\phi$ suc $\Lambda$ ) b).1)

```

lemma join_fstFieldOfContract_joinLiftRight[\[source\]](#)

```

@[simp]
lemma join_fstFieldOfContract_joinLiftRight { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.
  length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (a :  $\phi$ suc $\Lambda$ .1) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).fstFieldOfContract (joinLiftRight a) =
  uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a)

```

lemma join_sndFieldOfContract_joinLiftRight[\[source\]](#)

```

@[simp]
lemma join_sndFieldOfContract_joinLiftRight { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.
  length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (a :  $\phi$ suc $\Lambda$ .1) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).sndFieldOfContract (joinLiftRight a) =
  uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a)

```

lemma join_fstFieldOfContract_joinLiftLeft[\[source\]](#)

```

@[simp]
lemma join_fstFieldOfContract_joinLiftLeft { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.
  length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (a :  $\phi$ s $\Lambda$ .1) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).fstFieldOfContract (joinLiftLeft a) =
  ( $\phi$ s $\Lambda$ .fstFieldOfContract a)

```

lemma join_sndFieldOfContract_joinLift[\[source\]](#)

```

@[simp]
lemma join_sndFieldOfContract_joinLift { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (a :  $\phi$ s $\Lambda$ .1) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).sndFieldOfContract (joinLiftLeft a) =
  ( $\phi$ s $\Lambda$ .sndFieldOfContract a)

```

lemma mem_join_right_iff[\[source\]](#)

```

lemma mem_join_right_iff { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (a : Finset (Fin [ $\phi$ s $\Lambda$ ]uc.length)) :
  a  $\in$   $\phi$ suc $\Lambda$ .1  $\leftrightarrow$  a.map uncontractedListEmd  $\in$  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).1

```

lemma join_card[\[source\]](#)

```

lemma join_card { $\phi$ s : List  $\mathcal{F}$ .FieldOp} { $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length}
  { $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length} :

```

```
(join  $\phi s \Lambda$   $\phi suc \Lambda$ ).1.card =  $\phi s \Lambda$ .1.card +  $\phi suc \Lambda$ .1.card
```

lemma empty_join[\[source\]](#)

```
@[simp]
lemma empty_join { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction [empty (n :=  $\phi s$ .length)]uc.length) :
  join empty  $\phi s \Lambda$  = congr (by simp)  $\phi s \Lambda$ 
```

lemma join_empty[\[source\]](#)

```
@[simp]
lemma join_empty { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  join  $\phi s \Lambda$  empty =  $\phi s \Lambda$ 
```

lemma join_timeContract[\[source\]](#)

```
lemma join_timeContract { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc \Lambda$  : WickContraction [ $\phi s \Lambda$ ]uc.length) :
  (join  $\phi s \Lambda$   $\phi suc \Lambda$ ).timeContract =  $\phi s \Lambda$ .timeContract *  $\phi suc \Lambda$ .timeContract
```

lemma join_staticContract[\[source\]](#)

```
lemma join_staticContract { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc \Lambda$  : WickContraction [ $\phi s \Lambda$ ]uc.length) :
  (join  $\phi s \Lambda$   $\phi suc \Lambda$ ).staticContract =  $\phi s \Lambda$ .staticContract *  $\phi suc \Lambda$ .staticContract
```

lemma mem_join_uncontracted_of_mem_right_uncontracted[\[source\]](#)

```
lemma mem_join_uncontracted_of_mem_right_uncontracted { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc \Lambda$  : WickContraction [ $\phi s \Lambda$ ]uc.length) (i : Fin [ $\phi s \Lambda$ ]uc.length)
  (ha : i ∈  $\phi suc \Lambda$ .uncontracted) :
  uncontractedListEmd i ∈ (join  $\phi s \Lambda$   $\phi suc \Lambda$ ).uncontracted
```

lemma exists_mem_left_uncontracted_of_mem_join_uncontracted[\[source\]](#)

```
lemma exists_mem_left_uncontracted_of_mem_join_uncontracted { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc \Lambda$  : WickContraction [ $\phi s \Lambda$ ]uc.length) (i : Fin  $\phi s$ .length)
  (ha : i ∈ (join  $\phi s \Lambda$   $\phi suc \Lambda$ ).uncontracted) :
  i ∈  $\phi s \Lambda$ .uncontracted
```

lemma exists_mem_right_uncontracted_of_mem_join_uncontracted[\[source\]](#)

```
lemma exists_mem_right_uncontracted_of_mem_join_uncontracted { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc \Lambda$  : WickContraction [ $\phi s \Lambda$ ]uc.length) (i : Fin  $\phi s$ .length)
  (hi : i ∈ (join  $\phi s \Lambda$   $\phi suc \Lambda$ ).uncontracted) :
  ∃ a, uncontractedListEmd a = i ∧ a ∈  $\phi suc \Lambda$ .uncontracted
```

lemma join_uncontractedList[\[source\]](#)

```

lemma join_uncontractedList { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).uncontractedList = List.map uncontractedListEmd  $\phi$ suc $\Lambda$ .uncontractedList

```

lemma join_uncontractedList_get[\[source\]](#)

```

lemma join_uncontractedList_get { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) :
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).uncontractedList).get =
   $\phi$ s $\Lambda$ .uncontractedListEmd  $\circ$  ( $\phi$ suc $\Lambda$ .uncontractedList).get  $\circ$ 
  (Fin.cast (by rw [join_uncontractedList]; simp))

```

lemma join_uncontractedListGet[\[source\]](#)

```

lemma join_uncontractedListGet { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).uncontractedListGet =  $\phi$ suc $\Lambda$ .uncontractedListGet

```

lemma join_uncontractedListEmb[\[source\]](#)

```

lemma join_uncontractedListEmb { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).uncontractedListEmd =
  ((finCongr (congrArg List.length (join_uncontractedListGet _ _))).toEmbedding.trans
    $\phi$ suc $\Lambda$ .uncontractedListEmd).trans  $\phi$ s $\Lambda$ .uncontractedListEmd

```

lemma join_assoc[\[source\]](#)

```

lemma join_assoc { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) ( $\phi$ suc $\Lambda'$  : WickContraction [ $\phi$ s $\Lambda$ .join  $\phi$ suc $\Lambda$ ]uc.length) :
  join (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ) ( $\phi$ suc $\Lambda'$ ) = join  $\phi$ s $\Lambda$  (join  $\phi$ suc $\Lambda$  (congr
    (congrArg List.length (join_uncontractedListGet _ _))  $\phi$ suc $\Lambda'$ ))

```

lemma join_getDual?_apply_uncontractedListEmb_eq_none_iff[\[source\]](#)

```

lemma join_getDual?_apply_uncontractedListEmb_eq_none_iff { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (i : Fin [ $\phi$ s $\Lambda$ ]uc.length) :
  (join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).getDual? (uncontractedListEmd i) = none
   $\leftrightarrow$   $\phi$ suc $\Lambda$ .getDual? i = none

```

lemma join_getDual?_apply_uncontractedListEmb_isSome_iff[\[source\]](#)

```

lemma join_getDual?_apply_uncontractedListEmb_isSome_iff { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (i : Fin [ $\phi$ s $\Lambda$ ]uc.length) :
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).getDual? (uncontractedListEmd i)).isSome
   $\leftrightarrow$  ( $\phi$ suc $\Lambda$ .getDual? i).isSome

```

lemma join_getDual?_apply_uncontractedListEmb_some[\[source\]](#)

```

lemma join_getDual?_apply_uncontractedListEmb_some { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) (i : Fin [ $\phi$ s $\Lambda$ ]uc.length)
  (hi : ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).getDual? (uncontractedListEmd i)).isSome) :
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).getDual? (uncontractedListEmd i)) =
  some (uncontractedListEmd (( $\phi$ suc $\Lambda$ .getDual? i).get (by
  simp [join_getDual?_apply_uncontractedListEmb_isSome_iff]using hi)))

```

lemma join_getDual?_apply_uncontractedListEmb[\[source\]](#)

```

@[simp]
lemma join_getDual?_apply_uncontractedListEmb { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length)
  (i : Fin [ $\phi$ s $\Lambda$ ]uc.length) : ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).getDual? (uncontractedListEmd i)) =
  Option.map uncontractedListEmd ( $\phi$ suc $\Lambda$ .getDual? i)

```

lemma join_sub_quot[\[source\]](#)

```

lemma join_sub_quot (S : Finset (Finset (Fin  $\phi$ s.length))) (ha : S  $\subseteq$   $\phi$ s $\Lambda$ .1) :
  join (subContraction S ha) (quotContraction S ha) =  $\phi$ s $\Lambda$ 

```

lemma subContraction_card_plus_quotContraction_card_eq[\[source\]](#)

```

lemma subContraction_card_plus_quotContraction_card_eq (S : Finset (Finset (Fin  $\phi$ s.length)))
  (ha : S  $\subseteq$   $\phi$ s $\Lambda$ .1) :
  (subContraction S ha).1.card + (quotContraction S ha).1.card =  $\phi$ s $\Lambda$ .1.card

```

lemma join_singleton_getDual?_left[\[source\]](#)

```

@[simp]
lemma join_singleton_getDual?_left { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  (join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? i = some j

```

lemma join_singleton_getDual?_right[\[source\]](#)

```

@[simp]
lemma join_singleton_getDual?_right { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  (join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? j = some i

```

lemma exists_contraction_pair_of_card_ge_zero[\[source\]](#)

```

lemma exists_contraction_pair_of_card_ge_zero { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (h : 0 <  $\phi$ s $\Lambda$ .1.card) :
   $\exists$  a, a  $\in$   $\phi$ s $\Lambda$ .1

```

lemma exists_join_singleton_of_card_ge_zero[\[source\]](#)

```

lemma exists_join_singleton_of_card_ge_zero { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $h$  :  $0 < \phi s \Lambda.1.card$ ) ( $hc$  :  $\phi s \Lambda$ .GradingCompliant) :
   $\exists (i\ j : \text{Fin } \phi s.length) (h : i < j) (\phi suc \Lambda : \text{WickContraction } [\text{singleton } h]^{uc}.length),$ 
   $\phi s \Lambda = \text{join } (\text{singleton } h) \ \phi suc \Lambda \wedge (\mathcal{F} \mid_{>s} \phi s[i]) = (\mathcal{F} \mid_{>s} \phi s[j])$ 
   $\wedge \phi suc \Lambda$ .GradingCompliant  $\wedge \phi suc \Lambda.1.card + 1 = \phi s \Lambda.1.card$ 

```

lemma join_not_gradingCompliant_of_left_not_gradingCompliant[\[source\]](#)

```

lemma join_not_gradingCompliant_of_left_not_gradingCompliant { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $\phi suc \Lambda$  : WickContraction  $[\phi s \Lambda]^{uc}.length$ )
  ( $hc$  :  $\neg \phi s \Lambda$ .GradingCompliant) :  $\neg (\text{join } \phi s \Lambda \ \phi suc \Lambda)$ .GradingCompliant

```

2.48 PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Basic

[PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Basic.lean](#) on GitHub.

lemma sign_empty[\[source\]](#)

```

lemma sign_empty ( $\phi s$  : List  $\mathcal{F}$ .FieldOp) :
  sign  $\phi s$  empty = 1

```

lemma sign_congr[\[source\]](#)

```

lemma sign_congr { $\phi s \ \phi s'$  : List  $\mathcal{F}$ .FieldOp} ( $h$  :  $\phi s = \phi s'$ ) ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) :
  sign  $\phi s'$  (congr (by simp [h])  $\phi s \Lambda$ ) = sign  $\phi s \ \phi s \Lambda$ 

```

2.49 PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertNone

[PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean](#) on GitHub.

lemma sign_insert_none_eq_signInsertNone_mul_sign[\[source\]](#)

```

lemma sign_insert_none_eq_signInsertNone_mul_sign ( $\phi$  :  $\mathcal{F}$ .FieldOp) ( $\phi s$  : List  $\mathcal{F}$ .FieldOp)
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) :
  ( $\phi s \Lambda \hookrightarrow \phi \ i$  none).sign = ( $\phi s \Lambda$ .signInsertNone  $\phi \ \phi s \ i$ ) *  $\phi s \Lambda$ .sign

```

2.50 PhysLean.QFT.PerturbationTheory.WickContraction.Sign.InsertSome

[PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean](#) on GitHub.

lemma sign_insert_some_of_lt[\[source\]](#)

For a list $\phi s = \phi_{o...o} \phi_n$ of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}$.FieldOp, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$ such that $k < i$, the sign of $\phi s \Lambda \hookrightarrow \phi \ i$ (some k) is equal to the product of - the sign associated with moving ϕ through the $\phi s \Lambda$ -uncontracted FieldOp in $\phi_{o...o} \phi_k$, - the sign associated with moving ϕ through all FieldOp in $\phi_{o...o} \phi_{i-1}$, - the sign of $\phi s \Lambda$.

The proof of this result involves a careful consideration of the contributions of different FieldOp in ϕs to the sign of $\phi s \Lambda \hookrightarrow \phi \ i$ (some k).

```

lemma sign_insert_some_of_lt (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (k : φsΛ.uncontracted)
  (hk : i.succAbove k < i) (hg : GradingCompliant φs φsΛ ∧ ( $\mathcal{F}$  |>s φ) =  $\mathcal{F}$  |>s φs[k.1]) :
  (φsΛ ↪Λ φ i (some k)).sign =
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, φsΛ.uncontracted.filter (fun x => x ≤ ↑k)))
  * S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, Finset.univ.filter (fun x => i.succAbove x < i)))
  * φsΛ.sign

```

lemma sign_insert_some_of_not_lt

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}$.FieldOp, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$ such that $i \leq k$, the sign of $\phi s \Lambda \hookrightarrow \Lambda \phi i (some\ k)$ is equal to the product of - the sign associated with moving ϕ through the $\phi s \Lambda$ -uncontracted FieldOp in $\phi_0 \dots \phi_{k-1}$, - the sign associated with moving ϕ through all the FieldOp in $\phi_0 \dots \phi_{i-1}$, - the sign of $\phi s \Lambda$.

The proof of this result involves a careful consideration of the contributions of different FieldOp in ϕs to the sign of $\phi s \Lambda \hookrightarrow \Lambda \phi i (some\ k)$.

```

lemma sign_insert_some_of_not_lt (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (k : φsΛ.uncontracted)
  (hk : ¬ i.succAbove k < i) (hg : GradingCompliant φs φsΛ ∧ ( $\mathcal{F}$  |>s φ) =  $\mathcal{F}$  |>s φs[k.1]) :
  (φsΛ ↪Λ φ i (some k)).sign =
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, φsΛ.uncontracted.filter (fun x => x < ↑k)))
  * S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, Finset.univ.filter (fun x => i.succAbove x < i))) *
  φsΛ.sign

```

lemma sign_insert_some_zero

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}$.FieldOp, and a k in $\phi s \Lambda.uncontracted$, the sign of $\phi s \Lambda \hookrightarrow \Lambda \phi 0 (some\ k)$ is equal to the product of - the sign associated with moving ϕ through the $\phi s \Lambda$ -uncontracted FieldOp in $\phi_0 \dots \phi_{k-1}$, - the sign of $\phi s \Lambda$.

This is a direct corollary of `sign_insert_some_of_not_lt`.

```

lemma sign_insert_some_zero (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (φsΛ : WickContraction φs.length) (k : φsΛ.uncontracted)
  (hn : GradingCompliant φs φsΛ ∧ ( $\mathcal{F}$  |>s φ) =  $\mathcal{F}$  |>s φs[k.1]) :
  (φsΛ ↪Λ φ 0 k).sign = S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, (φsΛ.uncontracted.filter (fun x => x < ↑k))
  )) *
  φsΛ.sign

```

2.51 PhysLean.QFT.PerturbationTheory.WickContraction.Sign.Join

[PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Join.lean](#) on GitHub.

lemma stat_signFinset_right

[\[source\]](#)

```

lemma stat_signFinset_right {φs : List  $\mathcal{F}$ .FieldOp} (φsΛ : WickContraction φs.length)
  (φsucΛ : WickContraction [φsΛ]uc.length) (i j : Fin [φsΛ]uc.length) :
  ( $\mathcal{F}$  |>s ([φsΛ]uc.get, φsucΛ.signFinset i j)) =
  ( $\mathcal{F}$  |>s (φs.get, (φsucΛ.signFinset i j).map uncontractedListEmd))

```

lemma signFinset_right_map_uncontractedListEmd_eq_filter[\[source\]](#)

```

lemma signFinset_right_map_uncontractedListEmd_eq_filter { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length)
  (i j : Fin [ $\phi$ s $\Lambda$ ]uc.length) : ( $\phi$ suc $\Lambda$ .signFinset i j).map uncontractedListEmd =
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).signFinset (uncontractedListEmd i) (uncontractedListEmd j)).filter
  (fun c => c  $\in$   $\phi$ s $\Lambda$ .uncontracted)

```

lemma sign_right_eq_prod_mul_prod[\[source\]](#)

```

lemma sign_right_eq_prod_mul_prod { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length)
  ( $\phi$ suc $\Lambda$  : WickContraction [ $\phi$ s $\Lambda$ ]uc.length) :
   $\phi$ suc $\Lambda$ .sign = ( $\prod$  a,  $S(\mathcal{F}|>_s$  [ $\phi$ s $\Lambda$ ]uc[ $\phi$ suc $\Lambda$ .sndFieldOfContract a],  $\mathcal{F}|>_s$  ( $\phi$ s.get,
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).signFinset (uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a))
  (uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a))))).filter
  (fun c =>  $\neg$  c  $\in$   $\phi$ s $\Lambda$ .uncontracted)) *
  ( $\prod$  a,  $S(\mathcal{F}|>_s$  [ $\phi$ s $\Lambda$ ]uc[ $\phi$ suc $\Lambda$ .sndFieldOfContract a],  $\mathcal{F}|>_s$  ( $\phi$ s.get,
  ((join  $\phi$ s $\Lambda$   $\phi$ suc $\Lambda$ ).signFinset (uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a))
  (uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a))))))

```

lemma join_singleton_signFinset_eq_filter[\[source\]](#)

```

lemma join_singleton_signFinset_eq_filter { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  (join (singleton h)  $\phi$ suc $\Lambda$ ).signFinset i j =
  ((singleton h).signFinset i j).filter (fun c =>  $\neg$ 
  (((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).isSome  $\wedge$ 
  ((h1 : ((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).isSome)  $\rightarrow$ 
  (((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).get h1) < i)))

```

lemma join_singleton_left_signFinset_eq_filter[\[source\]](#)

```

lemma join_singleton_left_signFinset_eq_filter { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  ( $\mathcal{F}|>_s$  ( $\phi$ s.get, (singleton h).signFinset i j))
  = ( $\mathcal{F}|>_s$  ( $\phi$ s.get, (join (singleton h)  $\phi$ suc $\Lambda$ ).signFinset i j)) *
  ( $\mathcal{F}|>_s$  ( $\phi$ s.get, ((singleton h).signFinset i j).filter (fun c =>
  (((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).isSome  $\wedge$ 
  ((h1 : ((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).isSome)  $\rightarrow$ 
  (((join (singleton h)  $\phi$ suc $\Lambda$ ).getDual? c).get h1) < i)))))

```

def joinSignRightExtra[\[source\]](#)

The difference in sign between ϕ suc Λ .sign and the direct contribution of ϕ suc Λ to (join (singleton h) ϕ suc Λ).

```

def joinSignRightExtra { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :  $\mathbb{C}$ 

```

def joinSignLeftExtra[\[source\]](#)

The difference in sign between (singleton h).sign and the direct contribution of (singleton h) to (join (singleton h) ϕ suc Λ).

```
def joinSignLeftExtra { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :  $\mathbb{C}$ 
```

lemma join_singleton_sign_left

[\[source\]](#)

```
lemma join_singleton_sign_left { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  (singleton h).sign =  $S(\mathcal{F} \mid_{>s} \phi$ s[j],
    ( $\mathcal{F} \mid_{>s} \langle \phi$ s.get, (join (singleton h)  $\phi$ suc $\Lambda$ ).signFinset i j))) * (joinSignLeftExtra h  $\phi$ suc $\Lambda$ )
```

lemma join_singleton_sign_right

[\[source\]](#)

```
lemma join_singleton_sign_right { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j)
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
   $\phi$ suc $\Lambda$ .sign =
  (joinSignRightExtra h  $\phi$ suc $\Lambda$ ) *
  ( $\prod$  a,  $S(\mathcal{F} \mid_{>s} [\text{singleton h}]^{uc}[\phi$ suc $\Lambda$ .sndFieldOfContract a],  $\mathcal{F} \mid_{>s} \langle \phi$ s.get,
    ((join (singleton h)  $\phi$ suc $\Lambda$ ).signFinset (uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a))
      (uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a))))))
```

lemma joinSignRightExtra_eq_i_j_finset_eq_if

[\[source\]](#)

```
lemma joinSignRightExtra_eq_i_j_finset_eq_if { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j) ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  joinSignRightExtra h  $\phi$ suc $\Lambda$  =  $\prod$  a,
   $S((\mathcal{F} \mid_{>s} [\text{singleton h}]^{uc}[\phi$ suc $\Lambda$ .sndFieldOfContract a]),
     $\mathcal{F} \mid_{>s} \langle \phi$ s.get, (if uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a) < j  $\wedge$ 
      j < uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a)  $\wedge$ 
      uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a) < i then {j} else  $\emptyset$ )
       $\cup$  (if uncontractedListEmd ( $\phi$ suc $\Lambda$ .fstFieldOfContract a) < i  $\wedge$ 
      i < uncontractedListEmd ( $\phi$ suc $\Lambda$ .sndFieldOfContract a) then {i} else  $\emptyset$ )))
```

lemma joinSignLeftExtra_eq_joinSignRightExtra

[\[source\]](#)

```
lemma joinSignLeftExtra_eq_joinSignRightExtra { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j) (hs : ( $\mathcal{F} \mid_{>s} \phi$ s[i]) = ( $\mathcal{F} \mid_{>s} \phi$ s[j]))
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  joinSignLeftExtra h  $\phi$ suc $\Lambda$  = joinSignRightExtra h  $\phi$ suc $\Lambda$ 
```

lemma join_sign_singleton

[\[source\]](#)

```
lemma join_sign_singleton { $\phi$ s : List  $\mathcal{F}$ .FieldOp}
  {i j : Fin  $\phi$ s.length} (h : i < j) (hs : ( $\mathcal{F} \mid_{>s} \phi$ s[i]) = ( $\mathcal{F} \mid_{>s} \phi$ s[j]))
  ( $\phi$ suc $\Lambda$  : WickContraction [singleton h]uc.length) :
  (join (singleton h)  $\phi$ suc $\Lambda$ ).sign = (singleton h).sign *  $\phi$ suc $\Lambda$ .sign
```

lemma join_sign_induction

[\[source\]](#)

```

lemma join_sign_induction { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s\Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc\Lambda$  : WickContraction [ $\phi s\Lambda$ ]uc.length) (hc :  $\phi s\Lambda$ .GradingCompliant) :
  (n :  $\mathbb{N}$ ) → (hn :  $\phi s\Lambda$ .l.card = n) →
  (join  $\phi s\Lambda$   $\phi suc\Lambda$ ).sign =  $\phi s\Lambda$ .sign *  $\phi suc\Lambda$ .sign
| 0, hn => by
  rw [@card_zero_iff_empty] at hn
  subst hn
  simp only [empty_join, sign_empty, one_mul]
  apply sign_congr
  simp
| Nat.succ n, hn => by
  obtain (i, j, hij,  $\phi suc\Lambda'$ , rfl, h1, h2, h3)

```

lemma join_sign

[\[source\]](#)

For a list ϕs of $\mathcal{F}$.FieldOp, a grading compliant Wick contraction $\phi s\Lambda$ of ϕs , and a Wick contraction $\phi suc\Lambda$ of [$\phi s\Lambda$]^{uc}, the following relation holds (join $\phi s\Lambda$ $\phi suc\Lambda$).sign = $\phi s\Lambda$.sign * $\phi suc\Lambda$.sign.

In $\phi s\Lambda$.sign the sign is determined by starting with the contracted pair whose first element occurs at the left-most position. This lemma manifests that this choice does not matter, and that contracted pairs can be brought together in any order.

```

lemma join_sign { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s\Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc\Lambda$  : WickContraction [ $\phi s\Lambda$ ]uc.length) (hc :  $\phi s\Lambda$ .GradingCompliant) :
  (join  $\phi s\Lambda$   $\phi suc\Lambda$ ).sign =  $\phi s\Lambda$ .sign *  $\phi suc\Lambda$ .sign

```

lemma join_sign_timeContract

[\[source\]](#)

For a list ϕs of $\mathcal{F}$.FieldOp, a Wick contraction $\phi s\Lambda$ of ϕs , and a Wick contraction $\phi suc\Lambda$ of [$\phi s\Lambda$]^{uc}, (join $\phi s\Lambda$ $\phi suc\Lambda$).sign • (join $\phi s\Lambda$ $\phi suc\Lambda$).timeContract is equal to the product of - $\phi s\Lambda$.sign • $\phi s\Lambda$.timeContract and - $\phi suc\Lambda$.sign • $\phi suc\Lambda$.timeContract.

```

lemma join_sign_timeContract { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s\Lambda$  : WickContraction  $\phi s$ .length)
  ( $\phi suc\Lambda$  : WickContraction [ $\phi s\Lambda$ ]uc.length) :
  (join  $\phi s\Lambda$   $\phi suc\Lambda$ ).sign • (join  $\phi s\Lambda$   $\phi suc\Lambda$ ).timeContract.1 =
  ( $\phi s\Lambda$ .sign •  $\phi s\Lambda$ .timeContract.1) * ( $\phi suc\Lambda$ .sign •  $\phi suc\Lambda$ .timeContract.1)

```

2.52 PhysLean.QFT.PerturbationTheory.WickContraction.Singleton

PhysLean/QFT/PerturbationTheory/WickContraction/Singleton.lean on GitHub.

def singleton

[\[source\]](#)

The Wick contraction formed from a single ordered pair.

```

def singleton {i j : Fin n} (hij : i < j) : WickContraction n

```

lemma mem_singleton

[\[source\]](#)

```

lemma mem_singleton {i j : Fin n} (hij : i < j) :
  {i, j} ∈ (singleton hij).1

```

lemma mem_singleton_iff[\[source\]](#)

```
lemma mem_singleton_iff {i j : Fin n} (hij : i < j) {a : Finset (Fin n)} :
  a ∈ (singleton hij).1 ↔ a = {i, j}
```

lemma of_singleton_eq[\[source\]](#)

```
lemma of_singleton_eq {i j : Fin n} (hij : i < j) (a : (singleton hij).1) :
  a = ({i, j}, mem_singleton hij)
```

lemma singleton_prod[\[source\]](#)

```
lemma singleton_prod {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i < j)
  (f : (singleton hij).1 → M) [CommMonoid M] :
  [] a, f a = f ({i,j}, mem_singleton hij)
```

lemma singleton_fstFieldOfContract[\[source\]](#)

```
@[simp]
lemma singleton_fstFieldOfContract {i j : Fin n} (hij : i < j) :
  (singleton hij).fstFieldOfContract ({i, j}, mem_singleton hij) = i
```

lemma singleton_sndFieldOfContract[\[source\]](#)

```
@[simp]
lemma singleton_sndFieldOfContract {i j : Fin n} (hij : i < j) :
  (singleton hij).sndFieldOfContract ({i, j}, mem_singleton hij) = j
```

lemma singleton_sign_expand[\[source\]](#)

```
lemma singleton_sign_expand {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i < j) :
  (singleton hij).sign = S(ℱ |>_s φs[j], ℱ |>_s {φs.get, (singleton hij).signFinset i j})
```

lemma singleton_getDual?_eq_none_iff_neq[\[source\]](#)

```
lemma singleton_getDual?_eq_none_iff_neq {i j : Fin n} (hij : i < j) (a : Fin n) :
  (singleton hij).getDual? a = none ↔ (i ≠ a ∧ j ≠ a)
```

lemma singleton_uncontractedEmd_neq_left[\[source\]](#)

```
lemma singleton_uncontractedEmd_neq_left {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i
  < j)
  (a : Fin [singleton hij]uc.length) :
  (singleton hij).uncontractedListEmd a ≠ i
```

lemma singleton_uncontractedEmd_neq_right[\[source\]](#)

```
lemma singleton_uncontractedEmd_neq_right {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i
  < j)
  (a : Fin [singleton hij]uc.length) :
  (singleton hij).uncontractedListEmd a ≠ j
```

lemma mem_signFinset[\[source\]](#)

@[simp]

```
lemma mem_signFinset {i j : Fin n} (hij : i < j) (a : Fin n) :
  a ∈ (singleton hij).signFinset i j ↔ i < a ∧ a < j
```

lemma subContraction_singleton_eq_singleton[\[source\]](#)

```
lemma subContraction_singleton_eq_singleton {φs : List ℱ.FieldOp}
  (φsΛ : WickContraction φs.length)
  (a : φsΛ.1) : φsΛ.subContraction {a.1} (by simp) =
  singleton (φsΛ.fstFieldOfContract_lt_sndFieldOfContract a)
```

lemma singleton_timeContract[\[source\]](#)

```
lemma singleton_timeContract {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i < j) :
  (singleton hij).timeContract =
  (FieldOpAlgebra.timeContract φs[i] φs[j], timeContract_mem_center _ _)
```

lemma singleton_staticContract[\[source\]](#)

```
lemma singleton_staticContract {φs : List ℱ.FieldOp} {i j : Fin φs.length} (hij : i < j) :
  (singleton hij).staticContract.1 =
  [anPart φs[i], ofFieldOp φs[j]]_s
```

2.53 PhysLean.QFT.PerturbationTheory.WickContraction.StaticContract

[PhysLean/QFT/PerturbationTheory/WickContraction/StaticContract.lean](#) on GitHub.

lemma staticContract_insert_none[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, and a $i \leq \phi s.length$, then the following relation holds:

$$(\phi s \Lambda \hookrightarrow \phi \text{ i none}).staticContract = \phi s \Lambda.staticContract$$

The proof of this result ultimately is a consequence of definitions.

@[simp]

```
lemma staticContract_insert_none (φ : ℱ.FieldOp) (φs : List ℱ.FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) :
  (φsΛ ↪ φ i none).staticContract = φsΛ.staticContract
```

lemma staticContract_insert_some[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$, then $(\phi s \Lambda \hookrightarrow \phi \text{ i (some } k)).staticContract$ is equal to the product of - $[anPart \phi, \phi s[k]]_s$ if $i \leq k$ or $[anPart \phi s[k], \phi]_s$ if $k < i - \phi s \Lambda.staticContract$.

The proof of this result ultimately is a consequence of definitions.

```
lemma staticContract_insert_some
  (φ : ℱ.FieldOp) (φs : List ℱ.FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : φsΛ.uncontracted) :
```

```

(φsΛ ↦Λ φ i (some j)).staticContract =
  (if i < i.succAbove j then
    ([anPart φ, ofFieldOp φs[j.1]]s, superCommute_anPart_ofFieldOp_mem_center _ _)
  else ([anPart φs[j.1], ofFieldOp φ]s, superCommute_anPart_ofFieldOp_mem_center _ _)) *
  φsΛ.staticContract

```

lemma staticContract_insert_some_of_lt

[\[source\]](#)

```

lemma staticContract_insert_some_of_lt
  (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (k : φsΛ.uncontracted)
  (hik : i < i.succAbove k) :
  (φsΛ ↦Λ φ i (some k)).staticContract =
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (φs.get, (φsΛ.uncontracted.filter (fun x => x < k))))
  • (contractStateAtIndex φ [φsΛ]uc ((uncontractedFieldOpEquiv φs φsΛ) (some k)) *
    φsΛ.staticContract)

```

2.54 PhysLean.QFT.PerturbationTheory.WickContraction.SubContraction

[PhysLean/QFT/PerturbationTheory/WickContraction/SubContraction.lean](#) on GitHub.

def subContraction

[\[source\]](#)

Given a Wick contraction $\phi s \Lambda$, and a subset of $\phi s \Lambda.1$, the Wick contraction consisting of contracted pairs within that subset.

```

def subContraction (S : Finset (Finset (Fin φs.length))) (ha : S ⊆ φsΛ.1) :
  WickContraction φs.length

```

lemma mem_of_mem_subContraction

[\[source\]](#)

```

lemma mem_of_mem_subContraction {S : Finset (Finset (Fin φs.length))} {hs : S ⊆ φsΛ.1}
  {a : Finset (Fin φs.length)} (ha : a ∈ (φsΛ.subContraction S hs).1) : a ∈ φsΛ.1

```

def quotContraction

[\[source\]](#)

Given a Wick contraction $\phi s \Lambda$, and a subset S of $\phi s \Lambda.1$, the Wick contraction on the uncontracted list $[\phi s \Lambda.subContraction S ha]^{uc}$ consisting of the remaining contracted pairs of $\phi s \Lambda$ not in S .

```

def quotContraction (S : Finset (Finset (Fin φs.length))) (ha : S ⊆ φsΛ.1) :
  WickContraction [φsΛ.subContraction S ha]uc.length

```

lemma mem_of_mem_quotContraction

[\[source\]](#)

```

lemma mem_of_mem_quotContraction {S : Finset (Finset (Fin φs.length))} {hs : S ⊆ φsΛ.1}
  {a : Finset (Fin [φsΛ.subContraction S hs]uc.length)}
  (ha : a ∈ (quotContraction S hs).1) : Finset.map uncontractedListEmd a ∈ φsΛ.1

```

lemma mem_subContraction_or_quotContraction

[\[source\]](#)

```

lemma mem_subContraction_or_quotContraction {S : Finset (Finset (Fin φs.length))} {hs : S ⊆ φs
  Λ.1}

```

```

{a : Finset (Fin  $\phi$ s.length)} (ha : a  $\in$   $\phi$ s $\Lambda$ .1) :
a  $\in$  ( $\phi$ s $\Lambda$ .subContraction S hs).1  $\vee$ 
 $\exists$  a', Finset.map uncontractedListEmd a' = a  $\wedge$  a'  $\in$  (quotContraction S hs).1

```

lemma subContraction_uncontractedList_get[\[source\]](#)

```

@[simp]
lemma subContraction_uncontractedList_get {S : Finset (Finset (Fin  $\phi$ s.length))} {hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1}
(a : Fin [subContraction S hs]uc.length) :
[subContraction S hs]uc[a] =  $\phi$ s[uncontractedListEmd a]

```

lemma subContraction_fstFieldOfContract[\[source\]](#)

```

@[simp]
lemma subContraction_fstFieldOfContract {S : Finset (Finset (Fin  $\phi$ s.length))} {hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1}
(a : (subContraction S hs).1) :
(subContraction S hs).fstFieldOfContract a =
 $\phi$ s $\Lambda$ .fstFieldOfContract (a.1, mem_of_mem_subContraction a.2)

```

lemma subContraction_sndFieldOfContract[\[source\]](#)

```

@[simp]
lemma subContraction_sndFieldOfContract {S : Finset (Finset (Fin  $\phi$ s.length))} {hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1}
(a : (subContraction S hs).1) :
(subContraction S hs).sndFieldOfContract a =
 $\phi$ s $\Lambda$ .sndFieldOfContract (a.1, mem_of_mem_subContraction a.2)

```

lemma quotContraction_fstFieldOfContract_uncontractedListEmd[\[source\]](#)

```

@[simp]
lemma quotContraction_fstFieldOfContract_uncontractedListEmd {S : Finset (Finset (Fin  $\phi$ s.length))}
{hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1} (a : (quotContraction S hs).1) :
uncontractedListEmd ((quotContraction S hs).fstFieldOfContract a) =
( $\phi$ s $\Lambda$ .fstFieldOfContract
(Finset.map uncontractedListEmd a.1, mem_of_mem_quotContraction a.2))

```

lemma quotContraction_sndFieldOfContract_uncontractedListEmd[\[source\]](#)

```

@[simp]
lemma quotContraction_sndFieldOfContract_uncontractedListEmd {S : Finset (Finset (Fin  $\phi$ s.length))}
{hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1} (a : (quotContraction S hs).1) :
uncontractedListEmd ((quotContraction S hs).sndFieldOfContract a) =
( $\phi$ s $\Lambda$ .sndFieldOfContract
(Finset.map uncontractedListEmd a.1, mem_of_mem_quotContraction a.2))

```

lemma quotContraction_gradingCompliant[\[source\]](#)

```

lemma quotContraction_gradingCompliant {S : Finset (Finset (Fin  $\phi$ s.length))} {hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1}
(hs $\Lambda$  :  $\phi$ s $\Lambda$ .GradingCompliant) :
GradingCompliant [ $\phi$ s $\Lambda$ .subContraction S hs]uc (quotContraction S hs)

```

lemma mem_quotContraction_iff[\[source\]](#)

```

lemma mem_quotContraction_iff {S : Finset (Finset (Fin  $\phi$ s.length))} {hs : S  $\subseteq$   $\phi$ s $\Lambda$ .1}
  {a : Finset (Fin [ $\phi$ s $\Lambda$ .subContraction S hs]uc.length)} :
  a  $\in$  (quotContraction S hs).1  $\leftrightarrow$  a.map uncontractedListEmd  $\in$   $\phi$ s $\Lambda$ .1
   $\wedge$  a.map uncontractedListEmd  $\notin$  (subContraction S hs).1

```

2.55 PhysLean.QFT.PerturbationTheory.WickContraction.TimeCond

[PhysLean/QFT/PerturbationTheory/WickContraction/TimeCond.lean](#) on GitHub.

def EqTimeOnly[\[source\]](#)

The condition on a Wick contraction which is true iff and only if every contraction is between two fields of equal time.

```

def EqTimeOnly { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) : Prop

```

instance Decidable (EqTimeOnly ϕ s Λ)[\[source\]](#)

```

instance { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) :
  Decidable (EqTimeOnly  $\phi$ s $\Lambda$ )

```

lemma timeOrderRel_of_eqTimeOnly_pair[\[source\]](#)

```

lemma timeOrderRel_of_eqTimeOnly_pair {i j : Fin  $\phi$ s.length} (h : {i, j}  $\in$   $\phi$ s $\Lambda$ .1)
  (hc : EqTimeOnly  $\phi$ s $\Lambda$ ) :
  timeOrderRel  $\phi$ s[i]  $\phi$ s[j]

```

lemma timeOrderRel_both_of_eqTimeOnly[\[source\]](#)

```

lemma timeOrderRel_both_of_eqTimeOnly {i j : Fin  $\phi$ s.length} (h : {i, j}  $\in$   $\phi$ s $\Lambda$ .1)
  (hc : EqTimeOnly  $\phi$ s $\Lambda$ ) :
  timeOrderRel  $\phi$ s[i]  $\phi$ s[j]  $\wedge$  timeOrderRel  $\phi$ s[j]  $\phi$ s[i]

```

lemma eqTimeOnly_iff_forall_finset[\[source\]](#)

```

lemma eqTimeOnly_iff_forall_finset { $\phi$ s : List  $\mathcal{F}$ .FieldOp} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) :
   $\phi$ s $\Lambda$ .EqTimeOnly  $\leftrightarrow$   $\forall$  (a :  $\phi$ s $\Lambda$ .1),
    timeOrderRel ( $\phi$ s[ $\phi$ s $\Lambda$ .fstFieldOfContract a]) ( $\phi$ s[ $\phi$ s $\Lambda$ .sndFieldOfContract a])
     $\wedge$  timeOrderRel ( $\phi$ s[ $\phi$ s $\Lambda$ .sndFieldOfContract a]) ( $\phi$ s[ $\phi$ s $\Lambda$ .fstFieldOfContract a])

```

lemma empty_mem[\[source\]](#)

```

@[simp]
lemma empty_mem { $\phi$ s : List  $\mathcal{F}$ .FieldOp} : empty (n :=  $\phi$ s.length).EqTimeOnly

```

lemma staticContract_eq_timeContract_of_eqTimeOnly[\[source\]](#)

Let ϕ s be a list of $\mathcal{F}$.FieldOp and ϕ s Λ a WickContraction of ϕ s within which every contraction involves two $\mathcal{F}$.FieldOps that have the same time, then ϕ s Λ .staticContract = ϕ s Λ .timeContract.

```

lemma staticContract_eq_timeContract_of_eqTimeOnly (h :  $\phi s \Lambda$ .EqTimeOnly) :
   $\phi s \Lambda$ .staticContract =  $\phi s \Lambda$ .timeContract

```

lemma eqTimeOnly_congr

[\[source\]](#)

```

lemma eqTimeOnly_congr { $\phi s \phi s'$  : List  $\mathcal{F}$ .FieldOp} (h :  $\phi s = \phi s'$ ) ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  (congr (by simp [h])  $\phi s \Lambda$ ).EqTimeOnly ( $\phi s := \phi s'$ )  $\leftrightarrow$   $\phi s \Lambda$ .EqTimeOnly

```

lemma quotContraction_eqTimeOnly

[\[source\]](#)

```

lemma quotContraction_eqTimeOnly { $\phi s$  : List  $\mathcal{F}$ .FieldOp} { $\phi s \Lambda$  : WickContraction  $\phi s$ .length}
  (h :  $\phi s \Lambda$ .EqTimeOnly) (S : Finset (Finset (Fin  $\phi s$ .length))) (ha :  $S \subseteq \phi s \Lambda$ .1) :
  ( $\phi s \Lambda$ .quotContraction S ha).EqTimeOnly

```

lemma exists_join_singleton_of_card_ge_zero

[\[source\]](#)

```

lemma exists_join_singleton_of_card_ge_zero { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  (h :  $0 < \phi s \Lambda$ .1.card) (h1 :  $\phi s \Lambda$ .EqTimeOnly) :
   $\exists$  (i j : Fin  $\phi s$ .length) (h : i < j) ( $\phi s \text{uc} \Lambda$  : WickContraction [singleton h]uc.length),
   $\phi s \Lambda = \text{join} (\text{singleton } h) \phi s \text{uc} \Lambda \wedge (\text{timeOrderRel } \phi s[i] \phi s[j] \wedge \text{timeOrderRel } \phi s[j] \phi s[i])$ 
   $\wedge \phi s \text{uc} \Lambda$ .EqTimeOnly  $\wedge \phi s \text{uc} \Lambda$ .1.card + 1 =  $\phi s \Lambda$ .1.card

```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_mid_induction

[\[source\]](#)

```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_mid_induction { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  (h1 :  $\phi s \Lambda$ .EqTimeOnly) (a b :  $\mathcal{F}$ .FieldOpAlgebra) : (n :  $\mathbb{N}$ )  $\rightarrow$  (hn :  $\phi s \Lambda$ .1.card = n)  $\rightarrow$ 
   $\mathcal{T}(a * \phi s \Lambda$ .timeContract.1 * b) =  $\phi s \Lambda$ .timeContract.1 *  $\mathcal{T}(a * b)$ 
| 0, hn => by
  rw [@card_zero_iff_empty] at hn
  subst hn
  simp
| Nat.succ n, hn => by
  obtain (i, j, hij,  $\phi s \text{uc} \Lambda$ , rfl, h2, h3, h4)

```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_mid

[\[source\]](#)

```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_mid { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  (h1 :  $\phi s \Lambda$ .EqTimeOnly) (a b :  $\mathcal{F}$ .FieldOpAlgebra) :
   $\mathcal{T}(a * \phi s \Lambda$ .timeContract.1 * b) =  $\phi s \Lambda$ .timeContract.1 *  $\mathcal{T}(a * b)$ 

```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_left

[\[source\]](#)

Let ϕs be a list of $\mathcal{F}$.FieldOp, $\phi s \Lambda$ a WickContraction of ϕs within which every contraction involves two $\mathcal{F}$.FieldOps that have the same time and b a general element in $\mathcal{F}$.FieldOpAlgebra. Then $\mathcal{T}(\phi s \Lambda$.timeContract.1 * b) = $\phi s \Lambda$.timeContract.1 * $\mathcal{T}(b)$.

This follows from properties of orderings and the ideal defining $\mathcal{F}$.FieldOpAlgebra.


```

lemma timeOrder_timeContract_mul_of_eqTimeOnly_left { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $hl$  :  $\phi s \Lambda$ .EqTimeOnly) ( $b$  :  $\mathcal{F}$ .FieldOpAlgebra) :
 $\mathcal{T}(\phi s \Lambda.timeContract.1 * b) = \phi s \Lambda.timeContract.1 * \mathcal{T}(b)$ 

```

lemma exists_join_singleton_of_not_eqTimeOnly

[\[source\]](#)

```

lemma exists_join_singleton_of_not_eqTimeOnly { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) ( $hl$  :  $\neg \phi s \Lambda$ .EqTimeOnly) :
 $\exists (i\ j : \text{Fin } \phi s.length) (h : i < j) (\phi suc \Lambda : \text{WickContraction } [\text{singleton } h]^{uc}.length),$ 
 $\phi s \Lambda = \text{join } (\text{singleton } h) \ \phi suc \Lambda \wedge (\neg \text{timeOrderRel } \phi s[i] \ \phi s[j] \vee \neg \text{timeOrderRel } \phi s[j] \ \phi s[i])$ 

```

lemma timeOrder_timeContract_of_not_eqTimeOnly

[\[source\]](#)

```

lemma timeOrder_timeContract_of_not_eqTimeOnly { $\phi s$  : List  $\mathcal{F}$ .FieldOp}
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $hl$  :  $\neg \phi s \Lambda$ .EqTimeOnly) :  $\mathcal{T}(\phi s \Lambda.timeContract.1) = 0$ 

```

lemma timeOrder_staticContract_of_not_mem

[\[source\]](#)

Let ϕs be a list of $\mathcal{F}$.FieldOp and $\phi s \Lambda$ a WickContraction with at least one contraction between $\mathcal{F}$.FieldOp that do not have the same time. Then $\mathcal{T}(\phi s \Lambda.staticContract.1) = 0$.

```

lemma timeOrder_staticContract_of_not_mem { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $hl$  :  $\neg \phi s \Lambda$ .EqTimeOnly) :  $\mathcal{T}(\phi s \Lambda.staticContract.1) = 0$ 

```

def HaveEqTime

[\[source\]](#)

The condition on a Wick contraction which is true if it has at least one contraction which is between two equal time fields.

```

def HaveEqTime { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) : Prop

```

instance Decidable (HaveEqTime $\phi s \Lambda$)

[\[source\]](#)

```

noncomputable instance { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  Decidable (HaveEqTime  $\phi s \Lambda$ )

```

lemma haveEqTime_iff_finset

[\[source\]](#)

```

lemma haveEqTime_iff_finset { $\phi s$  : List  $\mathcal{F}$ .FieldOp} ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  HaveEqTime  $\phi s \Lambda \leftrightarrow \exists (a : \text{Finset } (\text{Fin } \phi s.length)) (h : a \in \phi s \Lambda.1),$ 
 $\text{timeOrderRel } \phi s[\phi s \Lambda.fstFieldOfContract\ a, h] \ \phi s[\phi s \Lambda.sndFieldOfContract\ a, h]$ 
 $\wedge \text{timeOrderRel } \phi s[\phi s \Lambda.sndFieldOfContract\ a, h] \ \phi s[\phi s \Lambda.fstFieldOfContract\ a, h]$ 

```

lemma empty_not_haveEqTime

[\[source\]](#)

```

@[simp]
lemma empty_not_haveEqTime { $\phi s$  : List  $\mathcal{F}$ .FieldOp} :
   $\neg \text{HaveEqTime } (\text{empty} : \text{WickContraction } \phi s.length)$ 

```

def eqTimeContractSet[\[source\]](#)

Given a Wick contraction the subset of contracted pairs between equal time fields.

```
def eqTimeContractSet {φs : List ℱ.FieldOp} (φsΛ : WickContraction φs.length) :
  Finset (Finset (Fin φs.length))
```

lemma eqTimeContractSet_subset[\[source\]](#)

```
lemma eqTimeContractSet_subset {φs : List ℱ.FieldOp} (φsΛ : WickContraction φs.length) :
  eqTimeContractSet φsΛ ⊆ φsΛ.1
```

lemma mem_of_mem_eqTimeContractSet[\[source\]](#)

```
lemma mem_of_mem_eqTimeContractSet {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.length}
  {a : Finset (Fin φs.length)} (h : a ∈ eqTimeContractSet φsΛ) : a ∈ φsΛ.1
```

lemma join_eqTimeContractSet[\[source\]](#)

```
lemma join_eqTimeContractSet {φs : List ℱ.FieldOp} (φsΛ : WickContraction φs.length)
  (φsucΛ : WickContraction [φsΛ]uc.length) :
  eqTimeContractSet (join φsΛ φsucΛ) = φsΛ.eqTimeContractSet ∪
  φsucΛ.eqTimeContractSet.map (Finset.mapEmbedding uncontractedListEmd).toEmbedding
```

lemma eqTimeContractSet_of_not_haveEqTime[\[source\]](#)

```
lemma eqTimeContractSet_of_not_haveEqTime {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.
  length}
  (h : ¬ HaveEqTime φsΛ) : eqTimeContractSet φsΛ = ∅
```

lemma eqTimeContractSet_of_mem_eqTimeOnly[\[source\]](#)

```
lemma eqTimeContractSet_of_mem_eqTimeOnly {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.
  length}
  (h : φsΛ.EqTimeOnly) : eqTimeContractSet φsΛ = φsΛ.1
```

lemma subContraction_eqTimeContractSet_eqTimeOnly[\[source\]](#)

```
lemma subContraction_eqTimeContractSet_eqTimeOnly {φs : List ℱ.FieldOp}
  (φsΛ : WickContraction φs.length) (h : {i, j} ∈ φsΛ.1) :
  (φsΛ.subContraction (eqTimeContractSet φsΛ) (eqTimeContractSet_subset φsΛ)).EqTimeOnly
```

lemma pair_mem_eqTimeContractSet_iff[\[source\]](#)

```
lemma pair_mem_eqTimeContractSet_iff {φs : List ℱ.FieldOp} {i j : Fin φs.length}
  (φsΛ : WickContraction φs.length) (h : {i, j} ∈ φsΛ.1) :
  {i, j} ∈ φsΛ.eqTimeContractSet ↔ timeOrderRel φs[i] φs[j] ∧ timeOrderRel φs[j] φs[i]
```

lemma subContraction_eqTimeContractSet_not_empty_of_haveEqTime[\[source\]](#)

```
lemma subContraction_eqTimeContractSet_not_empty_of_haveEqTime
  {φs : List ℱ.FieldOp} (φsΛ : WickContraction φs.length) (h : HaveEqTime φsΛ) :
  φsΛ.subContraction (eqTimeContractSet φsΛ) (eqTimeContractSet_subset φsΛ) ≠ empty
```

lemma quotContraction_eqTimeContractSet_not_haveEqTime[\[source\]](#)

```

lemma quotContraction_eqTimeContractSet_not_haveEqTime { $\phi s$  : List  $\mathcal{F}.FieldOp$ }
  ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ ) :
   $\neg$  HaveEqTime ( $\phi s\Lambda.quotContraction$  (eqTimeContractSet  $\phi s\Lambda$ ) (eqTimeContractSet_subset  $\phi s\Lambda$ ))

```

lemma join_haveEqTime_of_eqTimeOnly_nonEmpty[\[source\]](#)

```

lemma join_haveEqTime_of_eqTimeOnly_nonEmpty { $\phi s$  : List  $\mathcal{F}.FieldOp$ } ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ )
  ( $h1$  :  $\phi s\Lambda.EqTimeOnly$ ) ( $h2$  :  $\phi s\Lambda \neq \text{empty}$ )
  ( $\phi suc\Lambda$  : WickContraction [ $\phi s\Lambda.1$ ] $uc$ .length) :
  HaveEqTime (join  $\phi s\Lambda$   $\phi suc\Lambda$ )

```

lemma hasEqTimeEquiv_ext_sigma[\[source\]](#)

```

lemma hasEqTimeEquiv_ext_sigma { $\phi s$  : List  $\mathcal{F}.FieldOp$ } { $x1$   $x2$  :
   $\Sigma$  ( $\phi s\Lambda$  : { $\phi s\Lambda$  : WickContraction  $\phi s.length$  //  $\phi s\Lambda.EqTimeOnly \wedge \phi s\Lambda \neq \text{empty}$ }),
  { $\phi ssuc\Lambda$  : WickContraction [ $\phi s\Lambda.1$ ] $uc$ .length //  $\neg$  HaveEqTime  $\phi ssuc\Lambda$ }}
  ( $h1$  :  $x1.1.1 = x2.1.1$ ) ( $h2$  :  $x1.2.1 = \text{congr (by simp [h1]) } x2.2.1$ ) :  $x1 = x2$ 

```

def hasEqTimeEquiv[\[source\]](#)

The equivalence which separates a Wick contraction which has an equal time contraction into a non-empty contraction only between equal-time fields and a Wick contraction which does not have equal time contractions.

```

def hasEqTimeEquiv ( $\phi s$  : List  $\mathcal{F}.FieldOp$ ) :
  { $\phi s\Lambda$  : WickContraction  $\phi s.length$  // HaveEqTime  $\phi s\Lambda$ }  $\approx$ 
   $\Sigma$  ( $\phi s\Lambda$  : { $\phi s\Lambda$  : WickContraction  $\phi s.length$  //  $\phi s\Lambda.EqTimeOnly \wedge \phi s\Lambda \neq \text{empty}$ }),
  { $\phi ssuc\Lambda$  : WickContraction [ $\phi s\Lambda.1$ ] $uc$ .length //  $\neg$  HaveEqTime  $\phi ssuc\Lambda$ } where

```

lemma sum_haveEqTime[\[source\]](#)

```

lemma sum_haveEqTime ( $\phi s$  : List  $\mathcal{F}.FieldOp$ )
  ( $f$  : WickContraction  $\phi s.length \rightarrow M$ ) [AddCommMonoid  $M$ ]:
   $\Sigma$  ( $\phi s\Lambda$  : { $\phi s\Lambda$  : WickContraction  $\phi s.length$  // HaveEqTime  $\phi s\Lambda$ }),  $f$   $\phi s\Lambda$  =
   $\Sigma$  ( $\phi s\Lambda$  : { $\phi s\Lambda$  : WickContraction  $\phi s.length$  //  $\phi s\Lambda.EqTimeOnly \wedge \phi s\Lambda \neq \text{empty}$ }),
   $\Sigma$  ( $\phi ssuc\Lambda$  : { $\phi ssuc\Lambda$  : WickContraction [ $\phi s\Lambda.1$ ] $uc$ .length //  $\neg$  HaveEqTime  $\phi ssuc\Lambda$ }),
   $f$  (join  $\phi s\Lambda.1$   $\phi ssuc\Lambda.1$ )

```

2.56 PhysLean.QFT.PerturbationTheory.WickContraction.TimeContract

[PhysLean/QFT/PerturbationTheory/WickContraction/TimeContract.lean](#) on GitHub.

lemma timeContract_insert_none[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s\Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, and a $i \leq \phi s.length$ the following relation holds

$$(\phi s\Lambda \hookrightarrow \Lambda \ \phi \ i \ \text{none}).timeContract = \phi s\Lambda.timeContract$$

The proof of this result ultimately is a consequence of definitions.

@[simp]

```
lemma timeContract_insert_none (φ : F.FieldOp) (φs : List F.FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) :
  (φsΛ ↪Λ φ i none).timeContract = φsΛ.timeContract
```

lemma timeContract_insertAndContract_some

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$, then $(\phi s \Lambda \hookrightarrow \Lambda \phi i \text{ (some } k)).timeContract$ is equal to the product of $- timeContract \phi \phi s[k]$ if $i \leq k$ or $timeContract \phi s[k] \phi$ if $k < i - \phi s \Lambda.timeContract$.

The proof of this result ultimately is a consequence of definitions.

```
lemma timeContract_insertAndContract_some
  (φ : F.FieldOp) (φs : List F.FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : φsΛ.uncontracted) :
  (φsΛ ↪Λ φ i (some j)).timeContract =
  (if i < i.succAbove j then
    (FieldOpAlgebra.timeContract φ φs[j.1], timeContract_mem_center _ _)
  else (FieldOpAlgebra.timeContract φs[j.1] φ, timeContract_mem_center _ _)) *
  φsΛ.timeContract
```

lemma timeContract_empty

[\[source\]](#)

@[simp]

```
lemma timeContract_empty (φs : List F.FieldOp) :
  (@empty φs.length).timeContract = 1
```

lemma timeContract_insert_some_of_lt

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$ such that $i \leq k$, with the condition that ϕ has greater or equal time to $\phi s[k]$, then $(\phi s \Lambda \hookrightarrow \Lambda \phi i \text{ (some } k)).timeContract$ is equal to the product of $- [anPart \phi, \phi s[k]]_s - \phi s \Lambda.timeContract$ - two copies of the exchange sign of ϕ with the uncontracted fields in $\phi_0 \dots \phi_{k-1}$. These two exchange signs cancel each other out but are included for convenience.

The proof of this result ultimately is a consequence of definitions and `timeContract_of_timeOrderRel`.

```
lemma timeContract_insert_some_of_lt
  (φ : F.FieldOp) (φs : List F.FieldOp)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (k : φsΛ.uncontracted)
  (ht : F.timeOrderRel φ φs[k.1]) (hik : i < i.succAbove k) :
  (φsΛ ↪Λ φ i (some k)).timeContract =
  S(F |>_s φ, F |>_s (φs.get, (φsΛ.uncontracted.filter (fun x => x < k))))
  • (contractStateAtIndex φ [φsΛ]uc ((uncontractedFieldOpEquiv φs φsΛ) (some k))) *
  φsΛ.timeContract
```

lemma timeContract_insert_some_of_not_lt

[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$ such that $k < i$, with the condition that $\phi s[k]$ does not have time greater or equal to ϕ , then $(\phi s \Lambda \hookrightarrow \Lambda \phi i \text{ (some } k)).timeContract$ is equal to $timeContract \phi s \Lambda$.

k)).timeContract is equal to the product of - [anPart φ , $\varphi_s[k]$] $_s$ - $\varphi_s\Lambda$.timeContract - the exchange sign of φ with the uncontracted fields in $\varphi_{\emptyset \dots \varphi_{k-1}}$. - the exchange sign of φ with the uncontracted fields in $\varphi_{\emptyset \dots \varphi_k}$.

The proof of this result ultimately is a consequence of definitions and timeContract_of_not_timeOrderRel_ex

```

lemma timeContract_insert_some_of_not_lt
  (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp)
  (ψsΛ : WickContraction ψs.length) (i : Fin ψs.length.succ) (k : ψsΛ.uncontracted)
  (ht : ¬  $\mathcal{F}$ .timeOrderRel ψs[k.1] φ) (hik : ¬ i < i.succAbove k) :
  (ψsΛ  $\hookleftarrow$ Λ φ i (some k)).timeContract =
  S( $\mathcal{F}$  |> $_s$  φ,  $\mathcal{F}$  |> $_s$  (ψs.get, (ψsΛ.uncontracted.filter (fun x => x ≤ k))))
  • (contractStateAtIndex φ [ψsΛ] $^{uc}$ 
    ((uncontractedFieldOpEquiv ψs ψsΛ) (some k)) * ψsΛ.timeContract)

```

2.57 PhysLean.QFT.PerturbationTheory.WickContraction.Uncontracted

[PhysLean/QFT/PerturbationTheory/WickContraction/Uncontracted.lean](#) on GitHub.

lemma getDual?_eq_none_iff_mem_uncontracted

[\[source\]](#)

```

lemma getDual?_eq_none_iff_mem_uncontracted (i : Fin n) :
  c.getDual? i = none ↔ i ∈ c.uncontracted

```

lemma mem_uncontracted_empty

[\[source\]](#)

```

lemma mem_uncontracted_empty (i : Fin n) : i ∈ empty.uncontracted

```

lemma getDual?_empty_eq_none

[\[source\]](#)

```

@[simp]
lemma getDual?_empty_eq_none (i : Fin n) : empty.getDual? i = none

```

lemma uncontracted_empty

[\[source\]](#)

```

@[simp]
lemma uncontracted_empty {n : ℕ} : (@empty n).uncontracted = Finset.univ

```

lemma uncontracted_card_le

[\[source\]](#)

```

lemma uncontracted_card_le (c : WickContraction n) : c.uncontracted.card ≤ n

```

lemma uncontracted_card_eq_iff

[\[source\]](#)

```

lemma uncontracted_card_eq_iff (c : WickContraction n) :
  c.uncontracted.card = n ↔ c = empty

```

2.58 PhysLean.QFT.PerturbationTheory.WickContraction.UncontractedList

[PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList.lean](#) on GitHub.

lemma fin_finset_sort_map_monotone[\[source\]](#)

```

lemma fin_finset_sort_map_monotone {n m : ℕ} (a : Finset (Fin n)) (f : Fin n → Fin m)
  (hf : StrictMono f) : (Finset.sort (· ≤ ·) a).map f =
  (Finset.sort (· ≤ ·) (a.map f))

```

lemma uncontractedList_empty[\[source\]](#)

```

@[simp]
lemma uncontractedList_empty : (empty (n := n)).uncontractedList = List.finRange n

```

lemma uncontractedList_sorted_lt[\[source\]](#)

```

lemma uncontractedList_sorted_lt : List.Sorted (· < ·) c.uncontractedList

```

lemma uncontractedListGet_empty[\[source\]](#)

```

@[simp]
lemma uncontractedListGet_empty {φs : List ℱ.FieldOp} :
  (empty (n := φs.length)).uncontractedListGet = φs

```

def uncontractedFieldOpEquiv[\[source\]](#)

The equivalence between the type `Option c.uncontracted` for `WickContraction φs.length` and `Option (Fin (c.uncontractedList.map φs.get).length)`, that is optional positions of `c.uncontractedList.map φs.get` induced by `uncontractedIndexEquiv`.

```

def uncontractedFieldOpEquiv (φs : List ℱ.FieldOp) (φsΛ : WickContraction φs.length) :
  Option φsΛ.uncontracted ≈ Option (Fin [φsΛ]uc.length)

```

lemma uncontractedFieldOpEquiv_none[\[source\]](#)

```

@[simp]
lemma uncontractedFieldOpEquiv_none (φs : List ℱ.FieldOp) (φsΛ : WickContraction φs.length) :
  (uncontractedFieldOpEquiv φs φsΛ).toFun none = none

```

lemma uncontractedFieldOpEquiv_list_sum[\[source\]](#)

```

lemma uncontractedFieldOpEquiv_list_sum [AddCommMonoid α] (φs : List ℱ.FieldOp)
  (φsΛ : WickContraction φs.length) (f : Option (Fin [φsΛ]uc.length) → α) :
  ∑ (i : Option (Fin [φsΛ]uc.length)), f i =
  ∑ (i : Option φsΛ.uncontracted), f (φsΛ.uncontractedFieldOpEquiv φs i)

```

def uncontractedListEmd[\[source\]](#)

The embedding of `Fin [φsΛ]uc.length` into `Fin φs.length`.

```

def uncontractedListEmd {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.length} :
  Fin [φsΛ]uc.length → Fin φs.length

```

lemma uncontractedListEmd_congr[\[source\]](#)

```
lemma uncontractedListEmd_congr {φs : List ℱ.FieldOp} {φsΛ φsΛ' : WickContraction φs.length}
  (h : φsΛ = φsΛ') : φsΛ.uncontractedListEmd =
  (finCongr (by simp [h])).toEmbedding.trans φsΛ'.uncontractedListEmd
```

lemma uncontractedListEmd_toFun_eq_get[\[source\]](#)

```
lemma uncontractedListEmd_toFun_eq_get (φs : List ℱ.FieldOp) (φsΛ : WickContraction φs.length)
  :
  (uncontractedListEmd (φsΛ := φsΛ)).toFun =
  φsΛ.uncontractedList.get ∘ (finCongr (by simp [uncontractedListGet]))
```

lemma uncontractedListEmd_strictMono[\[source\]](#)

```
lemma uncontractedListEmd_strictMono {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.length}
  {i j : Fin [φsΛ]uc.length} (h : i < j) : uncontractedListEmd i < uncontractedListEmd j
```

lemma uncontractedListEmd_mem_uncontracted[\[source\]](#)

```
lemma uncontractedListEmd_mem_uncontracted {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.
  length}
  (i : Fin [φsΛ]uc.length) : uncontractedListEmd i ∈ φsΛ.uncontracted
```

lemma uncontractedListEmd_surjective_mem_uncontracted[\[source\]](#)

```
lemma uncontractedListEmd_surjective_mem_uncontracted {φs : List ℱ.FieldOp}
  {φsΛ : WickContraction φs.length} (i : Fin φs.length) (hi : i ∈ φsΛ.uncontracted) :
  ∃ j, φsΛ.uncontractedListEmd j = i
```

lemma uncontractedListEmd_finset_disjoint_left[\[source\]](#)

```
@[simp]
lemma uncontractedListEmd_finset_disjoint_left {φs : List ℱ.FieldOp}
  {φsΛ : WickContraction φs.length} (a : Finset (Fin [φsΛ]uc.length))
  (b : Finset (Fin φs.length)) (hb : b ∈ φsΛ.1) : Disjoint (a.map uncontractedListEmd) b
```

lemma uncontractedListEmd_finset_not_mem[\[source\]](#)

```
lemma uncontractedListEmd_finset_not_mem {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.
  length}
  (a : Finset (Fin [φsΛ]uc.length)) :
  a.map uncontractedListEmd ∉ φsΛ.1
```

lemma getElem_uncontractedListEmd[\[source\]](#)

```
@[simp]
lemma getElem_uncontractedListEmd {φs : List ℱ.FieldOp} {φsΛ : WickContraction φs.length}
  (k : Fin [φsΛ]uc.length) : φs[(uncontractedListEmd k).1] = [φsΛ]uc[k.1]
```

lemma uncontractedListEmd_empty[\[source\]](#)

```
@[simp]
lemma uncontractedListEmd_empty {φs : List  $\mathcal{F}$ .FieldOp} :
  (empty (n := φs.length)).uncontractedListEmd = (finCongr (by simp)).toEmbedding
```

2.59 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Basic

[PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Basic.lean](#) on GitHub.

lemma ofReal_hasDerivAt[\[source\]](#)

```
lemma Complex.ofReal_hasDerivAt : HasDerivAt Complex.ofReal 1 x
```

lemma deriv_ofReal[\[source\]](#)

```
@[simp]
lemma Complex.deriv_ofReal : deriv Complex.ofReal x = 1
```

lemma differentiableAt_ofReal[\[source\]](#)

```
@[fun_prop]
lemma Complex.differentiableAt_ofReal : DifferentiableAt  $\mathbb{R}$  Complex.ofReal x
```

structure HarmonicOscillator[\[source\]](#)

A quantum harmonic oscillator specified by a three real parameters: the mass of the particle m , a value of Planck's constant $\hbar$, and an angular frequency ω . All three of these parameters are assumed to be positive.

```
structure HarmonicOscillator where
  m :  $\mathbb{R}$ 
   $\hbar$  :  $\mathbb{R}$ 
   $\omega$  :  $\mathbb{R}$ 
   $\hbar \hbar$  :  $0 < \hbar$ 
   $\hbar \omega$  :  $0 < \omega$ 
   $\hbar m$  :  $0 < m$ 
```

lemma m_mul_ω_div_two_ħ_pos[\[source\]](#)

```
@[simp]
lemma m_mul_ω_div_two_ħ_pos :  $0 < Q.m * Q.\omega / (2 * Q.\hbar)$ 
```

lemma m_mul_ω_div_ħ_pos[\[source\]](#)

```
@[simp]
lemma m_mul_ω_div_ħ_pos :  $0 < Q.m * Q.\omega / Q.\hbar$ 
```

lemma m_ne_zero[\[source\]](#)

```
lemma m_ne_zero :  $Q.m \neq 0$ 
```


lemma $\hbar_ne_zero$ [\[source\]](#)**lemma** $\hbar_ne_zero : Q.\hbar \neq 0$ **def ξ** [\[source\]](#)*The characteristic length ξ of the harmonic oscillator is defined as $\sqrt{(\hbar / (m \omega))}$.***noncomputable def** $\xi : \mathbb{R}$ **lemma ξ_nonneg** [\[source\]](#)**lemma** $\xi_nonneg : 0 \leq Q.\xi$ **lemma ξ_pos** [\[source\]](#)

@[simp]

lemma $\xi_pos : 0 < Q.\xi$ **lemma ξ_ne_zero** [\[source\]](#)

@[simp]

lemma $\xi_ne_zero : Q.\xi \neq 0$ **lemma ξ_sq** [\[source\]](#)**lemma** $\xi_sq : Q.\xi^2 = Q.\hbar / (Q.m * Q.\omega)$ **lemma ξ_abs** [\[source\]](#)

@[simp]

lemma $\xi_abs : \text{abs } Q.\xi = Q.\xi$ **lemma one_over_xi** [\[source\]](#)**lemma** $one_over_xi : 1/Q.\xi = \sqrt{(Q.m * Q.\omega / Q.\hbar)}$ **lemma $\xi_inverse$** [\[source\]](#)**lemma** $\xi_inverse : Q.\xi^{-1} = \sqrt{(Q.m * Q.\omega / Q.\hbar)}$ **lemma $one_over_xi_sq$** [\[source\]](#)**lemma** $one_over_xi_sq : (1/Q.\xi)^2 = Q.m * Q.\omega / Q.\hbar$ **def momentumOperator**[\[source\]](#)*The momentum operator is defined as the map from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking ψ to $-i \hbar \psi'$.**The notation $P^{\circ P}$ can be used for the momentum operator.*

```
noncomputable def momentumOperator (ψ : ℝ → ℂ) : ℝ → ℂ
```

def positionOperator

[\[source\]](#)

The position operator is defined as the map from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking ψ to $x \psi$.

The notation $X \circ P$ can be used for the momentum operator.

```
noncomputable def positionOperator (ψ : ℝ → ℂ) : ℝ → ℂ
```

def schrodingerOperator

[\[source\]](#)

For a harmonic oscillator, the Schrodinger Operator corresponding to it is defined as the function from $\mathbb{R} \rightarrow \mathbb{C}$ to $\mathbb{R} \rightarrow \mathbb{C}$ taking

$$\psi \mapsto -\hbar^2 / (2 * m) * \psi'' + 1/2 * m * \omega^2 * x^2 * \psi.$$

```
noncomputable def schrodingerOperator (ψ : ℝ → ℂ) : ℝ → ℂ
```

lemma schrodingerOperator_eq

[\[source\]](#)

```
lemma schrodingerOperator_eq (ψ : ℝ → ℂ) :
  Q.schrodingerOperator ψ = (- Q.ħ ^ 2 / (2 * Q.m)) •
  (deriv (deriv ψ)) + (1/2 * Q.m * Q.ω^2) • ((fun x => Complex.ofReal x ^2) * ψ)
```

lemma schrodingerOperator_eq_ξ

[\[source\]](#)

The schrodinger operator written in terms of ξ .

```
lemma schrodingerOperator_eq_ξ (ψ : ℝ → ℂ) : Q.schrodingerOperator ψ =
  fun x => (Q.ħ ^ 2 / (2 * Q.m)) * (- (deriv (deriv ψ) x) + (1/Q.ξ^2) ^2 * x^2 * ψ x)
```

lemma schrodingerOperator_parity

[\[source\]](#)

The Schrodinger operator commutes with the parity operator.

```
lemma schrodingerOperator_parity (ψ : ℝ → ℂ) :
  Q.schrodingerOperator (parity ψ) = parity (Q.schrodingerOperator ψ)
```

2.60 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Completeness

[PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Completeness.lean](#) on GitHub.

lemma mul_eigenfunction_integrable

[\[source\]](#)

```
lemma mul_eigenfunction_integrable (f : ℝ → ℂ) (hf : MemHS f) :
  MeasureTheory.Integrable (fun x => Q.eigenfunction n x * f x)
```

lemma mul_physHermite_integrable

[\[source\]](#)

```
lemma mul_physHermite_integrable (f : ℝ → ℂ) (hf : MemHS f) (n : ℕ) :
  MeasureTheory.Integrable (fun x => physHermite n (x/Q.ξ) *
    (f x * ↑(Real.exp (- x ^ 2 / (2 * Q.ξ^2)))))
```

lemma mul_polynomial_integrable[\[source\]](#)

```
lemma mul_polynomial_integrable (f : ℝ → ℂ) (hf : MemHS f) (P : Polynomial ℤ) :
  MeasureTheory.Integrable (fun x => (P (x / Q.ξ)) *
    (f x * Real.exp (- x^2 / (2 * Q.ξ^2)))) volume
```

lemma mul_power_integrable[\[source\]](#)

```
lemma mul_power_integrable (f : ℝ → ℂ) (hf : MemHS f) (r : ℕ) :
  MeasureTheory.Integrable (fun x => x ^ r * (f x * Real.exp (- x^2 / (2 * Q.ξ^2)))) volume
```

lemma orthogonal_eigenfunction_of_mem_orthogonal[\[source\]](#)

```
lemma orthogonal_eigenfunction_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (n : ℕ) : ∫ (x : ℝ), Q.eigenfunction n x * f x = 0
```

lemma orthogonal_physHermite_of_mem_orthogonal[\[source\]](#)

```
lemma orthogonal_physHermite_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (n : ℕ) : ∫ (x : ℝ), (physHermite n (x / Q.ξ)) * (f x * ↑(Real.exp (- x ^ 2 / (2 * Q.ξ ^ 2)
  )))
  = 0
```

lemma orthogonal_polynomial_of_mem_orthogonal[\[source\]](#)

```
lemma orthogonal_polynomial_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (P : Polynomial ℤ) :
  ∫ x : ℝ, (P (x / Q.ξ)) * (f x * Real.exp (- x^2 / (2 * Q.ξ^2))) = 0
```

lemma orthogonal_power_of_mem_orthogonal[\[source\]](#)

If f is a function $\mathbb{R} \rightarrow \mathbb{C}$ satisfying $\text{MemHS } f$ such that it is orthogonal to all eigenfunction n then it is orthogonal to

$$x^r * e^{(-x^2 / (2 \xi^2))}$$

the proof of this result relies on the fact that Hermite polynomials span polynomials.

```
lemma orthogonal_power_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (r : ℕ) : ∫ x : ℝ, (x ^ r * (f x * Real.exp (- x^2 / (2 * Q.ξ^2)))) = 0
```

lemma orthogonal_exp_of_mem_orthogonal[\[source\]](#)

If f is a function $\mathbb{R} \rightarrow \mathbb{C}$ satisfying $\text{MemHS } f$ such that it is orthogonal to all eigenfunction n then it is orthogonal to

$$e^{(I c x)} * e^{(-x^2 / (2 \xi^2))}$$

for any real c . The proof of this result relies on the expansion of $e^{(I c x)}$ in terms of $x^r/r!$ and using $\text{orthogonal_power_of_mem_orthogonal}$ along with integrability conditions.

```

lemma orthogonal_exp_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (c : ℝ) : ∫ x : ℝ, Complex.exp (Complex.I * c * x) *
    (f x * Real.exp (- x^2 / (2 * Q.ξ^2))) = 0

```

lemma fourierIntegral_zero_of_mem_orthogonal

[\[source\]](#)

If f is a function $\mathbb{R} \rightarrow \mathbb{C}$ satisfying `MemHS f` such that it is orthogonal to all eigenfunction n then the fourier transform of

$$f(x) * e^{(-x^2 / (2 \xi^2))}$$

is zero.

The proof of this result relies on `orthogonal_exp_of_mem_orthogonal`.

```

lemma fourierIntegral_zero_of_mem_orthogonal (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0) :
   $\mathcal{F}$  (fun x => f x * Real.exp (- x^2 / (2 * Q.ξ^2))) = 0

```

lemma zero_of_orthogonal_mk

[\[source\]](#)

```

lemma zero_of_orthogonal_mk (f : ℝ → ℂ) (hf : MemHS f)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), HilbertSpace.mk hf⟩⟩_ℂ = 0)
  (plancherel_theorem : ∀ {f : ℝ → ℂ} (hf : Integrable f volume) (_ : Memℒp f 2),
    eLpNorm ( $\mathcal{F}$  f) 2 volume = eLpNorm f 2 volume) :
  HilbertSpace.mk hf = 0

```

lemma zero_of_orthogonal_eigenVector

[\[source\]](#)

```

lemma zero_of_orthogonal_eigenVector (f : HilbertSpace)
  (hOrth : ∀ n : ℕ, ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n), f⟩⟩_ℂ = 0)
  (plancherel_theorem : ∀ {f : ℝ → ℂ} (hf : Integrable f volume) (_ : Memℒp f 2),
    eLpNorm ( $\mathcal{F}$  f) 2 volume = eLpNorm f 2 volume) : f = 0

```

theorem eigenfunction_completeness

[\[source\]](#)

Assuming Plancherel's theorem (which is not yet in Mathlib), the topological closure of the span of the eigenfunctions of the harmonic oscillator is the whole Hilbert space.

The proof of this result relies on `fourierIntegral_zero_of_mem_orthogonal` and Plancherel's theorem which together give us that the norm of

$$f(x) * e^{(-x^2 / (2 * \xi^2))}$$

is zero for f orthogonal to all eigenfunctions, and hence the norm of f is zero.

```

theorem eigenfunction_completeness
  (plancherel_theorem : ∀ {f : ℝ → ℂ} (hf : Integrable f volume) (_ : Memℒp f 2),
    eLpNorm ( $\mathcal{F}$  f) 2 volume = eLpNorm f 2 volume) :
  (Submodule.span ℂ
    (Set.range (fun n => HilbertSpace.mk (Q.eigenfunction_memHS n))))
    .topologicalClosure =  $\mathbb{C}$ 

```

2.61 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Eigenfunction

[PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Eigenfunction.lean](#) on GitHub.

def eigenfunction[\[source\]](#)

The n th eigenfunction of the Harmonic oscillator is defined as the function $\mathbb{R} \rightarrow \mathbb{C}$ taking $x : \mathbb{R}$ to

$$1/\sqrt{(2^n n!)} \ 1/\sqrt{(\pi \xi)} * \text{physHermite } n \ (x / \xi) * e^{(-x^2 / (2 \xi^2))}.$$

```
noncomputable def eigenfunction (n : ℕ) : ℝ → ℂ
```

lemma eigenfunction_eq[\[source\]](#)

```
lemma eigenfunction_eq (n : ℕ) :
  Q.eigenfunction n = fun (x : ℝ) => (1/√(2 ^ n * n !)) * (1/ √(√Real.pi * Q.ξ))) *
  Complex.ofReal (physHermite n (x / Q.ξ) * Real.exp (- x^2 / (2 * Q.ξ^2)))
```

lemma eigenfunction_zero[\[source\]](#)

```
lemma eigenfunction_zero : Q.eigenfunction 0 = fun (x : ℝ) =>
  (1/ √(√Real.pi * Q.ξ)) * Complex.exp (- x^2 / (2 * Q.ξ^2))
```

lemma eigenfunction_eq_mul_eigenfunction_zero[\[source\]](#)

```
lemma eigenfunction_eq_mul_eigenfunction_zero (n : ℕ) :
  Q.eigenfunction n = fun x => Complex.ofReal (1/√(2 ^ n * n !))
  * Complex.ofReal (physHermite n (x / Q.ξ)) * Q.eigenfunction 0 x
```

lemma eigenfunction_integrable[\[source\]](#)

The eigenfunctions are integrable.

```
@[fun_prop]
lemma eigenfunction_integrable (n : ℕ) : Integrable (Q.eigenfunction n)
```

lemma eigenfunction_conj[\[source\]](#)

The eigenfunctions are real.

```
@[simp]
lemma eigenfunction_conj (n : ℕ) (x : ℝ) :
  (starRingEnd ℂ) (Q.eigenfunction n x) = Q.eigenfunction n x
```

lemma eigenfunction_point_norm[\[source\]](#)

```
lemma eigenfunction_point_norm (n : ℕ) (x : ℝ) :
  ||Q.eigenfunction n x|| = (1/√(2 ^ n * n !)) * (1/ √(√Real.pi * Q.ξ))) *
  (|physHermite n (x / Q.ξ)| * Real.exp (- x ^ 2 / (2 * Q.ξ ^ 2)))
```

lemma eigenfunction_point_norm_sq[\[source\]](#)

```
lemma eigenfunction_point_norm_sq (n : ℕ) (x : ℝ) :
  ||Q.eigenfunction n x|| ^ 2 = (1/√(2 ^ n * n !)) * (1/ √(√Real.pi * Q.ξ))) ^ 2 *
  ((physHermite n (x / Q.ξ)) ^ 2 * Real.exp (- x^2 / Q.ξ ^ 2))
```

lemma eigenfunction_square_integrable[\[source\]](#)*The eigenfunctions are square integrable.*

```
@[fun_prop]
lemma eigenfunction_square_integrable (n : ℕ) :
  MeasureTheory.Integrable (fun x => ||Q.eigenfunction n x|| ^ 2)
```

lemma eigenfunction_aeStronglyMeasurable[\[source\]](#)*The eigenfunctions are almost everywhere strongly measurable.*

```
@[fun_prop]
lemma eigenfunction_aeStronglyMeasurable (n : ℕ) :
  MeasureTheory.AEStronglyMeasurable (Q.eigenfunction n)
```

lemma eigenfunction_memHS[\[source\]](#)*The eigenfunctions are members of the Hilbert space.*

```
lemma eigenfunction_memHS (n : ℕ) : MemHS (Q.eigenfunction n)
```

lemma eigenfunction_differentiableAt[\[source\]](#)*The eigenfunctions are differentiable.*

```
@[fun_prop]
lemma eigenfunction_differentiableAt (x : ℝ) (n : ℕ) :
  DifferentiableAt ℝ (Q.eigenfunction n) x
```

lemma eigenfunction_continuous[\[source\]](#)*The eigenfunctions are continuous.*

```
@[fun_prop]
lemma eigenfunction_continuous (n : ℕ) : Continuous (Q.eigenfunction n)
```

lemma eigenfunction_parity[\[source\]](#)*The n th eigenfunction is an eigenfunction of the parity operator with the eigenvalue $(-1)^n$.*

```
lemma eigenfunction_parity (n : ℕ) :
  parity (Q.eigenfunction n) = (-1) ^ n * Q.eigenfunction n
```

lemma eigenfunction_mul[\[source\]](#)*A simplification of the product of two eigen-functions.*

```
lemma eigenfunction_mul (n p : ℕ) :
  (Q.eigenfunction n x) * (Q.eigenfunction p x) =
  1/√(2 ^ n * n !) * 1/√(2 ^ p * p !) * (1/ (√Real.pi * Q.ξ)) * Complex.ofReal
  (physHermite n (x / Q.ξ) * physHermite p (x / Q.ξ) * Real.exp (- x^2 / Q.ξ^2))
```

lemma eigenfunction_mul_self[\[source\]](#)

```
lemma eigenfunction_mul_self (n : ℕ) : (Q.eigenfunction n x) * (Q.eigenfunction n x) =
  (1 / (2 ^ n * n!)) * (1 / (√Real.pi * Q.ξ)) *
  Complex.ofReal ((physHermite n (x / Q.ξ))^2 * Real.exp (- x^2 / Q.ξ^2))
```

lemma eigenfunction_normalized[\[source\]](#)

The eigenfunction are normalized.

```
lemma eigenfunction_normalized (n : ℕ) : ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n),
  HilbertSpace.mk (Q.eigenfunction_memHS n)⟩⟩_C = 1
```

lemma eigenfunction_orthogonal[\[source\]](#)

The eigen-functions of the quantum harmonic oscillator are orthogonal.

```
lemma eigenfunction_orthogonal {n p : ℕ} (hnp : n ≠ p) :
  ⟨⟨HilbertSpace.mk (Q.eigenfunction_memHS n),
  HilbertSpace.mk (Q.eigenfunction_memHS p)⟩⟩_C = 0
```

lemma eigenfunction_orthonormal[\[source\]](#)

The eigenfunctions are orthonormal within the Hilbert space.

```
lemma eigenfunction_orthonormal :
  Orthonormal C (fun n => HilbertSpace.mk (Q.eigenfunction_memHS n))
```

lemma eigenfunction_linearIndependent[\[source\]](#)

The eigenfunctions are linearly independent.

```
lemma eigenfunction_linearIndependent :
  LinearIndependent C (fun n => HilbertSpace.mk (Q.eigenfunction_memHS n))
```

2.62 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.TISE

[PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/TISE.lean](#) on GitHub.

def eigenValue[\[source\]](#)

*The n th eigenvalues for a Harmonic oscillator is defined as $(n + 1/2) * \hbar * \omega$.*

```
noncomputable def eigenValue (n : ℕ) : ℝ
```

lemma deriv_eigenfunction_zero[\[source\]](#)

```
lemma deriv_eigenfunction_zero : deriv (Q.eigenfunction 0) =
  Complex.ofReal (- 1 / Q.ξ ^2) • Complex.ofReal * Q.eigenfunction 0
```

lemma deriv_eigenfunction_zero'[\[source\]](#)

```
lemma deriv_eigenfunction_zero' : deriv (Q.eigenfunction 0) =
  (- √2 / (2 * Q.ξ) : ℂ) • Q.eigenfunction 1
```

lemma deriv_physHermite_succ[\[source\]](#)

```

lemma deriv_physHermite_succ (n : ℕ) :
  deriv (fun x => Complex.ofReal (physHermite (n + 1) (x/Q.ξ))) = fun x =>
    Complex.ofReal (1/Q.ξ) * 2 * (n + 1) * physHermite n (x/Q.ξ)

```

lemma deriv_eigenfunction_succ[\[source\]](#)

```

lemma deriv_eigenfunction_succ (n : ℕ) :
  deriv (Q.eigenfunction (n + 1)) = fun x =>
    Complex.ofReal (1/√(2 ^ (n + 1) * (n + 1)!) * (1/Q.ξ)) *
    ((2 * (n + 1) * physHermite n (x/Q.ξ)
      - (x/Q.ξ) * physHermite (n + 1) (x/Q.ξ)) * Q.eigenfunction 0 x)

```

lemma deriv_deriv_eigenfunction_zero[\[source\]](#)

```

lemma deriv_deriv_eigenfunction_zero (x : ℝ) : deriv (deriv (Q.eigenfunction 0)) x =
  - (1/Q.ξ^2) * (Q.eigenfunction 0 x +
    x * (- (1/Q.ξ^2) * x * Q.eigenfunction 0 x))

```

lemma deriv_deriv_eigenfunction_succ[\[source\]](#)

```

lemma deriv_deriv_eigenfunction_succ (n : ℕ) (x : ℝ) :
  deriv (fun x => deriv (Q.eigenfunction (n + 1)) x) x =
    Complex.ofReal (1/√(2 ^ (n + 1) * (n + 1)!) * (1/Q.ξ)) *
    ((2 * (n + 1) * deriv (fun x => ↑(physHermite n (x/Q.ξ))) x +
      (- (1/Q.ξ^2)) * (4 * (n + 1) * x) *
      (physHermite n (x/Q.ξ)) + (- (1/Q.ξ)) * (1 + (- (1/Q.ξ^2)) * x ^ 2) *
      (physHermite (n + 1) (x/Q.ξ))) * Q.eigenfunction 0 x)

```

lemma deriv_deriv_eigenfunction_one[\[source\]](#)

```

lemma deriv_deriv_eigenfunction_one (x : ℝ) :
  deriv (fun x => deriv (Q.eigenfunction 1) x) x =
    Complex.ofReal (1/Real.sqrt (2 ^ 1 * 1!) * (1/Q.ξ)) *
    ((((- (1/ Q.ξ^2)) * (4 * x) + (- (1/ Q.ξ)) * (1 + (- (1/ Q.ξ^2)) * x ^ 2) *
      (2 * ((1/ Q.ξ) * x)))) * Q.eigenfunction 0 x)

```

lemma deriv_deriv_eigenfunction_succ_succ[\[source\]](#)

```

lemma deriv_deriv_eigenfunction_succ_succ (n : ℕ) (x : ℝ) :
  deriv (fun x => deriv (Q.eigenfunction (n + 2)) x) x = (- 1 / Q.ξ^2) * (2 * (n + 2)
    + (1 + (- 1/ Q.ξ^2) * x ^ 2)) * Q.eigenfunction (n + 2) x

```

lemma deriv_deriv_eigenfunction[\[source\]](#)

```

lemma deriv_deriv_eigenfunction (n : ℕ) (x : ℝ) :
  deriv (fun x => deriv (Q.eigenfunction n) x) x = (- 1 / Q.ξ^2) * ((2 * n + 1)
    + ((- 1/ Q.ξ^2) * x ^ 2)) * Q.eigenfunction n x

```

lemma schrodingerOperator_eigenfunction[\[source\]](#)

The n th eigenfunction satisfies the time-independent Schrodinger equation with respect to the n th eigenvalue. That is to say for Q a harmonic scillator,

*$Q.\text{schrodingerOperator } (Q.\text{eigenfunction } n) \ x = Q.\text{eigenValue } n * Q.\text{eigenfunction } n \ x.$*

The proof of this result is done by explicit calculation of derivatives.

```
lemma schrodingerOperator_eigenfunction (n : ℕ) (x : ℝ) :
  Q.schrodingerOperator (Q.eigenfunction n) x = Q.eigenValue n * Q.eigenfunction n x
```

2.63 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Basic

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Basic.lean](#) on GitHub.

abbrev HilbertSpace

[\[source\]](#)

The Hilbert space for a one dimensional quantum system is defined as the space of almost-everywhere equal equivalence classes of square integrable functions from $\mathbb{R}$ to $\mathbb{C}$.

```
noncomputable abbrev HilbertSpace
```

def MemHS

[\[source\]](#)

The proposition $\text{MemHS } f$ for a function $f : \mathbb{R} \rightarrow \mathbb{C}$ is defined to be true if the function f can be lifted to the Hilbert space.

```
def MemHS (f : ℝ → ℂ) : Prop
```

lemma aeStronglyMeasurable_of_memHS

[\[source\]](#)

```
lemma aeStronglyMeasurable_of_memHS {f : ℝ → ℂ} (h : MemHS f) :
  AESTronglyMeasurable f
```

lemma memHS_iff

[\[source\]](#)

A function f statisfies $\text{MemHS } f$ if and only if it is almost everywhere strongly measurable, and square integrable.

```
lemma memHS_iff {f : ℝ → ℂ} : MemHS f ↔
  AESTronglyMeasurable f ∧ Integrable (fun x => ||f x|| ^ 2)
```

lemma aeEqFun_mk_mem_iff

[\[source\]](#)

```
lemma aeEqFun_mk_mem_iff (f : ℝ → ℂ) (hf : AESTronglyMeasurable f volume) :
  AEEqFun.mk f hf ∈ HilbertSpace ↔ MemHS f
```

def mk

[\[source\]](#)

Given a function $f : \mathbb{R} \rightarrow \mathbb{C}$ such that $\text{MemHS } f$ is true via hf , then $\text{HilbertSpace.mk } hf$ is the element of the HilbertSpace defined by f .

```
def mk {f : ℝ → ℂ} (hf : MemHS f) : HilbertSpace
```

lemma coe_hilbertSpace_memHS[\[source\]](#)

```
lemma coe_hilbertSpace_memHS (f : HilbertSpace) : MemHS (f : ℝ → ℂ)
```

lemma mk_surjective[\[source\]](#)

```
lemma mk_surjective (f : HilbertSpace) : ∃ (g : ℝ → ℂ), ∃ (hg : MemHS g), mk hg = f
```

lemma coe_mk_ae[\[source\]](#)

```
lemma coe_mk_ae {f : ℝ → ℂ} (hf : MemHS f) : (mk hf : ℝ → ℂ) = [MeasureTheory.volume] f
```

lemma inner_mk_mk[\[source\]](#)

```
lemma inner_mk_mk {f g : ℝ → ℂ} {hf : MemHS f} {hg : MemHS g} :
  inner (mk hf) (mk hg) = ∫ x : ℝ, starRingEnd ℂ (f x) * g x
```

lemma eLpNorm_mk[\[source\]](#)

```
@[simp]
lemma eLpNorm_mk {f : ℝ → ℂ} {hf : MemHS f} :
  eLpNorm (mk hf) 2 volume = eLpNorm f 2 volume
```

lemma mem_iff'[\[source\]](#)

```
lemma mem_iff' {f : ℝ → ℂ} (hf : MeasureTheory.AEStronglyMeasurable f MeasureTheory.volume) :
  MeasureTheory.AEEqFun.mk f hf ∈ HilbertSpace
  ↔ MeasureTheory.Integrable (fun x => ||f x|| ^ 2)
```

lemma gaussian_integrable[\[source\]](#)

```
lemma gaussian_integrable {b : ℝ} (c : ℝ) (hb : 0 < b) :
  MeasureTheory.Integrable (fun x => (Real.exp (- b * (x - c)^ 2) : ℂ))
```

lemma gaussian_aestronglyMeasurable[\[source\]](#)

```
lemma gaussian_aestronglyMeasurable {b : ℝ} (c : ℝ) (hb : 0 < b) :
  AEStronglyMeasurable (fun x => (Real.exp (- b * (x - c) ^2) : ℂ)) volume
```

lemma gaussian_memHS[\[source\]](#)

```
lemma gaussian_memHS {b : ℝ} (c : ℝ) (hb : 0 < b) :
  MemHS (fun x => (Real.exp (- b * (x - c) ^2) : ℂ))
```

lemma exp_mul_gaussian_integrable[\[source\]](#)

```
lemma exp_mul_gaussian_integrable (b c : ℝ) (hb : 0 < b) :
  MeasureTheory.Integrable (fun x => Real.exp (c * x) * Real.exp (- b * x ^ 2))
```

lemma exp_abs_mul_gaussian_integrable[\[source\]](#)

```
lemma exp_abs_mul_gaussian_integrable (b c : ℝ) (hb : 0 < b) :
  MeasureTheory.Integrable (fun x => Real.exp (|c * x|) * Real.exp (- b * x ^ 2))
```

lemma mul_gaussian_mem_Lp_one[\[source\]](#)

```
lemma mul_gaussian_mem_Lp_one (f : ℝ → ℂ) (hf : MemHS f) (b c : ℝ) (hb : 0 < b) :
  MeasureTheory.MemLp (fun x => f x * Real.exp (- b * (x - c) ^ 2)) 1 volume
```

lemma mul_gaussian_mem_Lp_two[\[source\]](#)

```
lemma mul_gaussian_mem_Lp_two (f : ℝ → ℂ) (hf : MemHS f) (b c : ℝ) (hb : 0 < b) :
  MeasureTheory.MemLp (fun x => f x * Real.exp (- b * (x - c) ^ 2)) 2 volume
```

lemma abs_mul_gaussian_integrable[\[source\]](#)

```
lemma abs_mul_gaussian_integrable (f : ℝ → ℂ) (hf : MemHS f) (b c : ℝ) (hb : 0 < b) :
  MeasureTheory.Integrable (fun x => Complex.abs (f x) * Real.exp (- b * (x - c)^2))
```

lemma exp_mul_abs_mul_gaussian_integrable[\[source\]](#)

```
lemma exp_mul_abs_mul_gaussian_integrable (f : ℝ → ℂ) (hf : MemHS f)
  (b c : ℝ) (hb : 0 < b) : MeasureTheory.Integrable
  (fun x => Real.exp (c * x) * Complex.abs (f x) * Real.exp (- b * x ^ 2))
```

lemma exp_abs_mul_abs_mul_gaussian_integrable[\[source\]](#)

```
lemma exp_abs_mul_abs_mul_gaussian_integrable (f : ℝ → ℂ) (hf : MemHS f) (b c : ℝ) (hb : 0 < b) :
  :
  MeasureTheory.Integrable
  (fun x => Real.exp (|c * x|) * Complex.abs (f x) * Real.exp (- b * x ^ 2))
```

2.64 PhysLean.QuantumMechanics.OneDimension.HilbertSpace.Parity

[PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Parity.lean](#) on GitHub.

def parity[\[source\]](#)

The parity operator is defined as linear map from $\mathbb{R} \rightarrow \mathbb{C}$ to itself, such that ψ is taken to $\text{fun } x \Rightarrow \psi (-x)$.

```
def parity : (ℝ → ℂ) →l [ℂ] (ℝ → ℂ) where
```

lemma memHS_of_parity[\[source\]](#)

The parity operator acting on a member of the Hilbert space is in Hilbert space.

```
lemma memHS_of_parity (ψ : ℝ → ℂ) (hψ : MemHS ψ) : MemHS (parity ψ)
```

2.65 PhysLean.Relativity.Lorentz.ComplexTensor.Basic

[PhysLean/Relativity/Lorentz/ComplexTensor/Basic.lean](#) on GitHub.

instance CharZero complexLorentzTensor.k[\[source\]](#)

```
instance : CharZero complexLorentzTensor.k
```

lemma k_instSub[\[source\]](#)

```
@[simp]
lemma k_instSub : @HSub.hSub complexLorentzTensor.k complexLorentzTensor.k complexLorentzTensor.k
  k
  instHSub = @HSub.hSub ℂ ℂ ℂ instHSub
```

lemma k_instAdd[\[source\]](#)

```
@[simp]
lemma k_instAdd : @HAdd.hAdd complexLorentzTensor.k
  complexLorentzTensor.k complexLorentzTensor.k instHAdd = @HAdd.hAdd ℂ ℂ ℂ instHAdd
```

lemma k_neg[\[source\]](#)

```
@[simp]
lemma k_neg : @Neg.neg complexLorentzTensor.k = @Neg.neg ℂ
```

2.66 PhysLean.Relativity.Lorentz.ComplexTensor.Basis

[PhysLean/Relativity/Lorentz/ComplexTensor/Basis.lean](#) on GitHub.

lemma basisVector_eq_tensorBasis[\[source\]](#)

```
lemma basisVector_eq_tensorBasis {n : ℕ} (c : Fin n → complexLorentzTensor.C)
  (b : Π j, Fin (complexLorentzTensor.repDim (c j))) :
  basisVector c b = complexLorentzTensor.tensorBasis c b
```

2.67 PhysLean.Relativity.Lorentz.ComplexTensor.Metrics.Basis

[PhysLean/Relativity/Lorentz/ComplexTensor/Metrics/Basis.lean](#) on GitHub.

lemma coMetric_tensorBasis[\[source\]](#)

```
lemma coMetric_tensorBasis : η' =
  complexLorentzTensor.tensorBasis ![Color.down, Color.down] (fun _ => 0)
  - complexLorentzTensor.tensorBasis ![Color.down, Color.down] (fun _ => 1)
  - complexLorentzTensor.tensorBasis ![Color.down, Color.down] (fun _ => 2)
  - complexLorentzTensor.tensorBasis ![Color.down, Color.down] (fun _ => 3)
```

lemma coMetric_eq_ofRat[\[source\]](#)

```
lemma coMetric_eq_ofRat : η' = ofRat fun f =>
  if f 0 = 0 ∧ f 1 = 0 then 1 else
  if f 0 = f 1 then - 1 else 0
```

lemma contrMetric_basis_expand[\[source\]](#)

The expansion of the Lorentz contravariant metric in terms of basis vectors.

```
lemma contrMetric_basis_expand : {η | μ ν}^T.tensor =
  basisVector ![Color.up, Color.up] (fun _ => 0)
- basisVector ![Color.up, Color.up] (fun _ => 1)
- basisVector ![Color.up, Color.up] (fun _ => 2)
- basisVector ![Color.up, Color.up] (fun _ => 3)
```

lemma contrMetric_tensorBasis[\[source\]](#)

```
lemma contrMetric_tensorBasis : η =
  complexLorentzTensor.tensorBasis ![Color.up, Color.up] (fun _ => 0)
- complexLorentzTensor.tensorBasis ![Color.up, Color.up] (fun _ => 1)
- complexLorentzTensor.tensorBasis ![Color.up, Color.up] (fun _ => 2)
- complexLorentzTensor.tensorBasis ![Color.up, Color.up] (fun _ => 3)
```

lemma contrMetric_eq_ofRat[\[source\]](#)

```
lemma contrMetric_eq_ofRat : η = ofRat fun f =>
  if f 0 = 0 ∧ f 1 = 0 then 1 else
  if f 0 = f 1 then - 1 else 0
```

lemma contrMetric_basis_expand_tree[\[source\]](#)

The expansion of the Lorentz contravariant metric in terms of basis vectors as a structured tensor tree.

```
lemma contrMetric_basis_expand_tree : {η | μ ν}^T.tensor =
  (TensorTree.add (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 0))) <|
  TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 1)))) <|
  TensorTree.add (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 2)))) <|
  (smul (-1) (tensorNode (basisVector ![Color.up, Color.up] (fun _ => 3)))).tensor
```

lemma leftMetric_tensorBasis[\[source\]](#)

```
lemma leftMetric_tensorBasis : εL =
  - complexLorentzTensor.tensorBasis ![Color.upL, Color.upL] (fun | 0 => 0 | 1 => 1)
+ complexLorentzTensor.tensorBasis ![Color.upL, Color.upL] (fun | 0 => 1 | 1 => 0)
```

lemma leftMetric_eq_ofRat[\[source\]](#)

```
lemma leftMetric_eq_ofRat : εL = ofRat fun f =>
  if f 0 = 0 ∧ f 1 = 1 then - 1 else
  if f 1 = 0 ∧ f 0 = 1 then 1 else 0
```

lemma altLeftMetric_tensorBasis[\[source\]](#)

```
lemma altLeftMetric_tensorBasis : εL' =
  complexLorentzTensor.tensorBasis ![Color.downL, Color.downL] (fun | 0 => 0 | 1 => 1)
- complexLorentzTensor.tensorBasis ![Color.downL, Color.downL] (fun | 0 => 1 | 1 => 0)
```

lemma altLeftMetric_eq_ofRat[\[source\]](#)

```

lemma altLeftMetric_eq_ofRat :  $\varepsilon L' = \text{ofRat}$  fun f =>
  if f 0 = 0  $\wedge$  f 1 = 1 then 1 else
  if f 1 = 0  $\wedge$  f 0 = 1 then - 1 else 0

```

lemma rightMetric_tensorBasis[\[source\]](#)

```

lemma rightMetric_tensorBasis :  $\varepsilon R =$ 
  - complexLorentzTensor.tensorBasis ![Color.upR, Color.upR] (fun | 0 => 0 | 1 => 1)
  + complexLorentzTensor.tensorBasis ![Color.upR, Color.upR] (fun | 0 => 1 | 1 => 0)

```

lemma rightMetric_eq_ofRat[\[source\]](#)

```

lemma rightMetric_eq_ofRat :  $\varepsilon R = \text{ofRat}$  fun f =>
  if f 0 = 0  $\wedge$  f 1 = 1 then - 1 else
  if f 1 = 0  $\wedge$  f 0 = 1 then 1 else 0

```

lemma altRightMetric_tensorBasis[\[source\]](#)

```

lemma altRightMetric_tensorBasis :  $\varepsilon R' =$ 
  complexLorentzTensor.tensorBasis ![Color.downR, Color.downR] (fun | 0 => 0 | 1 => 1)
  - complexLorentzTensor.tensorBasis ![Color.downR, Color.downR] (fun | 0 => 1 | 1 => 0)

```

lemma altRightMetric_eq_ofRat[\[source\]](#)

```

lemma altRightMetric_eq_ofRat :  $\varepsilon R' = \text{ofRat}$  fun f =>
  if f 0 = 0  $\wedge$  f 1 = 1 then 1 else
  if f 1 = 0  $\wedge$  f 0 = 1 then - 1 else 0

```

2.68 PhysLean.Relativity.Lorentz.ComplexTensor.OfRat

[PhysLean/Relativity/Lorentz/ComplexTensor/OfRat.lean](#) on GitHub.

def ofRat[\[source\]](#)

A complex Lorentz tensor from a map $(\Pi j, \text{Fin } (\text{complexLorentzTensor.repDim } (c j))) \rightarrow \text{RatComplexNum}$. All complex Lorentz tensors with rational coefficients with respect to the basis are of this form.

```

noncomputable def ofRat {n :  $\mathbb{N}$ } {c :  $\text{Fin } n \rightarrow \text{complexLorentzTensor.C}$ }
  (f : ( $\Pi j, \text{Fin } (\text{complexLorentzTensor.repDim } (c j))$ )  $\rightarrow$  RatComplexNum) :
  complexLorentzTensor.F.obj (OverColor.mk c)

```

lemma ofRat_tensorBasis_repr_apply[\[source\]](#)

```

@[simp]
lemma ofRat_tensorBasis_repr_apply {n :  $\mathbb{N}$ } {c :  $\text{Fin } n \rightarrow \text{complexLorentzTensor.C}$ }
  (f : ( $\Pi j, \text{Fin } (\text{complexLorentzTensor.repDim } (c j))$ )  $\rightarrow$  RatComplexNum)
  (b :  $\Pi j, \text{Fin } (\text{complexLorentzTensor.repDim } (c j))$ ) :
  (complexLorentzTensor.tensorBasis c).repr (ofRat f) b = toComplexNum (f b)

```

lemma ofRat_add[\[source\]](#)

```

lemma ofRat_add {n : ℕ} {c : Fin n → complexLorentzTensor.C}
  (f f1 : (Π j, Fin (complexLorentzTensor.repDim (c j))) → RatComplexNum) :
  ofRat (f + f1) = ofRat f + ofRat f1

```

lemma tensorBasis_eq_ofRat[\[source\]](#)

```

lemma tensorBasis_eq_ofRat {n : ℕ} {c : Fin n → complexLorentzTensor.C}
  (b : Π j, Fin (complexLorentzTensor.repDim (c j))) :
  complexLorentzTensor.tensorBasis c b
  = ofRat (fun b' => if b = b' then (1, 0) else (0, 0))

```

lemma contr_basis_ratComplexNum[\[source\]](#)

```

lemma contr_basis_ratComplexNum {c : complexLorentzTensor.C}
  (i : Fin (complexLorentzTensor.repDim c))
  (j : Fin (complexLorentzTensor.repDim (complexLorentzTensor.τ c))) :
  complexLorentzTensor.castToField
    ((complexLorentzTensor.contr.app (Discrete.mk c)).hom
     (complexLorentzTensor.basis c i ⊗τ
      complexLorentzTensor.basis (complexLorentzTensor.τ c) j))
  = toComplexNum (if i.val = j.val then 1 else 0)

```

lemma prod_ofRat_ofRat[\[source\]](#)

```

lemma prod_ofRat_ofRat {n n1 : ℕ} {c : Fin n → complexLorentzTensor.C}
  (f : (Π j, Fin (complexLorentzTensor.repDim (c j))) → RatComplexNum)
  {c1 : Fin n1 → complexLorentzTensor.C}
  (f1 : (Π j, Fin (complexLorentzTensor.repDim (c1 j))) → RatComplexNum) :
  (TensorTree.prod (tensorNode (ofRat f)) (tensorNode (ofRat f1))).tensor =
  (tensorNode (ofRat (fun b => f (TensorSpecies.TensorBasis.prodEquiv b).1 *
    f1 (TensorSpecies.TensorBasis.prodEquiv b).2))).tensor

```

lemma contr_ofRat[\[source\]](#)

```

lemma contr_ofRat {n : ℕ} {c : Fin (n + 1 + 1) → complexLorentzTensor.C} {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = complexLorentzTensor.τ (c i)}
  (f : (Π k, Fin (complexLorentzTensor.repDim (c k))) → RatComplexNum) :
  (contr i j h (tensorNode (ofRat f))).tensor = (tensorNode (ofRat (fun b =>
    (∑ x : { x // x ∈ TensorSpecies.TensorBasis.ContrSection b },
      f x.1 * if (x.1 i).1 = (x.1 (i.succAbove j)).1 then 1 else 0))))).tensor

```

lemma smul_nat_ofRat[\[source\]](#)

```

lemma smul_nat_ofRat {c : Fin n → complexLorentzTensor.C} (n : ℕ)
  (f1 : (Π j, Fin (complexLorentzTensor.repDim (c j))) → RatComplexNum) :
  (TensorTree.smul n (tensorNode (ofRat f1))).tensor =
  (tensorNode (ofRat (fun b => n * f1 b))).tensor

```

lemma perm_ofRat[\[source\]](#)

```

lemma perm_ofRat {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
  {c1 : Fin m → complexLorentzTensor.C}

```

```

{σ : (OverColor.mk c) → (OverColor.mk c1)}
(f : (Π j, Fin (complexLorentzTensor.repDim (c j))) → RatComplexNum) :
(perm σ (tensorNode (ofRat f))).tensor =
(tensorNode (ofRat (fun b => f ((TensorSpecies.TensorBasis.congr (Hom.toEquiv σ)
(OverColor.Hom.toEquiv_comp_apply σ)) b)))).tensor

```

lemma add_ofRat[\[source\]](#)

```

lemma add_ofRat {n : ℕ} {c : Fin n → complexLorentzTensor.C}
(f f1 : (Π j, Fin (complexLorentzTensor.repDim (c j))) → RatComplexNum) :
(add (tensorNode (ofRat f)) (tensorNode (ofRat f1))).tensor =
(tensorNode (ofRat (fun b => f b + f1 b))).tensor

```

2.69 PhysLean.Relativity.Lorentz.ComplexTensor.Units.Basis

[PhysLean/Relativity/Lorentz/ComplexTensor/Units/Basis.lean](#) on GitHub.

lemma coContrUnit_tensorBasis[\[source\]](#)

```

lemma coContrUnit_tensorBasis : coContrUnit =
complexLorentzTensor.tensorBasis ![Color.down, Color.up] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.down, Color.up] (fun _ => 1)
+ complexLorentzTensor.tensorBasis ![Color.down, Color.up] (fun _ => 2)
+ complexLorentzTensor.tensorBasis ![Color.down, Color.up] (fun _ => 3)

```

lemma contrCoUnit_tensorBasis[\[source\]](#)

```

lemma contrCoUnit_tensorBasis : contrCoUnit =
complexLorentzTensor.tensorBasis ![Color.up, Color.down] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.down] (fun _ => 1)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.down] (fun _ => 2)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.down] (fun _ => 3)

```

lemma altLeftLeftUnit_tensorBasis[\[source\]](#)

```

lemma altLeftLeftUnit_tensorBasis : altLeftLeftUnit =
complexLorentzTensor.tensorBasis ![Color.downL, Color.upL] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.downL, Color.upL] (fun _ => 1)

```

lemma leftAltLeftUnit_tensorBasis[\[source\]](#)

```

lemma leftAltLeftUnit_tensorBasis : leftAltLeftUnit =
complexLorentzTensor.tensorBasis ![Color.upL, Color.downL] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.upL, Color.downL] (fun _ => 1)

```

lemma altRightRightUnit_tensorBasis[\[source\]](#)

```

lemma altRightRightUnit_tensorBasis : altRightRightUnit =
complexLorentzTensor.tensorBasis ![Color.downR, Color.upR] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.downR, Color.upR] (fun _ => 1)

```


lemma rightAltRightUnit_tensorBasis[\[source\]](#)

```

lemma rightAltRightUnit_tensorBasis : rightAltRightUnit =
  complexLorentzTensor.tensorBasis ![Color.upR, Color.downR] (fun _ => 0)
+ complexLorentzTensor.tensorBasis ![Color.upR, Color.downR] (fun _ => 1)

```

lemma coContrUnit_eq_ofRat[\[source\]](#)

```

lemma coContrUnit_eq_ofRat : δ' = ofRat fun f =>
  if f 0 = f 1 then 1 else 0

```

lemma contrCoUnit_eq_ofRat[\[source\]](#)

```

lemma contrCoUnit_eq_ofRat : δ = ofRat fun f =>
  if f 0 = f 1 then 1 else 0

```

2.70 PhysLean.Relativity.Lorentz.ComplexVector.Basic

[PhysLean/Relativity/Lorentz/ComplexVector/Basic.lean](#) on GitHub.

lemma complexContrBasisFin4_apply_zero[\[source\]](#)

```

@[simp]
lemma complexContrBasisFin4_apply_zero :
  complexContrBasisFin4 0 = complexContrBasis (Sum.inl 0)

```

lemma complexContrBasisFin4_apply_one[\[source\]](#)

```

@[simp]
lemma complexContrBasisFin4_apply_one :
  complexContrBasisFin4 1 = complexContrBasis (Sum.inr 0)

```

lemma complexContrBasisFin4_apply_two[\[source\]](#)

```

@[simp]
lemma complexContrBasisFin4_apply_two :
  complexContrBasisFin4 2 = complexContrBasis (Sum.inr 1)

```

lemma complexContrBasisFin4_apply_three[\[source\]](#)

```

@[simp]
lemma complexContrBasisFin4_apply_three :
  complexContrBasisFin4 3 = complexContrBasis (Sum.inr 2)

```

lemma complexContrBasisFin4_apply_succ[\[source\]](#)

```

@[simp]
lemma complexContrBasisFin4_apply_succ (i : Fin 3) :
  complexContrBasisFin4 i.succ = complexContrBasis (Sum.inr i)

```

lemma complexCoBasisFin4_apply_zero[\[source\]](#)

```
@[simp]
lemma complexCoBasisFin4_apply_zero :
  complexCoBasisFin4 0 = complexCoBasis (Sum.inl 0)
```

lemma complexCoBasisFin4_apply_one[\[source\]](#)

```
@[simp]
lemma complexCoBasisFin4_apply_one :
  complexCoBasisFin4 1 = complexCoBasis (Sum.inr 0)
```

lemma complexCoBasisFin4_apply_two[\[source\]](#)

```
@[simp]
lemma complexCoBasisFin4_apply_two :
  complexCoBasisFin4 2 = complexCoBasis (Sum.inr 1)
```

lemma complexCoBasisFin4_apply_three[\[source\]](#)

```
@[simp]
lemma complexCoBasisFin4_apply_three :
  complexCoBasisFin4 3 = complexCoBasis (Sum.inr 2)
```

2.71 PhysLean.Relativity.Lorentz.PauliMatrices.Basic

[PhysLean/Relativity/Lorentz/PauliMatrices/Basic.lean](#) on GitHub.

def pauliContr[\[source\]](#)

The Pauli matrices as the complex Lorentz tensor $\sigma^\mu \alpha^{\dot{\beta}}$.

```
def pauliContr
```

def pauliCo[\[source\]](#)

The Pauli matrices as the complex Lorentz tensor $\sigma_\mu \alpha^{\dot{\beta}}$.

```
def pauliCo
```

def pauliCoDown[\[source\]](#)

The Pauli matrices as the complex Lorentz tensor $\sigma_\mu \alpha_{\dot{\beta}}$.

```
def pauliCoDown
```

def pauliContrDown[\[source\]](#)

The Pauli matrices as the complex Lorentz tensor $\sigma^\mu \alpha_{\dot{\beta}}$.

```
def pauliContrDown
```

lemma tensorNode_pauliContr[\[source\]](#)*The definitional tensor node relation for pauliContr.*

```
lemma tensorNode_pauliContr : {pauliContr |  $\mu \alpha \beta$ }T.tensor =
  (TensorTree.constThreeNodeE complexLorentzTensor .up .upL .upR
   PauliMatrix.asConstTensor).tensor
```

lemma tensorNode_pauliCo[\[source\]](#)*The definitional tensor node relation for pauliCo.*

```
lemma tensorNode_pauliCo : {pauliCo |  $\mu \alpha \beta$ }T.tensor =
  { $\eta'$  |  $\mu \nu \otimes \text{pauliContr}$  |  $\nu \alpha \beta$ }T.tensor
```

lemma tensorNode_pauliCoDown[\[source\]](#)*The definitional tensor node relation for pauliCoDown.*

```
lemma tensorNode_pauliCoDown : {pauliCoDown |  $\mu \alpha \beta$ }T.tensor =
  {pauliCo |  $\mu \alpha \beta \otimes \epsilon R'$  |  $\beta \beta' \otimes \epsilon L'$  |  $\alpha \alpha'$ }T.tensor
```

lemma tensorNode_pauliContrDown[\[source\]](#)*The definitional tensor node relation for pauliContrDown.*

```
lemma tensorNode_pauliContrDown : {pauliContrDown |  $\mu \alpha \beta$ }T.tensor =
  {pauliContr |  $\mu \alpha \beta \otimes \epsilon R'$  |  $\beta \beta' \otimes \epsilon L'$  |  $\alpha \alpha'$ }T.tensor
```

lemma action_pauliContr[\[source\]](#)*The tensor pauliContr is invariant under the action of $SL(2, \mathbb{C})$.*

```
lemma action_pauliContr (g : SL(2,  $\mathbb{C}$ )) : {g •a pauliContr |  $\mu \alpha \beta$ }T.tensor =
  {pauliContr |  $\mu \alpha \beta$ }T.tensor
```

lemma action_pauliCo[\[source\]](#)*The tensor pauliCo is invariant under the action of $SL(2, \mathbb{C})$.*

```
lemma action_pauliCo (g : SL(2,  $\mathbb{C}$ )) : {g •a pauliCo |  $\mu \alpha \beta$ }T.tensor =
  {pauliCo |  $\mu \alpha \beta$ }T.tensor
```

lemma action_pauliCoDown[\[source\]](#)*The tensor pauliCoDown is invariant under the action of $SL(2, \mathbb{C})$.*

```
lemma action_pauliCoDown (g : SL(2,  $\mathbb{C}$ )) : {g •a pauliCoDown |  $\mu \alpha \beta$ }T.tensor =
  {pauliCoDown |  $\mu \alpha \beta$ }T.tensor
```

lemma action_pauliContrDown[\[source\]](#)*The tensor pauliContrDown is invariant under the action of $SL(2, \mathbb{C})$.*

```
lemma action_pauliContrDown (g : SL(2,ℂ)) : {g •a pauliContrDown | μ α β}T.tensor =
  {pauliContrDown | μ α β}T.tensor
```

2.72 PhysLean.Relativity.Lorentz.PauliMatrices.Basis

[PhysLean/Relativity/Lorentz/PauliMatrices/Basis.lean](#) on GitHub.

lemma pauliContr_in_basis

[\[source\]](#)

The expansion of the Pauli matrices $\sigma^\mu a^{\dot{a}}$ in terms of basis vectors.

```
lemma pauliContr_in_basis : {pauliContr | μ α β}T.tensor =
  basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 0 | 2 => 0)
+ basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 1 | 2 => 1)
+ basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 0 | 2 => 1)
+ basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 1 | 2 => 0)
- I • basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 0 | 2 => 1)
+ I • basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 1 | 2 => 0)
+ basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 0 | 2 => 0)
- basisVector ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

lemma pauliContr_tensorBasis

[\[source\]](#)

```
lemma pauliContr_tensorBasis : pauliContr =
  complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 0 | 1 => 0 | 2 => 0)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 0 | 1 => 1 | 2 => 1)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 1 | 1 => 0 | 2 => 1)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 1 | 1 => 1 | 2 => 0)
- I • complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 2 | 1 => 0 | 2 => 1)
+ I • complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 2 | 1 => 1 | 2 => 0)
+ complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 3 | 1 => 0 | 2 => 0)
- complexLorentzTensor.tensorBasis ![Color.up, Color.upL, Color.upR]
    (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

def pauliCoMap

[\[source\]](#)

The map to color one gets when lowering the indices of pauli matrices.

```
def pauliCoMap
```

lemma pauliContr_ofRat

[\[source\]](#)

```
lemma pauliContr_ofRat : pauliContr = ofRat (fun b =>
  if b = (fun | (0 : Fin 3) => 0 | 1 => 0 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 0 | 1 => 1 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 0 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 1 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 2 | 1 => 0 | 2 => 1) then (0, -1) else
```

```

if b = (fun | 0 => 2 | 1 => 1 | 2 => 0) then (0, 1) else
if b = (fun | 0 => 3 | 1 => 0 | 2 => 0) then (1, 0) else
if b = (fun | 0 => 3 | 1 => 1 | 2 => 1) then (-1, 0) else 0)

```

lemma pauliCo_ofRat[\[source\]](#)

```

lemma pauliCo_ofRat : pauliCo = ofRat (fun b =>
  if b = (fun | (0 : Fin 3) => 0 | 1 => 0 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 0 | 1 => 1 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 0 | 2 => 1) then (-1, 0) else
  if b = (fun | 0 => 1 | 1 => 1 | 2 => 0) then (-1, 0) else
  if b = (fun | 0 => 2 | 1 => 0 | 2 => 1) then (0, 1) else
  if b = (fun | 0 => 2 | 1 => 1 | 2 => 0) then (0, -1) else
  if b = (fun | 0 => 3 | 1 => 0 | 2 => 0) then (-1, 0) else
  if b = (fun | 0 => 3 | 1 => 1 | 2 => 1) then (1, 0) else (0, 0))

```

lemma pauliCoDown_ofRat[\[source\]](#)

```

lemma pauliCoDown_ofRat : pauliCoDown = ofRat (fun b =>
  if b = (fun | (0 : Fin 3) => 0 | 1 => 1 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 0 | 1 => 0 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 0 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 1 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 2 | 1 => 0 | 2 => 1) then (0, -1) else
  if b = (fun | 0 => 2 | 1 => 1 | 2 => 0) then (0, 1) else
  if b = (fun | 0 => 3 | 1 => 1 | 2 => 1) then (-1, 0) else
  if b = (fun | 0 => 3 | 1 => 0 | 2 => 0) then (1, 0) else (0, 0))

```

lemma pauliContrDown_ofRat[\[source\]](#)

```

lemma pauliContrDown_ofRat : pauliContrDown = ofRat (fun b =>
  if b = (fun | (0 : Fin 3) => 0 | 1 => 1 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 0 | 1 => 0 | 2 => 0) then (1, 0) else
  if b = (fun | 0 => 1 | 1 => 0 | 2 => 1) then (-1, 0) else
  if b = (fun | 0 => 1 | 1 => 1 | 2 => 0) then (-1, 0) else
  if b = (fun | 0 => 2 | 1 => 0 | 2 => 1) then (0, 1) else
  if b = (fun | 0 => 2 | 1 => 1 | 2 => 0) then (0, -1) else
  if b = (fun | 0 => 3 | 1 => 1 | 2 => 1) then (1, 0) else
  if b = (fun | 0 => 3 | 1 => 0 | 2 => 0) then (-1, 0) else 0)

```

2.73 PhysLean.Relativity.Lorentz.PauliMatrices.Relations

[PhysLean/Relativity/Lorentz/PauliMatrices/Relations.lean](#) on GitHub.

lemma pauliCo_contr_pauliContr[\[source\]](#)

The statement that $\sigma_v^{ab} \sigma^{va'b'} = 2 \epsilon^{aa'} \epsilon^{bb'}$.

```

lemma pauliCo_contr_pauliContr : {σ_^^ | v α β ⊗ σ_^^ | v α' β' = 2 •_t εL | α α' ⊗ εR | β β'}T

```

lemma pauliCoDown_trace_pauliCo[\[source\]](#)

```

lemma pauliCoDown_trace_pauliCo : {σ_ | μ β α ⊗ σ_^^ | v α β = 2 •_t η' | μ v}T

```

lemma pauliCo_trace_pauliCoDown[\[source\]](#)

```
lemma pauliCo_trace_pauliCoDown: {σ^__ | μ α β ⊗ σ^__ | ν β α = 2 •_τ η' | μ ν}^T
```

lemma pauliContr_mul_pauliContrDown_add[\[source\]](#)

```
lemma pauliContr_mul_pauliContrDown_add :
  {((σ^__ | μ α β ⊗ σ^__ | ν β α') + (σ^__ | ν α β ⊗ σ^__ | μ β α')) =
   2 •_τ η | μ ν ⊗ δL | α α'}^T
```

lemma auliContrDown_pauliContr_mul_add[\[source\]](#)

```
lemma auliContrDown_pauliContr_mul_add :
  {((σ^__ | μ β α ⊗ σ^__ | ν α β') + (σ^__ | ν β α ⊗ σ^__ | μ α β')) =
   2 •_τ η | μ ν ⊗ δR' | β β'}^T
```

2.74 PhysLean.Relativity.Tensors.TensorSpecies.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

lemma FD_map_basis[\[source\]](#)

```
lemma FD_map_basis {c c1 : S.C} (h : c = c1) (i : Fin (S.repDim c)) :
  (S.FD.map (Discrete.eqToHom h)).hom.hom (S.basis c i)
  = S.basis c1 (Fin.cast (by simp [h]) i)
```

def liftTensor[\[source\]](#)

The lift of a linear map $(S.F.obj \text{ (OverColor.mk } c) \rightarrow_l[S.k] E)$ to a multi-linear map from $\text{fun } i \Rightarrow S.FD.obj \text{ (Discrete.mk } (c \ i))$ (i.e. pure tensors) to E .

```
def liftTensor {n : ℕ} {c : Fin n → S.C} {E : Type} [AddCommMonoid E] [Module S.k E] :
  MultilinearMap S.k (fun i => S.FD.obj (Discrete.mk (c i))) E ≈l[S.k]
  (S.F.obj (OverColor.mk c) →l[S.k] E)
```

2.75 PhysLean.Relativity.Tensors.TensorSpecies.Basis

[PhysLean/Relativity/Tensors/TensorSpecies/Basis.lean](#) on GitHub.

def coordinateMultiLinearSingle[\[source\]](#)

The multi-linear map from $(\text{fun } i \Rightarrow S.FD.obj \text{ (Discrete.mk } (c \ i)))$ to $S.k$ giving the coordinate with respect to the basis described by b .

```
def coordinateMultiLinearSingle {n : ℕ} (c : Fin n → S.C) (b : Π j, Fin (S.repDim (c j))) :
  MultilinearMap S.k (fun i => S.FD.obj (Discrete.mk (c i))) S.k where
```

def coordinateMultiLinear[\[source\]](#)

The multi-linear map from $(\text{fun } i \Rightarrow S.FD.obj \text{ (Discrete.mk } (c \ i)))$ to $(\Pi j, \text{Fin } (S.repDim (c j))) \rightarrow S.k$ giving the coordinates with respect to the basis defined in S .

```
def coordinateMultiLinear {n : ℕ} (c : Fin n → S.C) :
  MultilinearMap S.k (fun i => S.FD.obj (Discrete.mk (c i)))
  ((Π j, Fin (S.repDim (c j))) → S.k) where
```

def coordinate[\[source\]](#)

The linear map from tensors to coordinates.

```
def coordinate {n : ℕ} (c : Fin n → S.C) :
  S.F.obj (OverColor.mk c) →l[S.k] ((Π j, Fin (S.repDim (c j))) → S.k)
```

lemma coordinate_tprod[\[source\]](#)

```
lemma coordinate_tprod {n : ℕ} (c : Fin n → S.C) (x : (i : Fin n) → S.FD.obj (Discrete.mk (c i)
)) :
  S.coordinate c (PiTensorProduct.tprod S.k x) = S.coordinateMultiLinear c x
```

def basisVector[\[source\]](#)

The basis vector of tensors given a $b : \prod j, \text{Fin } (S.\text{repDim } (c j))$.

```
def basisVector {n : ℕ} (c : Fin n → S.C) (b : Π j, Fin (S.repDim (c j))) :
  S.F.obj (OverColor.mk c)
```

lemma coordinate_basisVector[\[source\]](#)

```
lemma coordinate_basisVector {n : ℕ} (c : Fin n → S.C) (b1 b2 : Π j, Fin (S.repDim (c j))) :
  S.coordinate c (S.basisVector c b1) b2 = if b1 = b2 then 1 else 0
```

def fromCoordinates[\[source\]](#)

The linear map taking a $((\prod j, \text{Fin } (S.\text{repDim } (c j))) \rightarrow S.k)$ to a tensor, defined by summing over basis.

```
def fromCoordinates {n : ℕ} (c : Fin n → S.C) :
  ((Π j, Fin (S.repDim (c j))) → S.k) →l[S.k] S.F.obj (OverColor.mk c) where
```

lemma coordinate_fromCoordinate_left_inv[\[source\]](#)

```
lemma coordinate_fromCoordinate_left_inv {n : ℕ} (c : Fin n → S.C) :
  Function.LeftInverse (S.fromCoordinates c) (S.coordinate c)
```

lemma coordinate_fromCoordinate_right_inv[\[source\]](#)

```
lemma coordinate_fromCoordinate_right_inv {n : ℕ} (c : Fin n → S.C) :
  Function.RightInverse (S.fromCoordinates c) (S.coordinate c)
```

def tensorBasis[\[source\]](#)

The basis of tensors.

```
def tensorBasis {n : ℕ} (c : Fin n → S.C) :
  Basis (Π j, Fin (S.repDim (c j))) S.k (S.F.obj (OverColor.mk c)) where
```

lemma tensorBasis_eq_basisVector

[\[source\]](#)

```
lemma tensorBasis_eq_basisVector {n : ℕ} (c : Fin n → S.C) (b : Π j, Fin (S.repDim (c j))) :
  S.tensorBasis c b = S.basisVector c b
```

lemma tensorBasis_repr_tprod

[\[source\]](#)

```
lemma tensorBasis_repr_tprod {n : ℕ} {c : Fin n → S.C}
  (x : (i : Fin n) → S.FD.obj (Discrete.mk (c i)))
  (b : Π j, Fin (S.repDim (c j))) :
  (S.tensorBasis c).repr (PiTensorProduct.tprod S.k x) b =
  Π i, (S.basis (c i)).repr (x i) (b i)
```

def congr

[\[source\]](#)

The equivalence between the indexing set of basis of Lorentz tensors induced by an equivalence on indices.

```
def congr {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin n ≈ Fin m) (h : ∀ i, c i = c1 (σ i)) :
  (Π j, Fin (S.repDim (c1 j))) ≈ Π j, Fin (S.repDim (c j)) where
```

def elimEquiv

[\[source\]](#)

The equivalence between the coordinate parameters $(\Pi j, \text{Fin } (S.\text{repDim } (\text{Sum.elim } c \text{ } c1 \text{ } j)))$ and $(\Pi j, \text{Fin } (S.\text{repDim } (c \text{ } j))) \times (\Pi j, \text{Fin } (S.\text{repDim } (c1 \text{ } j)))$ formed by splitting up based on j .

```
def elimEquiv {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C} :
  (Π j, Fin (S.repDim (Sum.elim c c1 j))) ≈
  (Π j, Fin (S.repDim (c j))) × (Π j, Fin (S.repDim (c1 j))) where
```

def prodEquiv

[\[source\]](#)

The equivalence between the coordinate parameters $(\Pi j, \text{Fin } (S.\text{repDim } (\text{Sum.elim } c \text{ } c1 \text{ } (\text{finSumFinEquiv.symm } j))))$ and $(\Pi j, \text{Fin } (S.\text{repDim } (c \text{ } j))) \times (\Pi j, \text{Fin } (S.\text{repDim } (c1 \text{ } j)))$ formed by splitting up based on j .

```
def prodEquiv {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C} :
  (Π j, Fin (S.repDim (Sum.elim c c1 (finSumFinEquiv.symm j)))) ≈
  (Π j, Fin (S.repDim (c j))) × (Π j, Fin (S.repDim (c1 j)))
```

lemma prodEquiv_apply_fst

[\[source\]](#)

```
@[simp]
lemma prodEquiv_apply_fst {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (b : Π j, Fin (S.repDim (Sum.elim c c1 (finSumFinEquiv.symm j))))
  (j : Fin n) :
  (prodEquiv b).1 j = (Fin.cast (by simp) (b (Fin.castAdd m j)))
```


lemma prodEquiv_apply_snd[\[source\]](#)

```
@[simp]
lemma prodEquiv_apply_snd {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (b : Π j, Fin (S.repDim (Sum.elim c c1 (finSumFinEquiv.symm j))))
  (j : Fin m) :
  (prodEquiv b).2 j = (Fin.cast (by simp) (b (Fin.natAdd n j)))
```

lemma tensorBasis_prod[\[source\]](#)

```
lemma tensorBasis_prod {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (b : Π j, Fin (S.repDim (Sum.elim c c1 (finSumFinEquiv.symm j)))) :
  S.tensorBasis (Sum.elim c c1 ∘ finSumFinEquiv.symm) b =
  (S.F.map (OverColor.equivToIso finSumFinEquiv).hom).hom
  ((Functor.LaxMonoidal.μ S.F _ _).hom
   (S.tensorBasis c (prodEquiv b).1 ⊗ₜ[S.k] S.tensorBasis c1 (prodEquiv b).2))
```

lemma map_tensorBasis[\[source\]](#)

```
lemma map_tensorBasis {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : (OverColor.mk c) → (OverColor.mk c1)} (b : Π j, Fin (S.repDim (c j))) :
  (S.F.map σ).hom.hom (S.tensorBasis c b) =
  S.tensorBasis c1 ((congr (OverColor.Hom.toEquiv σ)
    (OverColor.Hom.toEquiv_comp_apply σ)).symm b)
```

lemma contrMap_tensorBasis[\[source\]](#)

```
lemma contrMap_tensorBasis {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ} {h : c (i.succAbove j) = S.τ (c i)}
  (b : Π j, Fin (S.repDim (c j))) :
  (S.contrMap c i j h).hom (S.tensorBasis c b) =
  (S.contr.app (Discrete.mk (c i))).hom
  (S.basis (c i) (b i) ⊗ₜ[S.k] S.basis (S.τ (c i)) (Fin.cast (by rw [h]) (b (i.succAbove j)))
   ) •
  (S.tensorBasis (c ∘ i.succAbove ∘ j.succAbove)
   (fun k => b ((i.succAbove ∘ j.succAbove) k)))
```

def ContrSection[\[source\]](#)

Given a coordinate parameter $b : \Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$, the coordinate parameter $\Pi k, \text{Fin } (S.\text{repDim } (c k))$ which map down to b .

```
def ContrSection {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i : Fin n.succ.succ} {j : Fin n.succ}
  (b : Π k, Fin (S.repDim ((c ∘ i.succAbove ∘ j.succAbove) k))) :
  Finset (Π k, Fin (S.repDim (c k)))
```

lemma pairIsoSep_tensorBasis_repr[\[source\]](#)

```
lemma pairIsoSep_tensorBasis_repr {c c1 : S.C}
  (t : (S.FD.obj { as := c } ⊗ S.FD.obj { as := c1 }).V)
  (b : ((j : Fin (Nat.succ 0).succ) → Fin (S.repDim (![c, c1] j)))) :
  (S.tensorBasis ![c, c1]).repr
  ((OverColor.Discrete.pairIsoSep S.FD).hom.hom t) b =
  (Basis.tensorProduct (S.basis c) (S.basis c1)).repr t (b 0, b 1)
```

2.76 PhysLean.Relativity.Tensors.TensorSpecies.OfInt

[PhysLean/Relativity/Tensors/TensorSpecies/OfInt.lean](#) on GitHub.

def tensorOfInt [\[source\]](#)

A tensor from a $(\prod j, \text{Fin } (S.\text{repDim } (c j))) \rightarrow \mathbb{Z}$. All tensors which have integer coefficients with respect to `tensorBasis` are of this form.

```
noncomputable def tensorOfInt {n : ℕ} {c : Fin n → S.C}
  (f : (∏ j, Fin (S.repDim (c j))) → ℤ) :
  S.F.obj (OverColor.mk c)
```

lemma tensorOfInt_tensorBasis_repr_apply [\[source\]](#)

@[simp]

```
lemma tensorOfInt_tensorBasis_repr_apply {n : ℕ} {c : Fin n → S.C}
  (f : (∏ j, Fin (S.repDim (c j))) → ℤ) (b : ∏ j, Fin (S.repDim (c j))) :
  (S.tensorBasis c).repr (S.tensorOfInt f) b = Int.cast (f b)
```

lemma tensorBasis_eq_ofInt [\[source\]](#)

```
lemma tensorBasis_eq_ofInt {n : ℕ} {c : Fin n → S.C}
  (b : ∏ j, Fin (S.repDim (c j))) :
  S.tensorBasis c b
  = S.tensorOfInt (fun b' => if b = b' then 1 else 0)
```

2.77 PhysLean.Relativity.Tensors.Tree.Basic

[PhysLean/Relativity/Tensors/Tree/Basic.lean](#) on GitHub.

lemma tensorNode_tensorBasis_repr [\[source\]](#)

```
lemma tensorNode_tensorBasis_repr {c : Fin n → S.C} (T : S.F.obj (OverColor.mk c)) :
  (S.tensorBasis c).repr (tensorNode T).tensor = (S.tensorBasis c).repr T
```

lemma add_tensorBasis_repr [\[source\]](#)

@[simp]

```
lemma add_tensorBasis_repr (t1 t2 : TensorTree S c) :
  (S.tensorBasis c).repr (add t1 t2).tensor = (S.tensorBasis c).repr t1.tensor +
  (S.tensorBasis c).repr t2.tensor
```

lemma prod_tensorBasis_repr_apply [\[source\]](#)

```
lemma prod_tensorBasis_repr_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (t : TensorTree S c) (t1 : TensorTree S c1)
  (b : ∏ j, Fin (S.repDim ((Sum.elim c c1) ∘ finSumFinEquiv.symm) j))) :
  (S.tensorBasis (Sum.elim c c1) ∘ finSumFinEquiv.symm).repr (prod t t1).tensor b =
  (S.tensorBasis c).repr t.tensor (TensorBasis.prodEquiv b).1 *
  (S.tensorBasis c1).repr t1.tensor (TensorBasis.prodEquiv b).2
```

lemma contr_tensorBasis_repr_apply[\[source\]](#)

```

lemma contr_tensorBasis_repr_apply {n : ℕ} {c : Fin (n + 1 + 1) → S.C} {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = S.τ (c i)} (t : TensorTree S c)
  (b : Π k, Fin (S.repDim (c (i.succAbove (j.succAbove k))))) :
  (S.tensorBasis (c ∘ i.succAbove ∘ j.succAbove)).repr (contr i j h t).tensor b =
  ∑ (b' : TensorBasis.ContrSection b),
  ((S.tensorBasis c).repr t.tensor b'.1) *
  S.castToField ((S.contr.app (Discrete.mk (c i))).hom
  (S.basis (c i) (b'.1 i) ⊗t[S.k] S.basis (S.τ (c i)) (Fin.cast (by rw [h])
  (b'.1 (i.succAbove j))))))

```

lemma contr_tensorBasis_repr_apply_eq[\[source\]](#)

```

lemma contr_tensorBasis_repr_apply_eq {n : ℕ} {c : Fin (n + 1 + 1) → S.C} {i : Fin (n + 1 + 1)}
  {j : Fin (n + 1)} {h : c (i.succAbove j) = S.τ (c i)} (t : TensorTree S c)
  (b : Π k, Fin (S.repDim (c (i.succAbove (j.succAbove k))))) (a : S.k) :
  (S.tensorBasis (c ∘ i.succAbove ∘ j.succAbove)).repr (contr i j h t).tensor b = a
  ↔
  ∑ (b' : TensorBasis.ContrSection b),
  ((S.tensorBasis c).repr t.tensor b'.1) *
  S.castToField ((S.contr.app (Discrete.mk (c i))).hom
  (S.basis (c i) (b'.1 i) ⊗t[S.k] S.basis (S.τ (c i)) (Fin.cast (by rw [h])
  (b'.1 (i.succAbove j)))))) = a

```

lemma perm_tensorBasis_repr_apply[\[source\]](#)

```

@[simp]
lemma perm_tensorBasis_repr_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : (OverColor.mk c) → (OverColor.mk c1)} (t : TensorTree S c)
  (b : Π j, Fin (S.repDim (c1 j))) :
  (S.tensorBasis c1).repr (perm σ t).tensor b =
  (S.tensorBasis c).repr t.tensor
  (TensorBasis.congr (OverColor.Hom.toEquiv σ) (OverColor.Hom.toEquiv_comp_apply σ) b)

```

lemma smul_tensorBasis_repr[\[source\]](#)

```

@[simp]
lemma smul_tensorBasis_repr {c : Fin n → S.C} (a : S.k) (T : TensorTree S c) :
  (S.tensorBasis c).repr (smul a T).tensor = a • (S.tensorBasis c).repr T.tensor

```

lemma smul_tensorBasis_repr_apply[\[source\]](#)

```

lemma smul_tensorBasis_repr_apply {c : Fin n → S.C} (a : S.k) (T : TensorTree S c)
  (b : Π j, Fin (S.repDim (c j))) :
  (S.tensorBasis c).repr (smul a T).tensor b = a * (S.tensorBasis c).repr T.tensor b

```

lemma neg_tensorBasis_repr[\[source\]](#)

```

@[simp]
lemma neg_tensorBasis_repr (t : TensorTree S c) :
  (S.tensorBasis c).repr (neg t).tensor = - (S.tensorBasis c).repr t.tensor

```

2.78 `scripts.check_file_imports`

[scripts/check_file_imports.lean](#) on GitHub.

def `expectedPhysLeanImports`

[\[source\]](#)

Compute imports expected by `PhysLean.lean` by looking at file structure.

```
def expectedPhysLeanImports : IO (Array Name)
```

2.79 `scripts.find_TODOs`

[scripts/find_TODOs.lean](#) on GitHub.

def `PhysLeanTODOItem`

[\[source\]](#)

Given a list of lines, outputs an error message and a line number.

```
def PhysLeanTODOItem : Type
```

2.80 `scripts.hepLean_style_lint`

[scripts/hepLean_style_lint.lean](#) on GitHub.

def `PhysLeanTextLinter`

[\[source\]](#)

Given a list of lines, outputs an error message and a line number.

```
def PhysLeanTextLinter : Type
```

structure `PhysLeanErrorContext`

[\[source\]](#)

```
structure PhysLeanErrorContext where
  error : String
  lineNumber : ℕ
  columnNumber : ℕ
  path : FilePath
```

2.81 `scripts.MetaPrograms.informal`

[scripts/MetaPrograms/informal.lean](#) on GitHub.

structure `Decl`

[\[source\]](#)

```
structure Decl where
  name : Name
  module : Name
  lineNo : Nat
  docString : String
```

inductive `InformalDeclKind`

[\[source\]](#)

```
inductive InformalDeclKind
| def | lemma
```

instance ToString InformalDeclKind[\[source\]](#)

```
instance : ToString InformalDeclKind where
```

structure InformalDecl[\[source\]](#)

```
structure InformalDecl extends Decl where
  kind : InformalDeclKind
  deps : Array Decl
```

structure DepDecls[\[source\]](#)

```
structure DepDecls where
  private mk::
  private decls : Std.HashMap Name Decl
  formalDecls : Array Decl
  informalModuleMap : Array (Name × Array InformalDecl)
```

abbrev DeclSM[\[source\]](#)

```
private abbrev DeclSM
```

def ofName[\[source\]](#)

```
private def Decl.ofName (name : Name) (module : Option Name := none) : DeclSM Decl
```

def ofName?[\[source\]](#)

```
private unsafe def InformalDecl.ofName? (name module : Name) : DeclSM (Option InformalDecl)
```

def ofRootModule[\[source\]](#)

```
unsafe def DepDecls.ofRootModule (rootModule : Name) : CoreM DepDecls
```

def mkMarkdown[\[source\]](#)

Making the Markdown file for dependency graph.

```
def mkMarkdown (depDecls : DepDecls) : IO Unit
```

def mkDOT[\[source\]](#)

Making the DOT file for dependency graph.

```
def mkDOT (depDecls : DepDecls) : IO Unit
```

def mkHTML[\[source\]](#)

Making the HTML file for dependency graph.

```
def mkHTML (depDecls : DepDecls) : IO Unit
```

def withStdoutRedirectedTo[\[source\]](#)

```
def IO.withStdoutRedirectedTo {α} (filePath : System.FilePath) (action : IO α) : IO α
```

2.82 scripts.MetaPrograms.notes

[scripts/MetaPrograms/notes.lean](#) on GitHub.

inductive NameStatus[\[source\]](#)

```
inductive NameStatus
| complete : NameStatus
| incomplete : NameStatus
```

instance ToString NameStatus[\[source\]](#)

```
instance : ToString NameStatus where
```

def harmonicOscillator[\[source\]](#)

```
def harmonicOscillator : Note where
```